Unidad 2 — Procesos, hilos, concurrencia y planificación

Sistemas Operativos — UTN FRM — Repaso de la clase completa en el chat: 10 temas, correcciones y cierre

> **Cómo leer este repaso.** Es la clase tal como se dio, tema por tema, con los mismos diagramas. Al final de cada tema están las **preguntas de control** y, a continuación, su **corrección**: ahí quedaron marcados los errores que conviene no repetir. La referencia al Tanenbaum está al final de cada tema. El resumen de todo está en el **Cierre de la Unidad 2**, en las últimas páginas.

Mapa de la unidad

Esta unidad es una sola historia con tres actos. Mirá cómo encadena:



La pregunta que recorre todo: si varias cosas se ejecutan a la vez sobre recursos compartidos, ¿cómo hacemos para que no se arruinen entre sí y para repartir la CPU con criterio?

Son **10 temas**. El 8 es el más pesado y el que más vale en tu examen.

Tema 1 — Qué es un proceso

La intuición

La mejor analogía es la del libro, y vale la pena desarrollarla entera:

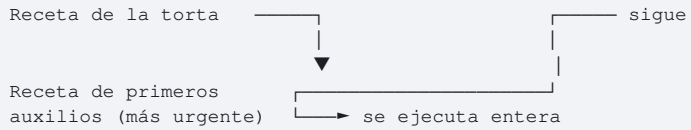
Un informático está haciendo una torta para su hija. Tiene:

```
LA RECETA       →  el programa (instrucciones escritas)
LOS INGREDIENTES →  los datos de entrada
ÉL COCINANDO     →  el PROCESO (la actividad de ejecutar)
ÉL               →  la CPU
```

De pronto el chico entra llorando: se cortó un dedo.

El informático **anota en qué paso de la receta estaba** (eso es el estado), deja la torta, busca el manual de primeros auxilios y **empieza a seguir otra receta**: la de curar heridas.

Cuando termina, vuelve a la cocina y retoma la torta **exactamente en el paso donde la había dejado**.



Lo que se guardó y se restauró no fue la receta — esa no cambió nunca. Fue **el estado de la actividad**.

El concepto formal

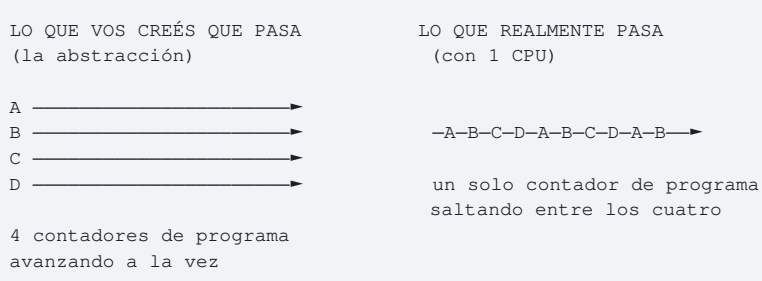
> **Un proceso es una instancia de un programa en ejecución**, incluyendo los valores actuales del contador de programa, los registros y las variables.

La diferencia entre programa y proceso es **sutil pero crucial**:

Programa	Proceso
Algo **pasivo**: un archivo en el disco	Algo **activo**: una actividad en curso
No cambia	Tiene estado, y el estado cambia
Uno solo	**Varios procesos pueden correr el mismo programa**

Ese último punto: si abrís tres ventanas del navegador, hay **un** programa y **tres** procesos, cada uno con su propia memoria, sus propias variables y su propio contador de programa.

Pseudoparalelismo — esto se toma



El libro lo dice sin vueltas: *"hablando en sentido estricto, en cualquier instante la CPU está ejecutando sólo un proceso"*. La CPU conmuta entre procesos cada **décimas o centésimas de milisegundo**, y eso produce la apariencia de paralelismo.

```
PSEUDOPARALELISMO  →  la ILUSIÓN producida por conmutación rápida
                      sobre UNA CPU

PARALELISMO REAL   →  dos o más CPU ejecutando de verdad
                      al mismo instante
```

> Esta distinción vale puntos. Un sistema con 50 procesos sobre un solo núcleo es **concurrente**, no **paralelo**.

Creación de procesos: los 4 eventos

```
1. El arranque del sistema
2. Una llamada al sistema hecha por otro proceso      ← la más común
```

```
3. Una petición del usuario (escribir un comando, doble clic)
4. El inicio de un trabajo por lotes
```

Los procesos que arrancan con el sistema y se quedan en segundo plano —correo, web, impresión— se llaman **demonios**.

La llamada que crea procesos

Acá está una de las diferencias de diseño más importantes de la materia:

```
UNIX / LINUX                        WINDOWS
____________                        _______
fork()    → duplica el proceso      CreateProcess()
            (clon exacto)             hace LAS DOS COSAS
                                       en una sola llamada
execve()  → reemplaza su imagen       (10+ parámetros)
            por otro programa

DOS llamadas separadas              UNA sola llamada
```

Después de `fork`, padre e hijo tienen **la misma imagen de memoria, las mismas cadenas de entorno y los mismos archivos abiertos**. Son clones.

Ejemplo resuelto — ¿cómo sabe el hijo que es el hijo?

Problema: después del fork hay **dos procesos ejecutando el mismo código**. ¿Cómo hace cada uno para saber cuál es?

```
Paso 1   El padre ejecuta fork()
Paso 2   Ahora hay DOS procesos, los dos siguen en la línea siguiente,
         los dos tienen el mismo código

Paso 3   La diferencia está en el VALOR DE RETORNO de fork:

         ┌──────────────────────────────────────┐
         │  en el PADRE  →  devuelve el PID     │   (número > 0)
         │                  del hijo            │
         │  en el HIJO   →  devuelve 0          │
         │  si falló     →  devuelve -1         │
         └──────────────────────────────────────┘

Paso 4   El programa bifurca con ese número:

         si vale 0        → soy el hijo
         si vale > 0      → soy el padre, y ese número es el PID de mi hijo
         si vale -1       → error, no se creó nada
```

El código es idéntico en los dos. Lo único que los distingue es un número. Esa asimetría es la base de todo el modelo de procesos de UNIX.

Terminación: 4 razones, 2 y 2

```
VOLUNTARIAS (las decide el proceso)
  1. Salida normal      → terminó su trabajo.  exit() / ExitProcess()
  2. Salida por error   → pide compilar un archivo que no existe

INVOLUNTARIAS (se las imponen)
  3. Error fatal        → instrucción ilegal, división por cero,
                          referencia a memoria inexistente
  4. Eliminado por otro → kill() / TerminateProcess()
```

Jerarquía de procesos — otra diferencia UNIX/Windows

```
UNIX: un ÁRBOL real                 WINDOWS: sin jerarquía

        init  ← el primero                  A    B    C    D
       /  |  \                              todos iguales
      A   B   C
     / \      |                     El padre recibe un token para
    D   E     F                     controlar al hijo, pero puede
                                    pasárselo a otro → se rompe
Un proceso NO puede
```

```
desheredar a sus hijos
```

Al arrancar, UNIX crea **init**, que lee cuántas terminales hay y hace `fork` por cada una. **Todos los procesos del sistema cuelgan de init.** Por eso `pstree` dibuja un árbol.

Referencia

(Tanenbaum, cap. 2.1.1 a 2.1.4, págs. impresas **83 a 90** → PDF **113 a 120**).

Preguntas de control

1. Un programa ejecuta **tres `fork()` seguidos**, sin ningún `if` en el medio. ¿Cuántos procesos hay al final? Justificá el razonamiento, no solo el número.

2. Abrís el mismo editor de texto tres veces. ¿Cuántos **programas** hay y cuántos **procesos**? ¿Qué comparten y qué no?

Corrección

2 — Bien. 1 programa, 3 procesos. Precisemos qué comparten y qué no, porque eso vale puntos:

```
COMPARTEN              NO COMPARTEN (cada uno el suyo)
─────────              ──────────────────────────────
el código del          · PID
programa               · espacio de direcciones (su memoria)
(incluso puede estar   · variables y sus valores
 físicamente una sola  · contador de programa
 vez en memoria)       · pila y registros
                       · archivos abiertos
```

Por eso podés tener un documento distinto en cada ventana: **el código es el mismo, el estado es propio de cada uno.**

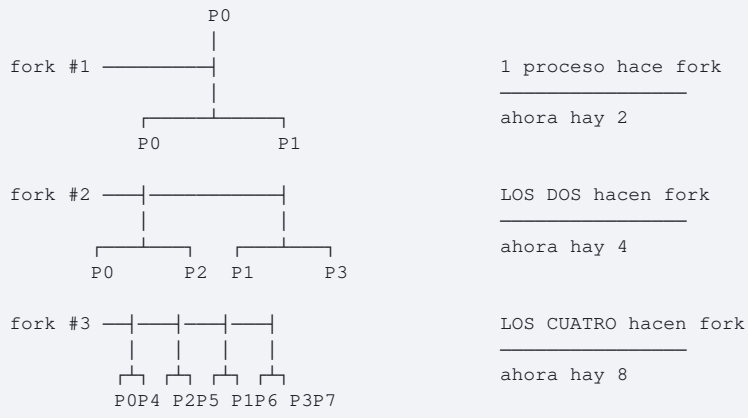
---

1 — Acá caíste justo en la trampa. Son **8**, no 4.

Tu razonamiento fue: cada fork agrega un proceso. Pero lo que pasa es otra cosa:

> **Después del primer fork hay dos procesos ejecutando el mismo código. Los dos llegan al segundo fork. Y los dos lo ejecutan.**

Mirá el árbol:



La cuenta:

```
antes del fork 1:  1 proceso
después:           2        ← se duplicó  (1 × 2)
después del 2:     4        ← se duplicó  (2 × 2)
después del 3:     8        ← se duplicó  (4 × 2)

REGLA:  2ⁿ procesos, donde n = cantidad de fork
```

La clave del error: pensaste en el fork como "crear un proceso". Es más preciso pensarlo como **"duplicar todo lo que hay"**. No suma uno — multiplica por dos.

Y el detalle que lo hace contraintuitivo: el proceso hijo **no empieza desde el principio del programa**. Arranca justo después del fork que lo creó. Por eso el hijo del primer fork también se topa con el segundo y el tercero.

> **Para el examen:** si ves 3 `fork()` seguidos sin `if`, la respuesta es **8 procesos** (7 hijos + el original).

Un caso que te puede aparecer con trampa:

```
fork();
if (fork() == 0) {      ← este fork lo ejecutan 2 procesos
    fork();             ← este SOLO lo ejecutan los hijos
}
```

Ahí ya no es 2^3. Hay que seguir el árbol rama por rama.

Tema 2 — Estados del proceso y el BCP

La intuición

Pensá en la guardia de un hospital. Un paciente puede estar en tres situaciones:

```
EN EL CONSULTORIO     → lo está atendiendo el médico ahora
EN LA SALA DE ESPERA  → listo para ser atendido, solo falta que lo llamen
EN EL LABORATORIO     → esperando el resultado de un análisis;
                        aunque el médico esté libre, NO puede atenderlo
```

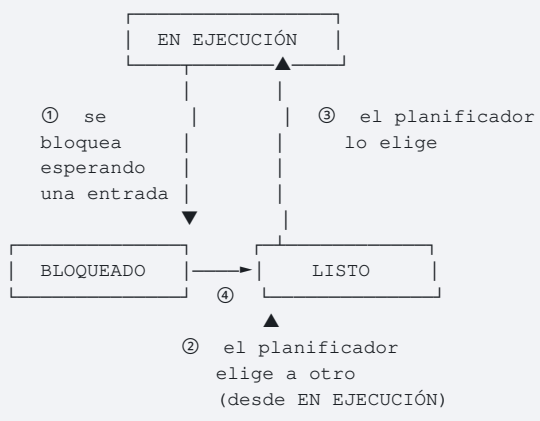
Esa tercera situación es la clave. El paciente del laboratorio **no puede ser atendido aunque el consultorio esté vacío**, porque le falta algo externo. El de la sala de espera sí podría: lo único que le falta es el turno.

El concepto formal: los tres estados

Estado	Qué significa	¿Podría ejecutarse ya?
En ejecución	Está usando la CPU en este instante	Ya lo está
Listo (preparado)	Es ejecutable; se detuvo para que corra otro	**Sí** — solo le falta la CPU
Bloqueado	No puede continuar hasta que ocurra un evento externo	**No** — le falta un dato

> **La confusión que más se cobra en los exámenes es listo vs. bloqueado.**
> **Bloqueado** = no puede por lógica. Está esperando una entrada que no existe todavía.
> **Listo** = sí podría. Lo único que le falta es que el planificador lo elija.

El diagrama — dibujalo de memoria



Las cuatro transiciones, con **quién las provoca**:

#	Transición	¿Quién la provoca?
①	Ejecución → **Bloqueado**	**El propio proceso**: pidió algo que no está
②	Ejecución → **Listo**	**El planificador**: se acabó el quantum o llegó alguien mejor
③	Listo → **Ejecución**	**El planificador**: te toca
④	Bloqueado → **Listo**	**Un evento externo**: llegó el dato (interrupción)

☆ Lo que más se pregunta de este diagrama

No existe la transición Bloqueado → En ejecución.

```
BLOQUEADO  ──✗──►  EN EJECUCIÓN

BLOQUEADO  ──④──►  LISTO  ──③──►  EN EJECUCIÓN
```

Cuando llega el dato que esperabas, **no volvés a ejecutarte de inmediato**: pasás a la cola de listos y esperás tu turno. La CPU puede estar ocupada con otro, o puede haber diez procesos listos antes que vos.

Y las transiciones ① y ④ son **opuestas pero asimétricas**:

- Bloquearse es **inmediato** (vos lo pedís)
- Desbloquearse te deja **en la fila** (te lo conceden)

Ejemplo resuelto — seguir un proceso que lee del disco

```
Paso 1   P está EN EJECUCIÓN, usando la CPU.

Paso 2   P ejecuta read(). Los datos no están en memoria:
         hay que ir al disco, que tarda milisegundos.

Paso 3   ① El SO pasa a P a BLOQUEADO y le pide al driver
         que inicie la operación.

Paso 4   ③ El planificador elige a Q, que estaba LISTO,
         y lo pone EN EJECUCIÓN.
         ← la CPU no queda ociosa: ESTO es la multiprogramación

Paso 5   El disco termina y genera una INTERRUPCIÓN.
         ④ El manejador pasa a P de BLOQUEADO a LISTO.

         ⚠ P NO vuelve a ejecutarse acá. Solo queda habilitado.

Paso 6   ③ Cuando el planificador lo decida, P pasa a
         EN EJECUCIÓN y continúa después del read().
```

El detalle fino: entre el paso 5 y el 6 puede pasar bastante tiempo. Desde adentro, el proceso cree que su `read()` tardó todo eso. Pero una parte fue **espera de disco** (bloqueado) y otra **espera de CPU** (listo). Son dos esperas distintas y es importante no confundirlas.

El BCP — dónde se guarda todo eso

Para que el paso 6 funcione, el sistema tiene que haber guardado **todo** lo necesario para reanudar a P como si nunca se hubiera detenido.

Eso vive en la **tabla de procesos**: un arreglo con **una entrada por proceso**. A cada entrada se la llama **bloque de control de proceso (BCP)**.

```
┌──────────────────────────────────────────────────────┐
│                    BCP  (una entrada)                │
├──────────────────┬──────────────────┬────────────────┤
│ ADMIN. DE        │ ADMIN. DE        │ ADMIN. DE      │
│ PROCESOS         │ MEMORIA          │ ARCHIVOS       │
├──────────────────┼──────────────────┼────────────────┤
│ registros        │ ptr. al segmento │ directorio raíz│
│ contador de      │   de texto       │ directorio de  │
│   programa       │ ptr. al segmento │   trabajo      │
```

```
| PSW              |   de datos       | descriptores de   |
| apuntador de pila| ptr. al segmento |   archivos        |
| ESTADO           |   de pila        | UID, GID          |
| prioridad        |                  |                   |
| PID, PPID        |                  |                   |
| tiempo de CPU    |                  |                   |
| señales          |                  |                   |
└──────────────────┴──────────────────┴───────────────────┘
```

Fijate que los tres primeros campos —registros, contador de programa, PSW— son exactamente los que viste en el **tema 2 de la Unidad 1**. Ahí decíamos *"el SO debe estar al tanto de todos los registros"*. Acá se ve para qué: **para poder guardarlos y restaurarlos**.

El cambio de contexto

```
1. Llega la interrupción
2. El HARDWARE apila el contador de programa y el PSW
3. El hardware carga un nuevo PC desde el VECTOR DE INTERRUPCIONES
4. Rutina en ensamblador: guarda los registros en el BCP del actual
5. Se ejecuta el servicio de la interrupción
6. El PLANIFICADOR decide cuál proceso sigue
7. Se cargan registros y mapa de memoria del nuevo proceso
8. Arranca el nuevo proceso
```

Y esto no es gratis: pasar a modo kernel, guardar y restaurar registros, cambiar el mapa de memoria y, muchas veces, **invalidar la caché**. Demasiadas conmutaciones degradan el rendimiento.

> Guardá este dato: es exactamente el motivo por el que el **tamaño del quantum** de Round Robin es un problema real de diseño, y no un detalle. Lo vemos en el tema 8.

Los comandos de tu cátedra para este tema

Comando	Qué te muestra
`ps -ef`	Todos los procesos: **UID, PID, PPID**, C, STIME, TTY, TIME, CMD
`pstree`	El árbol padre-hijo
`top` / `htop`	En tiempo real. `S` = **estado** del proceso
`pgrep firefox`	El PID buscando por nombre
`cat /proc/PID/status`	Nombre, **estado**, PID, PPID, memoria, cantidad de hilos
`ls -la /proc/PID/fd`	Los descriptores de archivos abiertos
`kill -STOP PID`	**Suspende** un proceso
`kill -CONT PID`	Lo **reanuda**

Esos dos últimos aparecen textualmente en tu examen final práctica. Y `/proc/PID/status` es, literalmente, **asomarse al BCP** desde el espacio de usuario.

Referencia

(Tanenbaum, cap. 2.1.5 y 2.1.6, págs. impresas **90 a 93** → PDF **120 a 123**).

Preguntas de control

1. Un proceso está **bloqueado** esperando que el usuario escriba algo por teclado. El usuario aprieta una tecla. Describí, **usando los números de las transiciones**, qué pasa desde ese momento hasta que el proceso vuelve a ejecutar su código.

---

2. Te dicen: *"como el proceso A tiene más prioridad que B, cuando A se desbloquea, A pasa directamente a*

ejecutarse y desplaza a B".

¿Qué hay de cierto y qué de incorrecto en esa frase?

Corrección

2 — Correcta. Lo cierto y lo incorrecto bien separados.

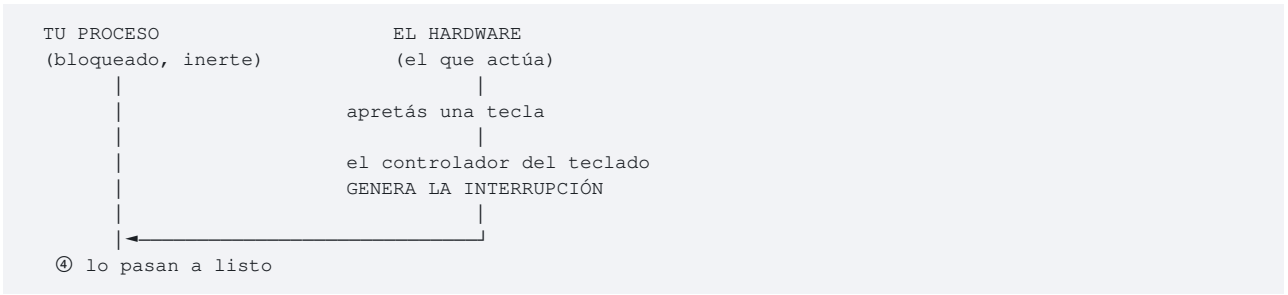
Agrego el final de la secuencia: si A tiene más prioridad, lo que pasa es que el planificador aplica ② a B (lo saca de la CPU) y después ③ a A. Son **dos transiciones**, no un salto. Y en un sistema **no apropiativo** ni siquiera eso: A espera a que B se bloquee o termine, por más prioridad que tenga.

---

1 — Bien la secuencia, pero hay un error de autoría importante.

Dijiste: *"el proceso bloqueado hace una interrupción"*.

Un proceso bloqueado no puede hacer nada. Está bloqueado justamente porque no se ejecuta. La interrupción la genera **el controlador del teclado**, que es hardware:



La diferencia no es un detalle: **el proceso es pasivo en todo este asunto**. Alguien de afuera lo despierta. Por eso la transición ④ se describe como *"ocurre el evento esperado"* y no como *"el proceso decide despertarse"*.

Y un matiz sobre lo de "para la CPU del proceso actual": la interrupción **sí interrumpe momentáneamente** al proceso que está corriendo, pero eso no significa que lo saque. La secuencia completa:

```
1. El teclado genera la interrupción
2. Se interrumpe al proceso actual y corre el MANEJADOR DE INTERRUPCIÓN
3. El manejador aplica ④: el proceso bloqueado pasa a LISTO
4. El planificador decide:
      · si el desbloqueado tiene más prioridad → ② al actual, ③ al desbloqueado
      · si no → el proceso actual SIGUE donde estaba, como si nada
```

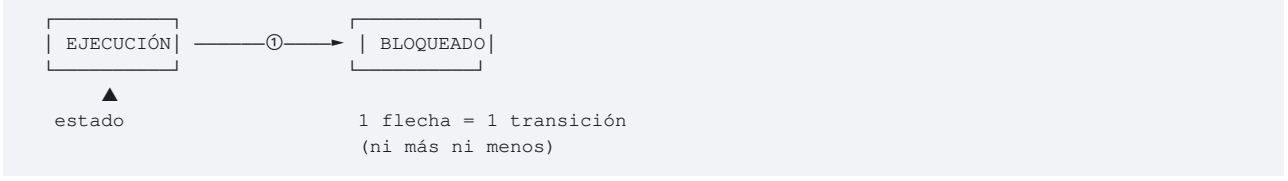
O sea: el proceso actual puede retomar tranquilamente. El manejador es un paréntesis de microsegundos, no una expulsión.

> **Para el examen:** el proceso que se bloquea actúa (①). El que se desbloquea **es actuado** (④). Esa asimetría es el corazón del diagrama.

Una **transición es UN solo cambio de estado**: el salto de un estado a otro. No es un conjunto.

```
ESTADO      →  dónde está el proceso ahora        (una foto)
TRANSICIÓN  →  el salto de un estado a otro       (el movimiento)
```

Es literalmente **una flecha** del diagrama.



Por qué son 4 y no 6

Con 3 estados, en teoría habría 6 saltos posibles (cada estado a cada otro). Pero **solo 4 existen**:

```
De EJECUCIÓN a BLOQUEADO    ①  ✓ existe
De EJECUCIÓN a LISTO        ②  ✓ existe
De LISTO     a EJECUCIÓN    ③  ✓ existe
De BLOQUEADO a LISTO        ④  ✓ existe
──────────────────────────────────────
De BLOQUEADO a EJECUCIÓN       ✗ NO existe  ← la famosa
De LISTO     a BLOQUEADO       ✗ NO existe
```

La última también vale la pena: **un proceso listo no puede bloquearse**, porque para bloquearse tiene que *pedir* algo, y para pedir algo tiene que estar ejecutándose. Si no tiene la CPU, no puede hacer nada — ni siquiera bloquearse.

Cuando un proceso hace varios saltos

Eso es una **secuencia de transiciones**, y se escribe encadenando los números:

```
"el proceso volvió a ejecutarse después de leer del disco"

   ①    →    ④    →    ③
bloqueó   llegó el   lo eligió
          dato       el planificador

EJECUCIÓN → BLOQUEADO → LISTO → EJECUCIÓN

3 transiciones, 4 estados recorridos
```

> **La regla:** contá las flechas, no los estados. En una secuencia siempre hay **una transición menos** que estados visitados.

Por eso, cuando en un examen te preguntan *"¿cómo vuelve a ejecutarse un proceso bloqueado?"*, la respuesta con todas las letras es: ④ **y después** ③. Dos transiciones. Nunca una sola.

Tema 3 — Hilos: el modelo clásico

La intuición

Volvamos a la cocina del tema 1. Un cocinero haciendo una torta es **un proceso con un hilo**.

Ahora imaginá un restaurante: **una cocina** (con sus ingredientes, sus ollas, su heladera) y **tres cocineros trabajando adentro**.

```
PROCESO con 1 hilo              PROCESO con 3 hilos
(cocina + 1 cocinero)           (cocina + 3 cocineros)

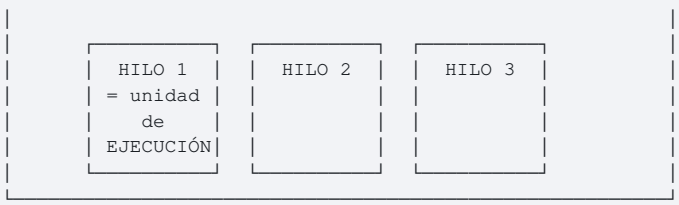
┌──────────────────┐            ┌──────────────────┐
│    LA COCINA     │            │    LA COCINA     │
│  ingredientes    │            │  ingredientes    │  ← compartidos
│  ollas, heladera │            │  ollas, heladera │
│                  │            │                  │
│       ♟          │            │   ♟  ♟  ♟        │  ← cada uno con su
│   1 cocinero     │            │   3 cocineros    │     propia tarea
└──────────────────┘            └──────────────────┘
```

Los tres cocineros **comparten la cocina entera**. Si uno saca la leche de la heladera, los otros ven que ya no está. No hay paredes entre ellos, y eso es **a propósito**: están para cooperar.

El concepto formal

El modelo clásico de hilos **separa dos cosas** que en el proceso tradicional estaban mezcladas:

```
┌──────────────────────────────────────────────────┐
│  EL PROCESO = agrupamiento de RECURSOS           │
│  (la cocina: espacio de direcciones, archivos,   │
│   señales, procesos hijos)                       │
```



El proceso agrupa. El hilo ejecuta.

Por eso a los hilos se los llama también **procesos ligeros** (lightweight processes).

☆ **La tabla que más se toma del tema**

Por PROCESO (compartido por todos sus hilos)	Por HILO (propio de cada uno)
Espacio de direcciones	**Contador de programa**
Variables globales	**Registros**
Archivos abiertos	**Pila**
Procesos hijos	**Estado**
Alarmas pendientes	
Señales y manejadores de señales	
Información contable	

Memorizá la columna derecha: son solo 4 cosas. Todo lo demás es del proceso.

Y hay una lógica detrás: lo propio del hilo es exactamente **lo que hace falta para saber dónde vas en la ejecución**. Lo del proceso es **con qué trabajás**.

Por qué cada hilo necesita su propia pila

Esta es la pregunta de control clásica.

```
HILO 1                          HILO 2
main()                          main()
  └─ procesarPedido()             └─ actualizarPantalla()
       └─ validarDatos()               └─ dibujarBoton()
            └─ ← acá está                   └─ ← acá está

Cada uno tiene un HISTORIAL DE LLAMADAS distinto.
La pila guarda ese historial: direcciones de retorno
y variables locales de cada procedimiento activo.
```

Si compartieran la pila, cuando el hilo 2 hiciera `return`, volvería a… ¿dónde? Al procedimiento del hilo 1. **Caos total.**

Acordate del tema 6b de la Unidad 1: `CALL` apila la dirección de retorno. Si dos hilos apilan en el mismo lugar, se pisan los retornos.

Una propiedad incómoda: no hay protección entre hilos

```
PROCESOS                        HILOS (del mismo proceso)
────────                        ─────────────────────────
compiten entre sí               cooperan
están PROTEGIDOS                NO hay protección:
unos de otros                   un hilo puede leer, escribir
                                o BORRAR la pila de otro
```

Y eso no es un error de diseño, es intencional. Los hilos de un proceso siempre son del mismo usuario y están pensados para trabajar juntos. Poner barreras entre ellos sería contraproducente — y además imposible, porque comparten el espacio de direcciones.

El precio: si un hilo corrompe memoria, **se lleva puesto a todo el proceso**. Por eso Chrome usa un proceso por pestaña y no un hilo: para que una pestaña que falla no tire el navegador entero.

Las 4 razones para usar hilos

```
1. Varias actividades a la vez, y algunas se bloquean
   → dividir en hilos secuenciales simplifica el modelo de programación

2. COMPARTEN el espacio de direcciones y los datos
   → esencial donde procesos separados no servirían

3. Son más LIGEROS: se crean y destruyen mucho más rápido
   → unas 10 a 100 veces más rápido que un proceso

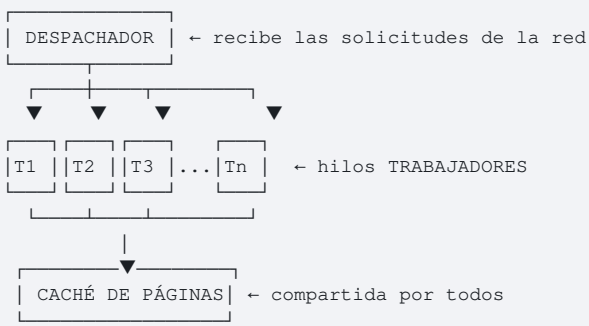
4. Permiten TRASLAPAR cálculo y E/S
   → y con varias CPU, paralelismo REAL
```

Ejemplo concreto 1 — el procesador de palabras

```
┌──────────────── el documento (compartido) ────────────────┐
│                                                           │
│   HILO 1              HILO 2              HILO 3          │
│   interactúa con      reformatea el       hace respaldos  │
│   el usuario          documento en        automáticos     │
│   (recibe teclas)     segundo plano       a disco         │
└───────────────────────────────────────────────────────────┘
```

¿Por qué no procesos? Porque no compartirían el documento. **¿Por qué no un solo hilo?** Porque tendrías que esperar a que termine el reformateo antes de poder seguir escribiendo.

Ejemplo concreto 2 — el servidor web, y la comparación que se toma



Las **tres formas** de construir ese servidor:

Modelo	Paralelismo	Llamadas al sistema	Veredicto
Hilos	Sí	con bloqueo	✓ Programación simple **y** buen rendimiento
Un solo hilo	**No**	con bloqueo	Simple pero **lento**: mientras espera el disco, la CPU está ociosa
Máquina de estados finitos	Sí	**sin** bloqueo	Rápido pero **muy difícil de programar**

Esa última merece una línea: sin hilos pero queriendo rendimiento, hay que usar **E/S sin bloqueo** y guardar a mano el estado de cada petición en una tabla, reaccionando a eventos. Funciona, pero es un infierno de programar.

La frase del libro: los hilos combinan lo mejor — **paralelismo con llamadas al sistema con bloqueo**.

Estados y llamadas de los hilos

Los hilos tienen **los mismos estados que los procesos**: en ejecución, bloqueado, listo, terminado. Todo el tema 2 aplica igual.

```
thread_create   → crea un hilo
thread_exit     → el hilo termina
thread_join     → espera a que otro hilo termine
thread_yield    → cede la CPU voluntariamente a otro hilo
```

`thread_yield` es importante y volvés a verlo en el tema 7. Como dentro de un proceso no hay interrupciones de reloj, los hilos **tienen que cederse la CPU educadamente**. No hay nadie que los obligue.

Las complicaciones (sí, se preguntan)

```
✗ fork() en un proceso multihilo
     ¿el hijo hereda TODOS los hilos o solo uno?

✗ Variables globales como errno
     dos hilos las pisan

✗ Bibliotecas no reentrantes
     fallan si las llaman dos hilos a la vez

✗ Señales
     se envían a PROCESOS, no a hilos. ¿Quién la atiende?

✗ Múltiples pilas
     si el kernel no sabe que hay hilos, no las administra bien
```

La conclusión del libro: agregar hilos **no es automático**. Convertir un programa de un hilo a multihilo exige rediseñar bibliotecas, la semántica de las llamadas al sistema y el manejo de recursos.

Referencia

(Tanenbaum, cap. 2.2.1 y 2.2.2, págs. impresas **95 a 104** → PDF **125 a 134**).

Preguntas de control

1. Un hilo de un proceso ejecuta `close(fd)`, cerrando un archivo. **¿Qué ven los otros hilos de ese proceso?** ¿Y si fueran procesos separados en vez de hilos?

---

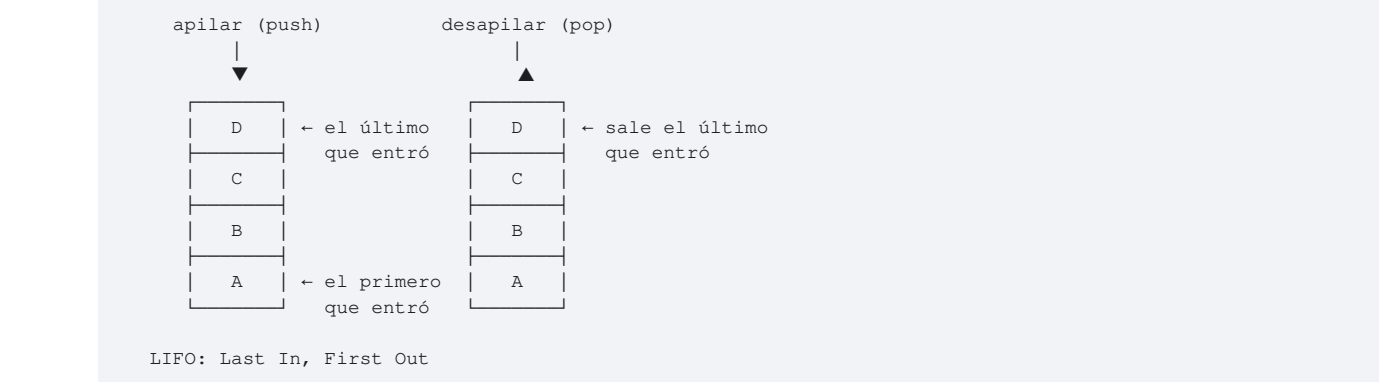
2. ¿Por qué Chrome usa **un proceso por pestaña** en lugar de un hilo por pestaña, si los hilos son mucho más ligeros y rápidos de crear?

Las dos preguntas son muy buenas. La segunda es **exactamente** el tema 4, así que te la respondo ahora y después la desarrollamos.

---

1. Qué es la pila (stack)

Es **una zona de memoria** donde el programa va apilando y desapilando cosas. Se llama pila porque funciona como una pila de platos:



```
   el último que entra es el primero que sale
```

Dónde vive

La memoria de un proceso se divide en zonas:

```
   direcciones altas
   ┌──────────────────────┐
   │       PILA           │  ← crece HACIA ABAJO
   │         ↓            │     (variables locales,
   │                      │      direcciones de retorno)
   │   (espacio libre)    │
   │                      │
   │         ↑            │
   │       HEAP           │  ← crece HACIA ARRIBA
   ├──────────────────────┤     (malloc, new)
   │       DATOS          │  ← variables globales
   ├──────────────────────┤
   │       TEXTO          │  ← el código del programa
   └──────────────────────┘
   dirección 0
```

El registro **SP (apuntador de pila)** que viste en la Unidad 1 apunta siempre al tope de la pila. Cada vez que se apila algo, SP se mueve.

Qué guarda exactamente

Tres cosas, y todas por **cada llamada a procedimiento activa**:

```
1. La DIRECCIÓN DE RETORNO    ← a dónde volver
2. Los PARÁMETROS             ← lo que le pasaste
3. Las VARIABLES LOCALES      ← las que declara adentro
```

Ejemplo concreto

```
main() {
    int x = 5;
    calcular(x);     ← línea 3
}

calcular(int n) {
    int doble = n * 2;
    imprimir(doble);  ← línea 8
}

imprimir(int v) { ... }
```

Mientras se ejecuta `imprimir`, la pila está así:

```
     ┌──────────────────────┐
  SP→ │ v = 10               │  ← marco de imprimir()
     │ volver a: línea 8    │
     ├──────────────────────┤
     │ doble = 10           │  ← marco de calcular()
     │ n = 5                │
     │ volver a: línea 3    │
     ├──────────────────────┤
     │ x = 5                │  ← marco de main()
     └──────────────────────┘
```

A cada bloque se lo llama **marco de pila** (stack frame). Cuando `imprimir` hace `return`, su marco se desapila, SP sube, y la ejecución continúa en la línea 8.

Y ahora se entiende por qué cada hilo necesita la suya

```
HILO 1                      HILO 2
┌──────────────┐            ┌──────────────┐
│ validarDatos() │           │ dibujarBoton() │
│ procesarPedido()│          │ actualizarPant()│
│ main()       │            │ main()       │
└──────────────┘            └──────────────┘
```

```
   su propia pila               su propia pila
```

Si compartieran una sola pila, cuando el hilo 2 hiciera `return`, desapilaría **el marco que puso el hilo 1** y saltaría a una dirección que no le corresponde. El programa se rompería de formas imposibles de depurar.

> **En una línea:** la pila es el *"dónde estoy y cómo vuelvo"* de un hilo. Por eso es lo primero de la lista de cosas propias de cada hilo.

---

2. Si un hilo se bloquea, ¿se bloquea el proceso?

Depende de dónde estén implementados los hilos. Y esa es, literalmente, la pregunta central del tema 4.

Te adelanto la respuesta corta, porque mañana la desarrollamos:

```
HILOS DE USUARIO              HILOS DE KERNEL
(el kernel no sabe            (el kernel los conoce
 que existen)                  uno por uno)

El kernel ve UN proceso.      El kernel ve 3 hilos.
Si un hilo pide E/S, el       Si el hilo 2 pide E/S,
kernel bloquea al ÚNICO       bloquea SOLO al hilo 2.
proceso que conoce.
                              Los hilos 1 y 3 siguen
→ SE BLOQUEAN LOS 3 HILOS     ejecutándose normalmente.

   el defecto central          ✓ la ventaja central
```

En dibujo:

```
HILOS DE USUARIO:
   hilo 2 pide E/S
        │
        ▼
   el kernel bloquea al proceso
        │
   ┌────┴─────┬──────────┐
   ▼          ▼          ▼
 hilo 1    hilo 2    hilo 3
BLOQUEADO BLOQUEADO BLOQUEADO
   ↑                    ↑
   no pidieron nada, pero caen igual


HILOS DE KERNEL:
   hilo 2 pide E/S
        │
        ▼
   el kernel bloquea solo al hilo 2
        │
   ┌────┴─────┬──────────┐
   ▼          ▼          ▼
 hilo 1    hilo 2    hilo 3
EJECUTANDO BLOQUEADO  LISTO
```

Por qué pasa eso con los hilos de usuario: el kernel solo sabe bloquear lo que conoce, y lo que conoce es el proceso. No tiene forma de bloquear "una parte" de un proceso, porque para él el proceso es indivisible.

Esto tiene una consecuencia práctica fuerte: los hilos de usuario **funcionan mal justamente en las aplicaciones que hacen muchas llamadas al sistema**, que son las que más necesitarían hilos. Esa paradoja es lo que te van a pedir que expliques.

Corrección

1 — Bien encaminado, pero afiná el efecto. No es que "lo ven": es más fuerte.

Los **archivos abiertos son del proceso** (columna izquierda de la tabla). Así que cuando un hilo hace

`close(fd)`:

```
EL DESCRIPTOR DESAPARECE PARA TODOS

   hilo 1  ──┐
   hilo 2  ──┼──► fd 7 ──► archivo.txt
   hilo 3  ──┘

   hilo 2 hace close(7)
            ▼
   hilo 1  ──┐
   hilo 2  ──┼──► fd 7  ✗ ya no existe
   hilo 3  ──┘

   Si el hilo 1 estaba leyendo de fd 7 → su read() FALLA
```

No es que se enteren: es que **se quedan sin el archivo**, lo hayan pedido o no. Un hilo puede romperle el trabajo a otro sin querer. Eso es consecuencia directa de que no haya protección entre hilos.

Con procesos separados: cada uno tiene **su propia tabla de descriptores**. El proceso A cierra el suyo y el de B sigue funcionando perfecto. Están protegidos.

---

2 — La intuición es correcta, pero el argumento fuerte es otro.

Dijiste "para no mezclar y romper tareas". Vas bien. Lo que falta es nombrar **las dos cosas concretas** que se ganan:

```
① AISLAMIENTO DE FALLAS
   un hilo que corrompe memoria → se lleva TODO el proceso
   una pestaña que crashea con hilos → se cae el navegador entero
   una pestaña que crashea con procesos → se cae ESA pestaña

② PROTECCIÓN DE MEMORIA
   los hilos COMPARTEN el espacio de direcciones
   → una pestaña con código malicioso podría leer la memoria
     de la pestaña donde tenés abierto el homebanking

   los procesos NO comparten memoria
   → no puede, el hardware se lo impide (modo usuario, U1)
```

Y acá está lo elegante del asunto: **Chrome paga el precio a propósito.** Sabe que los procesos son más pesados, que consumen más memoria y que comunicarse entre ellos es más caro. Lo paga igual, porque el aislamiento vale más que el rendimiento.

> Es el mismo intercambio de la Unidad 1, tema 9: **monolítico = rápido y frágil, microkernel = lento y robusto.** Chrome eligió microkernel.

Fijate que en toda la materia vuelve la misma tensión. **Compartir es rápido pero peligroso; aislar es seguro pero caro.** Hilos y procesos son otra instancia de eso.

Correcto. Dos precisiones chicas:

El verbo no es "no poder ver": es "no poder usar". El descriptor deja de existir para todos. No es que quede oculto — desaparece.

Y ojo con "trabajan en conjunto": eso explica *por qué está diseñado así* (comparten a propósito, para cooperar), pero **no garantiza que lo hagan bien**. El hilo que cerró el archivo probablemente no tenía idea de que otro lo estaba usando.

```
comparten  ≠  se coordinan

compartir lo hace el sistema  →  gratis
coordinarse lo hace el programador  →  hay que escribirlo
```

Y esa distinción es justo el puente al **acto 2** de la unidad (temas 5 a 7): compartir sin coordinarse es exactamente lo que produce las **condiciones de carrera**.

Un **descriptor de archivo** (file descriptor, `fd`) es **un número** que identifica un archivo abierto dentro de un proceso.

La analogía

Es el **número del ticket del guardarropa**. Dejás el abrigo y te dan el ticket 7. Vos no manejás el abrigo: manejás el 7. Cuando querés el abrigo, mostrás el 7.

```
open("datos.txt")  →  devuelve 7      ← te dan el ticket
read(7, buf, 100)  →  lee del archivo ← usás el ticket
close(7)           →  cierra          ← devolvés el ticket
```

Qué es por dentro

Es un **índice en una tabla que el proceso tiene en su BCP**:

```
TABLA DE DESCRIPTORES del proceso
┌────┬──────────────────────────────┐
│ fd │ a qué apunta                 │
├────┼──────────────────────────────┤
│ 0  │ entrada estándar (teclado)   │  ← siempre
│ 1  │ salida estándar (pantalla)   │  ← siempre
│ 2  │ error estándar               │  ← siempre
│ 3  │ /home/usuario/datos.txt      │
│ 4  │ un socket de red             │
│ 7  │ /tmp/log.txt                 │
└────┴──────────────────────────────┘
```

Los **0, 1 y 2 están siempre abiertos** en todo proceso de UNIX. Por eso `printf` funciona sin que abras nada: escribe en el fd 1.

Y acá se cierra el círculo con la redirección

Acordate del tema 1, cuando vimos por qué `fork` y `exec` están separados. **Esto es lo que el shell hace en el medio:**

```
ls -la > salida.txt

1. fork()           → se crea el hijo
2. en el hijo:
      close(1)      → cierra la salida estándar
      open("salida.txt")  → el SO le asigna el fd LIBRE más bajo... el 1
3. execve("ls")     → ahora ls escribe en el fd 1 como siempre,
                      pero el fd 1 es el archivo
```

`ls` no se entera de nada. Escribe en el 1 como toda la vida. Le cambiaron el abrigo del guardarropa sin avisarle.

Volviendo a tu pregunta del hilo

```
Tabla de descriptores → vive en el BCP → es DEL PROCESO
                                              ↓
                      todos los hilos usan LA MISMA tabla
                                              ↓
                      un close(7) borra el renglón 7 para todos
```

Por eso lo podés ver con `ls -la /proc/PID/fd` — ese comando de tu cartilla lista exactamente esa tabla.

Exacto. Y la clave está en que **lo dijiste con la palabra justa: compartir**.

```
HILOS  →  comparten el espacio de direcciones  →  un hilo corrompe memoria
                                                     → cae TODO el proceso

PROCESOS → cada uno su espacio, aislado         →  uno corrompe memoria
                                                     → cae SOLO ese proceso
```

Es la misma tensión de siempre, ahora con nombre y apellido:

	Compartir (hilos)	Aislar (procesos)
Velocidad	✓ rápido, liviano	✗ pesado
Comunicación	✓ directa, por memoria	✗ hay que pasar mensajes
Si falla uno	✗ **caen todos**	✓ **cae solo ese**

Chrome eligió la columna derecha **sabiendo** que paga más memoria y más lentitud. Para un navegador, donde cada pestaña ejecuta código de un sitio desconocido, el aislamiento vale más.

> Y fijate que es la tercera vez que aparece el mismo intercambio en la materia: monolítico vs microkernel (U1), hilos vs procesos (acá), y lo vas a volver a ver en memoria compartida vs paso de mensajes.

Tema 3 cerrado. Vas **3 de 10**.

Tema 4 — Implementación de hilos

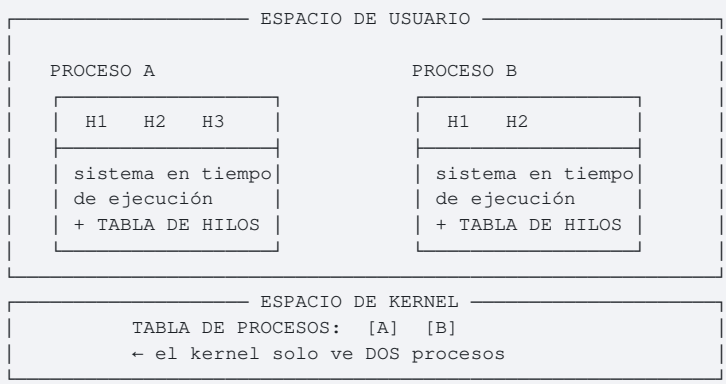
La pregunta que define todo

> **¿El kernel sabe que los hilos existen?**

De la respuesta a esa pregunta se desprende absolutamente todo el tema.

Opción A — Hilos en el espacio de usuario

El kernel **no sabe que existen**. Ve un proceso normal, con un solo hilo, como siempre.



Los hilos los maneja **una biblioteca** dentro del proceso, con su propia **tabla de hilos**.

Ventajas

```
✓ RAPIDÍSIMO
  conmutar de hilo = unas pocas instrucciones de máquina
  (guardar registros, cargar otros)
  NO hay trap al kernel, no hay cambio de modo

✓ PLANIFICADOR PROPIO POR APLICACIÓN
  cada proceso elige su algoritmo, adaptado a lo que hace

✓ ESCALAN MEJOR
  podés tener miles de hilos sin consumir tablas del kernel
```

Desventajas — y son graves

```
✗ UNA LLAMADA CON BLOQUEO BLOQUEA TODO EL PROCESO
  ← el defecto central (lo que preguntaste)

✗ UN FALLO DE PÁGINA también bloquea a todos los hilos

✗ NO HAY INTERRUPCIONES DE RELOJ dentro del proceso
  → un hilo puede quedarse con la CPU para siempre
    si no la cede voluntariamente (thread_yield)

✗ Para evitar los bloqueos hacen falta llamadas sin bloqueo,
  select y envolturas → código complejo e ineficiente
```

La paradoja que te van a pedir explicar

> Los hilos de usuario son **rápidos pero difíciles de usar bien**, y funcionan mal **justamente en las aplicaciones que hacen muchas llamadas al sistema**… que son exactamente las que más necesitarían hilos.

```
¿Para qué querés hilos?  → para que mientras uno espera E/S,
                           otro trabaje

¿Qué pasa con hilos de   → cuando uno espera E/S,
usuario?                   SE BLOQUEAN TODOS

→ te dan la herramienta justo donde NO funciona
```

Opción B — Hilos en el kernel

El kernel **los conoce uno por uno** y mantiene una **tabla global de hilos**.

```
┌──────────────────── ESPACIO DE USUARIO ────────────────────┐
│   PROCESO A                    PROCESO B                   │
│   ┌──────────────┐             ┌──────────────┐            │
│   │  H1   H2   H3 │            │  H1   H2     │            │
│   └──────────────┘             └──────────────┘            │
└────────────────────────────────────────────────────────────┘
┌──────────────────── ESPACIO DE KERNEL ─────────────────────┐
│   TABLA DE PROCESOS:  [A]  [B]                             │
│   TABLA DE HILOS:  [A-H1][A-H2][A-H3][B-H1][B-H2]          │
│                  ← el kernel ve CINCO hilos                │
└────────────────────────────────────────────────────────────┘
```

Ventajas

```
✓ SI UN HILO SE BLOQUEA, el kernel ejecuta OTRO HILO
  del mismo proceso                    ← la ventaja central

✓ No hacen falta llamadas sin bloqueo ni envolturas

✓ Funcionan bien en aplicaciones con muchas llamadas al sistema

✓ Aprovechan varias CPU (paralelismo real)
```

Desventaja

```
✗ ALTA SOBRECARGA
  crear, destruir y conmutar hilos requiere LLAMADAS AL SISTEMA
  cada conmutación = cambio de contexto COMPLETO
                     (cambiar mapa de memoria, invalidar caché)
  → varias ÓRDENES DE MAGNITUD más lento
```

Para mitigarla, algunos sistemas **reciclan hilos**: en vez de destruirlos, los marcan como inactivos y los reutilizan.

Lo que tampoco resuelven

Los hilos de kernel **no arreglan** dos problemas:

- Qué hacer con los hilos al ejecutar `fork`
- Cómo manejar las **señales**, que se envían a procesos y no a hilos

---

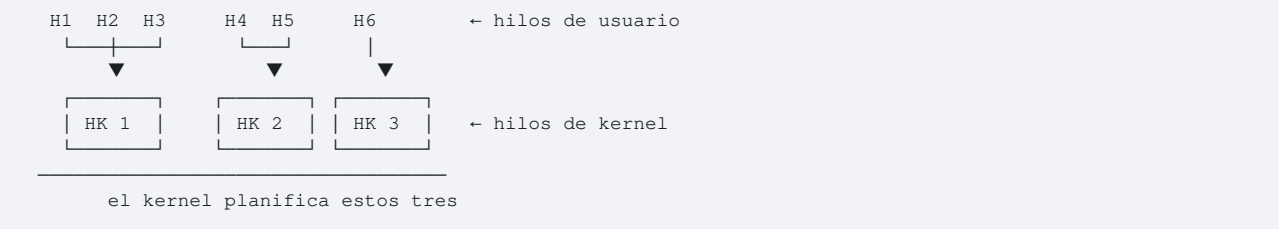
La comparación — esta tabla es la que cae

	Usuario	**Kernel**
¿El kernel los conoce?	No	Sí
Dónde está la tabla de hilos	**En el proceso**	**En el kernel**
Costo de conmutar	Bajísimo (instrucciones)	Alto (cambio de contexto)
Si un hilo se bloquea	**Se bloquea TODO el proceso**	**El kernel ejecuta otro hilo**
Interrupciones de reloj	No hay	Sí
Planificador propio	Sí	No
Aprovecha varias CPU	No	Sí

Los dos renglones en negrita son los que te preguntan. Son el intercambio del tema: velocidad contra independencia.

Opción C — Implementación híbrida

Lo mejor de los dos: el kernel conoce y planifica **hilos de kernel**, y sobre cada uno se multiplexan **varios hilos de usuario**.

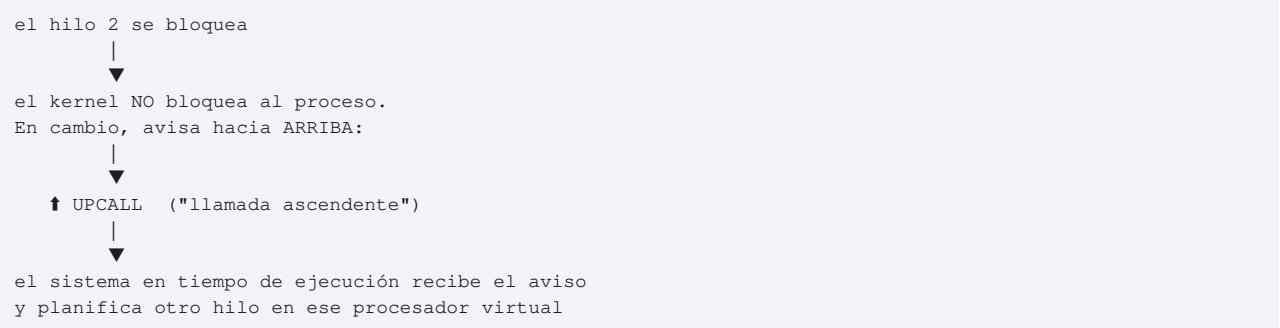


El programador decide cuántos hilos de usuario pone sobre cuántos de kernel. **Máxima flexibilidad**, combinando rendimiento y control.

Activaciones del planificador

Buscan lo mismo que la híbrida, pero de otra forma: **mantener el paralelismo real sin pagar toda la sobrecarga**.

La idea: el kernel asigna **procesadores virtuales** a cada proceso, y el sistema en tiempo de ejecución (en espacio de usuario) decide qué hilo correr en cada uno.



Cuando el evento se resuelve, **otra upcall** avisa que el hilo puede seguir.

La crítica que se menciona: las upcalls **rompen el modelo por capas**, porque el kernel (capa baja) llama a código de nivel superior. Va en contra de la estructura por capas que viste en la Unidad 1.

Hilos emergentes

Técnica de sistemas distribuidos: **la llegada de un mensaje crea automáticamente un hilo nuevo** para procesarlo.

```
llega un mensaje  →  se CREA un hilo desde cero  →  lo procesa  →  muere
```

La ventaja es la latencia: como el hilo empieza limpio, no hay que restaurar ningún estado previo, ni registros, ni pila. Es más rápido que despertar a un hilo existente.

Pueden correr en el kernel (más rápido y potente, pero **más riesgoso**) o en espacio de usuario (más seguro).

Referencia

(Tanenbaum, cap. 2.2.4 a 2.2.8, págs. impresas **106 a 114** → PDF **136 a 144**).

Preguntas de control

1. Un servidor web con 50 hilos trabajadores atiende peticiones que casi siempre requieren leer del disco.

¿Con hilos de **usuario** o de **kernel**? Justificá con el defecto o la ventaja concreta que aplica.

---

2. Un hilo de usuario entra en un bucle infinito calculando, y nunca llama a `thread_yield`.

¿Qué les pasa a los otros hilos de su proceso? ¿Y a los otros procesos del sistema? (Son dos respuestas distintas y ahí está la gracia.)

Tu objeción es **correcta**, y justamente por eso entendiste bien los extremos. Mirá:

```
RELACIÓN  hilos de usuario : hilos de kernel

   N : 1    →  TODOS los hilos de usuario sobre UN hilo de kernel
               = hilos de usuario puros     ← lo que decís vos

   1 : 1    →  un hilo de kernel por cada hilo de usuario
               = hilos de kernel puros      ← lo que decís vos

   M : N    →  M hilos de usuario sobre N hilos de kernel,
               con M > N > 1
               = ★ HÍBRIDA
```

La híbrida es lo que está en el medio. Tus dos casos son los bordes, y en los bordes degenera — tenés toda la razón.

El ejemplo concreto que lo aclara

Una máquina con **4 núcleos** y un servidor que quiere **1000 hilos** (uno por conexión).

```
✗ 1000 hilos de KERNEL (1:1)
  → 1000 entradas en la tabla del kernel
  → 1000 estructuras, 1000 pilas de kernel
  → muchísima memoria y sobrecarga
  → y para nada: solo 4 pueden correr DE VERDAD a la vez
                 (hay 4 núcleos, no 1000)

✗ 1000 hilos de USUARIO sobre 1 (N:1)
  → livianísimo
  → pero usás 1 solo núcleo de los 4
  → y si uno pide E/S, SE BLOQUEAN LOS 1000

✓ HÍBRIDA: 1000 hilos de usuario sobre 4 de kernel
```

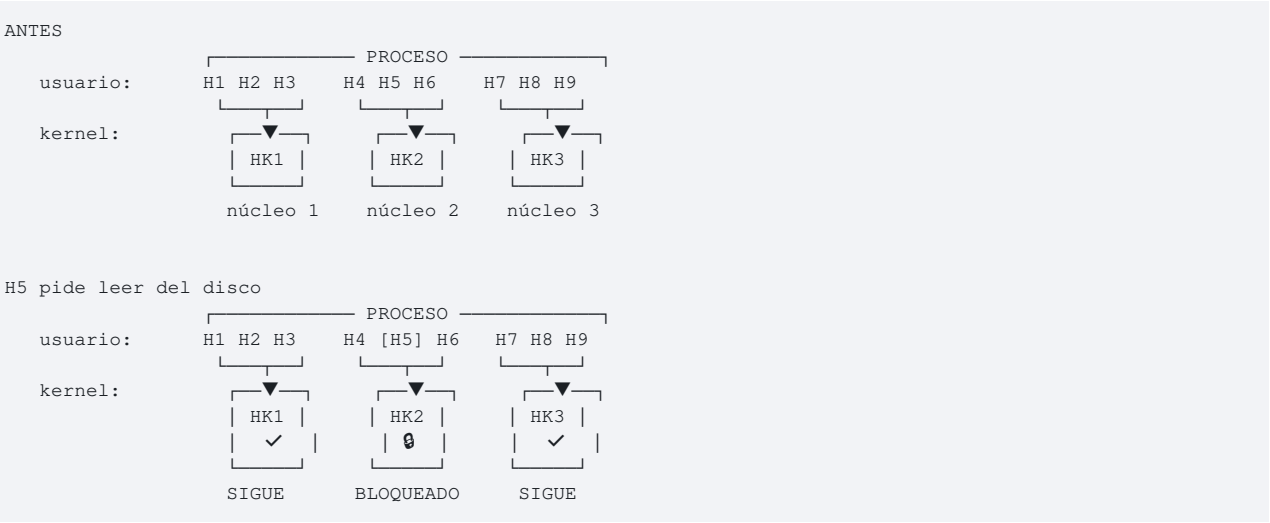
```
   → liviano (solo 4 estructuras en el kernel)
   → usás los 4 núcleos
   → y si uno pide E/S, NO se bloquean los 1000
```

La regla de oro:

```
hilos de USUARIO  →  cuántas tareas lógicas querés  (pueden ser miles)
hilos de KERNEL   →  cuánto paralelismo REAL podés tener  (≈ núcleos)
```

Poner más hilos de kernel que núcleos **no te da más paralelismo**. Solo te da más sobrecarga. Por eso no conviene 1:1.

Y ahora tu pregunta: ¿qué pasa cuando uno se bloquea?



El kernel bloquea el hilo de kernel sobre el que corría H5 — porque es lo único que el kernel conoce.

Resultado:

	Hilos de usuario puros	Híbrida	Hilos de kernel puros
Se bloquean	**los 9**	**3 de 9** (los de HK2)	**1 de 9**
Siguen	0	**6** (HK1 y HK3)	8

No es ni una cosa ni la otra: se bloquea una fracción. Eso es exactamente lo que significa "híbrido".

Y en las implementaciones buenas, el sistema en tiempo de ejecución puede **mover** H4 y H6 a HK1 o HK3, que están libres. Ahí ya casi no perdés nada.

El resumen

```
                liviano    paralelismo    aislamiento
                           real           ante bloqueo
usuario puro      ★★★          ✗              ✗
HÍBRIDA           ★★           ✓             ★★
kernel puro       ✗            ✓             ★★★
```

La híbrida **no es mejor en nada en particular** — es la que no es mala en nada. Y por eso en el libro aparece como "máxima flexibilidad": **vos elegís el punto** de la escala según tu aplicación.

> Si tu objeción fuera un reclamo —"entonces no sirve"— la respuesta sería: sirve porque **M y N los elegís vos**. Con M=1000 y N=4 tenés algo que ni el modelo de usuario ni el de kernel te pueden dar.

¿Te cierra ahora? Si sí, te quedan las dos preguntas del tema 4 (el servidor web con 50 hilos, y el hilo en bucle infinito).

Buenas preguntas, y la primera tiene un matiz importante.

1. No es un encapsulador — es un *vehículo*

El HK no **contiene** hilos de usuario. Los **ejecuta, de a uno por vez**.

```
EN UN INSTANTE DADO:

   H1  H2  H3  H4  H5  H6         ← 6 hilos de usuario
   ▲
   |  esperan: H2 H3 H4 H6
   |
┌──┴──┐              ┌─────┐
| HK1 |              | HK2 |
| corre H1|          | corre H5|   ← cada HK ejecuta UNO
└─────┘              └─────┘
```

Un HK ejecuta exactamente un HU a la vez. Los demás esperan su turno, igual que los procesos esperan la CPU.

La mejor analogía: el HK es **un asiento de trabajo**. Los hilos de usuario se turnan para sentarse. No "viven adentro" del asiento.

Por eso el libro los llama **procesadores virtuales**: para el runtime, un HK es "una CPU disponible donde poner a correr algo".

```
CPU real        → el SO reparte procesos/hilos de kernel entre ellas
HK (CPU virtual)→ el runtime reparte hilos de usuario entre ellos

MISMA IDEA, DOS NIVELES
```

---

2. No, en general no está programado cuántos HU por HK

Es **dinámico**. El runtime va asignando según quién esté listo:

```
t=0    HK1 corre H1,  HK2 corre H5
t=10   H1 cede → HK1 toma H2,  HK2 sigue con H5
t=20   H5 termina → HK2 toma H3
```

Ningún HU tiene "su" HK asignado de antemano. **Van rotando.**

Lo que sí decide el programador

```
✓ CUÁNTOS hilos de kernel hay        (N)  ← esto se elige
✓ CUÁNTOS hilos de usuario crea      (M)  ← esto se elige

✗ cuál HU va en cuál HK                   ← lo decide el runtime,
                                            en tiempo de ejecución
```

Y **N se elige por la cantidad de núcleos**, no por la de hilos de usuario. Si tenés 4 núcleos, poner 4 u 8 hilos de kernel. Poner 1000 sería tirar memoria.

La excepción

Algunas implementaciones permiten **fijar** un hilo de usuario a un hilo de kernel (se los llama *bound threads*). Se usa cuando ese hilo tiene que responder sí o sí, sin depender de que el runtime lo elija — por ejemplo un hilo de tiempo real.

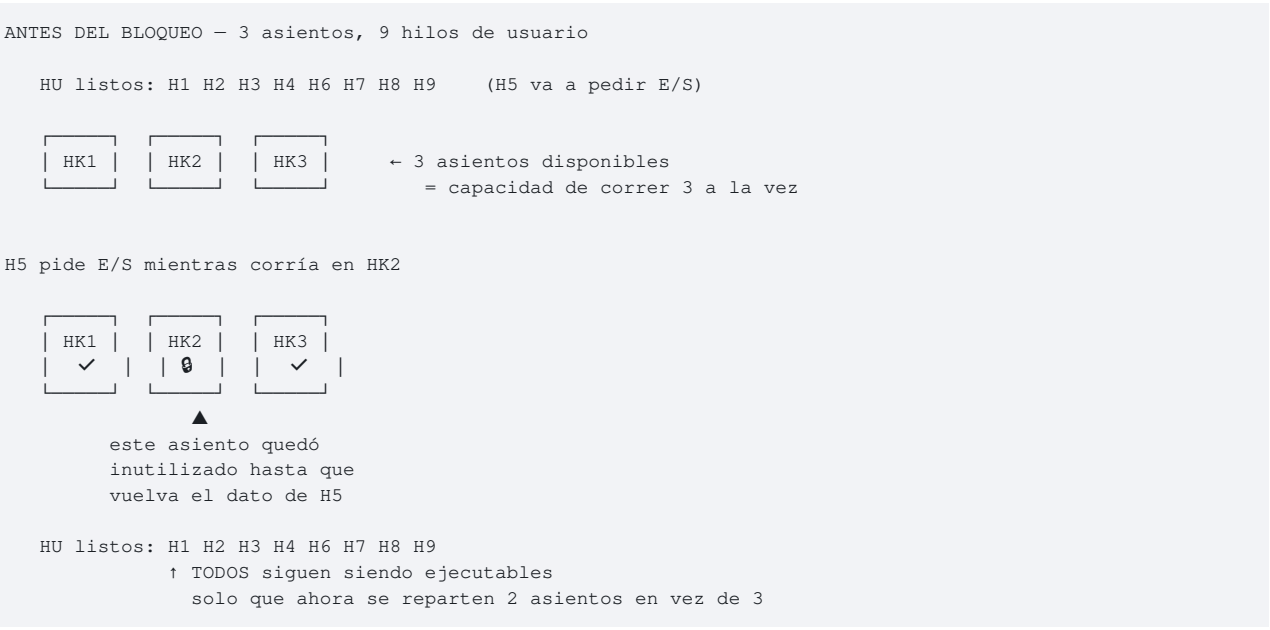
Pero es la excepción, no la regla.

En una línea: el HK es un puesto de ejecución que el runtime va llenando con el hilo de usuario que corresponda; no es una caja donde los hilos viven.

¿Vamos con las dos preguntas del tema 4?

Buena objeción — y tenés razón: **mi diagrama de antes estaba simplificado de más**. Lo corrijo.

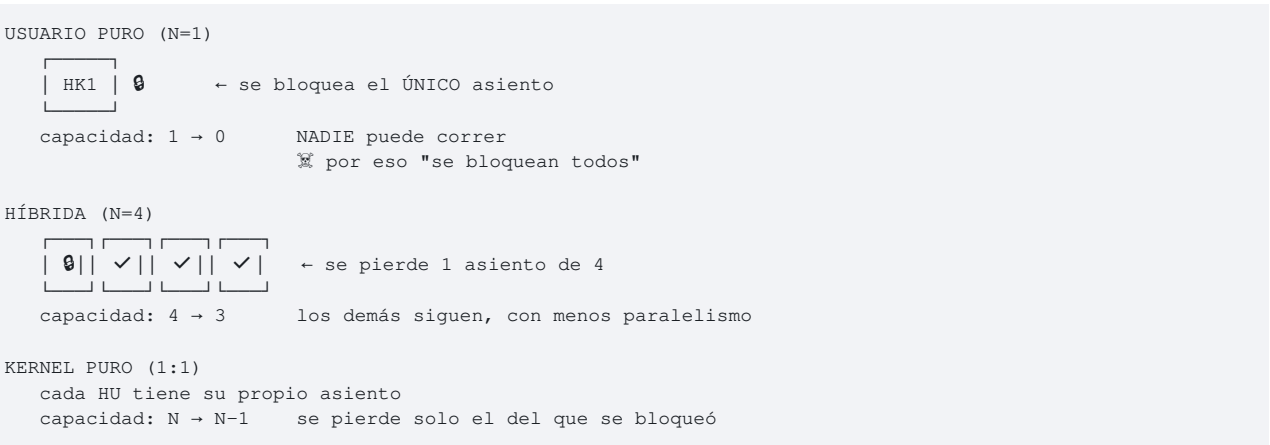
Lo que se bloquea no es un grupo de hilos: es un *asiento*



Ningún hilo de usuario queda atrapado. Lo que se perdió es **capacidad de ejecución**: pasaste de 3 a 2 en paralelo.

Eso responde tu objeción: no hace falta una asignación previa, porque **lo que se pierde no son hilos, es un asiento**.

Y ahora los tres modelos se entienden de una



La frase precisa, entonces, no es "se bloquea una fracción de los hilos". Es:

> **Se pierde un asiento de N. Con N=1 eso es catastrófico; con N=4, es el 25% de la capacidad.**

Esa es la verdadera razón por la que la híbrida funciona: no evita el bloqueo, **diluye su impacto**.

Perdón por la imprecisión del diagrama anterior — decía "se bloquean 3 de 9" y eso te hacía esperar una asignación fija que no existe. Era justamente lo que estabas detectando.

¿Ahora cierra?

Correcto. No se bloquea todo el proceso.

```
USUARIO PURO     →  un hilo se bloquea  →  SE BLOQUEA TODO EL PROCESO
HÍBRIDA          →  un hilo se bloquea  →  el proceso SIGUE funcionando
KERNEL PURO      →  un hilo se bloquea  →  el proceso SIGUE funcionando
```

El motivo, en una línea: el proceso se bloquea solo si se quedó **sin ningún hilo de kernel disponible**. Con N=1 eso pasa al primer bloqueo. Con N=4, tendrían que bloquearse los 4 a la vez.

```
¿Se bloquea el proceso?
   = ¿se quedó sin asientos?

   N=1  →  1 bloqueo basta        ☠
   N=4  →  hacen falta 4 bloqueos simultáneos
```

Y acá está la respuesta honesta: **sí, con N=4 el proceso puede bloquearse entero** — si los 4 hilos de kernel se bloquean al mismo tiempo. Pero es mucho menos probable y, sobre todo, es un problema de *capacidad*, no de *diseño*.

> En el modelo de usuario puro el bloqueo total es **inevitable por construcción**. En la híbrida es **un caso extremo**.

Esa es la ganancia real de la híbrida: convierte un defecto estructural en una cuestión de dimensionamiento.

Tema 4 cerrado. Vas **4 de 10**.

¿Querés contestar las dos preguntas que te dejé (el servidor web de 50 hilos y el hilo en bucle infinito), o seguimos al tema 5?

Corrección

1 — Correcto: hilos de kernel. Pero cambiemos una palabra que en el examen te costaría puntos.

```
✗ "el servidor se me caería"
✓ "el servidor se me BLOQUEARÍA"
```

Caerse = el proceso muere, termina, hay que reiniciarlo. **Bloquearse** = el proceso está vivo, pero no puede avanzar hasta que llegue un dato.

Acá no se cae nada: se **bloquea**. Y el detalle es brutal:

```
50 hilos de usuario, uno pide leer del disco
        ▼
el kernel bloquea al único proceso que conoce
        ▼
LOS 50 HILOS QUEDAN BLOQUEADOS
        ▼
el servidor no atiende NI UNA petición hasta que vuelva ese disco
```

Con hilos de kernel, se bloquea **ese hilo** y los otros 49 siguen atendiendo.

Y notá la ironía que vimos: es un servidor que hace **muchísima E/S**, que es exactamente el caso donde los hilos de usuario son peores. Si el servidor fuera de puro cálculo (sin disco), los hilos de usuario andarían bárbaro y serían más rápidos.

---

2 — Las dos partes correctas. Afino la justificación de la segunda, porque apuntaste a lo que no es.

Los otros hilos del proceso ✓ — quedan listos y nunca ejecutan. Y el motivo que diste es el correcto: **no hay interrupciones de reloj dentro del proceso**. Nadie puede forzar al hilo egoísta a soltar la CPU. Solo él puede cederla, y no lo hace.

Los otros procesos ✓ — no les pasa nada. Pero el motivo **no es** el espacio de memoria separado:

```
✗ "porque es otro espacio de memoria RAM"
  → eso explica la PROTECCIÓN (que no se pisen los datos),
    no la planificación

✓ "porque el KERNEL sí tiene interrupciones de reloj"
  → cuando al proceso se le agota el quantum, el kernel
    le saca la CPU por la fuerza (transición ②)
    y se la da a otro proceso
```

El dibujo lo deja claro:

```
NIVEL KERNEL          hay reloj     →  apropiación  →  nadie monopoliza
   (entre procesos)                    FORZADA

NIVEL USUARIO         NO hay reloj  →  cesión       →  un egoísta
   (entre hilos de un                  VOLUNTARIA      monopoliza
    mismo proceso)
```

La frase para el examen: el hilo egoísta mata a sus compañeros de proceso, pero **no afecta al resto del sistema**, porque el kernel le quita la CPU al proceso entero cuando se le acaba el quantum.

Tema 5 — Condiciones de carrera y regiones críticas

Empezamos por el problema

Hasta acá armamos procesos e hilos que corren "a la vez". Ahora aparece la cuenta a pagar.

La **comunicación entre procesos** (IPC) tiene tres cuestiones:

```
1. ¿Cómo un proceso le pasa información a otro?

2. ¿Cómo evitamos que se interpongan entre sí
   al usar recursos compartidos?              ← temas 5, 6 y 7

3. ¿Cómo aseguramos la secuencia correcta
   cuando hay dependencias?                   ← tema 7
```

Las cuestiones 2 y 3 **valen igual para hilos**. La 1 es más fácil en hilos, porque ya comparten memoria.

La intuición

Dos personas en una cuenta bancaria conjunta con $1000. Cada una va a un cajero distinto, al mismo tiempo, a sacar $600.

```
Cajero A                          Cajero B
────────                          ────────
lee el saldo: 1000
                                  lee el saldo: 1000
¿alcanza para 600? sí
                                  ¿alcanza para 600? sí
entrega 600
escribe saldo: 400                entrega 600
                                  escribe saldo: 400
```

Se entregaron $1200 de una cuenta que tenía $1000. Y el saldo final dice $400, que es coherente… con una sola de las dos operaciones.

Nadie detectó un error. Los dos cajeros hicieron exactamente lo que debían.

El concepto formal

Condición de carrera: dos o más procesos leen o escriben datos compartidos, y **el resultado final depende del orden y del momento preciso** en que se ejecutaron.

Por qué son tan difíciles de depurar: la mayoría de las veces el programa funciona bien. Solo falla cuando la conmutación cae justo en el peor momento. Puede andar meses y romperse un martes.

Ejemplo resuelto — el spooler de impresión

Este es el ejemplo del libro, y te lo tomarían tal cual.

El escenario: un directorio de spooler con ranuras numeradas. Dos variables compartidas:

```
sal = 0   → apunta al siguiente archivo A IMPRIMIR
ent = 4   → apunta a la siguiente ranura LIBRE

┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
│ 0   │ 1   │ 2   │ 3   │ 4   │ 5   │ 6   │
│ a.c │ b.c │ c.c │ d.c │     │     │     │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┘
  ▲                       ▲
 sal                     ent
```

Los procesos A y B quieren encolar un archivo casi al mismo tiempo.

```
Paso 1   A lee ent          →  obtiene 4, lo guarda en su variable local

Paso 2   ⏰ SE AGOTA EL QUANTUM DE A
         el planificador conmuta a B
         (A quedó con un 4 guardado y SIN HABER ESCRITO NADA)

Paso 3   B lee ent          →  obtiene 4  (A todavía no la actualizó)

Paso 4   B escribe su archivo en la ranura 4
         B actualiza ent = 5
         ← desde el punto de vista de B, todo salió perfecto

Paso 5   Vuelve a ejecutarse A, que retoma EXACTAMENTE donde quedó:
         con el 4 en su variable local

Paso 6   A escribe su archivo en la ranura 4   ← PISA EL DE B
         A actualiza ent = 5
```

El resultado:

```
┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
│ 0   │ 1   │ 2   │ 3   │ 4   │ 5   │ 6   │
│ a.c │ b.c │ c.c │ d.c │ A.c │     │     │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┘
                        ▲
                  el archivo de B
                  NUNCA ESTUVO ACÁ
```

- El directorio quedó **internamente consistente**
- El demonio de impresión **no va a notar nada raro**
- El archivo de B **nunca se imprime**
- **B jamás se entera**
- Nadie reportó un error

Lo decisivo del ejemplo

El código de A es correcto. El código de B es correcto. El problema es que la secuencia **leer → modificar → escribir** sobre una variable compartida **fue interrumpida en el medio**.

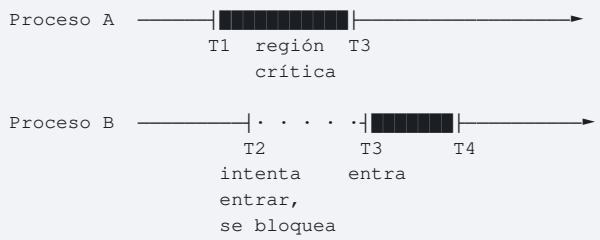
```
   leer ent  ──┬──  modificar  ──┬──  escribir ent
               │                 │
               └── ⚠ acá ────────┘
                         │
```

```
                si te interrumpen en esta zona,
                trabajaste con un valor viejo
```

La solución: regiones críticas

> **Región crítica** (o sección crítica): la parte del programa donde un proceso accede a memoria compartida, archivos compartidos, o cualquier recurso que pueda producir condiciones de carrera.

Si garantizamos que **nunca haya dos procesos simultáneamente dentro de sus regiones críticas**, se acabaron las condiciones de carrera. A eso se lo llama **exclusión mutua**.



☆ Las cuatro condiciones

Exclusión mutua sola **no alcanza**. Una solución correcta debe cumplir las cuatro. Esta lista aparece textual en los exámenes de tu cátedra:

```
1. No puede haber DOS PROCESOS A LA VEZ dentro
   de sus regiones críticas.

2. No deben hacerse SUPOSICIONES sobre las VELOCIDADES
   ni sobre la CANTIDAD DE CPU.

3. Ningún proceso que se ejecute FUERA de su región crítica
   puede BLOQUEAR a otros procesos.

4. Ningún proceso debe ESPERAR INDEFINIDAMENTE para entrar
   a su región crítica.
```

Por qué cada una:

#	Si no se cumple	Lo llamamos
1	Hay condiciones de carrera	—
2	Funciona en tu máquina y falla en otra	—
3	Alguien que no está usando nada te bloquea	—
4	Un proceso nunca entra	**inanición**

> Las soluciones que vamos a ver en el tema 6 fallan, **cada una, en una condición distinta**. Tener esta lista a mano es lo que te permite decir *por qué* falla cada una.

Referencia

(Tanenbaum, cap. 2.3.1 y 2.3.2, págs. impresas **117 a 120** → PDF **147 a 150**).

Esto se entiende mucho mejor viéndolo correr. Te armo la animación.

Preguntas de control

1. La "alternancia estricta" es una solución donde los procesos se turnan obligatoriamente: A, B, A, B...

Supongamos que **A es rapidísimo** y quiere entrar muchas veces, y **B casi no usa la región crítica**. A entra,

sale, y quiere volver a entrar — pero tiene que esperar a que B entre y salga.

¿Cuál de las cuatro condiciones se está violando?

---

2. En el spooler, ¿por qué la condición de carrera **no se detecta** como un error? ¿Qué quedó "roto" y qué quedó "sano"?

Corrección

2 — Bien. Lo esencial está. Afino el vocabulario para el examen:

```
QUEDÓ SANO (consistente)        QUEDÓ ROTO
────────────────────────        ──────────
· ent apunta bien (5)           · el archivo de B desapareció
· no hay ranuras corruptas      · B cree que su trabajo se encoló
· el demonio puede imprimir
  sin problemas
```

La clave de por qué no se detecta: no se violó ninguna *invariante* del sistema. El directorio es perfectamente válido. Para detectar el error habría que saber que B *había querido* escribir ahí — y eso no está registrado en ninguna parte.

> Por eso el libro dice que estos errores son "impredecibles e irreproducibles". No hay nada que chequear.

---

1 — Acá erraste la condición. Es la 3, no la 2.

Releé la condición 3:

> **Ningún proceso que se ejecute FUERA de su región crítica puede bloquear a otros procesos.**

Y mirá tu escenario:

```
A: entra, sale, quiere volver a entrar
B: está haciendo otra cosa, NO quiere entrar

   turno = B

   A espera... y espera... y espera
   ¿a quién? A B, que ni siquiera quiere entrar
   ¿la región crítica está ocupada? NO, está VACÍA
```

B bloquea a A sin estar usando nada. Eso es exactamente la condición 3.

Para qué sirve la condición 2

La 2 es otra cosa: *"no deben hacerse suposiciones sobre velocidades ni sobre la cantidad de CPU"*. Se viola cuando una solución **anda solo si el hardware se porta de cierta manera**:

```
Violar la condición 2:
  · "esto funciona porque A siempre es más rápido que B"
  · "esto funciona porque hay una sola CPU"
      ← deshabilitar interrupciones viola esta:
        en multiprocesador no sirve
```

La alternancia estricta **no supone nada** sobre velocidades: funciona igual sea A rápido o lento. Lo que hace mal es **obligar a turnarse**.

La tabla que te conviene tener

Solución	Falla en	Por qué
Variables de candado	**1**	Dos pueden entrar a la vez

Deshabilitar interrupciones	**2**	No sirve con varias CPU
Alternancia estricta	**3**	Uno que no entra, bloquea
Peterson, TSL	ninguna ✓	(pero usan espera ocupada)

El truco para no confundirlas: preguntate *¿quién está bloqueando a quién, y estaba usando algo?* Si el que bloquea **no estaba usando la región crítica**, es la condición 3.

Esa tabla es, literalmente, el tema 6. Vas **5 de 10**.

Tema 6 — Exclusión mutua con espera ocupada

El punto de partida

Ya sabemos **qué** queremos: que nunca haya dos procesos dentro de su región crítica, cumpliendo las 4 condiciones.

Ahora viene el **cómo**. Y todas las soluciones de este tema comparten un defecto:

Espera ocupada (busy waiting): el proceso que no puede entrar **se queda evaluando una variable en un ciclo**, quemando CPU, hasta que cambie.

A un candado que funciona así se lo llama **candado de giro** (spin lock).

```
mientras (no puedo entrar)
    ;        ← no hace NADA, pero consume CPU al 100%
```

Vamos una por una. **Tres fallan y dos funcionan.**

✗ Solución 1 — Deshabilitar interrupciones

La idea: el proceso apaga las interrupciones al entrar a su región crítica y las enciende al salir. Sin interrupciones no hay conmutación, porque la CPU solo cambia de proceso cuando ocurre una.

Por qué falla:

```
① Un proceso de usuario NO puede hacerlo
   (es instrucción privilegiada — Unidad 1)
   y si se lo permitieras y se olvida de reactivarlas,
   el sistema queda MUERTO

② En MULTIPROCESADOR solo afecta a SU CPU
   las otras siguen corriendo y entran igual a la región crítica
```

→ **Viola la condición 2** (suposiciones sobre la cantidad de CPU).

Dónde sí se usa: adentro del kernel, por unas pocas instrucciones, para proteger sus propias estructuras. Nunca para procesos de usuario.

✗ Solución 2 — Variables de candado

La idea: una variable compartida `candado`.

```
candado = 0  →  nadie adentro
candado = 1  →  alguien adentro

mientras (candado == 1) ;    ← esperar
```

```
candado = 1                  ← tomar
   ... región crítica ...
candado = 0                  ← soltar
```

Por qué falla — es exactamente el spooler otra vez:

```
Paso 1   A lee candado → 0. "Está libre, entro"
Paso 2   ⏰ CONMUTA
Paso 3   B lee candado → 0  (A todavía no lo escribió)
Paso 4   B pone candado = 1 y entra
Paso 5   A retoma, pone candado = 1 y entra TAMBIÉN
```

Los dos adentro. → Viola la condición 1.

> Y verificar el candado dos veces no arregla nada: la conmutación puede caer entre las dos verificaciones. **El problema es que leer-y-escribir no es atómico.**

✗ Solución 3 — Alternancia estricta

Ya la vimos. Una variable `turno` obliga a alternar: A, B, A, B...

```
mientras (turno != yo) ;     ← esperar mi turno
   ... región crítica ...
turno = el_otro              ← cederle
```

Funciona (no hay condiciones de carrera), **pero es inaceptable**:

```
B está tomando café, no quiere entrar
A quiere entrar por segunda vez seguida
   → A ESPERA, aunque la región esté VACÍA
```

→ **Viola la condición 3.** Un proceso fuera de su región crítica bloquea a otro.

✓ Solución 4 — El algoritmo de Peterson

Propuesto por G. L. Peterson en **1981**; reemplazó a la solución más compleja de **Dekker**.

Dos variables compartidas:

```
turno          → de quién es el turno
interesado[]   → un elemento por proceso: ¿quiere entrar?
```

El procedimiento de entrada, en 3 pasos:

```
1.  interesado[yo] = true      ← "quiero entrar"
2.  turno = yo                 ← "me anoto"
3.  mientras (turno == yo  Y  interesado[otro])  ;   ← esperar

    ... región crítica ...

    interesado[yo] = false     ← "ya salí"
```

Por qué funciona cuando los dos quieren entrar a la vez

Acá está la elegancia:

```
Los dos se marcan interesados.
Los dos escriben turno.

Pero turno es UNA sola variable:
el que escribe ÚLTIMO pisa al anterior.

        A escribe turno = A
        B escribe turno = B    ← queda B
```

```
A evalúa: ¿turno == A? NO (vale B) → ENTRA
B evalúa: ¿turno == B? SÍ, y A está interesado → ESPERA
```

La regla: el **último que escribe `turno` es el que cede el paso.** Suena al revés y por eso es ingenioso.

Verificá las cuatro condiciones

```
1. ¿Dos adentro?          NO, el mecanismo lo impide          ✓
2. ¿Supone velocidades?   NO, funciona con cualquier timing   ✓
3. ¿Bloquea el que está   NO: si B no está interesado,
   afuera?                   A entra directo                  ✓
4. ¿Espera indefinida?    NO, se alternan si ambos quieren    ✓
```

Defecto: sigue usando **espera ocupada**.

✓ Solución 5 — La instrucción TSL (hardware)

Las soluciones de software son complicadas. El hardware lo resuelve de una.

TSL = Test and Set Lock. Una instrucción del procesador que hace **dos cosas de manera indivisible y atómica**:

```
TSL registro, candado

   1. LEE el valor de candado y lo copia al registro
   2. ESCRIBE un valor distinto de cero en candado

   ← las dos cosas, SIN que nadie pueda meterse en el medio
```

El secreto: mientras se ejecuta TSL, el procesador **bloquea el bus de memoria**. Ningún otro procesador puede tocar esa variable.

```
TSL                              vs    deshabilitar interrupciones
———                                    ——————————————————————
bloquea el BUS DE MEMORIA              bloquea las interrupciones
→ ningún procesador puede              → solo afecta a ESTA CPU
  acceder a la variable                → las otras siguen

✓ sirve en multiprocesador             ✗ no sirve
```

El uso:

```
entrar:  TSL registro, candado
         si registro != 0 → volver a entrar     (espera ocupada)
         ... región crítica ...
salir:   candado = 0
```

XCHG hace lo mismo intercambiando atómicamente un registro con una posición de memoria. **Las CPU Intel x86 usan XCHG** para sincronización de bajo nivel.

⚠ El problema de fondo: inversión de prioridades

Peterson y TSL son correctos, pero la espera ocupada puede producir algo peor que desperdicio.

Escenario: proceso **H** de alta prioridad, proceso **L** de baja. La regla es que H se ejecuta siempre que esté listo.

```
Paso 1   L entra a su región crítica
Paso 2   H queda listo → por prioridad, DESPLAZA a L
Paso 3   H intenta entrar a la región crítica
         → está tomada → entra en ESPERA OCUPADA (gira)
Paso 4   Como H gira sin parar, L NUNCA vuelve a ejecutarse
Paso 5   L nunca sale de la región crítica
         L nunca libera el candado
         H gira PARA SIEMPRE
```

```
H  ████████████████████  girando, 100% CPU
L  ⣿                     nunca más se ejecuta
     ▲
   quedó adentro de la región crítica, congelado
```

El sistema se cuelga. Y la causa es que H tiene demasiada prioridad.

> Esto no es teórico: le pasó al **Mars Pathfinder** en 1997, en Marte. Se reiniciaba solo y hubo que parchearlo a distancia.

Resumen del tema

Solución	¿Funciona?	Falla en	Observación
Deshabilitar interrupciones	✗	**cond. 2**	Útil solo dentro del kernel
Variables de candado	✗	**cond. 1**	Es el spooler otra vez
Alternancia estricta	✗	**cond. 3**	Obliga a turnarse
Peterson	✓	—	Software puro, 2 procesos
TSL / **XCHG**	✓	—	Necesita apoyo del hardware

Las dos que funcionan comparten el defecto: espera ocupada. Eso es lo que resuelve el tema 7.

Referencia

(Tanenbaum, cap. 2.3.3, págs. impresas **120 a 125** → PDF **150 a 155**).

Preguntas de control

1. ¿Por qué **TSL funciona en un multiprocesador** y deshabilitar interrupciones no? Respondé diciendo **qué bloquea cada uno**.

---

2. En Peterson, los dos procesos quieren entrar al mismo tiempo. A escribe `turno = A` y después B escribe `turno = B`.

¿Cuál entra? ¿Por qué resulta contraintuitivo?

Muy buena pregunta — acá está el nudo del tema.

Primero: ¿cómo hace el SO para sacarle la CPU a un proceso?

El sistema operativo **es software**. Para hacer algo, **tiene que estar ejecutándose**. Y si el proceso A está usando la CPU... el SO **no está corriendo**.

```
CPU ejecutando proceso A
         │
         └─► el SO NO está ejecutándose. Está "dormido" en memoria.
```

Entonces, ¿quién lo despierta? **Una interrupción.** Es la única forma.

La interrupción de reloj

Hay un chip de reloj que genera una interrupción **cada pocos milisegundos**, sin falta. Esa es la que te importa acá:

```
cada ~10 ms
     │
```

```
     ▼
INTERRUPCIÓN DE RELOJ
     │
     ▼
se ejecuta el MANEJADOR DE RELOJ (que es parte del SO)
     │
     ├─► decrementa el quantum del proceso actual
     │
     └─► ¿llegó a cero?
              SÍ → llama al PLANIFICADOR → transición ② → cambia de proceso
              NO → vuelve al proceso, que sigue
```

El quantum no se descuenta solo. Lo descuenta el manejador de reloj, cada vez que se ejecuta.

Y ahora sí: qué pasa si las deshabilitás

```
El proceso ejecuta "deshabilitar interrupciones"
                    │
                    ▼
     la interrupción de reloj NO LLEGA
                    │
                    ▼
     el manejador de reloj NO SE EJECUTA
                    │
                    ▼
     nadie decrementa el quantum
     nadie llama al planificador
                    │
                    ▼
     ☠ EL SO NO SE EJECUTA NUNCA MÁS
        el proceso se queda con la CPU PARA SIEMPRE
```

Respondiendo tus preguntas directamente:

> *¿el planificador no saca el proceso?*

Exacto. **El planificador ni siquiera llega a ejecutarse.** No es que decide no sacarlo: es que nunca lo llaman.

> *¿no lo interrumpe hasta que termine?*

Hasta que **él mismo vuelva a habilitarlas**. Esa es toda la apuesta: confiás en que el proceso cumpla su palabra.

> *¿qué pasa con su quantum?*

Nada. Sigue en el valor que tenía. El reloj físico sigue andando, pero el SO no se entera porque nadie le avisa. Es como un despertador al que le sacaste las pilas: la hora pasa igual, pero no suena.

Por qué eso es tan peligroso

```
MOV AH, ...          ← deshabilitar interrupciones
   ... región crítica ...
   ← si acá hay un bug, un bucle infinito, o simplemente
     el programador se olvidó de rehabilitar...

     LA MÁQUINA SE MUERE. Ni Ctrl+C funciona:
     Ctrl+C también es una interrupción.
```

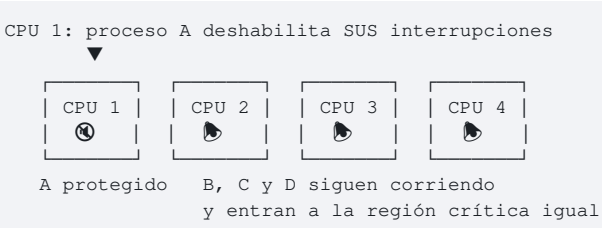
Y hay otro detalle: **se deshabilitan TODAS las interrupciones**, no solo la del reloj.

```
✗ reloj      → no hay planificación
✗ disco      → "terminé de leer"... nadie lo escucha
✗ teclado    → apretás teclas, se pierden
✗ red        → llegan paquetes, se descartan
```

Por eso se usa **solo dentro del kernel y por un puñado de instrucciones**: el kernel es código confiable y sabe

lo que hace. Un proceso de usuario, no.

Y el segundo defecto: multiprocesador



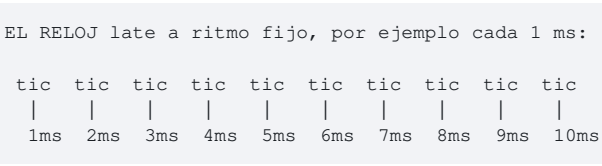
Deshabilitar interrupciones es una instrucción que afecta **al procesador que la ejecuta**. Los otros tres ni se enteran.

→ **Por eso viola la condición 2: supone que hay una sola CPU.**

El contraste que cierra el tema: TSL bloquea **el bus de memoria** (que es compartido por todas las CPU), así que sí funciona en multiprocesador. Deshabilitar interrupciones bloquea **la atención de una CPU**, que es local.

¿Se aclaró?

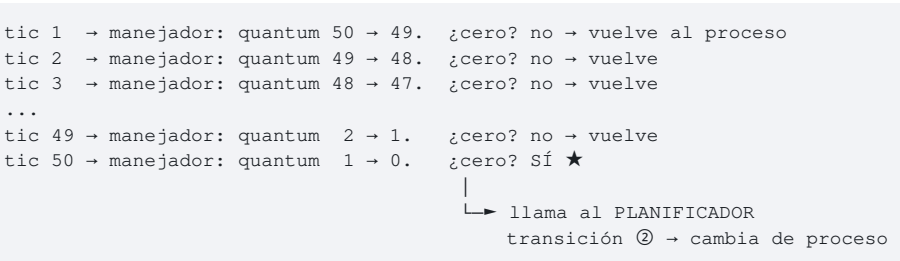
No, y la diferencia es importante. El reloj late **siempre**, pase lo que pase.



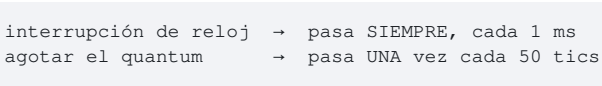
El quantum es un contador que se descuenta en cada tic.

Ejemplo con números

Reloj cada **1 ms**, quantum de **50 ms**:

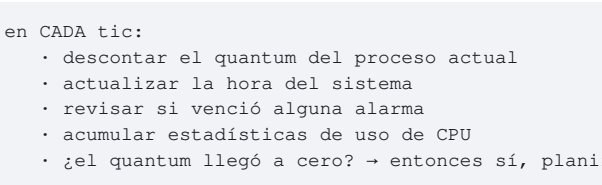


Hubo 50 interrupciones de reloj y 49 no cambiaron nada. Solo la última disparó el cambio.



Por qué está hecho así

Porque el manejador de reloj aprovecha para hacer varias cosas en cada tic, no solo el quantum:



Y ahora se entiende mejor la solución 1

```
CON interrupciones habilitadas:
   tic → el SO se ejecuta un instante → decide si sigue o cambia
   tic → el SO se ejecuta un instante → decide
   tic → ...
   ↑ el SO retoma el control 1000 veces por segundo


SIN interrupciones:
   (nada)
   (nada)
   (nada)
   ↑ el SO NO retoma el control NUNCA
```

> **La clave:** el reloj es la **correa** con la que el sistema operativo mantiene atado al proceso. Mil veces por segundo tira de la correa para ver cómo va. Deshabilitar interrupciones es **cortar la correa**.

Y por eso la planificación **apropiativa** necesita reloj y la **no apropiativa** no: sin reloj, el único momento en que el SO recupera el control es cuando el proceso hace una llamada al sistema o se bloquea.

¿Ahora sí?

Es la distinción central del **tema 8**, pero la adelanto porque acá se ve clarísimo.

```
¿QUIÉN DECIDE QUE EL PROCESO DEJE LA CPU?
```

No apropiativa — la decide el proceso

El proceso corre **hasta que él quiere soltarla**. El SO recupera el control solo cuando:

```
· el proceso TERMINA
· el proceso SE BLOQUEA (pide E/S)
· el proceso hace una LLAMADA AL SISTEMA
```

En los tres casos **el proceso entrega la CPU voluntariamente**. No hace falta ningún reloj: el SO se ejecuta porque lo llamaron.

```
proceso A ████████████████████  hasta que él decida
                               |
                          se bloquea o termina
                               |
                               ▼
                          ahí entra el SO
```

El riesgo: si A entra en un bucle infinito y no pide nada, **se queda con la máquina**. Nadie puede sacarlo.

Apropiativa — la decide el sistema

El SO le **quita** la CPU por la fuerza cuando se le agota el quantum. Para eso necesita **ejecutarse periódicamente**, y la única forma de lograrlo es la **interrupción de reloj**.

```
proceso A ██████|██████|██████|
               tic    tic    tic
                ↑      ↑      ↑
          el SO se asoma en cada tic
          y en algún momento dice "se acabó" → ②
```

```
apropiativa = necesita reloj = el SO puede recuperar el control
              sin pedir permiso
```

La tabla

	No apropiativa	Apropiativa

¿Necesita reloj?	**No**	**Sí**
Quién suelta la CPU	el proceso	el sistema
Un proceso egoísta	**monopoliza**	no puede
Sobrecarga	menor	mayor (cambios de contexto)
Dónde se usa	sistemas por lotes	sistemas interactivos

Y acá está lo bueno: ya viste esto en el tema 4

```
HILOS DE USUARIO                  HILOS DE KERNEL
────────────────                  ───────────────
no hay reloj dentro del proceso   el kernel tiene reloj
un hilo cede con thread_yield     el kernel le quita la CPU
   (voluntario)                      (forzado)
un hilo egoísta en bucle          no puede monopolizar
infinito mata a sus compañeros

= NO APROPIATIVO                  = APROPIATIVO
  entre hilos                       entre hilos
```

Es exactamente la misma idea, un nivel más abajo. Por eso aquel hilo del bucle infinito mataba a sus compañeros de proceso pero no al resto del sistema: adentro del proceso la planificación era **no apropiativa**, y entre procesos era **apropiativa**.

> **La regla que unifica todo:** sin un mecanismo que te devuelva el control por la fuerza, dependés de que el otro se porte bien. El reloj es ese mecanismo.

Vamos despacio, con los valores de las variables a la vista.

Las variables

```
turno          → UN número: de quién es el turno       (0 o 1)
interesado[]   → UN ARREGLO de 2 posiciones:
                   interesado[0] = ¿el proceso 0 quiere entrar?
                   interesado[1] = ¿el proceso 1 quiere entrar?
```

Llamemos **A** al proceso 0 y **B** al proceso 1.

El código

```
ENTRAR (yo, otro):
    interesado[yo] = true          ← paso 1
    turno = yo                     ← paso 2
    mientras (turno == yo  Y  interesado[otro] == true)
        ;                          ← paso 3: GIRAR acá

    ... REGIÓN CRÍTICA ...

SALIR (yo):
    interesado[yo] = false         ← paso 4  ★ acá está la clave
```

La traza, paso a paso

Los dos quieren entrar casi a la vez.

```
ESTADO INICIAL
   turno = ?        interesado[A] = false    interesado[B] = false
```

① **A ejecuta `interesado[A] = true`**

```
   turno = ?        interesado[A] = TRUE     interesado[B] = false
```

② **A ejecuta `turno = A`**

```
   turno = A        interesado[A] = true     interesado[B] = false
```

③ ⏰ **Conmuta a B. B ejecuta `interesado[B] = true`**

```
   turno = A        interesado[A] = true     interesado[B] = TRUE
```

④ **B ejecuta `turno = B`** ← pisa el valor de A

```
   turno = B        interesado[A] = true     interesado[B] = true
```

Ahora cada uno evalúa su condición de espera

```
B evalúa:  mientras (turno == B  Y  interesado[A])
                    ↑ turno vale B    ↑ A está interesado
                    VERDADERO    Y    VERDADERO
                    ──────────────────────────
                           = VERDADERO  →  B GIRA, espera 🔄

A evalúa:  mientras (turno == A  Y  interesado[B])
                    ↑ turno vale B, no A
                    FALSO        Y    (no importa)
                    ──────────────────────────
                           = FALSO  →  A NO espera  →  A ENTRA ✓
```

Con una sola condición falsa alcanza para entrar, porque es un `Y`.

★ Y acá está lo que preguntás: ¿cómo sale B de la espera?

B no cambia nada. El que lo libera es A, al salir.

```
A termina su región crítica y ejecuta:

   interesado[A] = false      ← paso 4

   turno = B        interesado[A] = FALSE    interesado[B] = true


B vuelve a evaluar su condición (está girando, la evalúa sin parar):

   mientras (turno == B  Y  interesado[A])
            VERDADERO   Y    FALSO        ← ¡cambió!
            ──────────────────────────
                   = FALSO  →  B DEJA DE GIRAR  →  B ENTRA ✓
```

> **B no sale porque cambie el turno. Sale porque A deja de estar interesado.**

Esa es la respuesta a tu pregunta: la salida la provoca **el que estaba adentro, al irse**.

Por qué hacen falta las DOS variables

Si sacás una, se rompe:

```
SOLO "turno"            →  es la ALTERNANCIA ESTRICTA
                           obliga a turnarse aunque el otro no quiera entrar
                           ✗ viola la condición 3

SOLO "interesado[]"     →  si los dos se marcan a la vez,
                           los dos ven al otro interesado
                           y los dos esperan para siempre
                           ✗ interbloqueo
```

```
interesado[] →  dice QUIÉN QUIERE entrar
```

```
turno        →  DESEMPATA cuando los dos quieren
```

`turno` solo se mira cuando hay empate. Si B no está interesado, A entra sin importar de quién sea el turno — y eso es lo que arregla la condición 3.

El caso sin competencia

```
A quiere entrar, B está tomando café (interesado[B] = false)

   A evalúa: (turno == A  Y  interesado[B])
                              ↑ FALSO
             = FALSO  →  A entra directo, sin esperar
```

Fijate que **ni miró el turno**. Puede entrar diez veces seguidas si B no aparece. Eso es exactamente lo que la alternancia estricta no permitía.

El resumen en una frase

```
"Cedo el paso si: es mi turno Y el otro también quiere entrar"
                  └──────┬──────┘      └──────────┬──────────┘
              el que escribió      hay competencia
              turno último
```

Suena al revés porque uno esperaría que "anotarse" sea una ventaja. Acá **anotarse último es una desventaja**: equivale a decir *"bueno, pasá vos primero"*.

¿Ahora cierra?

Perfecto, hagamos **la tabla completa**. Solo hay 4 combinaciones posibles.

```
CONDICIÓN DE ESPERA:   mientras ( turno == yo   Y   interesado[otro] )
                                  └──────┬──────┘       └──────┬──────┘
                                   ¿es mi turno?      ¿el otro quiere?
```

Las 4 combinaciones

Caso 1 — `turno == yo` ✓ y `interesado[otro]` ✓

```
   VERDADERO  Y  VERDADERO  =  VERDADERO  →  🔄 ESPERO
```

Cuándo pasa: los dos quieren entrar, y yo escribí `turno` último. **Traducción:** *"Hay empate y yo me anoté último → cedo."*

Caso 2 — `turno == yo` ✓ y `interesado[otro]` ✗

```
   VERDADERO  Y  FALSO  =  FALSO  →  ✓ ENTRO
```

Cuándo pasa: es mi turno, pero el otro ni siquiera quiere entrar. **Traducción:** *"¿Para qué voy a esperar a alguien que no viene?"*

> **Este caso es el que salva la condición 3.** Acá es donde la alternancia estricta fallaba.

Caso 3 — `turno == yo` ✗ y `interesado[otro]` ✓

```
   FALSO  Y  VERDADERO  =  FALSO  →  ✓ ENTRO
```

Cuándo pasa: los dos queremos entrar, pero **el otro escribió `turno` último**. **Traducción:** *"Él se anotó último,*

así que me cedió el paso. Entro yo."

Este es el espejo del caso 1. Si uno está en el caso 1, el otro está sí o sí en el caso 3.

Caso 4 — `turno == yo` ✗ y `interesado[otro]` ✗

```
FALSO  Y  FALSO  =  FALSO  →  ✓ ENTRO
```

Cuándo pasa: el otro no quiere entrar, y encima el turno ni es mío. **Traducción:** *"No hay nadie. Paso."*

La tabla de verdad

	`turno == yo`	`interesado[otro]`	Resultado	Qué hago
1	✓ V	✓ V	**VERDADERO**	🔄 **ESPERO**
2	✓ V	✗ F	FALSO	✓ Entro
3	✗ F	✓ V	FALSO	✓ Entro
4	✗ F	✗ F	FALSO	✓ Entro

De 4 combinaciones, solo UNA me hace esperar. Es un `Y`: con que una sola parte sea falsa, entro.

Ahora los dos procesos a la vez

Escenario A — solo A quiere entrar

```
turno = A      interesado[A] = true     interesado[B] = FALSE

A:  (turno==A ✓) Y (interesado[B] ✗)  =  FALSO  →  CASO 2  →  ENTRA ✓
B:  no está intentando entrar, está en otra cosa
```

Escenario B — solo A quiere entrar, pero el turno es de B

```
turno = B      interesado[A] = true     interesado[B] = FALSE

A:  (turno==A ✗) Y (interesado[B] ✗)  =  FALSO  →  CASO 4  →  ENTRA ✓
```

A entra igual, aunque el turno sea de B. Esto es lo que la alternancia estricta prohibía.

Escenario C — los dos quieren, A escribió turno último

```
turno = A      interesado[A] = true     interesado[B] = true

A:  (turno==A ✓) Y (interesado[B] ✓)  =  VERDADERO  →  CASO 1  →  ESPERA 🔄
B:  (turno==B ✗) Y (interesado[A] ✓)  =  FALSO      →  CASO 3  →  ENTRA ✓
```

Escenario D — los dos quieren, B escribió turno último

```
turno = B      interesado[A] = true     interesado[B] = true

A:  (turno==A ✗) Y (interesado[B] ✓)  =  FALSO      →  CASO 3  →  ENTRA ✓
B:  (turno==B ✓) Y (interesado[A] ✓)  =  VERDADERO  →  CASO 1  →  ESPERA 🔄
```

Escenario E — B estaba esperando y A sale

```
ANTES:   turno = B    interesado[A] = true    interesado[B] = true
```

```
         B está girando en el CASO 1

A sale y ejecuta:  interesado[A] = false

DESPUÉS: turno = B    interesado[A] = FALSE   interesado[B] = true

B reevalúa:  (turno==B ✓) Y (interesado[A] ✗)  =  FALSO  →  CASO 2  →  ENTRA ✓
```

Lo que garantiza que nunca entren los dos

Mirá los escenarios C y D:

```
          A          B
   C    CASO 1     CASO 3      uno espera, uno entra
   D    CASO 3     CASO 1      uno espera, uno entra
```

Nunca pueden estar los dos en el caso 1, porque `turno` es **una sola variable**: no puede valer A y B al mismo tiempo. Si `turno == A` es verdadero para A, entonces `turno == B` es falso para B, y B cae en el caso 3.

> **Esa es toda la demostración de Peterson.** `turno` guarda un solo valor, así que como mucho **uno** de los dos puede estar esperando.

Y para que no haya interbloqueo

```
¿Podrían esperar los dos?
   Haría falta que los dos estén en el CASO 1
   Haría falta que turno == A  Y  turno == B  al mismo tiempo
   IMPOSIBLE: turno es una sola variable
```

¿Ahora sí se arma el cuadro completo?

Dale, vamos de nuevo con TSL, pero arrancando por **el problema que viene a resolver**, no por la instrucción.

Volvamos a donde se rompió la solución 2

La solución de la **variable candado** era esta, y parecía perfecta:

```
mientras (candado == 1)      ← paso 1: MIRO si está ocupado
     ;
candado = 1                  ← paso 2: lo MARCO como ocupado
... REGIÓN CRÍTICA ...
candado = 0
```

Y fallaba por **una sola cosa**: son **dos pasos**.

```
Proceso A: lee candado → ve 0 → "¡libre!"
           ↓
     💥 INTERRUPCIÓN DE RELOJ (justo acá, antes de escribir)
           ↓
Proceso B: lee candado → ve 0 → "¡libre!"  (A todavía no lo marcó)
Proceso B: candado = 1  → entra ✓
           ↓
Proceso A: retoma donde quedó, candado = 1 → entra ✓

   LOS DOS ADENTRO. Violada la condición 1.
```

> **El hueco está entre leer y escribir.** No importa cuánto reordenes el código: mientras sean dos instrucciones, hay un hueco donde te pueden interrumpir.

La idea de TSL: tapar el hueco con hardware

No se puede arreglar con software. Si el problema es que **leer** y **escribir** son dos instrucciones, la solución es pedirle al fabricante de la CPU **una instrucción que haga las dos cosas de un golpe**.

```
TSL REGISTRO, CANDADO
```

Hace exactamente dos cosas, **en el mismo movimiento, indivisible**:

```
   1)  REGISTRO ← candado        (me traigo el valor viejo)
   2)  candado  ← 1              (lo dejo marcado en 1)
```

No existe un "entre" los pasos 1 y 2. No hay instante en el que te puedan interrumpir ahí. Es un paso solo para el resto del mundo.

> **La palabra de examen: TSL es ATÓMICA** (indivisible). Lee y escribe sin que nadie se pueda meter en el medio.

La analogía del baño con cartel

```
Baño con un cartel en la puerta:  [ LIBRE ]  /  [ OCUPADO ]
```

Sin TSL (solución 2):

```
Vos:  mirás el cartel → dice LIBRE
      estirás la mano para darlo vuelta...
      💥 te llaman por teléfono, te distraés
Otro: mira el cartel → todavía dice LIBRE → lo da vuelta y entra
Vos:  volvés, das vuelta el cartel (ya estaba dado vuelta) y entrás

      LOS DOS ADENTRO 😬
```

Con TSL:

```
Vos:  en UN SOLO movimiento:
      ├─ leés el cartel
      └─ lo dejás en OCUPADO

      Y mientras lo hacés, NADIE MÁS puede tocar ese cartel.

      Si lo que leíste decía LIBRE   → el baño era tuyo, entrás
      Si lo que leíste decía OCUPADO → ya había alguien, esperás
```

El truco fino: vos **siempre** lo dejás en OCUPADO, lo leas como lo leas.

- ¿Decía LIBRE y lo pusiste en OCUPADO? Perfecto, era tuyo.
- ¿Decía OCUPADO y lo pusiste en OCUPADO? No cambiaste nada, ya estaba así. Inofensivo.

> **Lo que te dice si ganaste o no es el valor que leíste, no el que escribiste.** El valor que escribís es siempre 1.

Ahora sí, el código (4 instrucciones)

```
entrar_region:
    TSL  REGISTRO, CANDADO    | leo candado al registro Y fijo candado = 1
    CMP  REGISTRO, #0         | ¿lo que leí era 0?
    JNE  entrar_region        | si NO era 0 → estaba ocupado → vuelvo a intentar
    RET                       | era 0 → gané el candado → entro a la región crítica

salir_region:
    MOVE CANDADO, #0          | libero: candado = 0
    RET
```

Línea por línea:

Instrucción	Qué hace
`TSL REGISTRO, CANDADO`	El golpe atómico: me guardo el valor viejo y dejo 1
`CMP REGISTRO, #0`	¿El valor viejo era 0 (libre)?
`JNE entrar_region`	Si **no** era 0, estaba cerrado → vuelvo arriba y reintento (**espera ocupada**)
`RET`	Era 0 → el candado es mío → entro

Y para salir no hace falta nada especial: **un `MOVE` común**. Escribir 0 es un solo paso, no hay nada que proteger.

La traza con dos procesos

```
Estado inicial:  candado = 0

 A:  TSL REG_A, CANDADO    REG_A = 0 ← lee LIBRE  │  candado = 1
 A:  CMP REG_A, #0         0 == 0 ✓
 A:  RET                   → A ENTRA a la región crítica

     ──────  justo ahora el planificador conmuta a B  ──────

 B:  TSL REG_B, CANDADO    REG_B = 1 ← lee OCUPADO │  candado = 1 (sigue 1)
 B:  CMP REG_B, #0         1 ≠ 0 ✗
 B:  JNE entrar_region     → vuelve arriba, GIRA 🔄
 B:  TSL REG_B, CANDADO    REG_B = 1 otra vez... gira, gira, gira

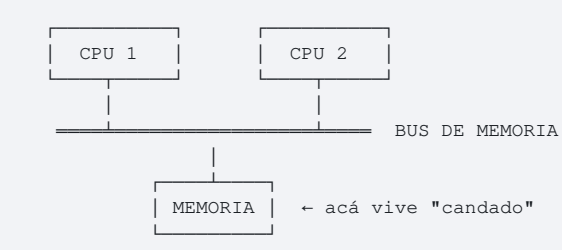
     ──────  A termina y sale  ──────

 A:  MOVE CANDADO, #0      candado = 0
 B:  TSL REG_B, CANDADO    REG_B = 0 ← ¡LIBRE!  │  candado = 1
 B:  CMP REG_B, #0         0 == 0 ✓
 B:  RET                   → B ENTRA
```

Fijate que en el segundo `TSL` de B, aunque `candado` ya valía 1, B igual escribió 1. **No rompe nada.** Y lo importante: B jamás pudo colarse.

La parte que más se pregunta en el final

¿Por qué TSL funciona en un multiprocesador y deshabilitar interrupciones no?



Deshabilitar interrupciones:

```
CPU 1 apaga SUS interrupciones
   → CPU 1 no se interrumpe. Bien.
   → CPU 2 NO SE ENTERÓ DE NADA. Sigue leyendo y escribiendo candado tranquila.

   💥 Las dos pueden leer 0 y las dos entran.
```

Apagar las interrupciones afecta **solo a la CPU que ejecuta la instrucción**. Es un asunto interno de ese procesador.

TSL:

```
CPU 1 ejecuta TSL
   → levanta una línea física del bus: "BUS BLOQUEADO"
   → CPU 2 queda FRENADA, no puede ni llegar a la memoria
   → CPU 1 lee + escribe
   → suelta el bus

   ✓ Exclusión mutua real, aunque haya 2, 4 o 64 CPUs.
```

TSL bloquea el bus de memoria. Deshabilitar interrupciones bloquea una sola CPU. Esa es la diferencia y es literalmente lo que dice el libro *(Tanenbaum, cap. 2.3.3, pág. 123)*.

El dato de la realidad

Las CPUs Intel x86 no usan `TSL`, usan **`XCHG`**, que intercambia un registro con una palabra de memoria de forma atómica. El código es idénticamente el mismo:

```
entrar_region:
    MOVE REGISTRO, #1
    XCHG REGISTRO, CANDADO    | intercambia atómicamente
    CMP  REGISTRO, #0
    JNE  entrar_region
    RET
```

Pongo 1 en el registro, lo intercambio con `candado`, y me fijo qué me vino de vuelta. Misma lógica.

(Tanenbaum, cap. 2.3.3, págs. 123–125, figuras 2-25 y 2-26)

¿Cumple las 4 condiciones?

Condición	TSL
1 — Nunca dos adentro	✓ atómica + bus bloqueado
2 — No supone nada del número de CPUs	✓ **funciona con varias CPUs**
3 — Ningún proceso afuera bloquea a otro	✓ si nadie tiene el candado, entrás directo
4 — Nadie espera para siempre	⚠ en teoría puede haber inanición si siempre gana otro

Lo que queda mal: sigue siendo **espera ocupada**. El proceso que gira quema CPU al 100% sin hacer nada útil. Y encima arrastra el problema de la **inversión de prioridades** que vimos.

Eso es lo que nos lleva al tema 7: **dormir en vez de girar** → semáforos y mutexes.

Preguntas de control

1) Un programador escribe esto en lugar de usar `TSL`:

```
entrar_region:
    MOVE REGISTRO, CANDADO    | lee candado
    MOVE CANDADO, #1          | fija candado = 1
    CMP  REGISTRO, #0
    JNE  entrar_region
    RET
```

Son las mismas operaciones, pero en dos instrucciones. ¿Funciona? Si no, ¿en qué punto exacto se rompe?

2) `TSL` escribe `1` en `candado` **siempre**, incluso cuando el candado ya estaba cerrado y el proceso va a tener que esperar. ¿Por qué esa escritura "de más" no causa ningún problema?

Pará, vamos a tirar el código a la basura por un rato. **Primero la idea, sin una sola línea de assembler.** Después te enseño a leer esas 4 líneas, una por una.

Parte 1 — La idea, sin código

Imaginate una sala de reuniones con **un papelito pegado en la puerta**. El papelito tiene un número:

```
        ┌──────────────────┐
        │                  │
        │   SALA DE        │     papelito:  [ 0 ]
        │   REUNIONES      │
        │                  │     0 = vacía
        └────────┬─────────┘     1 = ocupada
                 │
             [ puerta ]
```

La regla es simple: **si el papel dice 0, entrás y escribís 1. Cuando salís, escribís 0.**

Cómo lo hace Juan (la versión mala, en dos movimientos)

Juan tiene que hacer **dos cosas separadas**:

```
movimiento 1:   LEER   el papel
movimiento 2:   ESCRIBIR un 1 en el papel
```

Y ahí está la desgracia. Mirá qué pasa:

```
🕐  Juan se acerca, LEE el papel ......................  dice 0  → "¡libre!"

🕑  Juan se da media vuelta a buscar la lapicera
    💥  ACÁ SE LO INTERRUMPE (el planificador le saca la CPU)

🕒  Pedro se acerca, LEE el papel .....................  dice 0  → "¡libre!"
          ↑ CLARO QUE DICE 0: Juan nunca llegó a escribir

🕓  Pedro ESCRIBE 1 y entra ...........................  papel: [ 1 ]   Pedro adentro

🕔  Juan vuelve con la lapicera, ESCRIBE 1 y entra .....  papel: [ 1 ]     DOS adentro
```

El papel dice 1. Todo parece correctísimo. **Pero hay dos personas adentro.** Y nadie se dio cuenta.

> El problema no es el papel. El problema es **el ratito que pasa entre leer y escribir**. Ahí se te cuela el otro.

Cómo lo hace TSL

TSL es un **robot que hace las dos cosas de un tirón**, y mientras las hace **traba la puerta para que nadie se pueda acercar al papel**.

Vos le decís al robot: "andá". Y el robot:

```
   1. arranca el papel y te lo pone en la mano
   2. pega un papel nuevo que dice 1
   3. listo

   Y durante esos dos segundos, NADIE MÁS puede tocar el papel.
   No hay "ratito". No hay hueco. No existe un momento intermedio.
```

Ahora fijate en **las dos cosas raras** que hace el robot, porque son la clave de todo:

Rareza 1: el robot escribe 1 SIEMPRE

No importa qué decía el papel. **Siempre** deja un 1 pegado.

```
¿El papel decía 0?  → el robot pega 1.   (bien, acabás de ocupar la sala)
```

```
¿El papel decía 1?  → el robot pega 1.   (no cambió nada, ya decía 1)
```

¿Por qué eso no rompe nada? Porque **escribir 1 donde ya había un 1 no cambia nada**. Es como apagar una luz que ya estaba apagada.

Rareza 2: lo que te dice si ganaste es el papel VIEJO, el que tenés en la mano

Esto es lo que más cuesta. **No mirás la puerta. Mirás tu mano.**

```
   Mirás el papel que el robot te puso en la mano:

   ┌──────────────────────────────────────────────┐
   │  dice 0  →  la sala estaba VACÍA             │
   │             y YO acabo de ocuparla           │
   │             ✓ ENTRO                          │
   ├──────────────────────────────────────────────┤
   │  dice 1  →  la sala YA estaba ocupada        │
   │             por alguien que llegó antes      │
   │             🔄 ESPERO y vuelvo a intentar    │
   └──────────────────────────────────────────────┘
```

Ese papel que tenés en la mano es lo que en el código se llama "el registro". Es tuyo, privado, nadie más lo ve. Cada proceso tiene el suyo.

Ahora Juan y Pedro con el robot

```
papel en la puerta: [ 0 ]

🕐  Juan manda al robot
    ├─ el robot traba la puerta 🔒
    ├─ arranca el papel: dice 0  →  a la mano de Juan
    ├─ pega uno nuevo:   [ 1 ]
    └─ destraba la puerta 🔓

    Juan mira su mano: 0  →  "la sala estaba vacía, es mía"  →  ENTRA ✓

    💥  ahora sí se lo puede interrumpir, no importa, ya está adentro

🕑  Pedro manda al robot
    ├─ el robot traba la puerta 🔒
    ├─ arranca el papel: dice 1  →  a la mano de Pedro
    ├─ pega uno nuevo:   [ 1 ]      (no cambió nada, ya era 1)
    └─ destraba la puerta 🔓

    Pedro mira su mano: 1  →  "ya hay alguien"  →  ESPERA 🔄

🕒  Pedro manda al robot otra vez → mano: 1 → espera
🕓  Pedro manda al robot otra vez → mano: 1 → espera     ← ESPERA OCUPADA
       (sigue quemando CPU preguntando al robot)

🕓  Juan sale y escribe 0 .............................  papel: [ 0 ]

🕐  Pedro manda al robot → mano: 0  →  ENTRA ✓
```

No hay forma de que entren los dos. No existe el "ratito".

> **Y esto es lo único que hay que entender del tema.** Lo demás es notación.

¿Hasta acá vamos bien? Porque si esto cerró, el código es puro vocabulario.

Parte 2 — Cómo se lee ese código

Antes de mirarlo, **el diccionario**. Son 5 cosas nada más.

Las etiquetas (terminan en `:`)

```
entrar_region:
```

Esto **no es una instrucción**. Es un **nombre de un lugar del código**, como el nombre de una calle. Sirve para poder decir después "volvé a `entrar_region`". Es el cartelito que marca ese punto.

`MOVE` (o `MOV`) — copiar

Esta ya la conocés de la unidad 1, cuando hacíamos `mov dl, 'z'`.

```
MOVE CANDADO, #0      ←  "poné un 0 en candado"
```

El `#` quiere decir "el número literal", para distinguirlo del contenido de una dirección de memoria. `#0` es **el número cero**.

`CMP A, B` — comparar

```
CMP REGISTRO, #0      ←  "¿lo que tengo en el registro es igual a 0?"
```

`CMP` **no decide nada**. Solo compara y **anota el resultado al costado** (en unas banderas de la CPU). Es como alguien que mira y dice "son iguales" o "son distintos", y lo deja escrito en un pizarrón, pero no hace nada más.

`JNE etiqueta` — el "si no, volvé"

`JNE` = **J**ump if **N**ot **E**qual = "saltá si eran distintos".

```
JNE entrar_region     ←  "si la comparación de antes dijo DISTINTOS,
                          saltá al lugar llamado entrar_region"
```

Esta sí decide. Lee el pizarrón que dejó `CMP`:

```
   CMP dijo "distintos"  →  JNE SALTA     (me voy a entrar_region)
   CMP dijo "iguales"    →  JNE NO SALTA  (sigo con la línea de abajo)
```

`CMP` + `JNE` juntos son un `if`. En la vida real escribirías `if (registro != 0) volver al principio;`. En assembler hacen falta dos instrucciones: una que compara, otra que salta.

`RET` — volver

```
RET                   ←  "terminé, volvé al programa que me llamó"
```

Esto lo vimos en las llamadas de procedimiento: el programa principal llamó a esta rutina, y `RET` lo devuelve a la línea siguiente de donde llamó.

Ahora sí, el código, línea por línea

```
entrar_region:
    TSL  REGISTRO, CANDADO
    CMP  REGISTRO, #0
    JNE  entrar_region
    RET
```

Traducido al castellano, como si fuera el pensamiento de Pedro:

```
entrar_region:                  ← "acá empieza la rutina de pedir la sala"

    TSL REGISTRO, CANDADO       ← "robot, andá"
                                   (el papel viejo me queda en REGISTRO,
                                    el papel de la puerta queda en 1)

    CMP REGISTRO, #0            ← "¿el papel que tengo en la mano es 0?"
                                   (solo compara y anota el resultado)

    JNE entrar_region           ← "si NO era 0 (o sea, había alguien),
                                    volvé arriba y mandá al robot de nuevo"
```

```
                                ↑↑↑ ACÁ ESTÁ LA ESPERA OCUPADA:
                                    este salto es el que da vueltas

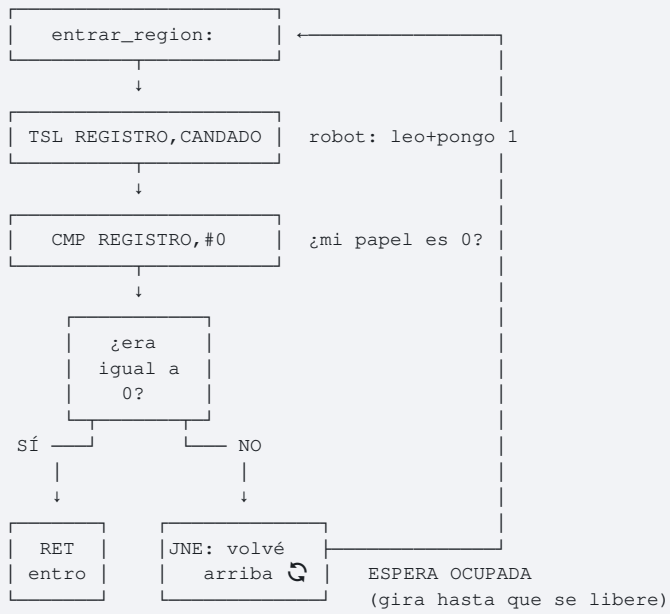
    RET                         ← "era 0, la sala es mía, volvé al programa
                                    así entra a la región crítica"
```

Y la salida:

```
salir_region:
    MOVE CANDADO, #0            ← "pegá un 0 en la puerta" (libero la sala)
    RET                         ← "volvé"
```

¿Por qué salir no necesita TSL? Porque escribir un 0 es **un solo movimiento**. No hay nada que leer antes, así que no hay ningún "ratito" que proteger.

El recorrido del código, dibujado



(Tanenbaum, cap. 2.3.3, págs. 123–125, figura 2-25)

Te armo la visualización para que lo manipules, porque este tema se entiende mucho mejor moviendo el momento de la interrupción.

Dale, vamos con **inversión de prioridades**, que es el punto que mata a la espera ocupada. De nuevo: primero la historia, después el nombre.

El escenario: dos procesos y una regla

```
H  →  proceso de prioridad ALTA
L  →  proceso de prioridad BAJA

REGLA DEL PLANIFICADOR:
«Si H está listo, H se ejecuta. Siempre. L solo corre cuando H no puede.»
```

Esa regla es **totalmente razonable**. H es importante (el control de un motor, una alarma, lo que quieras) y L es un proceso de relleno. Querés que H tenga prioridad absoluta.

Ahora mirá cómo esa regla razonable, combinada con espera ocupada, **cuelga la máquina**.

La historia: el jefe y el empleado

En una oficina hay **un archivo con una sola llave** (la región crítica).

Y hay una regla de la empresa: **mientras el jefe está pidiendo algo, nadie más trabaja. Todos quedan quietos hasta que el jefe esté satisfecho.**

```
🕐  El empleado agarra la llave y entra al archivo.
    Está adentro, trabajando. Todo normal.

🕑  Llega el jefe.
    Por la regla de la empresa, el empleado QUEDA CONGELADO en el acto.
    (no lo congelan afuera: lo congelan ADENTRO del archivo, con la llave en el bolsillo)

🕒  El jefe quiere entrar al archivo. Está cerrado.
    El jefe se pone a golpear la puerta:  toc toc toc toc toc toc...

🕓  El empleado, adentro, quiere salir y darle la llave.
    PERO NO PUEDE MOVERSE: la regla dice que mientras el jefe esté
    pidiendo algo, nadie más trabaja.

🕓  El jefe golpea.          toc toc toc toc...
🕓  El empleado congelado.
🕓  El jefe golpea.          toc toc toc toc...
🕓  El empleado congelado.

    ♾   PARA SIEMPRE. La oficina entera se murió.
```

La causa está en el círculo: el jefe no avanza porque el empleado no suelta la llave, y el empleado no suelta la llave porque el jefe no lo deja moverse.

Y lo peor: **el jefe se está causando el problema a sí mismo.** Si se fuera a tomar un café dos minutos, el empleado saldría, dejaría la llave y todo se arreglaría.

Lo mismo, con H y L

```
🕐  L está en su REGIÓN CRÍTICA
    candado = 1   (L tiene el candado)
    H está BLOQUEADO esperando una E/S

    ┌──────────────────────────────────────┐
    │  L: EJECUCIÓN   (dentro de la R.C.)  │
    │  H: BLOQUEADO                        │
    └──────────────────────────────────────┘


🕑  Termina la E/S de H  →  H pasa a LISTO

    El planificador ve que H (prioridad alta) está listo
    →  DESALOJA a L   (L pasa a LISTO, con el candado todavía agarrado)

    ┌──────────────────────────────────────┐
    │  H: EJECUCIÓN                        │
    │  L: LISTO       ⚠  ¡CON EL CANDADO!  │
    └──────────────────────────────────────┘


🕒  H ejecuta entrar_region:

        TSL REGISTRO, CANDADO    →  registro = 1  (ocupado)
        CMP REGISTRO, #0         →  1 ≠ 0
        JNE entrar_region        →  vuelve arriba  ↺
        TSL ... CMP ... JNE ...  →  vuelve arriba  ↺↺
        TSL ... CMP ... JNE ...  →  vuelve arriba  ↺↺↺

    H está girando en ESPERA OCUPADA.
```

```
🕓  ¿Y acá está el detalle que cambia todo?

    H, girando, está en estado EJECUCIÓN.
    El planificador NO SABE que H está esperando algo.
    Para el planificador, H está trabajando a toda máquina.

    →  Como H es de prioridad alta y está "trabajando", L nunca se planifica.
    →  L nunca llega a ejecutar  candado = 0
    →  el candado no se libera nunca
    →  H gira para siempre

    ♾   BLOQUEO TOTAL
```

¿Por qué se llama "inversión de prioridades"?

Porque en los hechos **las prioridades quedaron dadas vuelta**:

```
EN EL PAPEL:      H manda,  L obedece

EN LA REALIDAD:   L, el de prioridad BAJA, tiene paralizado
                  a H, el de prioridad ALTA.

                  El de abajo manda sobre el de arriba. INVERTIDO.
```

> El proceso de prioridad baja terminó teniendo poder de veto sobre el de prioridad alta. Y encima la máquina ni se da cuenta de que está colgada: **para el planificador, H está trabajando perfecto al 100% de CPU.**

La parte importante: esto se arregla con el diagrama de estados

Acá se ve por qué la espera ocupada es el verdadero culpable. Mirá la diferencia:

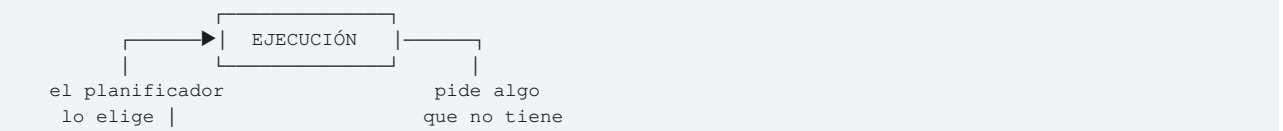
Con espera ocupada (TSL, Peterson)

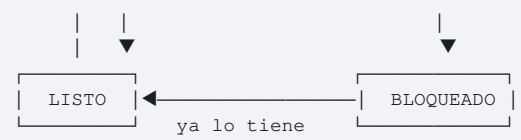
```
H no puede entrar  →  GIRA
                        ↓
                  H sigue en EJECUCIÓN
                        ↓
                  H no suelta la CPU
                        ↓
                  L nunca corre  →  💀 muerto
```

Si H se DURMIERA en lugar de girar

```
H no puede entrar  →  SE DUERME
                        ↓
                  H pasa a BLOQUEADO     ← ¡acá está la clave!
                        ↓
                  H SUELTA LA CPU
                        ↓
                  el planificador elige a L (es el único listo)
                        ↓
                  L sale de su región crítica, candado = 0
                        ↓
                  L despierta a H  →  H pasa a LISTO  →  EJECUCIÓN
                        ↓
                  H entra. ✓  TODO FUNCIONA
```

Y lo dibujo sobre el diagrama de estados que ya conocés:






```
ESPERA OCUPADA:  H se queda clavado acá  →  [ EJECUCIÓN ]  ← acapara la CPU
DORMIR:          H se va para acá        →  [ BLOQUEADO ]  ← libera la CPU
```

Girar es "esperar sin salir de EJECUCIÓN". Dormir es "esperar desde BLOQUEADO". Esa única diferencia de estado es lo que resuelve el problema.

(Tanenbaum, cap. 2.3.4, pág. 126)

Lo que tenés que poder decir en el final

```
┌──────────────────────────────────────────────────────────┐
| Inversión de prioridades:                                |
|                                                          |
| Un proceso de prioridad ALTA queda girando en espera ocupada |
| por un candado que tiene un proceso de prioridad BAJA.   |
| Como el de alta prioridad nunca suelta la CPU, el de baja |
| nunca se planifica, nunca libera el candado, y el de alta |
| gira indefinidamente.                                    |
|                                                          |
| Requiere: planificación por prioridades + espera ocupada. |
| Se resuelve: bloqueando en vez de girar (sleep/wakeup,   |
| semáforos, mutex).                                       |
└──────────────────────────────────────────────────────────┘
```

Ojo con el error típico: esto **no es un interbloqueo** (deadlock) del tema de la unidad 6. En un interbloqueo cada proceso espera un recurso que tiene el otro. Acá L **no está esperando nada**: L podría terminar perfectamente si lo dejaran correr. El problema es puramente de planificación.

> *(Dato fuera del libro, como complemento: esto le pasó de verdad al **Mars Pathfinder** en 1997. La sonda se reiniciaba sola en Marte por una inversión de prioridades, y el equipo de la NASA la arregló a distancia activando la herencia de prioridades.)*

Preguntas de control

Alguien propone este arreglo: *"que el proceso H, si después de girar 1000 veces no consiguió el candado, se duerma un rato".*

¿Resolvería la inversión de prioridades? ¿Por qué?

Sí, **la primera parte la tenés perfecta**. La segunda está bien encaminada pero con dos cosas dadas vuelta. Vamos por partes.

✓ TSL: lo dijiste exacto

> *"es lo mismo que la variable candado, solo que esa se hacía en dos pasos y había un hueco; TSL usa el hardware para hacer esos dos pasos en uno"*

Eso es TSL, literal. Mismo candado, misma variable compartida, misma idea. La única diferencia es que **leer + escribir se vuelven indivisibles** porque la CPU lo garantiza bloqueando el bus de memoria.

```
variable candado   →   leer ... [HUECO] ... escribir     ✗
```

```
TSL                →   [ leer + escribir ]              ✓ atómico
```

Si en el final te preguntan "¿qué aporta TSL sobre la variable candado?", esa frase tuya es la respuesta.

⚠ Inversión de prioridades: dos ajustes

Ajuste 1 — El planificador sí actúa. Lo que pasa es que vuelve a elegir a H.

Vos dijiste *"el que dice que va a salir es el proceso, no el planificador"*. La intuición es buena, pero el mecanismo es otro:

```
El reloj SÍ interrumpe a H cada quantum.
El planificador SÍ se ejecuta y SÍ elige de nuevo.

Pero cuando mira quién está listo:
     H  (prioridad ALTA)   ← lo elige a él
     L  (prioridad BAJA)

→  elige a H otra vez. Y otra. Y otra.
```

No es que nadie lo interrumpa: es que lo vuelven a elegir siempre. El planificador está haciendo su trabajo correctamente.

Y ahí está la clave de por qué la espera ocupada es el problema: **H nunca se declara bloqueado**, así que nunca sale de la lista de listos. Si se durmiera, desaparecería de esa lista y L sería el único candidato.

Ajuste 2 — Esto es lo más importante: la prioridad NO se descarta. Se cumple de más.

Vos dijiste *"nunca se va a poder el de mayor priorización en el momento que se requiere"*. Está al revés:

```
                      ¿Consigue la CPU?    ¿Puede avanzar?
H (alta)                   SÍ, siempre          NO ✗
L (baja)                   NO, nunca            NO ✗
```

- **H sí consigue la CPU.** La tiene toda, al 100%, en el momento exacto en que la necesita. La prioridad funciona perfecto.
- **El que queda afuera de la CPU es L**, el de prioridad baja.
- Lo que H no consigue **no es la CPU: es el candado**. Y el candado lo tiene L, que no puede correr para soltarlo.

La prioridad no falla. Se respeta demasiado bien. El sistema se cuelga precisamente *porque* le dio a H toda la CPU que pidió.

Reescribí tu frase así:

```
✗  "el de mayor prioridad nunca llega a ejecutarse"

✓  "el de mayor prioridad se ejecuta siempre, pero gira sin avanzar,
    y al acaparar la CPU impide que el de prioridad baja libere
    el candado que él mismo está esperando"
```

Es lo que hace que el nombre tenga sentido: **en los hechos, el de prioridad baja terminó mandando sobre el de prioridad alta.**

El resumen de los dos temas juntos

```
TSL resuelve la EXCLUSIÓN MUTUA.    ✓   (nunca dos adentro)
TSL no resuelve la ESPERA OCUPADA.  ✗   (el que espera quema CPU)

Y de la espera ocupada salen dos males:
```

```
    1. desperdicio de CPU
    2. inversión de prioridades  ← este es grave, cuelga el sistema

→ Por eso el próximo paso es DORMIR en lugar de girar:
  sleep/wakeup, semáforos, mutex.  (tema 7)
```

Te armo ahora la visualización, así lo ves moviendo el momento de la interrupción. Y me quedó pendiente la pregunta: el arreglo de *"que H gire 1000 veces y si no consiguió el candado, se duerma un rato"* — ¿funciona?

Ahí está el problema, y es mi culpa: **la analogía se presta a confusión**. Vos entendiste "estar adentro del archivo" = "estar trabajando". Y es justo lo que **no** pasa.

Hay **dos cosas distintas** que la analogía me mezcló en una sola:

```
1)  TENER EL CANDADO        (estar dentro de la región crítica)
2)  TENER LA CPU            (estar en estado EJECUCIÓN)
```

Son independientes. Un proceso puede tener el candado y **estar congelado sin la CPU**. Eso es exactamente lo que le pasa a L.

> **La región crítica no es un lugar donde estás activo. Es un permiso que tenés agarrado.** Podés estar completamente quieto con el permiso en el bolsillo.

Vamos de nuevo con una analogía que separa las dos cosas.

La oficina con UNA SOLA silla

En la oficina hay:

```
🪑  UNA SOLA SILLA con teclado  →  es la CPU
                                   para hacer CUALQUIER COSA
                                   hay que estar sentado ahí

🔑  UNA LLAVE del archivo       →  es el CANDADO
                                   se guarda en el bolsillo
```

La regla clave, y es la que lo explica todo:

```
┌──────────────────────────────────────────────────────┐
│  Para hacer cualquier cosa —INCLUSO DEVOLVER LA LLAVE— │
│  tenés que estar sentado en la silla.                │
└──────────────────────────────────────────────────────┘
```

Ahora la película:

```
🕐  L está SENTADO en la silla 🪑  y tiene la LLAVE 🔑 en el bolsillo.
    Está trabajando con los datos compartidos. Todo normal.

    🪑 silla: L        🔑 llave: L


🕑  Llega H (prioridad alta).
    H tiene más prioridad, entonces LEVANTA A L DE LA SILLA y se sienta él.

    🪑 silla: H        🔑 llave: L   ← ⚠  MIRÁ ESTO
                                       L está PARADO al costado,
                                       quieto, SIN la silla,
                                       PERO CON LA LLAVE EN EL BOLSILLO


🕒  H, sentado, quiere la llave. Ejecuta TSL:
       "¿está libre la llave?"  →  NO, la tiene L
       "¿está libre la llave?"  →  NO
       "¿está libre la llave?"  →  NO
```

```
    H se queda sentado preguntando. No se levanta.


🕓  L quiere devolver la llave. De verdad quiere, es su próxima instrucción.
    Pero para devolverla TIENE QUE SENTARSE EN LA SILLA.
    Y H no se levanta.

    L está PARADO, CONGELADO, con la llave en el bolsillo.
    No porque no quiera devolverla: porque NO PUEDE EJECUTAR NADA.


🕓  H sentado preguntando.    L parado con la llave.
🕐  H sentado preguntando.    L parado con la llave.
🕑  H sentado preguntando.    L parado con la llave.

    ♾   Nadie avanza nunca.
```

El círculo, en una línea

```
   H no se levanta de la silla     porque espera la llave
                                         ↑
   L no devuelve la llave          porque no tiene la silla
```

Cada uno traba al otro, y **el que rompería el empate es H**: si se levantara dos segundos, L se sienta, devuelve la llave, y listo. Pero H no se levanta porque está esperando.

La versión técnica, con las dos columnas separadas

Esto es lo que te conviene fijar:

Momento	🪑 CPU (estado)	🔑 Candado
1 — L trabaja tranquilo	**L** (EJECUCIÓN)	**L**
2 — H pasa a listo y desaloja a L	**H** (EJECUCIÓN)	**L** ⚠ *L está en LISTO*
3 — H gira en espera ocupada	**H** (EJECUCIÓN)	**L**
4 — sigue igual para siempre	**H** (EJECUCIÓN)	**L**

Fijate en la fila 2: **L tiene el candado pero está en estado LISTO.** Esa combinación es la que no habías visto, y es toda la trampa del tema.

```
   L  →  estado LISTO     +  tiene el candado
         ↑                   ↑
         no puede ejecutar   nadie más puede entrar a la región crítica

         Es un proceso congelado con la puerta cerrada por dentro.
```

La frase que lo resuelve

```
┌──────────────────────────────────────────────────────────┐
│  Para liberar el candado, L tiene que EJECUTAR           │
│  la instrucción  candado = 0                             │
│                                                          │
│  Y para ejecutar cualquier instrucción, L necesita LA CPU.  │
│                                                          │
│  Si nunca le dan la CPU, nunca libera el candado.        │
└──────────────────────────────────────────────────────────┘
```

Eso es todo el tema. Soltar un candado **no es gratis**: es código, y el código necesita CPU.

(Tanenbaum, cap. 2.3.4, pág. 126)

Preguntas de control

En el momento 2 de la tabla, cuando H desaloja a L:

¿En qué estado queda L: LISTO o BLOQUEADO? ¿Y por qué eso importa para entender por qué el sistema se cuelga?

Sí. Ahora sí lo tenés. Lo que escribiste es correcto. Dejame confirmártelo y afinarte dos cosas chiquitas, porque las dos se preguntan.

Tu frase, confirmada

```
L estaba usando un recurso y no terminó
        ↓
entra H, de mayor prioridad, y lo desaloja
        ↓
pero lo desaloja CON EL RECURSO AGARRADO
        ↓
H no puede trabajar porque no tiene el recurso
```

✓ **Correcto, tal cual.** Eso es la inversión de prioridades.

Precisión 1 — La culpa no es de TSL. Es de la espera ocupada.

Vos arrancaste con *"en TSL hay un problema"*. Ojo con eso, porque es un punto de examen:

```
TSL  →  ¿garantiza exclusión mutua?   SÍ, perfecta ✓
     →  ¿es la causa del problema?    NO ✗
```

La inversión de prioridades le pasa **igual** a Peterson, a la variable candado, a XCHG y a cualquier solución que **espere girando**. El culpable es el **método de espera**, no el mecanismo del candado.

```
┌──────────────────────────────────────────┐
│  CULPABLE:  la ESPERA OCUPADA            │
│                                          │
│  Peterson, TSL, XCHG  →  todos la usan   │
│                       →  todos sufren esto │
└──────────────────────────────────────────┘
```

Precisión 2 — Lo fatal no es que H no pueda trabajar. Es que H no suelta la CPU.

Esta es la parte fina, y es la que hace que el sistema se cuelgue **para siempre** en vez de solo demorarse.

Separá las dos desgracias:

```
Desgracia 1:  H no tiene el recurso, así que no puede avanzar.
              ──────────────────────────────────────────────
              Esto solo sería ESPERA NORMAL. Molesto, pero sano.
              Se arregla solo: L termina y le da el recurso.

Desgracia 2:  H, mientras no puede avanzar, SE QUEDA CON LA CPU
              (gira preguntando, no se duerme)
              ──────────────────────────────────────────────
              ESTO es lo que lo hace eterno.
```

Mirá el contraste. **El mismo escenario, cambiando solo qué hace H al no conseguir el candado:**

Si H gira (espera ocupada)

```
H: EJECUCIÓN girando 🔄        L: LISTO con el candado 🔑
H: EJECUCIÓN girando 🔄🔄       L: LISTO con el candado 🔑
H: EJECUCIÓN girando 🔄🔄🔄      L: LISTO con el candado 🔑

♾️  ETERNO. Nadie avanza.
```

Si H se duerme (se bloquea)

```
H: no consigue el candado  →  pasa a BLOQUEADO 💤
                              ↓
                         SUELTA LA CPU
                              ↓
L: el planificador lo elige (es el único listo)  →  EJECUCIÓN
L: ejecuta  candado = 0       →  🔑 libre
L: despierta a H              →  H pasa a LISTO
                              ↓
H: EJECUCIÓN  →  agarra el candado  →  entra ✓

✓ FUNCIONA. Y H igual terminó siendo el prioritario.
```

La diferencia es una sola: si H, al no poder avanzar, **se saca de la lista de listos** o no. Girando, sigue en la lista y lo eligen siempre. Durmiendo, desaparece de la lista y L tiene su chance.

El resumen de los tres temas encadenados

```
variable candado  →  falla la exclusión mutua (hueco entre leer y escribir)
        ↓
TSL / Peterson    →  exclusión mutua ✓  ... pero esperan GIRANDO
        ↓
espera ocupada    →  quema CPU  +  inversión de prioridades 💀
        ↓
DORMIR            →  sleep/wakeup, semáforos, mutex   ← tema 7
```

Ahí se ve por qué el tema 7 existe: no es una solución más elegante del mismo problema, es **la única que no cuelga el sistema**.

(Tanenbaum, cap. 2.3.3 y 2.3.4, págs. 123–126)

Te armo la visualización ahora mismo, con las dos columnas separadas (quién tiene la CPU / quién tiene el candado) y un interruptor para que compares "H gira" contra "H duerme". Dame un minuto.

Dos preguntas muy buenas, y la segunda toca justo el punto que más se confunde. Vamos una por una.

---

1) ¿Si H no quiere ese recurso, todo normal?

Sí, todo normal. No pasa absolutamente nada.

```
🕐  L: EJECUCIÓN, dentro de su región crítica, con el candado 🔑

🕑  H pasa a LISTO  →  desaloja a L
    L queda en LISTO con el candado 🔑

🕒  H ejecuta lo suyo. NUNCA llama a entrar_region.
    No pide el candado, no gira, no pregunta nada.
    Hace su trabajo tranquilo.

🕓  H termina (o se bloquea esperando una E/S)  →  suelta la CPU

🕔  El planificador elige a L  →  L: EJECUCIÓN
    L sigue donde quedó, termina su región crítica, candado = 0 🔓

    ✓  Todo funcionó. H fue prioritario, L terminó. Nadie se colgó.
```

Que a L le saquen la CPU con el candado agarrado **no es el problema**. Eso pasa todo el tiempo y es normal. El problema aparece **solo** cuando el que lo desalojó quiere **ese mismo candado**.

Los 3 ingredientes de la inversión de prioridades

Esto conviene tenerlo así, porque en el final te pueden dar un escenario y preguntarte si ocurre o no:

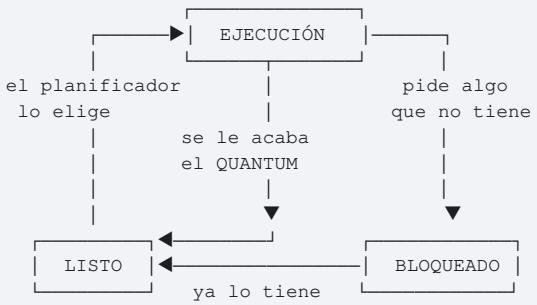
```
┌──────────────────────────────────────────────────────────┐
│  1.  Planificación POR PRIORIDADES                       │
│      (si no, el reloj le da su turno a L y se arregla)   │
│                                                          │
│  2.  ESPERA OCUPADA                                      │
│      (si H durmiera, soltaría la CPU y se arregla)       │
│                                                          │
│  3.  Los dos compiten por el MISMO candado               │
│      (si H no lo quiere, no hay conflicto)               │
└──────────────────────────────────────────────────────────┘

   Faltando CUALQUIERA de los tres  →  NO hay inversión de prioridades.
```

---

2) ¿Cuando a H se le acaba el quantum pasa a bloqueado?

No. Y acá está el corazón del tema. Mirá el diagrama de estados:



Fin de quantum → LISTO. Nunca a BLOQUEADO. Esas son dos transiciones completamente distintas:

Qué pasó	A dónde va	Quién lo decide
Se le acabó el quantum	**LISTO**	el **planificador** (se lo saca)
Pide algo que no tiene (E/S, `sleep`)	**BLOQUEADO**	el **proceso mismo** (se va solo)

Y ahora la secuencia real

```
🕐  H: EJECUCIÓN, girando en TSL 🔄
    L: LISTO, con el candado 🔑

🕑  💥 Se le acaba el quantum a H
    H pasa a LISTO.     ✓ esto SÍ pasa, tenías razón en que lo interrumpen

    Ahora en la cola de listos hay:
        H  (prioridad ALTA)
        L  (prioridad BAJA)

🕒  El planificador elige... a H. Es el de mayor prioridad.
    H: EJECUCIÓN otra vez, girando 🔄
    L: sigue en LISTO con el candado 🔑

🕓  💥 Se le acaba el quantum a H  →  LISTO
🕔  El planificador elige a H otra vez  →  EJECUCIÓN, girando 🔄
🕕  💥 quantum  →  LISTO  →  eligen a H  →  girando 🔄

    ♾   L nunca es elegido. Nunca ejecuta  candado = 0.
```

El quantum se acaba una y otra vez, y no sirve de nada, porque cada vez que se vuelve a repartir la CPU, H gana por prioridad. L está en la cola pero nunca llega al frente.

La clave: con espera ocupada, H NO TIENE CÓMO BLOQUEARSE

Mirá otra vez las 4 instrucciones y buscame una que bloquee:

```
entrar_region:
    TSL  REGISTRO, CANDADO    ← lee y escribe memoria. No bloquea.
    CMP  REGISTRO, #0         ← compara. No bloquea.
    JNE  entrar_region        ← salta. No bloquea.
    RET                       ← vuelve. No bloquea.
```

No hay ninguna. Ninguna de esas 4 instrucciones es una llamada al sistema. H nunca le dice al SO "ponéme en BLOQUEADO". Solo da vueltas.

```
   Para pasar a BLOQUEADO hace falta una LLAMADA AL SISTEMA.
   TSL no tiene ninguna. Peterson tampoco.
   Por eso se llama ESPERA OCUPADA: el proceso espera SIN bloquearse.
```

Tu intuición está anticipando el tema 7

Fijate lo que preguntaste: *"¿no debería pasar a bloqueado para que el otro suelte el recurso?"*

Eso es exactamente lo que debería pasar. Y es exactamente lo que falta. Esa es la solución, y se llama `sleep`/`wakeup`, y es lo que hacen los semáforos y los mutexes:

```
   TSL:        "¿libre?" ... "¿libre?" ... "¿libre?"   🔄 EJECUCIÓN
               → no se bloquea nunca

   Semáforo:   "¿libre? no → DESPERTAME CUANDO SE LIBERE"  💤 BLOQUEADO
               → se bloquea, suelta la CPU, L avanza
```

Le pusiste el dedo al problema justo donde va. Ese es el tema 7.

(Tanenbaum, cap. 2.3.4, pág. 126)

Ahora sí te armo la visualización, porque esto de "H pierde el quantum pero lo vuelven a elegir" se entiende mucho mejor viéndolo correr.

Estas dos, del cierre del tema 6:

Pregunta 1

¿Por qué `TSL` funciona en una máquina con varias CPUs y deshabilitar las interrupciones no?

Pregunta 2

Dos procesos usan la solución de Peterson. Pasa esto, en este orden:

```
   A ejecuta:  interesado[A] = true
   A ejecuta:  turno = A
   B ejecuta:  interesado[B] = true
   B ejecuta:  turno = B        ← B escribió turno DESPUÉS
```

¿Cuál de los dos entra a la región crítica y cuál se queda girando? ¿Por qué?

Y si querés, una tercera sobre lo que acabamos de ver recién:

Pregunta 3 (opcional)

Alguien propone este arreglo para la inversión de prioridades: *"que H gire 1000 veces y, si no consiguió el candado, se duerma un rato"*.

¿Resolvería el problema?

Dos de tres bien, y la tercera tiene un error interesante. Vamos.

---

1) TSL en multiprocesador — ✓ CORRECTA

La idea central la dijiste bien: **TSL bloquea el bus de memoria**, y deshabilitar interrupciones es **local a una sola CPU**.

Dos precisiones de vocabulario para el final:

```
✗  "otras CPUs no pueden meterse a otras CPUs"
✓  "otras CPUs no pueden ACCEDER A LA MEMORIA mientras dura el TSL"
```

TSL no frena a las CPUs: frena el **camino hacia la memoria**. Las otras CPUs siguen vivas, pero si quieren leer o escribir `candado`, se encuentran el bus tomado.

Y la segunda mitad, ordenada:

```
CPU 1 deshabilita las interrupciones
     → afecta SOLO a la CPU 1 (es un bit de SU registro de estado)
     → la CPU 2 NO SE ENTERÓ DE NADA
     → la CPU 2 lee y escribe candado cuando quiere

💥 las dos leen 0, las dos entran. Exclusión mutua violada.
```

> La frase de examen: **deshabilitar interrupciones protege una CPU de sí misma; TSL protege la memoria de todas las CPUs.**

---

2) Peterson — ✓ CORRECTA

Entra **A**, y la razón que diste es la correcta: **el último que escribe `turno` es el que cede**.

Te muestro la justificación con las evaluaciones, porque así la querés escribir en el examen:

```
Estado final:   turno = B      interesado[A] = true      interesado[B] = true

A evalúa:  (turno == A  Y  interesado[B])
            └─ FALSO ─┘       └─ true ─┘
           turno vale B, no A
          = FALSO  →  A NO entra al while  →  A ENTRA a la región crítica ✓

B evalúa:  (turno == B  Y  interesado[A])
            └─ VERDADERO ┘    └─ true ─┘
          = VERDADERO  →  B GIRA 🔄
```

Y B se libera recién cuando A ejecuta `interesado[A] = false` al salir. No cuando cambia `turno`.

---

3) El arreglo de "girar 1000 veces y dormir un rato" — ⚠ PARCIAL

Tu razonamiento del mecanismo es correcto: **si H suelta la CPU, L puede correr y liberar el candado.** Eso está bien visto. En el caso de dos procesos, el sistema efectivamente deja de colgarse.

Pero **no resuelve el problema**, y por cuatro motivos. Te los doy porque es exactamente el tipo de pregunta de final.

a) ¿Cuánto es "un rato"? No hay respuesta posible

```
duerme POCO   →  L no alcanzó a terminar su región crítica
                 →  H despierta, gira 1000 veces de nuevo, duerme...
                 →  ciclo infinito de girar y dormir

duerme MUCHO  →  H, el proceso CRÍTICO, está dormido de gusto
                 →  si el candado se liberó al instante, H perdió todo ese tiempo
```

Para elegir bien el número habría que saber **cuánto tarda L en su región crítica**, y eso no se puede saber. Es el mismo pecado de la condición 2: **estarías haciendo suposiciones sobre las velocidades.**

b) Sacrificaste justamente lo que querías proteger

H es de prioridad alta por algo. Si es el control del ABS, "dormí un rato" es inaceptable:

```
Querías:  H responde YA
Lograste: H responde dentro de un rato indeterminado
```

Rompiste la garantía de prioridad para arreglar un problema de prioridad.

c) Seguís quemando 1000 vueltas de CPU cada vez

Sigue siendo espera ocupada, solo que acotada. El desperdicio no desapareció.

d) El golpe final: con TRES procesos vuelve a fallar

Este es el que lo mata. Agregá un proceso **M**, de prioridad media:

```
H  (ALTA)     quiere el candado
M  (MEDIA)    no le interesa el candado, solo computa
L  (BAJA)     tiene el candado

🕐 H gira 1000 veces, se duerme 💤  →  suelta la CPU

🕑 El planificador mira quién está listo:
       M  (MEDIA)   ← lo elige a él
       L  (BAJA)

🕒 M se ejecuta. L sigue sin la CPU. El candado sigue tomado.

🕓 H despierta → es el de mayor prioridad → desaloja a M → gira 1000 veces → duerme
🕔 El planificador elige a M otra vez. L nunca corre.

   ♾  Volvimos al cuelgue.
```

> Dormir solo garantiza que H suelta la CPU. **No garantiza que L sea el que la reciba.**

La distinción que cierra el tema

```
✖  DORMIR POR TIEMPO     →  "despertame en 50 ms"
                            es una apuesta, no una garantía

✓  DORMIR POR EVENTO     →  "despertame CUANDO SE LIBERE EL CANDADO"
                            lo despierta el que libera, en el instante exacto
```

Eso segundo es un **semáforo** / **mutex**, y es el tema 7. H no adivina: se encola esperando ese candado puntual, y el que sale lo despierta.

Y para el problema de prioridades propiamente dicho, la solución se llama **herencia de prioridades**: mientras L tenga un candado que H necesita, **L hereda temporalmente la prioridad de H** para que pueda terminar rápido y soltarlo. (Es lo que activaron en el Mars Pathfinder; esto es complemento, no está en el Tanenbaum.)

Tema 7.1 — Dormir y despertar (sleep y wakeup)

Este es el tema que resuelve todo lo que vimos en el 6. Antes de meternos, el mapa:

```
7.1  dormir y despertar (sleep/wakeup)     ← arrancamos acá
     └─ el problema del productor-consumidor
     └─ la señal de despertar perdida  💀

7.2  SEMÁFOROS  (down / up)
     └─ productor-consumidor resuelto con 3 semáforos

7.3  MUTEX  (el semáforo binario)
     └─ mutex en espacio de usuario, para hilos

7.4  MONITORES
     └─ variables de condición, wait / signal
```

La cadena es: cada uno **arregla el problema que dejó abierto el anterior**. Igual que en el tema 6.

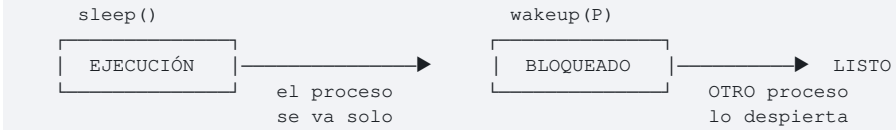
La idea

En el tema 6, el que no podía entrar **giraba**. Ahora queremos que **se duerma**. Para eso el SO ofrece dos llamadas al sistema:

```
sleep()            →  "ponéme en BLOQUEADO hasta que alguien me despierte"
                      el proceso suelta la CPU

wakeup(proceso)    →  "pasá a ese proceso de BLOQUEADO a LISTO"
```

Son **llamadas al sistema**, no instrucciones. Es decir: hay un TRAP, el proceso cambia a modo kernel, y el SO le toca el estado en el BCP.

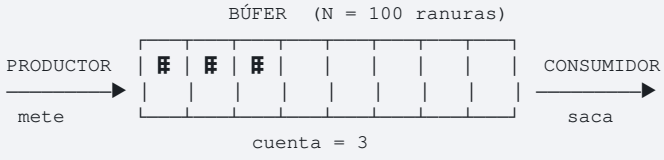


Diferencia clave con lo del tema 6: el que duerme **sale de la cola de listos**. Ya no se lo puede elegir. Esa es toda la ganancia.

(Tanenbaum, cap. 2.3.4, pág. 126)

El problema donde se usa: productor-consumidor

Es **el** problema clásico de la materia. Dos procesos y un búfer de tamaño fijo:



Las dos reglas obvias:

```
El PRODUCTOR,  si el búfer está LLENO  (cuenta == N)  →  se duerme
El CONSUMIDOR, si el búfer está VACÍO  (cuenta == 0)  →  se duerme

Y cada uno despierta al otro cuando la situación cambia.
```

La panadería

Para que se te quede:

```
     PANADERO   (productor)  →  pone panes en la vitrina
   🧺 VENDEDOR   (consumidor)  →  los saca y los vende
  🪟 VITRINA de 100 bandejas  →  el búfer

  Vitrina LLENA   →  el panadero se va a dormir a la trastienda
  Vitrina VACÍA   →  el vendedor se va a dormir

  Y cuando el panadero pone el primer pan en una vitrina vacía,
  grita: "¡VENDEDOR, DESPERTATE, HAY PAN!"
```

Suena perfecto. Y está roto.

El código (así lo da el libro)

```
#define N 100
int cuenta = 0;              /* cuántos elementos hay en el búfer */

void productor(void) {
    while (TRUE) {
        elemento = producir_elemento();
        if (cuenta == N) sleep();          /* lleno → me duermo */
        insertar_elemento(elemento);
        cuenta = cuenta + 1;
        if (cuenta == 1) wakeup(consumidor);   /* estaba vacío → lo despierto */
    }
}

void consumidor(void) {
    while (TRUE) {
        if (cuenta == 0) sleep();          /* vacío → me duermo */
        elemento = quitar_elemento();
        cuenta = cuenta - 1;
        if (cuenta == N-1) wakeup(productor);  /* estaba lleno → lo despierto */
        consumir_elemento(elemento);
    }
}
```

Dos cosas para que leas esto tranquilo:

- `cuenta` es **una variable compartida** entre los dos procesos. Guardá eso.
- `if (cuenta == 1) wakeup(consumidor)` se lee: *"si después de meter el mío hay exactamente 1, quiere decir que antes había 0, o sea que el consumidor debe estar dormido → lo despierto."*

💀 Dónde se rompe: la señal de despertar perdida

Fijate que `if (cuenta == 0) sleep();` **son dos pasos**: leer `cuenta` y después dormirse. Otra vez el hueco.

```
  cuenta = 0    (búfer vacío)

🕐 CONSUMIDOR:  lee cuenta  →  ve 0  →  "me voy a dormir"
                el vendedor empieza a caminar hacia la trastienda

   💥 ACÁ SE LO INTERRUMPE. Todavía NO está dormido.
      Está en el pasillo, en estado LISTO.

🕑 PRODUCTOR:   inserta un pan
                cuenta = 1
                ¿cuenta == 1?  SÍ
                →  wakeup(consumidor)
                   el panadero grita: "¡DESPERTATE, HAY PAN!"

   ⚠  PERO EL CONSUMIDOR NO ESTÁ DORMIDO. Está en el pasillo.
      El grito no lo despierta (ya está despierto)
      y NO SE GUARDA EN NINGÚN LADO.
```

```
        ☞  LA SEÑAL SE PERDIÓ.

🕒  CONSUMIDOR:  retoma donde quedó. Su próxima instrucción es sleep().
                 Se acuesta. 💤
                 (nunca volvió a mirar cuenta: ya la había leído, valía 0)

🕒  PRODUCTOR:   sigue metiendo panes... cuenta = 2, 3, ... 100
                 ¿cuenta == N?  SÍ  →  sleep()  💤

    ♾   LOS DOS DORMIDOS PARA SIEMPRE.
        La vitrina llena de pan y nadie que lo venda.
        Y nadie que los despierte, porque los únicos que podían
        despertarse entre sí están los dos dormidos.
```

La esencia del problema, textual del libro: *"una señal que se envía para despertar a un proceso que no está dormido (todavía) se pierde."*

Las dos causas, separadas

```
CAUSA 1:  cuenta es una variable compartida SIN PROTECCIÓN
          → condición de carrera (igual que el spooler)

CAUSA 2:  wakeup NO TIENE MEMORIA
          → si el destinatario está despierto, la señal se tira a la basura
```

(Tanenbaum, cap. 2.3.4, págs. 126–128, figura 2-27)

El parche que casi funciona: la alcancía

El libro propone un arreglo rápido: agregarle a cada proceso un **bit de espera de despertar**.

```
wakeup a un proceso DESPIERTO  →  en vez de perderse, PRENDE EL BIT

Después, cuando ese proceso llama a sleep():
     ¿bit prendido?  →  lo apaga y SIGUE DESPIERTO (no se duerme)
     ¿bit apagado?   →  se duerme normalmente
```

En nuestra película:

```
🕒  El panadero grita, el vendedor está en el pasillo
    →  el grito queda ANOTADO en un papelito 📝 (el bit)

🕒  El vendedor llega a la cama, pero antes mira el papelito:
    →  "hay un grito pendiente" → lo rompe y NO SE ACUESTA
    →  vuelve a la vitrina, encuentra el pan, lo vende ✓

    ✓  ARREGLADO
```

El libro lo llama **"una alcancía para almacenar señales de despertar"**. Guardá esa palabra, porque es exactamente lo que es un semáforo.

¿Y por qué no alcanza?

Con **un bit** salvás el caso de dos procesos. Pero:

```
¿3 procesos?   → te hacen falta más bits
¿10 procesos?  → más bits todavía
¿llegan 5 señales de despertar seguidas?
               → 1 bit solo recuerda "hubo alguna", no CUÁNTAS
```

Un bit guarda *"pasó algo"*. No guarda *"pasaron 5 cosas"*.

```
┌──────────────────────────────────────────────────────┐
```

```
┌──────────────────────────────────────────────────────────┐
│  ¿Y si en lugar de un BIT ponemos un CONTADOR?           │
│                                                          │
│  Una alcancía que cuente cuántas señales de despertar    │
│  hay guardadas.                                          │
│                                                          │
│           ☞  ESO ES UN SEMÁFORO                          │
└──────────────────────────────────────────────────────────┘
```

Ese es el tema 7.2, y ahí te armo la visualización del productor-consumidor con la cola de bloqueados.

Preguntas de control

1) En la película del wakeup perdido, el consumidor se durmió **después** de que el productor ya había metido un pan en el búfer. O sea: se durmió con `cuenta = 1`, con comida disponible.

¿Por qué no se dio cuenta? ¿Qué fue lo que no volvió a hacer?

2) `sleep()` y `wakeup()` son **llamadas al sistema**, no instrucciones de máquina como `TSL`.

¿Por qué tienen que ser llamadas al sistema? ¿Qué es lo que necesitan hacer que un programa en modo usuario no puede hacer por su cuenta?

Una perfecta y una a medias. Vamos.

1) ✓ Correcta, y muy bien explicada

> *"ya había leído la variable cuenta, y se quedó con ese valor... no volvió a leer ese valor de cuenta"*

Exacto. El consumidor **tomó la decisión con un dato viejo**:

```
🕐  lee cuenta  →  0        ← la foto que se guardó
    💥 interrupción
🕑  el productor mete un elemento  →  cuenta = 1     ← el mundo cambió
🕒  el consumidor retoma. Su siguiente instrucción es sleep().
    NO vuelve a mirar cuenta. Se duerme por una foto vencida. 💤
```

Y fijate que es **el mismo hueco del tema 6**, movido de lugar:

```
tema 6:   leer candado   ...  [HUECO]  ...  escribir candado
tema 7:   leer cuenta    ...  [HUECO]  ...  sleep()
```

Siempre la misma enfermedad: **consultar una condición y actuar sobre ella en dos pasos separados.**

---

2) ⚠ Parcial — la mitad está bien, y hay un error concreto

Lo que está bien

> *"tienen que ser llamadas al sistema porque el kernel tiene que cambiar el estado"*

✓ Correcto. Ese es el motivo de fondo.

El error: no es un bit del PSW

```
✗  "cambian el bit del PSW en el BCP"
✓  "cambian el CAMPO ESTADO del BCP"
```

Estás fusionando **dos cosas distintas** de la Unidad 1. Separémoslas:

	Qué es	Qué indica

Bit de modo del PSW	un bit de un **registro de la CPU**	si la **CPU** está en modo kernel o modo usuario
Campo estado del BCP	un campo de una **estructura en memoria del kernel**	si el **proceso** está en ejecución, listo o bloqueado

```
PSW  →  habla de la CPU.      "¿puedo ejecutar instrucciones privilegiadas?"
BCP  →  habla del proceso.    "¿está corriendo, esperando turno, o dormido?"
```

Son dos preguntas diferentes. El PSW **no sabe** si un proceso está bloqueado.

> **Ojo con la trampa:** el PSW **sí se guarda dentro del BCP** cuando hay cambio de contexto (es uno de los registros que se archivan). Pero está ahí como **foto de un registro**, no como el estado del proceso. Son dos campos distintos del BCP.

Y hay algo cierto escondido en tu respuesta

Mirá, el bit del PSW **sí cambia** durante un `sleep()`. Pero por otra razón:

```
el proceso ejecuta sleep()
     ↓
TRAP  →  cambia el bit de modo del PSW:  usuario → KERNEL   ← este sí cambia
     ↓
ahora en modo kernel, el SO puede escribir estructuras protegidas
     ↓
el SO cambia el CAMPO ESTADO del BCP:  ejecución → BLOQUEADO  ← este es el estado
```

Dos cambios, en dos lugares, por dos motivos. El del PSW es el *permiso* para hacer el trabajo; el del BCP es el *trabajo*.

Lo que faltaba en tu respuesta

`sleep()` no es solo cambiar un campo. Hace **tres cosas**, y todas necesitan modo kernel:

```
1.  campo estado del BCP:  EJECUCIÓN  →  BLOQUEADO
2.  SACA el BCP de la cola de listos  y  lo mete en la cola de bloqueados
3.  LLAMA AL PLANIFICADOR para que elija otro proceso  →  cambio de contexto
```

Y ahí está la respuesta completa a "por qué no puede hacerlo solo":

```
┌──────────────────────────────────────────────────────┐
│  El BCP y las colas del planificador viven en MEMORIA    │
│  DEL KERNEL, protegida por el hardware.                  │
│                                                          │
│  Un proceso en modo usuario no puede escribir ahí,       │
│  ni puede invocar al planificador.                       │
│                                                          │
│  Necesita un TRAP. Por eso es una LLAMADA AL SISTEMA.    │
└──────────────────────────────────────────────────────┘
```

Marcador del 7.1: 1 correcta, 1 parcial. Vamos a semáforos.

Tema 7.2 — Semáforos

De dónde sale la idea

Quedamos en que el bit de espera de despertar era *"una alcancía para guardar señales"*, y que un bit no alcanza porque **recuerda "pasó algo" pero no "pasaron 5 cosas"**.

Dijkstra, en 1965, propuso lo obvio: **que la alcancía sea un contador.**

```
┌──────────────────────────────────────────────────┐
│  Un SEMÁFORO es una variable entera que cuenta   │
│  cuántas señales de despertar hay guardadas.     │
│                                                  │
│     0   →  no hay ninguna guardada               │
│     3   →  hay 3 guardadas                       │
└──────────────────────────────────────────────────┘
```

La lata de fichas

Pensá un semáforo como **una lata con fichas**:

```
        ╔═══════════╗
        ‖  ●  ●  ●  ‖     ← 3 fichas:  el semáforo vale 3
        ‖           ‖
        ╚═══════════╝
```

Y tiene **dos operaciones**, nada más:

```
down(s)   →  "SACO una ficha"
             ├─ ¿hay fichas?  →  saco una, sigo de largo
             └─ ¿está vacía?  →  ME DUERMO al lado de la lata 💤

up(s)     →  "PONGO una ficha"
             ├─ ¿hay alguien dormido al lado?  →  lo DESPIERTO
             │                                    y se lleva la ficha
             └─ ¿no hay nadie?  →  la ficha QUEDA GUARDADA en la lata
```

Ahí está la diferencia con `wakeup`. Si hacés `up` y nadie está esperando, **la señal no se pierde: queda en la lata**. Por eso los semáforos resuelven el problema del wakeup perdido.

```
wakeup   a nadie  →  SE PIERDE 💀
up(s)    a nadie  →  SE GUARDA ✓
```

Lo formal, para el examen

```
down(s):
     si  s > 0   →  s = s - 1   y continúa
     si  s == 0  →  el proceso se BLOQUEA sin completar el down

up(s):
     s = s + 1
     si había procesos bloqueados en s  →  el SO elige UNO
                                           y lo deja completar su down
```

Las dos operaciones son atómicas (indivisibles). Esto es obligatorio, y es lo mismo que pedimos con TSL: comprobar el valor, modificarlo y dormirse tienen que ser **un solo acto**.

¿Y cómo se logra que sean atómicas?

```
Son LLAMADAS AL SISTEMA.
El SO deshabilita brevemente las interrupciones mientras
evalúa el semáforo, lo actualiza y duerme al proceso si hace falta.

En un MULTIPROCESADOR eso no alcanza (lo vimos en el tema 6):
cada semáforo se protege además con TSL o XCHG.
```

Pregunta clásica de final: *"¿No es contradictorio usar TSL (espera ocupada) para implementar semáforos, si los semáforos existen para evitar la espera ocupada?"*
No, y el libro lo aclara explícitamente: el TSL protege **la operación del semáforo**, que dura **unos microsegundos**. La espera ocupada del tema 6 protegía **la región crítica entera**, que puede durar **lo que sea**. No es lo mismo girar 2 microsegundos que girar media hora.

(Tanenbaum, cap. 2.3.5, págs. 128–129)

Productor-consumidor resuelto: tres semáforos

Acá está la solución, y es **de las cosas que más se preguntan**:

```
#define N 100

semaforo mutex  = 1;     /* permiso para tocar el búfer */
semaforo vacias = N;     /* cuántas ranuras LIBRES hay */
semaforo llenas = 0;     /* cuántas ranuras OCUPADAS hay */

void productor(void) {
    while (TRUE) {
        elemento = producir_elemento();
        down(&vacias);          /* ¿hay lugar? si no, me duermo */
        down(&mutex);           /* entro a la región crítica */
        insertar_elemento(elemento);
        up(&mutex);             /* salgo de la región crítica */
        up(&llenas);            /* aviso: hay una ranura más ocupada */
    }
}

void consumidor(void) {
    while (TRUE) {
        down(&llenas);          /* ¿hay mercadería? si no, me duermo */
        down(&mutex);           /* entro a la región crítica */
        elemento = quitar_elemento();
        up(&mutex);             /* salgo de la región crítica */
        up(&vacias);            /* aviso: hay una ranura más libre */
        consumir_elemento(elemento);
    }
}
```

Las tres latas

```
┌──────────┐   vacias = N       "tickets de LUGAR DISPONIBLE"
│ ●●●●●●●●● │                        el productor gasta uno al meter
└──────────┘                     el consumidor devuelve uno al sacar

┌──────────┐   llenas = 0       "tickets de MERCADERÍA LISTA"
│          │                     el productor pone uno al meter
└──────────┘                     el consumidor gasta uno al sacar

┌──────────┐   mutex = 1        "LA llave del búfer"
│ ●        │                     una sola: entra uno a la vez
└──────────┘
```

Fijate la simetría:

```
PRODUCTOR:    gasta  vacias   →   produce  llenas
CONSUMIDOR:   gasta  llenas   →   produce  vacias

vacias + llenas = N    siempre     ← invariante lindo para recordar
```

☆ Los dos usos del semáforo — esto se toma

El libro lo recalca explícitamente, porque son **dos cosas distintas** usando el mismo mecanismo:

```
mutex            →  EXCLUSIÓN MUTUA
                    "que no toquen el búfer dos a la vez"
                    empieza en 1,  semáforo BINARIO

vacias, llenas   →  SINCRONIZACIÓN
                    "que el productor pare si está lleno,
                     que el consumidor pare si está vacío"
                    son CONTADORES, empiezan en N y en 0
```

Si te preguntan *"¿para qué sirven los semáforos?"*, la respuesta completa son **esos dos usos**, no solo la exclusión mutua.

Y de ahí sale la definición de **semáforo binario**: *un semáforo inicializado en 1, usado por dos o más procesos para garantizar que solo uno entre a su región crítica.* Si cada proceso hace `down` antes de entrar y `up` al salir, **tenés exclusión mutua garantizada**.

(Tanenbaum, cap. 2.3.5, págs. 129–130, figura 2-28)

💀 El orden de los `down` NO es decorativo

Esta es **la pregunta trampa del tema**. ¿Qué pasa si el productor invierte sus dos primeras líneas?

```
    down(&mutex);      /* ← primero agarra la llave */
    down(&vacias);     /* ← después pregunta si hay lugar */
```

```
🕐  El búfer está LLENO.  vacias = 0,  llenas = N,  mutex = 1

🕑  PRODUCTOR:  down(&mutex)   →  mutex = 0.  Se guarda LA LLAVE 🔑
🕒  PRODUCTOR:  down(&vacias)  →  vacias está en 0
                               →  SE DUERME 💤  ... CON LA LLAVE ENCIMA

🕓  CONSUMIDOR: down(&llenas)  →  hay mercadería, pasa ✓
🕔  CONSUMIDOR: down(&mutex)   →  mutex = 0, la llave la tiene el productor
                               →  SE DUERME 💤

    ♾   INTERBLOQUEO. Los dos dormidos.
        El productor espera lugar, que solo el consumidor puede crear.
        El consumidor espera la llave, que solo el productor puede devolver.
```

☞ Y esto es exactamente lo que sufriste ayer

```
Inversión de prioridades:   L se queda SIN LA CPU   con el candado agarrado
Este interbloqueo:          el productor SE DUERME  con el mutex agarrado

Mismo pecado:  QUEDAR INACTIVO SIN SOLTAR EL RECURSO.
```

La regla que sale de esto:

```
┌──────────────────────────────────────────────────────────┐
│  Primero el down de SINCRONIZACIÓN (vacias / llenas)     │
│  Después el down de EXCLUSIÓN MUTUA (mutex)              │
│                                                          │
│  Nunca te duermas esperando algo con el mutex tomado.    │
└──────────────────────────────────────────────────────────┘
```

Ahora sí te armo la visualización, que para este tema es la que de verdad vale la pena. Dame un momento.

Las dos preguntas del 7.2

1) En el modo correcto, el productor hace `down(&vacias)` y se bloquea porque el búfer está lleno. En ese momento el `mutex` vale **1**, o sea que está libre.

¿Por qué es imprescindible que el `mutex` haya quedado libre? ¿Qué pasaría si se hubiera bloqueado teniéndolo?

2) Un compañero dice: *"los semáforos `vacias` y `llenas` están al revés. Si `vacias` cuenta las ranuras libres, el productor debería hacerle `up` cuando mete algo, porque está usando una ranura."*

¿Dónde está el error de razonamiento? ¿Qué significa realmente un `up` sobre un semáforo?

Corrección

Pregunta 1 — correcta, con una palabra que hay que sacar.

El razonamiento está completo: el productor se duerme esperando una ranura libre, pero se lleva el `mutex` puesto. El consumidor es el único que puede liberar una ranura, y para hacerlo necesita entrar a la región crítica… que está cerrada con la llave que tiene el productor dormido. Cada uno espera algo que solo el otro puede dar.

Lo que hay que corregir: **"ocupando tiempo de CPU" no.** Están *bloqueados*, no girando. Un proceso bloqueado está fuera de la cola de listos: el planificador no lo elige, no consume ni un ciclo.

```
ESPERA OCUPADA       →  no avanza  Y  gasta CPU
INTERBLOQUEO         →  no avanza  pero NO gasta CPU
```

Esa distinción se toma. El interbloqueo es peor que la espera ocupada en un sentido (nunca se sale solo) y mejor en otro (no roba CPU). Si en el examen escribís "interbloqueo consumiendo CPU", te lo marcan.

Y la regla general, que es la misma que viste en inversión de prioridades:

> **Nunca te quedes inactivo —ni girando ni dormido— teniendo tomado un recurso que otro necesita para desbloquearte.**

Pregunta 2 — bien el mecanismo, falta el concepto.

Dijiste bien qué hace el productor: `down(&vacias)` y `up(&llenas)`. Pero la pregunta era *dónde está el error de razonamiento del compañero*, y ahí está lo que se toma.

El compañero piensa que `up` y `down` significan **sumar y restar**. Y entonces razona: "está usando una ranura, hay un cambio en `vacias`, algo se mueve". No. Mirá qué significan de verdad:

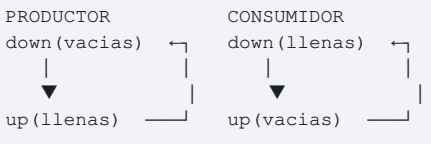
Operación	Qué significa	¿Puede bloquear?
`down(&s)`	**pido** una unidad del recurso que `s` cuenta	**Sí**
`up(&s)`	**entrego** una unidad del recurso que `s` cuenta	**Nunca**

Con eso solo, el error del compañero se cae sin hacer cuentas: si el productor hiciera `up(&vacias)`, estaría diciendo *"acabo de crear una ranura libre"*. Y es al revés: la está gastando.

Y hay un segundo argumento, todavía más fuerte, que no usa el significado sino la mecánica:

> Un `up` **nunca bloquea**. Si el productor pidiera lugar con `up(&vacias)`, pasaría de largo aunque el búfer estuviera lleno y escribiría encima de un elemento sin consumir. **No hay forma de esperar con un `up`.**

Entonces cada proceso **consume de un semáforo y produce en el otro**:



Cruzado. Lo que uno toma, el otro lo repone. Por eso se mantiene `vacias + llenas = N`.

Nota: `vacias` y `llenas` cuentan **recursos**, no eventos. `vacias` = "cuántos lugares para escribir tengo disponibles". `llenas` = "cuántos datos listos para leer hay". Si te acordás de esa frase, nunca más confundís el sentido.

Tema 7.3 — Mutexes

La intuición

Fijate en el productor-consumidor: de los tres semáforos, **el `mutex` nunca pasa de 1**. Vale 1 o 0, nada más. No está contando nada: está diciendo *libre* u *ocupado*.

Y si un semáforo solo va a tomar dos valores, el contador sobra. Para eso está el **mutex**: un semáforo al que le sacaron la capacidad de contar.

> **Mutex = variable compartida en dos estados: *abierto* (0) o *cerrado* (1).**

Alcanza **un solo bit**, aunque en la práctica se use un entero (0 = abierto, cualquier otro valor = cerrado).

Las dos operaciones

```
mutex_lock(&m)     → si está abierto, lo cierra y entra.
                     Si está cerrado, el hilo se BLOQUEA.

mutex_unlock(&m)   → lo abre. Si hay hilos esperando,
                     uno de ellos entra.
```

No sirve para sincronizar (no puede contar señales guardadas). **Sirve solo para exclusión mutua.** Un mutex es la herramienta para proteger una región crítica, y nada más.

Lo que de verdad importa: cómo se implementa

Acá está la parte que se toma, porque junta todo lo del tema 6. Mirá el código del Tanenbaum:

```
mutex_lock:
        TSL REGISTRO, MUTEX        | copia mutex al registro y lo pone en 1
        CMP REGISTRO, #0           | ¿era cero?
        JZE ok                     | si era cero, estaba abierto → entra
        CALL thread_yield          | está cerrado → cedé la CPU
        JMP mutex_lock             | cuando volvés, intentá de nuevo
ok:     RET                        | tenés el mutex

mutex_unlock:
        MOVE MUTEX, #0             | abrilo
        RET
```

Comparalo línea por línea con el `enter_region` con TSL del tema 6. Son **idénticos** hasta la línea 3. La única diferencia:

	Si el candado está ocupado
`enter_region` (TSL puro)	`JNE enter_region` → **gira** quemando CPU
`mutex_lock`	`CALL thread_yield` → **cede la CPU** y después reintenta

Una sola instrucción cambia, y con eso se cae el problema de la inversión de prioridades. Porque `thread_yield` le entrega la CPU a otro hilo — y entre esos otros está el que tiene el candado, que ahora puede terminar y soltarlo.

Por qué los mutexes son baratos

Esto es clave y es la razón por la que existen:

> **`thread_yield` es una llamada al planificador de hilos en el espacio de usuario. No es una llamada al sistema. No hay TRAP, no hay cambio a modo kernel.**

Por eso un mutex de usuario es **extremadamente rápido** y por eso los paquetes de hilos de usuario los usan en vez de semáforos. Un semáforo necesita al kernel (hay que mover el BCP entre colas); un mutex de usuario, en el caso en que el candado está libre, son tres instrucciones de máquina y listo.

```
EL CASO FELIZ (candado libre):
mutex_lock  →  TSL, CMP, RET      ← 3 instrucciones, cero kernel

EL CASO CON CONFLICTO:
mutex_lock  →  TSL, CMP, yield    ← planificador de usuario, igual sin kernel
```

Un primo que conviene conocer: `mutex_trylock`

Igual a `mutex_lock`, pero **si el candado está ocupado no bloquea: devuelve un código de falla.** El hilo decide qué hacer: hacer otra cosa útil y volver más tarde, o avisar un error. Es la opción "no me cuelgues, decime si puedo o no".

Pthreads: el estándar real

En Pthreads (el estándar POSIX de hilos) tenés:

Llamada	Qué hace
`pthread_mutex_init`	crea el mutex
`pthread_mutex_destroy`	lo destruye
`pthread_mutex_lock`	adquiere, bloqueándose si hace falta
`pthread_mutex_trylock`	intenta adquirir, no bloquea
`pthread_mutex_unlock`	libera

Pero Pthreads reconoce que el mutex **no alcanza**, porque no sabe sincronizar. Entonces agrega las **variables de condición**:

Llamada	Qué hace
`pthread_cond_wait`	se bloquea esperando una señal
`pthread_cond_signal`	despierta a **uno** de los que esperan
`pthread_cond_broadcast`	despierta a **todos** los que esperan

Y acá viene la trampa que se toma:

> ⚠ **Las variables de condición, a diferencia de los semáforos, NO tienen memoria.** Si mandás un `signal` a una variable de condición en la que no hay nadie esperando, **la señal se pierde**.

Reconocés el problema, ¿no? Es exactamente el `wakeup` perdido del tema anterior. Por eso una variable de condición **siempre se usa junto a un mutex**: se toma el mutex, se chequea la condición con la variable protegida, y `cond_wait` **suelta el mutex atómicamente al dormirse** (resolviendo justo el interbloqueo que vimos en la pregunta 1).

```
SEMÁFORO             ← tiene contador → RECUERDA las señales
VARIABLE DE CONDICIÓN ← no tiene nada  → la señal sin oyente SE PIERDE
```

Mutex vs. semáforo binario

Un semáforo binario (uno que solo vale 0 o 1) hace lo mismo que un mutex. Tanenbaum los trata casi como sinónimos: el mutex es "una versión simplificada del semáforo".

Complemento elaborado (no está en el Tanenbaum, pero se pregunta): en la mayoría de las implementaciones reales el mutex tiene **dueño** — solo el hilo que lo cerró puede abrirlo — mientras que un semáforo binario lo puede subir cualquiera. Esa propiedad es la que permite la **herencia de prioridades**: el sistema sabe a quién hay que subirle la prioridad porque sabe quién tiene el candado.

(Tanenbaum, cap. 2.3.6, págs. 130–134)

Preguntas de control

1) `mutex_lock` tiene un `JMP mutex_lock`: es un bucle. Un compañero dice: *"entonces un mutex también hace espera ocupada, solo que más lenta"*.

¿Tiene razón? ¿Qué diferencia esencial hay entre girar en `enter_region` y girar en `mutex_lock`?

2) Tenés un búfer compartido entre un productor y un consumidor, y alguien te dice: *"usá dos mutexes en lugar de los semáforos `vacias` y `llenas`, que es más barato"*.

¿Por qué no se puede? ¿Qué le falta al mutex que el semáforo sí tiene?

Verificado en el libro, así que te puedo contestar la segunda parte con la cita.

Pero primero hay que separar algo, porque en tu pregunta hay **dos atomicidades distintas** pegadas:

Qué	Qué par de cosas tiene que ser atómico	¿Lo garantiza qué?
`mutex_lock`	**leer** el candado + **escribirlo**	**TSL** (hardware)
`pthread_cond_wait`	**soltar** el mutex + **dormirse**	el **kernel** (llamada al sistema)

Vos las mezclaste: dijiste "lee el valor del mutex, lo toma, pero no alcanza a dormirse". Eso junta la primera mitad de una con la segunda mitad de la otra. Vamos de a una, porque la respuesta es distinta.

Caso A — la interrupción cae durante `mutex_lock`

Tu pregunta textual: *"¿entra el otro sin el mutex?"*

No. Y la razón es la clave de todo el tema 6. Mirá los tres momentos posibles:

```
mutex_lock:
                        ← 🕐 interrupción ACÁ
        TSL REGISTRO, MUTEX
                        ← 🕑 interrupción ACÁ
        CMP REGISTRO, #0
        JZE ok
                        ← 🕒 interrupción ACÁ
        CALL thread_yield
        JMP mutex_lock
ok:     RET
```

🕐 **Antes del TSL.** No tocaste nada. El candado está como estaba. Otro puede tomarlo tranquilamente. Cuando vos volvés, ejecutás el TSL, encontrás 1, y cedés. Perfecto.

🕑 **Justo después del TSL, y ganaste** (el registro tiene 0). Acá está lo importante:

> **El candado ya está en 1 en la memoria.** La interrupción de reloj no lo deshace.

Vos todavía no sabés que ganaste —no ejecutaste el `CMP`—, pero **la puerta ya está cerrada para todos los demás**. Cualquier otro hilo que corra va a hacer su TSL, va a encontrar 1 y va a ceder. Y cuando a vos te devuelvan la CPU, ejecutás `CMP`, `JZE`, y entrás.

🕒 **Justo después del TSL, y perdiste.** Escribiste un 1 sobre un 1. Inofensivo — ésa era la "rareza" de TSL que

ya vimos. La interrupción no cambia nada.

La idea que vale la pena guardar

> **Tomar el candado no es un trámite con pasos. Termina en el instante exacto en que termina el TSL.**

No existe "a medio tomar". Por eso el `mutex_lock` es a prueba de interrupciones: lo que te protege **no es llegar a dormirte**, es que la escritura ya ocurrió.

Y fijate que el momento 🕑 es exactamente la situación del tema 6 que te costó: **un hilo que tiene el candado pero no tiene la CPU.** Es normal, es legal, y es lo que pasa todo el tiempo.

Caso B — la interrupción cae entre soltar y dormirse

Acá sí tenés razón: éste es un problema real. Y de hecho es el problema que ya conocés.

Supongamos que `cond_wait` estuviera implementado como dos operaciones separadas:

```
mutex_unlock(&m);      ← suelto el mutex
                       ← 🕑 INTERRUPCIÓN DE RELOJ
cond_sleep(&c);        ← me duermo
```

Mirá la película:

```
🕑  El hilo A suelta el mutex. Todavía no se durmió. Lo interrumpen.

🕑  El hilo B toma el mutex, cambia la condición
    (mete un dato, lo que sea) y hace cond_signal(&c).

    ☞ ¿Quién está dormido en esa variable de condición?
      NADIE. A no llegó a dormirse.

    ☞ Y las variables de condición NO TIENEN MEMORIA.
      LA SEÑAL SE PIERDE.

🕑  A retoma donde estaba: ejecuta cond_sleep(&c).
    Se duerme esperando una señal que ya pasó y nadie guardó.

    A duerme para siempre, con el dato esperándolo.
```

¿Lo reconocés? Es **el wakeup perdido del 7.1, letra por letra.** La misma estructura: una decisión tomada en dos pasos, con un hueco en el medio donde el mundo cambió.

Por eso `pthread_cond_wait` tiene que ser atómico. Y lo es porque es **una sola llamada al sistema**: el kernel suelta el mutex y encola al hilo en la variable de condición sin que nadie pueda ver el estado intermedio. Nadie puede meterse ahí porque, mientras lo hace, el planificador está fuera de juego.

> Esto es lo que te justifica por qué la variable de condición **no puede usarse sola**: necesita el mutex al lado, y necesita que soltarlo sea parte de la misma operación que dormirse.

Tu segunda pregunta — y acá la pegaste

> *"o como justo esto es en el espacio usuario no están los quantum"*

✓ **Esto es correcto, y es una observación muy fina.** El libro lo dice explícitamente:

> *"si un hilo empieza a ejecutarse, ningún otro hilo en ese proceso se ejecutará a menos que el primero renuncie de manera voluntaria a la CPU. **Dentro de un solo proceso no hay interrupciones de reloj**, lo*

cual hace que sea imposible planificar [los hilos] en el formato round robin."
(Tanenbaum, cap. 2.2.4, pág. 109)

Y más adelante, en planificación de hilos:

> *"el kernel… selecciona un proceso, por decir A, y otorga a este proceso el control de su quántum. El planificador de hilos dentro de A decide cuál hilo ejecutar, por decir A1.* ***Como no hay interrupciones de reloj para multiprogramar hilos****, este hilo puede continuar ejecutándose todo el tiempo que quiera."*
> *(Tanenbaum, cap. 2.4.6, págs. 161–162)*

Entonces, con hilos **puros de usuario**:

```
El QUANTUM es del PROCESO, no de los hilos.

La interrupción de reloj puede sacarle la CPU a TODO el proceso A
(y dársela a B), pero NUNCA va a conmutar de A1 a A2:
el kernel no sabe que A1 y A2 existen.
```

O sea: **un hilo de usuario no puede ser desalojado en favor de otro hilo de usuario del mismo proceso.** Sale solo si él quiere (`thread_yield`), o si se bloquea. Cuando el proceso vuelve, **A1 retoma exactamente donde estaba**.

Y ojo que esto ya lo vimos: es justo el punto que te corregí en el tema 4. Cuando preguntaste por qué un hilo egoísta no arruina a los otros *procesos*, la razón es que **el kernel sí tiene interrupciones de reloj** — las tiene para los procesos, no para los hilos de adentro.

Pero no te confíes: hay dos agujeros

1) Multiprocesador. Si hay varios núcleos, otro hilo del mismo proceso puede estar corriendo **de verdad al mismo tiempo**, en otra CPU. No hace falta ninguna interrupción para que te pise. Por eso el TSL/XCHG sigue siendo imprescindible:

```
1 CPU, hilos de usuario  →  nadie te interrumpe en favor de un hermano
N CPUs                   →  tu hermano corre EN PARALELO, de verdad
```

2) Pthreads en Linux es 1:1. Los `pthread_mutex_*` que miramos corren, en la práctica, sobre **hilos de kernel**: cada hilo es planificable por separado y **sí tiene su propio quantum**. Así que en el caso real sí te pueden desalojar en cualquier instrucción.

Tu intuición es correcta para el **modelo** de hilos de usuario puro, pero no es lo que te salva en la implementación real.

La regla que hay que llevarse al examen

> **Atomicidad no significa "terminar antes de que llegue la interrupción".** Significa **que nadie pueda observar el estado intermedio.**

Y hay exactamente **dos formas** de conseguirla:

Herramienta	Cómo lo logra	Para qué par de operaciones
TSL / XCHG	bloquea el bus de memoria	leer + escribir el candado
Llamada al sistema	el kernel deshabilita interrupciones un instante	soltar + dormirse; `down`; `up`

Y la pregunta que conviene hacerse siempre es la tuya, literalmente: *"¿qué pasa si me interrumpen justo acá?"*. Si la respuesta es "alguien ve algo a medio hacer", **necesitás atomicidad**. Si la respuesta es "el efecto ya está

escrito en memoria", no hace falta nada.

En el 🕑 del `mutex_lock` el efecto ya está escrito → no hace falta nada. En el hueco del `cond_wait` alguien ve el mutex libre **y** a nadie dormido → hace falta atomicidad.

Volviendo a lo que quedó pendiente: esta pregunta tuya es casi la respuesta de la **pregunta 1 del 7.3**, así que te la repito ahora que tenés más herramientas.

`mutex_lock` tiene un `JMP mutex_lock`, o sea un bucle. Un compañero dice: *"entonces un mutex también hace espera ocupada, solo que más lenta"*.

¿Tiene razón? Y si no, ¿cuál es la diferencia esencial entre girar en `enter_region` y girar en `mutex_lock`?

Pregunta 1 — ✓ correcta

Diste la diferencia esencial: **`enter_region` gira sin soltar la CPU; `mutex_lock` cede la CPU en cada vuelta.** Eso es lo que se pregunta y lo tenés.

Tres cosas para afinar, porque son los detalles que se pueden tomar.

1️⃣ El bucle sigue existiendo — tu compañero no está del todo equivocado

Mirá el código: el `JMP mutex_lock` está ahí. **El mutex sí gira.** Lo que desaparece no es el bucle, es **el daño** del bucle.

```
enter_region   →  gira SIN ceder   →  monopoliza la CPU
mutex_lock     →  gira CEDIENDO    →  no monopoliza nada
```

Lo que hace dañina a la espera ocupada no es dar vueltas: es **dar vueltas quedándose con la CPU**. Si cedés en cada vuelta, el bucle te cuesta algunas instrucciones y nada más.

Entonces la respuesta fina en un examen es: *"el bucle existe, pero no es espera ocupada en el sentido problemático, porque el hilo no retiene la CPU mientras espera"*.

Y el remate, que es la frase que conviene memorizar:

> **Girar sin ceder bloquea justo al que te tiene que desbloquear.**

Por eso con `enter_region` aparece la **inversión de prioridades** y con `mutex_lock` no. Es el mismo diagnóstico de siempre: *quedarse inactivo sin soltar el recurso que el otro necesita* — y acá el recurso retenido es la CPU.

2️⃣ `thread_yield` no es una llamada al sistema

Escribiste "ejecutar una llamada `thread_yield`". Ojo con cómo lo decís en el examen, porque si suena a *llamada al sistema* te comés el punto. Es:

> una llamada al **planificador de hilos en el espacio de usuario**. **Sin TRAP, sin modo kernel.**

Y ésa es la razón de ser del mutex. Compará:

	Costo
`down` de un semáforo	llamada al sistema → TRAP → modo kernel → tocar el BCP y las colas
`mutex_lock` sin conflicto	`TSL`, `CMP`, `RET` → **tres instrucciones de máquina**
`mutex_lock` con conflicto	+ `thread_yield` → planificador de usuario, **sigue sin kernel**

Por eso Tanenbaum dice que los mutexes de usuario son *económicos* y que los paquetes de hilos de usuario los prefieren. **Si `thread_yield` fuera una llamada al sistema, el mutex perdería toda su ventaja sobre el semáforo.**

3️⃣ Un detalle de precisión: "para que el proceso anterior termine"

Dos correcciones chicas de vocabulario:

a) Son **hilos**, no procesos. Los mutexes de usuario viven dentro de un proceso, entre sus hilos. El planificador de usuario solo puede repartir entre hermanos.

b) `thread_yield` **no elige** al que tiene el candado. Le entrega la CPU al **planificador de hilos**, que elige alguno de los listos — y podría elegir a un tercero. La diferencia es:

```
✗ "cedo la CPU PARA QUE el que tiene el candado termine"
✓ "cedo la CPU, con lo cual el que tiene el candado PUEDE llegar a ejecutarse"
```

Parece un detalle, pero es exactamente la distinción que viste en el arreglo de *"girar 1000 veces y dormir"*: **ceder la CPU garantiza que la sueltes, no garantiza quién la recibe.** La diferencia es que acá, tarde o temprano, el que tiene el candado la va a recibir y lo va a soltar.

Bonus: los tres niveles de espera

Para que quede ordenado, porque en la realidad hay tres comportamientos distintos:

Mecanismo	Qué hace al no conseguir el candado	Estado del hilo
`enter_region` con TSL	gira sin ceder	**EN EJECUCIÓN** (quemando quantum)
`mutex_lock` de usuario	gira cediendo	**LISTO** (vuelve a intentar)
mutex con bloqueo real	no gira, se encola	**BLOQUEADO**

El tercero es el que usa `pthread_mutex_lock` en un sistema con hilos de kernel, y es el que el libro describe cuando dice que el hilo "se bloquea hasta que el otro llame a `mutex_unlock`". *(Complemento elaborado: Linux lo implementa con **futexes** — gira un poquito en espacio de usuario y solo baja al kernel si hay conflicto real. No está en el Tanenbaum 3ª ed.)*

Fijate que la columna de la derecha son tres estados distintos del diagrama del tema 2. **Toda esta unidad es la misma pregunta: cuando no podés avanzar, ¿en qué estado te quedás?**

Te falta la **pregunta 2**, que es la que de verdad separa los dos conceptos:

> Tenés el búfer compartido entre un productor y un consumidor, y alguien te dice: *"usá dos mutexes en lugar de los semáforos `vacias` y `llenas`, que es más barato"*.
> ¿Por qué no se puede? ¿Qué le falta al mutex que el semáforo sí tiene?

Pista, por si la necesitás: pensá qué pasa si el productor mete **5 elementos seguidos** antes de que el consumidor corra una sola vez. ¿Qué tendría que valer `llenas`? ¿Puede un mutex valer eso?

Pregunta 2 — ✓ correcta

El centro lo tenés: **el mutex no cuenta, solo distingue libre de ocupado.** Dos ajustes.

Da vuelta la frase

Dijiste: *"el semáforo además de un mutex tiene el contador"*. Es al revés:

```
✗  semáforo = mutex + contador
```

```
✓  mutex = semáforo al que le sacaron el contador
```

El libro lo dice así: el mutex es *"una versión simplificada del semáforo"*. El semáforo es la primitiva general (contador + cola de bloqueados); el mutex es el **caso particular** en que el contador solo necesita valer 0 o 1. No se construye un semáforo poniéndole un contador a un mutex.

Para qué sirve el contador (esto te faltaba)

No es solo "poder valer más de 1". El contador hace **dos** cosas:

a) Recordar una cantidad. `vacias` tiene que poder valer de 0 a N. Un mutex no tiene dónde guardar "quedan 3 ranuras".

b) Acumular señales. Ésta es la importante, y es la pista que te di. Mirá:

```
El productor mete 5 elementos seguidos, el consumidor no corrió nunca.

CON SEMÁFORO              CON MUTEX
up(llenas) → 1            unlock → abierto
up(llenas) → 2            unlock → abierto
up(llenas) → 3            unlock → abierto
up(llenas) → 4            unlock → abierto
up(llenas) → 5            unlock → abierto
     ↓                         ↓
llenas = 5                 "abierto"
el consumidor hace         el consumidor entra UNA vez
5 down sin bloquearse      y pierde los otros 4 datos
```

Cinco `unlock` dejan el mutex igual que uno solo. No hay memoria de cuántas veces pasó.

¿Y eso qué es? **Es el wakeup perdido.** La misma carencia del bit de espera de despertar del 7.1 (un bit recuerda "pasó algo", no "pasaron 5 cosas") y la misma de las variables de condición. Son todos el mismo defecto:

> **No acumular es perder información.** El semáforo es la única primitiva que la guarda.

El corolario para el examen

	Exclusión mutua	Sincronización
Mutex	✓	✗
Semáforo	✓	✓

Si en el examen te preguntan "¿cuál es la diferencia entre mutex y semáforo?", esa tabla más la frase *"el mutex no cuenta, así que no puede hacer esperar por una cantidad"* es la respuesta completa.

Tema 7.3 cerrado: 2 de 2 correctas. Vas 7 temas y 3/4 de 10.

Tema 7.4a — Monitores

La intuición: ¿y si el programador no tuviera que acordarse?

Pará y mirá todo lo que vimos. Los semáforos funcionan, sí. Pero fijate de qué depende que funcionen:

```
✗ Un down que te olvidás      →  condición de carrera
✗ Un up que te olvidás        →  interbloqueo
✗ Los dos down al revés       →  interbloqueo
```

```
✗ Un down de más              →  bloqueo eterno
```

Todo descansa en que **el programador no se equivoque**, en código que se ejecuta intercalado de formas que nadie puede probar a mano. Y vos ya lo viviste: para ver el interbloqueo de los `down` invertidos hubo que armar un simulador.

> Brinch Hansen (1973) y Hoare (1974) propusieron otra cosa: **que la exclusión mutua no la escriba el programador. Que la ponga el compilador.**

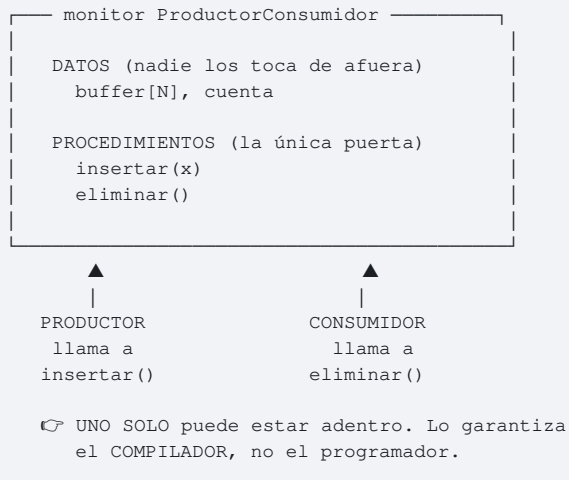
Ésa es toda la idea del monitor.

Qué es, formalmente

> Un **monitor** es una **colección de procedimientos, variables y estructuras de datos** agrupados en un módulo o paquete. Los procesos pueden **llamar a los procedimientos** del monitor cuando quieran, pero **no pueden acceder a sus estructuras de datos internas** desde afuera.

Y la propiedad que lo hace útil, que es la frase que hay que saber textual:

> **"Sólo puede haber un proceso activo en un monitor en cualquier instante."**



☆ Lo que más se toma: es una construcción DEL LENGUAJE

Esto es lo que hay que tener clarísimo, porque es la diferencia de categoría con todo lo anterior:

Primitiva	Qué es	Quién la provee
`TSL`	instrucción de máquina	**el hardware**
`down`/`up`, `sleep`	llamada al sistema	**el sistema operativo**
monitor	**construcción del lenguaje**	**el compilador**

Por eso el libro usa un lenguaje imaginario, **Pidgin Pascal**, para mostrarlo:

> *"Aquí no podemos utilizar C debido a que los monitores son un concepto de lenguaje… C no los conoce."*

¿Cómo lo hace el compilador? Cuando ve una llamada a un procedimiento de monitor, **le inserta código al principio**: *"¿hay alguien adentro? Si hay, suspendete; si no, entrá"*. Normalmente lo implementa con un mutex o un semáforo binario — pero eso es asunto del compilador:

> *"la persona que escribe el monitor no tiene que saber acerca de cómo el compilador hace los arreglos para la exclusión mutua."*

Ése es el aporte del monitor: no agrega potencia, agrega seguridad. No podés hacer nada que no pudieras hacer con semáforos; simplemente ya no te podés equivocar en la exclusión mutua, porque no la escribís vos.

Pero la exclusión mutua no alcanza (otra vez)

Ya lo sabés: te falta **sincronización**. Si el productor entra al monitor y encuentra el búfer lleno, ¿qué hace? No puede quedarse adentro esperando, porque entonces el consumidor nunca podría entrar a vaciarlo. **Sería el interbloqueo del `mutex` puesto, de nuevo.**

La solución son las **variables de condición**, con dos operaciones:

Operación	Qué hace
`wait(c)`	**bloquea** al proceso que llama **y lo saca del monitor**, dejando entrar a otro
`signal(c)`	despierta a **uno** de los que esperan en `c`

La parte crítica de `wait`, que es la que resuelve el problema:

> `wait` no solo te duerme: **te saca del monitor.** Deja la puerta libre para que entre el otro.

¿Te suena? Es exactamente `pthread_cond_wait`, que **suelta el mutex al dormirse**. Es la misma idea: *dormite, pero no te lleves la llave*.

El problema del `signal`: Hoare vs. Brinch Hansen

Acá hay un detalle fino. Si el consumidor hace `signal` y despierta al productor… **quedarían dos procesos activos en el monitor a la vez**, y eso rompe la regla. Hay que decidir quién sigue:

Propuesta	Qué pasa después del `signal`
Hoare	corre el **despertado**; el que señalizó se suspende
Brinch Hansen	el que hace `signal` **debe salir del monitor de inmediato** → `signal` solo puede ser **la última instrucción** del procedimiento
Tercera (ni uno ni otro)	sigue el **señalizador**, y el despertado arranca **después** de que éste salga del monitor

El libro adopta Brinch Hansen, *"debido a que, en concepto, es más simple y también más fácil de implementar"*.

De ahí sale el dato que se toma:

> **Con la regla de Brinch Hansen, `signal` tiene que ser la última instrucción del procedimiento de monitor.**

Y un detalle más: si hay varios esperando en la misma variable, `signal` revive a **uno solo**, *"que determine el planificador del sistema"*. No elegís vos.

⚠ Las variables de condición no son contadores

Esto ya lo tenés del 7.3, pero el libro lo dice acá con todas las letras:

> *"Las variables de condición **no son contadores**; no acumulan señales para su uso posterior como hacen los semáforos. Por ende, si una variable de condición se señaliza y no hay ningún proceso esperándola, **la señal se pierde para siempre**."*

De lo cual sale la regla:

> **`wait` debe ir antes de `signal`.**

Y no es un problema en la práctica, dice el libro, porque es fácil llevar registro del estado con variables: *"un proceso que de lo contrario realizaría una operación `signal` puede ver que esta operación no es necesaria con sólo analizar las variables"*. Ya vas a ver el truco en el código.

☆☆ La pregunta que te vas a hacer: ¿no es sleep/wakeup otra vez?

`wait` y `signal` se parecen peligrosamente a `sleep` y `wakeup`. **¿Por qué no falla igual?** El libro se adelanta y lo contesta — y ésta es *literalmente* la pregunta que me hiciste vos antes sobre la atomicidad:

> *"sleep y wakeup fallaban debido a que, mientras un proceso estaba tratando de dormir, el otro estaba tratando de despertarlo. **Con los monitores, esto no puede ocurrir. La exclusión mutua automática en los procedimientos de monitor garantiza** que si el productor dentro de un procedimiento de monitor descubre que el búfer está lleno, podrá **completar la operación wait sin tener que preocuparse** por la posibilidad de que el planificador pueda conmutar al consumidor justo antes de que se complete la operación wait. **Al consumidor ni siquiera se le va a permitir entrar al monitor** hasta que wait sea terminado y el productor se haya marcado como no ejecutable."*

Leelo despacio, porque es la respuesta exacta a tu pregunta de ayer:

```
   sleep/wakeup:   el hueco entre "decido dormirme" y "estoy dormido"
                   está ABIERTO. El otro puede entrar y gritar al vacío.

   monitor:        ese hueco está DENTRO del monitor, y el monitor
                   está CERRADO. El otro no puede ni entrar.
                   El wait termina SIEMPRE antes de que alguien pueda
                   señalizar.
```

El monitor no hace atómico al `wait`: hace inaccesible al que podría arruinarlo. Es otra forma de conseguir lo mismo — nadie observa el estado intermedio, no porque sea instantáneo, sino porque nadie puede mirar.

Y fijate que ahora tenés **tres** mecanismos para el mismo fin:

Mecanismo	Cómo evita el estado intermedio
`TSL`	el hardware bloquea el bus
llamada al sistema	el kernel deshabilita interrupciones un instante
monitor	**el compilador impide que otro entre**

Ejemplo resuelto: productor-consumidor con monitor

Éste es el código del libro (fig. 2-34), en Pidgin Pascal:

```
monitor ProductorConsumidor
    condition llenas, vacias;
    integer cuenta;

    procedure insertar(elemento: integer);
    begin
```

```
        if cuenta = N then wait(llenas);      ← lleno: me duermo y SALGO
        insertar_elemento(elemento);
        cuenta := cuenta + 1;
        if cuenta = 1 then signal(vacias)     ← última instrucción
    end;

    function eliminar: integer;
    begin
        if cuenta = 0 then wait(vacias);      ← vacío: me duermo y SALGO
        eliminar = eliminar_elemento;
        cuenta := cuenta - 1;
        if cuenta = N - 1 then signal(llenas) ← última instrucción
    end;

    cuenta := 0;
end monitor;
```

Comparalo con la versión de semáforos: **desapareció el `mutex`**. No hay `down(&mutex)` ni `up(&mutex)` en ninguna parte. Eso es lo que te regala el monitor.

Tres cosas para mirar en ese código

1️⃣ `cuenta` ya no necesita protección. Es una variable *interna* del monitor: solo se toca desde `insertar` y `eliminar`, y nunca hay dos adentro. Acordate de que `cuenta` sin proteger era **una de las dos causas** de la falla en el 7.1. Resuelta gratis.

2️⃣ Los `signal` están condicionados, y no es casual.

```
   if cuenta = 1 then signal(vacias)
```

¿Por qué solo cuando `cuenta` vale 1? Porque `cuenta = 1` significa **que antes valía 0** — y si valía 0, es el único caso en que un consumidor pudo haberse dormido en `vacias`. Si `cuenta` valía 7, nadie está esperando y la señal **se perdería para siempre**.

> Ése es el truco que anunciaba el libro: *"analizar las variables"* para no señalizar al vacío. **Es el parche obligatorio de una primitiva sin memoria.** Con semáforos no hacía falta: un `up` siempre se guarda.

3️⃣ Los `signal` son la última instrucción de cada procedimiento. Brinch Hansen, tal como vimos.

Traza, con N = 3

t	quién y qué	cuenta	en el monitor	dormidos
1	C llama eliminar()	0	CONSUMIDOR	—
2	cuenta=0 → wait(vacias)	0	nadie ← SALIÓ	C
3	P llama insertar() (pudo entrar porque el wait liberó)	0	PRODUCTOR	C
4	inserta, cuenta := 1	1	PRODUCTOR	C
5	cuenta=1 → signal(vacias) (última instrucción)	1	PRODUCTOR sale	—
6	C despierta, completa su wait y sigue	1	CONSUMIDOR	—
7	elimina, cuenta := 0	0	CONSUMIDOR	—
8	cuenta=N-1? 0≠2 → no señaliza. Sale.	0	nadie	—

Fijate **t=2**: el `wait` saca a C del monitor. Si no lo hiciera, P nunca habría podido entrar en t=3 y tendrías interbloqueo. **Es la pieza esencial.**

Y fijate **t=8**: no señaliza, y está bien, porque nadie está esperando lugar.

En la vida real: Java

> *"Java… permite agrupar métodos en clases. Al agregar la palabra clave **`synchronized`** a la declaración de un método, Java garantiza que, una vez un hilo ha empezado a ejecutar ese método, no se permitirá que ningún otro hilo empiece a ejecutar ningún otro método `synchronized` de ese objeto."*

Eso es un monitor. En Java, `wait()` y `notify()` cumplen el papel de `wait` y `signal`.

```
Pidgin Pascal          Java
_____________          ____________________
monitor                class con métodos synchronized
wait(c)                wait()
signal(c)              notify()
```

(Complemento elaborado, no está en el libro: en Java y en Pthreads el `wait` se escribe dentro de un `while`, no de un `if` como en la fig. 2-34, porque esas implementaciones no garantizan la semántica estricta de Hoare/Brinch Hansen y existen despertares espurios. Si en el examen te piden el código del Tanenbaum, usá `if`.)

Lo que el monitor NO resuelve

Dos limitaciones, y la segunda es la que abre el próximo tema:

1) Dependen del lenguaje. Muy pocos los tienen. C, que es el lenguaje de los sistemas operativos, no. Por eso en la práctica se siguen usando semáforos y mutexes.

2) Necesitan memoria compartida. Monitores *y* semáforos suponen que todos los procesos ven las mismas variables. En una **máquina distribuida**, con varias CPU que no comparten memoria, **ninguno de los dos sirve**.

Y de ahí sale lo que viene: **paso de mensajes**.

(Tanenbaum, cap. 2.3.7, págs. 136–141, figs. 2-33, 2-34 y 2-35)

Preguntas de control

1) En la figura 2-34, el `signal(vacias)` está condicionado a `if cuenta = 1`. Supongamos que lo escribís sin la condición, así:

```
insertar_elemento(elemento);
cuenta := cuenta + 1;
signal(vacias)          ← siempre, sin preguntar
```

¿Se rompe algo? ¿Por qué pusieron la condición? Y la segunda parte: **con semáforos, ¿haría falta una condición equivalente antes del `up`?** ¿Por qué sí o por qué no?

2) Un compañero dice: *"el monitor es más potente que el semáforo, porque con el monitor no te podés equivocar"*.

¿Es más **potente**, o es otra cosa? ¿Qué puede hacer un semáforo que un monitor no?

Pregunta 1 — ⚠ mitad y mitad, y la segunda mitad está al revés

Esta pregunta era una trampa y caíste en las dos.

Parte A: ¿se rompe algo?

Dijiste que sí. **No se rompe nada.** Seguí la película:

```
cuenta pasa de 4 a 5. El código hace signal(vacias) sin preguntar.

¿Hay algún consumidor dormido en "vacias"?
☞ El consumidor solo se duerme si encuentra cuenta = 0.
```

```
   Con cuenta = 4 no se durmió nunca. NO HAY NADIE.

→ La señal se pierde.
→ Y no pasa absolutamente nada: nadie la necesitaba.
```

Tu instinto fue bueno (*"podría llamarlo sin tener nada"*) pero la conclusión no. Y acá está la distinción que importa, porque te va a servir para toda la unidad:

	¿Es un problema?
Señal perdida **con alguien que la necesitaba**	💥 **desastre** (el wakeup perdido del 7.1)
Señal perdida **sin nadie esperando**	☺ nada, solo trabajo al vacío

Una señal perdida no es un error en sí misma. El desastre del 7.1 no fue que la señal se perdiera: fue que se perdió justo cuando el consumidor estaba a punto de dormirse.

Entonces, ¿para qué está el `if cuenta = 1`? Es exactamente lo que anunciaba el libro:

> *"Un proceso, que de lo contrario realizaría una operación `signal`, puede ver que esta operación no es necesaria con sólo analizar las variables."*

Es una **optimización, no una condición de correctitud**. `cuenta = 1` significa *"antes valía 0"*, y 0 es el único estado en que un consumidor pudo haberse dormido. O sea: **señalizá solo cuando pueda haber alguien escuchando.**

*(Complemento elaborado: si hubiera **varios consumidores**, esa condición se queda corta y el `signal` incondicional sería más correcto — pueden quedar dos dormidos y señalizarse solo uno. La fig. 2-34 es un esqueleto con un productor y un consumidor. No lo digas en el examen salvo que te lo pregunten, pero tenelo claro.)*

Parte B: ¿haría falta lo mismo con semáforos? ✕

Dijiste que sí. **Es exactamente al revés, y ésta es LA pregunta del tema.**

Con semáforos **el `up` tiene que ser incondicional, siempre.** Si le pusieras una condición, romperías todo:

```
CON LA CONDICIÓN PUESTA SOBRE UN SEMÁFORO:

llenas = 0.  El productor mete un elemento → up(llenas) → llenas = 1
El productor mete otro → ¿cuenta = 1? NO → no hace up
                                            ↑
                          💥 llenas = 1 pero hay 2 elementos

El consumidor hace 1 down y cree que el búfer está vacío.
Un dato queda ahí para siempre.
Y se rompió el invariante: vacias + llenas ≠ N
```

🔑 La asimetría que hay que llevarse

Si nadie está esperando...	¿Qué pasa?	¿Conviene llamarlo?
signal (monitor)	SE PIERDE	no, es al vacío
wakeup	SE PIERDE	no, es al vacío
up (semáforo)	SE GUARDA	SÍ, SIEMPRE

Es la misma frase que te di en el 7.2 y que ahora cierra el círculo:

> **Un `up` sin destinatario se guarda. Un `signal` sin destinatario se pierde.**

Y de ahí sale la regla práctica:

- **Semáforo:** `up` siempre, sin preguntar. El contador se encarga.
- **Monitor / variable de condición:** `signal` solo si puede haber alguien. **Y `wait` tiene que ir antes que `signal`.**

El `if cuenta = 1` es, en el fondo, **el precio que pagás por no tener contador.** El monitor te regaló la exclusión mutua, pero te cobró tener que razonar a mano cuándo señalizar.

Pregunta 2 — ⚠ parcial, encontraste una diferencia real

Vamos por partes.

"¿Es más potente?" → No

```
✗  el monitor es MÁS POTENTE
✓  el monitor es MÁS SEGURO
```

No podés resolver con monitores ningún problema que no pudieras resolver con semáforos. Son equivalentes en potencia expresiva (en memoria compartida). Lo que cambia es de quién es la responsabilidad:

	Quién garantiza la exclusión mutua
Semáforo	**vos**, acordándote de cada `down` y cada `up`
Monitor	**el compilador**

Tu frase *"el monitor es automático"* apunta justo a eso. El aporte del monitor es **ergonómico**: te saca de las manos la parte en la que los humanos se equivocan. Nada más, y es muchísimo.

"¿Qué puede hacer un semáforo que un monitor no?"

Acá encontraste una diferencia real, así que te la acredito — pero con el vocabulario corregido:

```
✗  "el contador con las llamadas perdidas acumuladas"
✓  "el contador con las señales GUARDADAS"
```

Si se acumulan, **no están perdidas**. Es justo lo contrario. Pero el fondo es correcto y tiene una consecuencia concreta que vale la pena nombrar:

> **Con semáforos el orden no importa.** Podés hacer `up` antes de que nadie haya hecho `down`: queda guardado. Con variables de condición, **`wait` tiene que ir antes de `signal`**, o la señal muere.

Eso es una diferencia funcional de verdad, y la dijiste. ✓

Pero te faltan las **dos grandes**, que son las que se toman:

1️⃣ **El semáforo no depende del lenguaje.** El monitor necesita un compilador que lo entienda. **C no tiene monitores** — y C es el lenguaje con el que están escritos los sistemas operativos. El semáforo es una primitiva del SO: funciona desde cualquier lenguaje, siempre.

2️⃣ **Un manejador de interrupciones puede hacer `up`; no puede "entrar a un monitor".**

Esto es un uso de los semáforos que no habíamos visto y que conviene que tengas, porque es muy del libro:

```
El proceso que espera datos del disco:        down(&sem_disco)
                                                   ↓ se bloquea

Llega la interrupción del controlador.
El MANEJADOR hace:                            up(&sem_disco)
```

```
                                                ↓
El proceso se despierta y sigue.
```

Un semáforo por dispositivo, inicializado en 0. El manejador de interrupciones hace `up` y se va. Así el semáforo **oculta las interrupciones** detrás de una abstracción de sincronización.

Un manejador de interrupciones es un pedacito de código del kernel que tiene que terminar rapidísimo y **no puede bloquearse nunca**. Por eso le sirve el `up`, que no bloquea. No podría llamar a un procedimiento de monitor, porque eso **sí** se puede bloquear esperando entrar.

(Tanenbaum, cap. 2.3.5)

Y la limitación que comparten los dos

Ninguno sirve en una **máquina distribuida**, porque los dos suponen **memoria compartida**. Ahí hace falta otra cosa — y es lo que viene.

Tema 7.4a cerrado

Pregunta	Resultado
1a — ¿se rompe sin la condición?	⚠ instinto bueno, conclusión invertida
1b — ¿lo mismo con semáforos?	✗ es al revés: el `up` va siempre
2 — ¿más potente?	⚠ encontraste una diferencia real, faltaron dos

Lo que hay que fijar, que es lo único que te falló hoy:

`up` se guarda → llamalo siempre.
`signal` se pierde → llamalo solo si hay alguien.

Si te acordás de esa asimetría, las dos preguntas salen solas.

Para terminar el tema 7 queda un bloque corto: **paso de mensajes** (`send`/`receive`, que es lo que sí funciona en máquinas distribuidas), **RPC** y **barreras**. Son unos 10 minutos y con eso cerramos el tema y pasamos al **8, planificación** — el importante.

✓ **Sí, es exactamente así.** Dicho corto:

```
MONITOR     →  signal le habla a PERSONAS
               Si no hay nadie dormido, no le habla a nadie: se pierde.

SEMÁFORO    →  up cuenta COSAS
               La ficha queda guardada, haya o no alguien esperándola.
```

Por eso:

- En el monitor te tenés que preguntar **"¿hay alguien dormido?"** → de ahí el `if cuenta = 1`.
- En el semáforo te preguntás **"¿cuántas unidades hay?"** → el contador lo lleva solo, y por eso el `up` va siempre sin condición.

El contador del semáforo **no mira a nadie**: registra cuántas ranuras hay llenas y vacías, y punto. Si después aparece alguien a pedir, la ficha está ahí esperándolo. Si no aparece nunca, la ficha queda igual.

El `signal` del monitor, en cambio, **necesita un oyente en el momento exacto**. Sin oyente, no deja rastro.

Tema 7.4b — Paso de mensajes y barreras

El agujero que queda

Semáforos, mutexes y monitores tienen todos el mismo supuesto escondido: **que los procesos ven las mismas variables en memoria.** Un semáforo *es* una variable; un monitor *es* un módulo con datos compartidos.

```
¿Y si los procesos están en MÁQUINAS DISTINTAS, conectadas por red?

☞ No hay memoria compartida.
☞ No hay dónde poner el semáforo.
☞ Todo lo que vimos NO SIRVE.
```

Hace falta algo que no suponga memoria común. Ese algo es el **paso de mensajes**.

Las dos primitivas

```
send(destino, &mensaje);      /* enviar */
receive(origen, &mensaje);    /* recibir */
```

`receive` puede aceptar `CUALQUIERA` como origen si al receptor no le importa quién le escribe. Y si no hay mensaje disponible, el receptor **se bloquea** hasta que llegue uno — o vuelve con un código de error, si preferís la versión que no bloquea.

☆ Dónde se ubica en la clasificación

Esto es tabla de examen directa. El libro lo dice explícitamente: `send` y `receive` *"al igual que los semáforos y a diferencia de los monitores, son llamadas al sistema en vez de construcciones del lenguaje"*.

Primitiva	Qué es	Quién la provee	¿Necesita memoria compartida?
`TSL`	instrucción de máquina	hardware	sí
`down`/`up`	llamada al sistema	**SO**	sí
monitor	construcción del lenguaje	**compilador**	sí
`send`/`receive`	**llamada al sistema**	**SO**	**NO** ☆

Los problemas que aparecen y los otros no tenían

Mandar un mensaje por una red es poco confiable, y eso trae una cadena de problemas:

1️⃣ Se pierde el mensaje → el receptor devuelve un **acuse de recibo** (*acknowledgement*). Si el emisor no lo recibe en cierto tiempo, **retransmite**.

2️⃣ Se pierde el acuse → el emisor retransmite un mensaje que **ya había llegado bien**. El receptor lo recibe dos veces.

```
P ──mensaje──→ C      ✓ llegó
P ←──acuse──✗  C      ✗ se perdió el acuse
P ──mensaje──→ C      💥 duplicado
```

→ Se resuelve con **números de secuencia consecutivos**. Si llega un mensaje con el mismo número que el anterior, es un duplicado y se descarta.

3️⃣ Nomenclatura → hay que poder nombrar los procesos sin ambigüedad.

4️⃣ Autenticación → *"¿cómo puede saber el cliente que se está comunicando con el verdadero servidor de*

archivos y no con un impostor?"

5️⃣ **Rendimiento**, incluso en la misma máquina:

> *"La acción de copiar mensajes de un proceso a otro es **siempre más lenta** que realizar una operación con un semáforo o entrar a un monitor."*

Ése es el precio: **ganás funcionar sin memoria compartida, perdés velocidad.**

Ejemplo resuelto: productor-consumidor con mensajes

Acá está la idea linda del tema, y es contraintuitiva:

> **El consumidor arranca enviándole al productor N mensajes VACÍOS.**

```
Al inicio:   C ──[ ][ ][ ][ ]──→ P        (N = 4 mensajes vacíos)

Ciclo del PRODUCTOR:                Ciclo del CONSUMIDOR:
─────────────────────               ─────────────────────
produce un elemento                 receive(productor, &m)   ← espera uno lleno
receive(consumidor,&m) ← vacío      consume el elemento
arma el mensaje con el dato         send(productor, &m)      ← devuelve el vacío
send(consumidor, &m)   → lleno
```

El total de mensajes en el sistema es siempre N. No se crean ni se destruyen: circulan, cambiando de "vacío" a "lleno" y de vuelta.

¿Y qué pasa si uno va más rápido?

Situación	Qué ocurre
El **productor** es más rápido	todos los mensajes terminan **llenos**, esperando al consumidor → **el productor se bloquea** esperando que vuelva un vacío
El **consumidor** es más rápido	todos los mensajes terminan **vacíos** → **el consumidor se bloquea** esperando uno lleno

Fijate qué elegante: **el mensaje vacío cumple el papel del semáforo `vacias`, y el lleno el de `llenas`.** El invariante `vacias + llenas = N` se volvió literalmente *"hay N mensajes dando vueltas"*. Es el mismo problema con otra piel.

Y lo mejor: **no hay ni un semáforo, ni un mutex, ni una variable compartida.** Funciona entre máquinas.

Dos variantes de direccionamiento

a) A procesos: cada proceso tiene una dirección única y le escribís a él.

b) A buzones (*mailbox*): una estructura de datos que **almacena en búfer** cierta cantidad de mensajes, fijada al crearla. Los parámetros de `send`/`receive` son buzones, no procesos.

```
Si un proceso envía a un buzón LLENO → se suspende hasta que
alguien saque un mensaje de ahí.
```

c) El extremo opuesto — sin búfer, o *rendezvous* (cita):

> Si el `send` ocurre **antes** del `receive`, el emisor **se bloquea** hasta que el receptor haga `receive`. En ese momento el mensaje se copia **directo** del emisor al receptor, sin búfer. Y al revés también.

O sea: los dos tienen que **coincidir en el tiempo**. Es más simple de implementar pero *"menos flexible, debido a que el emisor y el receptor se ven obligados a ejecutarse en paso de bloqueo"*.

En la práctica: el sistema de paso de mensajes más conocido es **MPI** (*Message-Passing Interface*), muy usado en computación científica y programación paralela.

(Tanenbaum, cap. 2.3.8, págs. 140–144, fig. 2-36)

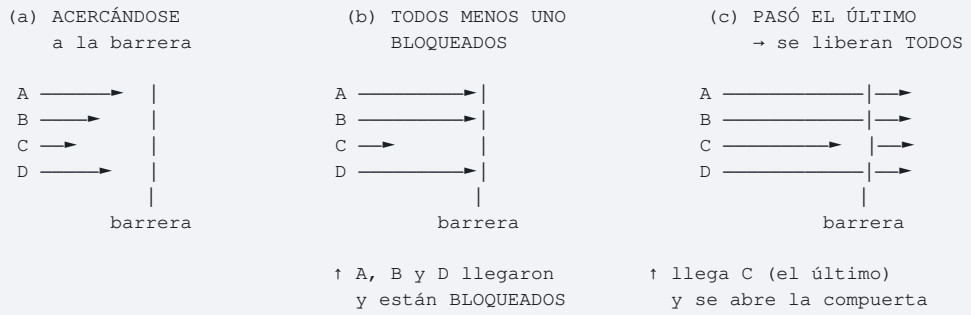
Barreras

El último mecanismo, y es distinto a todos los anteriores:

> Todos los que vimos eran para **dos** procesos peleándose por un recurso. **La barrera es para un GRUPO de procesos que avanzan juntos.**

El escenario

Algunas aplicaciones se dividen en **fases**, con la regla de que **ningún proceso puede pasar a la fase siguiente hasta que todos estén listos**. Se pone una **barrera** al final de cada fase.



El proceso llama a la primitiva `barrier` (normalmente un procedimiento de biblioteca) y **se suspende**. Cuando llega el último, **se liberan todos de golpe**.

Ejemplo típico

Un cálculo matricial por iteraciones: cada proceso calcula una franja de la matriz usando los valores de la iteración anterior.

```
Si un proceso arrancara la iteración 2 antes de que todos
terminen la 1, leería valores a medio actualizar.
☞ La barrera garantiza que la fase n+1 arranque solo
   cuando la fase n esté COMPLETA para todos.
```

(Tanenbaum, cap. 2.3.9, págs. 144–146, fig. 2-37)

🏁 Tema 7 cerrado — el cuadro completo

Mirá todo el recorrido, porque esta tabla es oro para el examen:

Mecanismo	Nivel	Al no poder seguir	¿Acumula señales?	¿Memoria compartida?
Deshabilitar interrupciones	hardware	—	—	sí
Variable candado	—	**gira** 🔥	—	sí

Alternancia estricta	—	**gira** 🔥	—	sí
Peterson	software	**gira** 🔥	—	sí
`TSL` / `XCHG`	hardware	**gira** 🔥	—	sí
`sleep`/`wakeup`	SO	se bloquea	✕ **se pierde**	sí
Semáforo	SO	se bloquea	✓ **contador**	sí
Mutex	SO / usuario	cede o se bloquea	✕ binario	sí
Monitor	**lenguaje**	`wait` y sale	✕ **se pierde**	sí
`send`/`receive`	SO	se bloquea	✓ (buzón)	✕ **NO**
Barrera	biblioteca	se bloquea hasta el último	—	sí

Las tres columnas de la derecha son las tres preguntas que te van a hacer, de cualquier forma que las disfracen:

1. ¿Gira o se bloquea?
2. ¿La señal se guarda o se pierde?
3. ¿Necesita memoria compartida?

Preguntas de control

1) En el productor-consumidor con mensajes, el consumidor arranca mandando **N mensajes vacíos**. Un compañero dice: *"es al revés, el productor tendría que arrancar mandando los datos, total el consumidor los va a recibir cuando pueda"*.

¿Qué se rompe si el productor arranca mandando sin esperar un vacío? ¿Qué papel cumplen esos N mensajes vacíos iniciales?

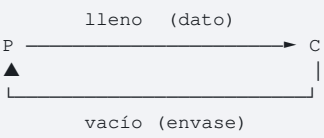
2) Tenés un servidor web con 4 hilos en una sola máquina, con memoria compartida, y hay que proteger la caché común. Alguien propone usar **paso de mensajes** en vez de un mutex, *"porque es más general y además funciona si después lo distribuimos"*.

¿Es técnicamente posible? ¿Lo harías? Justificá con **una** razón concreta del libro.

Con esto cerrás **8 de 10 temas**. Lo que viene es el **tema 8: planificación** — el más importante de la materia y el que más puntos vale en los parciales de tu cátedra. Ahí te debo el simulador de Gantt, que esta vez sí va junto con la clase.

Pregunta 1 — ⚠ parcial

La imagen la tenés bien: **"caja de transporte"** es exactamente la metáfora correcta. Los N mensajes son envases que **circulan** cambiando de estado:



Un ajuste chico de vocabulario: dijiste *"lo mande al emisor nuevamente"*. Ojo, que acá **los dos son emisor y**

receptor por turnos — no hay un "emisor" fijo. El productor recibe el vacío y envía el lleno; el consumidor recibe el lleno y envía el vacío.

Pero no contestaste lo que se preguntaba

La pregunta era: **¿qué se rompe si el productor arranca mandando sin esperar un vacío?** Y ahí está lo importante:

> **Los N mensajes vacíos son el semáforo `vacias`.** Cada mensaje vacío es **un permiso para producir**. Una ficha.

Si el productor manda sin pedir permiso:

```
El productor es 100 veces más rápido que el consumidor.

send, send, send, send, send, send... (10.000 veces)
                ↓
¿Dónde se guardan esos 10.000 mensajes?
☞ En memoria del SISTEMA OPERATIVO, que tiene que
   almacenar todo lo enviado-y-no-recibido.

💥 Consumo de memoria SIN LÍMITE.
```

Y eso es justo lo que el libro quiere evitar:

> *"el **número total de mensajes en el sistema permanece constante** en el tiempo, para que se puedan almacenar en **una cantidad de memoria específica que se conoce de antemano**."*

Lo que esos vacíos aportan

Función	Cómo
Control de flujo	el productor no puede ir más rápido que el consumidor
Memoria acotada	siempre hay exactamente N mensajes, nunca más
Bloqueo natural	sin vacíos disponibles, el productor se bloquea en su `receive`

Y fijate la equivalencia completa con lo que ya sabés:

```
mensaje vacío  =  ficha de "vacias"
mensaje lleno  =  ficha de "llenas"
N mensajes totales  =  invariante vacias + llenas = N
```

Es el mismo mecanismo, con envases en vez de contadores. Por eso el truco de los N vacíos iniciales es equivalente a inicializar `vacias = N`.

Pregunta 2 — ✕ incorrecta, y el error vale la pena

Dijiste que no es posible, que *"tienen que ser procesos de un hilo, no con varios"*.

Sí es posible. El paso de mensajes funciona perfectamente entre hilos del mismo proceso y entre procesos de la misma máquina. De hecho el libro dedica un párrafo entero a los problemas de diseño *"cuando el emisor y el receptor están en la misma máquina"*. Si no se pudiera, ese párrafo no existiría.

Dónde está la confusión

Estás leyendo la tabla al revés. Fijate:

```
✗  "paso de mensajes = para cuando NO hay memoria compartida"
✓  "paso de mensajes = NO NECESITA memoria compartida"
```

No necesitarla no es lo mismo que no poder usarse cuando la hay. Al contrario:

	¿Funciona sin memoria compartida?	¿Funciona con memoria compartida?
Semáforo / monitor / mutex	✗	✓
`send/receive`	✓	✓

> **El paso de mensajes es el mecanismo MÁS GENERAL de todos: funciona en los dos escenarios.**

Y eso significa que tu compañero **tiene razón en la primera mitad** de lo que dice. Es más general y sí seguiría funcionando si distribuís la aplicación. La pregunta era si eso lo hace conveniente.

La respuesta correcta

Sí, es posible. No, no lo haría. Y la razón concreta del libro es una sola:

> *"La acción de copiar mensajes de un proceso a otro es **siempre más lenta** que realizar una operación con un semáforo o entrar a un monitor."*

Con números:

```
mutex_lock (sin conflicto)  →  TSL, CMP, RET   ≈ 3 instrucciones
send/receive                →  copiar el mensaje entero, dos veces
                               (emisor → SO → receptor) + 2 TRAP
```

Y para una **caché**, que es el caso del enunciado, es peor todavía: la caché es *grande*. Con un mutex los 4 hilos **comparten la misma caché** y la tocan por puntero. Con mensajes tendrías que **copiar los datos** en cada consulta, o inventar un hilo dueño de la caché al que todos le escriben — que es un cuello de botella que serializa todo.

El argumento del "por si después lo distribuimos"

Es la parte más engañosa, y la respuesta es de criterio:

> **Estás pagando el costo HOY por un beneficio que tal vez nunca llegue.**

Si algún día lo distribuís, vas a tener que rediseñar un montón de cosas más que el candado de la caché. Mientras tanto, cada petición web paga copias de más.

La regla general: si tenés memoria compartida, **usala**. El paso de mensajes es la herramienta para cuando **no** la tenés. Pagar por generalidad que no necesitás es malgastar rendimiento.

🏁 Tema 7 cerrado

Pregunta	Resultado
1 — los N mensajes vacíos	⚠ buena imagen, faltó la función (son `vacias`: control de flujo y memoria acotada)
2 — mensajes en vez de mutex	✗ sí es posible; la razón para no hacerlo es el **costo de copiar**

Vas 8 de 10 temas. Lo que te quedó de hoy para fijar, en dos líneas:

Los N mensajes vacíos son fichas de `vacias` con otra forma.
El paso de mensajes funciona siempre, pero copiar es más caro que compartir un puntero.

Ahora viene el **tema 8: planificación**. Es el más largo y el más importante de la materia — en las fotos de tus parciales aparece Round Robin con quantum 1 y 2, el estado de la cola de preparados en un instante t, SJN y SRTF. Son puntos casi asegurados si lo tenés aceitado.

Te propongo arrancarlo así: primero los conceptos y las métricas (tiempo de retorno y tiempo de espera, que es lo que te piden calcular), y cuando lleguemos a los algoritmos te armo el **simulador de Gantt** en el mismo turno, con la regla de desempate de tu cátedra ya incorporada.

Tema 8.1 — Planificación: conceptos y métricas

Mapa del tema

Este tema es grande, así que lo partimos en cinco bloques:

```
8.1  CONCEPTOS Y MÉTRICAS          ← arrancamos acá
     ráfagas, cuándo planificar, apropiativa vs no
     tiempo de retorno y tiempo de espera ☆

8.2  ALGORITMOS POR LOTES          + 📱 SIMULADOR DE GANTT
     FCFS, SJF, SRTF

8.3  ALGORITMOS INTERACTIVOS       + 📱 el mismo simulador
     Round Robin ☆, prioridades, múltiples colas, lotería

8.4  TIEMPO REAL
     plazos y la condición de planificabilidad

8.5  POLÍTICA vs MECANISMO + planificación de hilos
```

El que más aparece en tus parciales es el **8.3 (Round Robin)**, y el 8.2. Pero sin el 8.1 no podés calcular nada, así que vamos de a poco.

---

1. Qué es el planificador

El **planificador** (*scheduler*) es la parte del SO que decide **cuál de los procesos listos se ejecuta a continuación**. El **algoritmo de planificación** es la regla que usa para decidir.

Dos aclaraciones de arranque:

a) Decide solo entre los que están LISTOS. Un proceso bloqueado no entra en la discusión — no se lo puede elegir. Esto es clave y vas a ver por qué cuando hagamos los ejercicios: *la cola de preparados* es el único universo del planificador.

b) En una PC moderna casi no importa; en un servidor sí. El libro es honesto con eso: en una PC *"la mayor parte del tiempo sólo hay un proceso activo"* y la CPU *"raras veces es un recurso escaso"*. Pero en un servidor web, donde compiten el proceso que atiende usuarios y el que calcula estadísticas, la planificación define si tus usuarios están contentos.

2. Por qué no se puede conmutar todo el tiempo

Antes de los algoritmos, hay un costo que condiciona todo. Un cambio de contexto implica:

```
1. pasar de modo usuario a modo kernel
2. guardar el estado del proceso actual (registros → BCP)
3. guardar el mapa de memoria
4. ejecutar el algoritmo de planificación
5. cargar el mapa de memoria del nuevo proceso en la MMU
6. arrancar el proceso nuevo
7. 💥 y además se INVALIDA toda la caché
```

> *"si se realizan muchas conmutaciones de procesos por segundo, se puede llegar a consumir una cantidad considerable de tiempo de la CPU, por lo cual se aconseja tener precaución."*

Guardate esto, porque es la mitad de la respuesta cuando te pregunten por el tamaño del quantum en Round Robin.

3. Comportamiento de un proceso: las ráfagas

Casi todos los procesos **alternan ráfagas de CPU con esperas de E/S**:

```
(a) PROCESO LIMITADO A CÁLCULOS (CPU-bound)

    ████████████████:::::████████████████:::::████████████
    ↑ ráfaga de CPU LARGA     ↑ espera de E/S (corta y rara)


(b) PROCESO LIMITADO A E/S (I/O-bound)

    ██::::::::::::::::██::::::::::::::::██::::::::::::::::██::
    ↑ ráfaga CORTA    ↑ espera de E/S (frecuente)

    ▉ = usando la CPU          ⣿ = bloqueado esperando E/S
```

⚠ La trampa clásica del examen

> *"Observe que **el factor clave es la longitud de la ráfaga de CPU, no de la ráfaga de E/S.**"*

Un proceso es limitado a E/S **no porque sus operaciones de E/S sean largas**, sino porque **hace pocos cálculos entre una petición y otra**. Pedir un bloque de disco cuesta lo mismo sin importar cuánto proceses después.

```
✗ "es I/O-bound porque sus E/S son muy largas"
✓ "es I/O-bound porque sus ráfagas de CPU son cortas"
```

Dos datos más que suelen preguntarse:

- **Copiar bits a la RAM de video cuenta como cálculo**, no como E/S, porque la CPU está trabajando. E/S es solo cuando el proceso **pasa a bloqueado** esperando un dispositivo externo.
- **A medida que las CPU se vuelven más rápidas, los procesos tienden a ser más limitados a E/S**, porque las CPU mejoran mucho más rápido que los discos.

4. ¿Cuándo hay que planificar?

Las cuatro situaciones (esto se pide enumerado):

#	Situación	Por qué hay que decidir

1	**Se crea un proceso**	padre e hijo quedan los dos listos: ¿cuál corre?
2	**Un proceso termina**	ya no existe, hay que elegir otro
3	**Un proceso se bloquea**	(E/S, semáforo, lo que sea) liberó la CPU
4	**Llega una interrupción de E/S**	alguien pasó de bloqueado a listo: ¿lo corremos ya?

Y además, **la interrupción de reloj**: si el reloj interrumpe a 50 o 60 Hz, se puede decidir en cada interrupción o cada k interrupciones.

Un detalle lindo del punto 3, que conecta con todo el tema 7: el libro dice que si A espera que B salga de su región crítica, **convendría ejecutar B**… pero *"el planificador comúnmente no tiene la información necesaria para tomar en cuenta esta dependencia"*. **Es la inversión de prioridades otra vez: el planificador no sabe quién tiene el candado.**

5. ☆ Apropiativa vs. no apropiativa

Ésta es *la* clasificación del tema:

	No apropiativa (*nonpreemptive*)	**Apropiativa (*preemptive*)**
Elige un proceso y…	lo deja correr **hasta que se bloquee o ceda voluntariamente**	lo deja correr **un máximo de tiempo fijo**
¿Lo puede sacar a la fuerza?	**No.** Aunque corra horas	**Sí**, al terminar el intervalo
En la interrupción de reloj	**no toma decisiones**	**toma la decisión**
¿Necesita reloj?	no	**sí, imprescindible**

Dos consecuencias que se preguntan:

a) *"Si no hay un reloj disponible, la planificación no apropiativa es la única opción."* Sin reloj no podés desalojar a nadie.

b) Fin de quantum → **el proceso va a LISTO**, no a bloqueado. (Esto ya lo vimos y te lo corregí; acá es donde vive la regla.)

6. Los tres entornos y sus metas

Entorno	**¿Apropiación?**	**Meta principal**
Por lotes (*batch*)	no hace falta	**rendimiento** y **tiempo de retorno**
Interactivo	**esencial**	**tiempo de respuesta**
Tiempo real	a veces no hace falta	**cumplir los plazos**

¿Por qué en interactivo la apropiación es *esencial*? Porque sin ella **un proceso puede acaparar la CPU y negarle el servicio a todos** — y no hace falta mala fe, basta un bug con un bucle infinito.

Y el dato fino de tiempo real: a veces no hace falta apropiación porque *"los procesos saben que no se pueden ejecutar durante periodos extensos"*. La diferencia con interactivo es que en tiempo real **solo corren programas hechos para esa aplicación**, no programas arbitrarios o maliciosos.

Las metas completas (fig. 2-39)

```
TODOS LOS SISTEMAS
  Equidad               dar a cada proceso una parte justa de la CPU
  Aplicación de políticas   que se cumplan las políticas establecidas
  Balance               mantener ocupadas todas las partes del sistema

POR LOTES
  Rendimiento           maximizar trabajos por hora
  Tiempo de retorno     minimizar entrega → terminación
  Utilización de CPU    mantenerla ocupada

INTERACTIVOS
  Tiempo de respuesta   responder rápido a las peticiones
  Proporcionalidad      cumplir las expectativas del usuario

TIEMPO REAL
  Cumplir los plazos
  Predictibilidad
```

Proporcionalidad es la más curiosa: es que el usuario *perciba* el tiempo como razonable. Si marcar un teléfono tarda 10 segundos, molesta — no porque sea mucho, sino porque el usuario cree que debería ser instantáneo.

---

7. ☆☆ LAS MÉTRICAS — esto es lo que te van a hacer calcular

Acá no te podés equivocar, porque de esto salen los puntos.

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│   TIEMPO DE RETORNO  =  Fin  −  Llegada              │
│   (turnaround)                                       │
│                                                      │
│   TIEMPO DE ESPERA   =  Retorno  −  Ráfaga           │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Pensalas así:

Métrica	Qué mide	La frase
Retorno	**todo** lo que el proceso vivió en el sistema	*"desde que llegué hasta que me fui"*
Espera	solo el tiempo **perdido**, sin trabajar	*"de todo eso, cuánto estuve al pedo"*

> **La espera es el retorno menos lo que realmente trabajó.** Si te acordás de eso, no necesitás memorizar nada más: la ráfaga es el trabajo útil, y todo lo demás del retorno fue espera.

Y una tercera, que es distinta:

Tiempo de respuesta	**desde la llegada hasta que empieza a ejecutarse por primera vez**

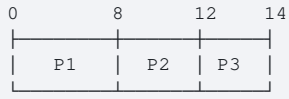
Ojo con no confundirlas: retorno mira el **final**, respuesta mira el **primer arranque**. En un sistema interactivo te importa la respuesta (que el cursor se mueva ya); en uno por lotes te importa el retorno (que la nómina esté lista).

Ejemplo resuelto — FCFS

Tres procesos. "Ráfaga" es cuánto necesitan de CPU:

Proceso	Llegada	Ráfaga
P1	0	8
P2	1	4
P3	2	2

Con FCFS (por orden de llegada, no apropiativo), el Gantt:



Y la tabla de métricas, **paso a paso**:

	Llegada	Ráfaga	Fin	Retorno = Fin−Lleg	Espera = Ret−Ráf
P1	0	8	8	8 – 0 = **8**	8 – 8 = **0**
P2	1	4	12	12 – 1 = **11**	11 – 4 = **7**
P3	2	2	14	14 – 2 = **12**	12 – 2 = **10**

```
Retorno promedio = (8 + 11 + 12) / 3 = 31/3 = 10,33
Espera promedio  = (0 +  7 + 10) / 3 = 17/3 =  5,67
```

*(Verificado con script. Si te interesa el detalle: `retorno − ráfaga` da lo mismo que `inicio − llegada` **solo cuando no hay apropiación**. Con Round Robin o SRTF el proceso se ejecuta a pedacitos y `inicio − llegada` ya no sirve; **la fórmula `retorno − ráfaga` sirve siempre**. Usá ésa y no te vas a equivocar nunca.)*

🔑 Fijate en P3

Necesitaba **2** unidades de CPU y tardó **12** en salir del sistema. Esperó **10** por culpa de P1, que es largo y llegó primero.

Ése es el defecto de FCFS, y es el motivo por el que existen todos los demás algoritmos. Lo vas a ver en el próximo bloque con el simulador.

(Tanenbaum, cap. 2.4.1, págs. 145–152, figs. 2-38 y 2-39. Apunte de la cátedra: mismas metas y métricas, con "tiempo de retorno: desde envío hasta terminación")

Preguntas de control

1) Un proceso A hace, en ciclo: 2 ms de cálculo y después espera 50 ms a que el disco le traiga un bloque. Un proceso B hace: 40 ms de cálculo y después espera 5 ms al disco.

¿Cuál es limitado a E/S y cuál a cálculos? Y ahora lo importante: **¿qué dato usaste para decidirlo, y qué dato es una distracción?**

2) En el ejemplo de arriba, P3 tuvo **tiempo de retorno 12** y **tiempo de espera 10**. Un compañero dice: *"está mal, el tiempo de espera no puede ser 10 si el proceso solo tardó 2 en ejecutarse; esperó 12 como máximo"*.

¿Dónde está el error de su razonamiento? Y aprovechando: **¿cuál fue el tiempo de respuesta de P3?**

Pregunta 1 — ✓ la clasificación, ⚠ el criterio, ✕ el distractor

A es I/O-bound y B es CPU-bound. Correcto.

Pero el criterio lo usaste a medias

Dijiste que A es I/O-bound *"porque tiene una ráfaga super corta de ejecución **y mayor de espera**"*. Esa segunda mitad es la que hay que sacar: **estás comparando la ráfaga contra la espera**, y eso es justo la trampa.

Mirá esta prueba:

```
Proceso A    2 ms de cálculo  +  50 ms de disco   →  I/O-bound
Proceso A'   2 ms de cálculo  +   5 ms de disco   →  I/O-bound TAMBIÉN
```

A' espera **10 veces menos** y sigue siendo limitado a E/S. ¿Por qué? Porque con 2 ms de cálculo entre pedido y pedido, **lo que hace es pedir, no calcular**. La duración del pedido no cambia su naturaleza.

Y al revés:

```
Proceso B    40 ms de cálculo +   5 ms de disco   →  CPU-bound
Proceso B'   40 ms de cálculo +  50 ms de disco   →  CPU-bound TAMBIÉN
```

> **Se mira UNA sola columna: la ráfaga de CPU.** La columna de E/S no se mira nunca.

El distractor ✕

Dijiste que la distracción es *"la palabra cálculo"*. No: **la distracción son los números de la E/S** — los 50 ms de A y los 5 ms de B. Están ahí para que los compares, y no hay que compararlos.

El libro lo dice explícito:

> *"Los procesos limitados a E/S están limitados a la E/S debido a que **no realizan muchos cálculos entre una petición de E/S y otra**, no debido a que tengan peticiones de E/S en especial largas. Se requiere el mismo tiempo para emitir la petición de hardware para leer un bloque de disco, sin importar qué tanto tiempo se requiera para procesar los datos."*

Un solo número decide: la ráfaga de CPU. Corta → I/O-bound. Larga → CPU-bound.

Pregunta 2 — ✓ la definición y la respuesta, ⚠ el diagnóstico

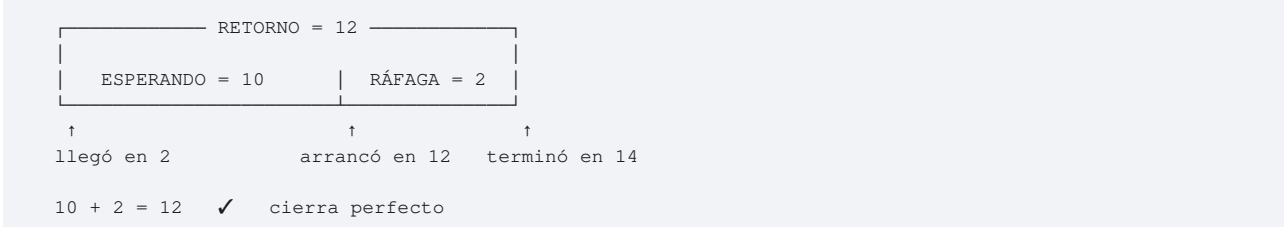
Lo que dijiste bien: **"el tiempo de espera es el tiempo en el que el proceso no estuvo haciendo nada"**. Definición perfecta.

Y **tiempo de respuesta = 10** ✓, con la razón correcta (*"lo que tardó para que le tocara"*).

Lo que falta: el compañero se refuta solo

Fijate lo que dijo: *"la espera no puede ser 10… esperó **12 como máximo**"*. Pero **10 es menor que 12**. ¡Su propia cota permite el 10! Está discutiendo contra un número que él mismo habilita.

El error de fondo: cree que la ráfaga y el retorno compiten, cuando en realidad **se suman**:



Y de paso quedate con la cota real, que es la que sirve para chequear tus propias cuentas en el examen:

> **0 ≤ espera ≤ retorno**, y siempre **espera + ráfaga = retorno**.

Si te da una espera mayor que el retorno, o negativa, **te equivocaste en la cuenta**.

Un detalle que se toma

En **FCFS la respuesta y la espera son iguales** (las dos dieron 10). No es casualidad: sin apropiación, el proceso arranca una sola vez y después no para, así que todo lo que esperó lo esperó antes de arrancar.

Con Round Robin o SRTF eso se rompe, porque el proceso arranca temprano pero se ejecuta a pedacitos. Lo vas a ver en un minuto.

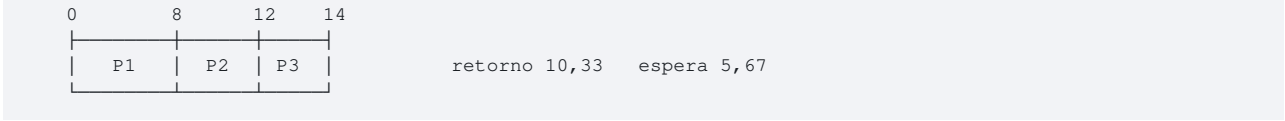
Tema 8.2 — Algoritmos por lotes: FCFS, SJF y SRTF

Vamos con los mismos tres procesos de ayer, así comparás sobre números que ya conocés:

Proceso	Llegada	Ráfaga
P1	0	8
P2	1	4
P3	2	2

1️⃣ FCFS — primero en llegar, primero en ser atendido

Ya lo viste. **No apropiativo**, una sola cola, orden de llegada.



La ventaja real es que es dificilísimo hacerlo mal: una sola cola, y cuando un proceso se bloquea se pone al final. *"Fácil de entender e igual de fácil de programar"*, dice el libro.

El defecto: el efecto convoy

P3 necesitaba **2** y tardó **12**. Quedó atrapado detrás de P1, que es cuatro veces más largo.

> **Un proceso largo que llega primero retrasa a todos los cortos que están detrás.** Como un camión en una ruta de un carril.

Y perjudica especialmente a los procesos **limitados a E/S**, que necesitan la CPU por ratitos cortos y seguidos.

2️⃣ SJF — el trabajo más corto primero

No apropiativo. Entre los que **ya llegaron**, elige el de **ráfaga total más chica**.



Fijate qué pasó: en t=0 **solo estaba P1**, así que no hubo nada que elegir — P1 corre igual sus 8. Pero en t=8 ya estaban P2 (ráfaga 4) y P3 (ráfaga 2), y eligió **P3**.

	FCFS	SJF
Retorno promedio	10,33	**9,67**
Espera promedio	5,67	**5,00**

P3 bajó su retorno de 12 a **8**. Mejoró, pero poco, porque el daño grande ya estaba hecho: **P1 ocupó la CPU los primeros 8 instantes y nadie pudo sacarlo.**

☆ "SJF minimiza el retorno promedio" — pero con una condición

Ésta es la frase que más se toma del tema, y tiene letra chica:

> **SJF es óptimo SOLO si todos los trabajos están disponibles al mismo tiempo.**

Por qué es óptimo cuando llegan juntos. Con 4 trabajos de duraciones *a, b, c, d* ejecutados en ese orden:

```
retorno promedio = (4a + 3b + 2c + d) / 4
                    ↑
            el PRIMERO pesa 4 veces
```

El primero entra en el retorno de todos; el último solo en el suyo. Entonces **conviene poner adelante al más corto**. Comprobado:

4 trabajos de 8, 4, 4, 4 llegando juntos	Retorno promedio
Orden 8, 4, 4, 4 (el largo primero)	(4·8 + 3·4 + 2·4 + 4)/4 = **14**
Orden 4, 4, 4, 8 (SJF)	(4·4 + 3·4 + 2·4 + 8)/4 = **11**

Y por qué NO es óptimo si llegan escalonados. Contraejemplo chiquito:

```
P1: llega 0, ráfaga 5      P2: llega 1, ráfaga 1

SJF:   P1[0-5] P2[5-6]        → retorno promedio 5,00
SRTF:  P1[0-1] P2[1-2] P1[2-6] → retorno promedio 3,50  ✓ mejor
```

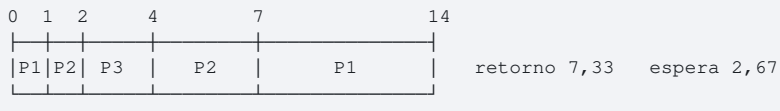
En t=0 SJF no tiene opción: solo está P1, y lo arranca. Cuando aparece P2, **ya es tarde: SJF no puede sacar a nadie.** Por eso existe el siguiente.

El otro problema de SJF

Hay que conocer la duración de antemano, y en general no se sabe. Funciona en procesamiento por lotes (donde el operador estima cuánto tarda la nómina) y no en un sistema interactivo.

3️⃣ SRTF — menor tiempo restante a continuación

Apropiativo. La versión con dientes de SJF: en cada instante corre el de **menor tiempo restante**, y **un proceso nuevo y corto desaloja al que está en la CPU**.



Seguí la película, que es lo que hay que saber explicar:

```
t=0   solo P1 (le faltan 8)            → corre P1
t=1   llega P2, le faltan 4.
      A P1 le faltan 7.  4 < 7         → 💥 P2 DESALOJA a P1
t=2   llega P3, le faltan 2.
      A P2 le faltan 3.  2 < 3         → 💥 P3 DESALOJA a P2
t=4   P3 termina. Quedan P2(3) y P1(7) → corre P2 (el menor)
t=7   P2 termina                       → corre P1 hasta el final
```

Los resultados son impresionantes

	FCFS	SJF	SRTF
Retorno promedio	10,33	9,67	**7,33**
Espera promedio	5,67	5,00	**2,67**
Respuesta promedio	5,67	5,00	**0,00**

Mirá la última fila: **respuesta 0 para los tres**. Cada proceso arrancó en el instante exacto en que llegó. Eso es lo que hace a SRTF buenísimo para trabajos cortos, y es donde se rompe la igualdad respuesta = espera de FCFS.

Por proceso:

	Retorno FCFS	Retorno SRTF
P3 (el cortito)	12	**2**
P2	11	**6**
P1 (el largo)	8	**14** ⚠

P3 pasó de 12 a 2. Pero P1 pasó de 8 a 14. Nada es gratis.

⚠ El peligro: inanición

Si cada tanto llega un proceso corto, **el largo nunca termina**: siempre hay alguien con menos tiempo restante que lo desaloja. Se queda esperando para siempre.

> **Inanición** (*starvation*): un proceso listo que nunca consigue la CPU porque el algoritmo siempre prefiere a otro.

Acordate del nombre, porque vuelve en prioridades y en lectores/escritores.

*(Tanenbaum, cap. 2.4.2, págs. 152–156, figs. 2-40 y 2-41. Apunte de la cátedra: le dice **SJN** al SJF y **SRTN** al SRTF — los parciales usan esas siglas.)*

🎮 Tres cosas para probar en el simulador

1 — El efecto convoy. Preset *"El de la clase"*, algoritmo **FCFS**. Ahora cambiale a P1 la ráfaga de 8 a **15** y mirá cómo explotan los retornos de P2 y P3. Volvé a 8 y pasá a SRTF: el daño desaparece.

2 — El contraejemplo de la optimalidad. Dejá dos procesos: P1(llegada 0, ráfaga 5) y P2(llegada 1, ráfaga 1). Compará **SJF** contra **SRTF** en la tabla de abajo. SJF da 5,00 y SRTF 3,50 — ahí ves con tus ojos que *"SJF es óptimo"* necesita la letra chica.

3 — La inanición de SRTF. Poné un proceso largo P1(0, 12) y después agregá varios cortitos que lleguen escalonados: (1,2), (3,2), (5,2), (7,2). Con **SRTF**, mirá el Gantt de P1: no arranca de verdad hasta el final.

Preguntas de control

1) En el Gantt de SRTF de arriba, P1 arranca en t=0 y termina en t=14. Su **tiempo de respuesta es 0** pero su **tiempo de espera es 6**.

En FCFS los dos valían lo mismo. ¿Por qué acá son distintos? ¿Qué está midiendo cada uno que en FCFS coincidía?

2) Un compañero mira la tabla comparativa y dice: *"entonces SRTF es el mejor algoritmo, gana en las tres métricas, habría que usarlo siempre"*.

Dale **dos razones distintas** por las que no se usa siempre. Una tiene que ver con un dato que el algoritmo necesita, y la otra con lo que le pasó a P1.

Corrección rápida

1) ✓ El mecanismo lo tenés. *"En SRTF otros procesos nuevos y cortos sí lo podían sacar, por lo que su tiempo de espera sí tenía"* — exacto. Al desalojarlo, P1 acumula espera **después** de haber arrancado, y la respuesta ya quedó congelada en 0.

⚠ Pero ojo con el **porqué** que diste: *"porque fue el primero en llegar y el primero en ser atendido"*. Eso no es la razón. Mirá FCFS completo:

FCFS	Respuesta	Espera
P1 (llegó 1º)	0	0
P2 (llegó 2º)	7	7
P3 (llegó 3º)	10	10

Los tres coinciden, y dos de ellos no llegaron primeros. La razón no es el orden de llegada:

> **La respuesta mide SOLO el primer tramo de espera. La espera mide TODOS los tramos.**
> Sin apropiación hay un solo tramo → coinciden. Con apropiación hay varios → la espera crece y la respuesta no.

De ahí sale la cota que ya usaste: **respuesta ≤ espera**, siempre.

2) ✓ Las dos razones. Inanición del largo y la necesidad de conocer las duraciones.

Un ajuste de palabras: no es que *"sea recomendable"* saber los tiempos — **es obligatorio, y es imposible**. SJF y SRTF necesitan un dato del futuro. Lo que se hace en la práctica es **estimarlo**: se va promediando lo que el proceso tardó en sus ráfagas anteriores. La técnica se llama **envejecimiento** (*aging*), y es con lo que arranca el tema que sigue.

Tema 8.3 — Round Robin

La intuición

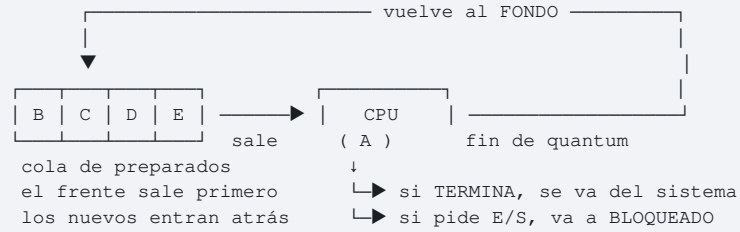
Ninguno de los tres anteriores sirve para un sistema interactivo: FCFS te deja esperando detrás de un camión, y los otros dos necesitan adivinar el futuro. Lo que querés cuando estás sentado frente a la máquina es simple: **que te atiendan rápido, aunque sea un ratito**.

Round Robin es la cola del banco con un reloj de arena en el mostrador: te atienden, se te termina la arena, **vas al fondo de la cola** y entra el siguiente.

El algoritmo

> **Round Robin (por turno circular)**: a cada proceso se le asigna un intervalo de tiempo llamado **quantum**. Si al terminar el quantum el proceso sigue en ejecución, se lo **desaloja** y se le da la CPU a otro. Si se bloquea o termina antes, se conmuta ahí mismo.

Es apropiativo, y lo único que mantiene es una lista enlazada de procesos listos.



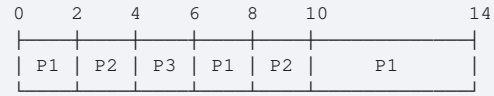
⚠ La trampa de examen

```
fin de quantum   →  pasa a LISTO     ✓   (sigue queriendo CPU)
pide E/S         →  pasa a BLOQUEADO ✓   (no puede usar la CPU)
```

Fin de quantum NUNCA es BLOQUEADO. Al proceso no le falta nada: le sobró trabajo. Esto se toma, y se toma seguido.

Ejemplo — los mismos tres procesos, q = 2

P1(llega 0, ráfaga 8), P2(1, 4), P3(2, 2):




```
t=0  entra P1 (cola vacía)
t=1  llega P2 → cola [P2]
t=2  fin de quantum de P1. Primero llega P3 → cola [P3].
     Después P1 se va al fondo → cola [P3, P1]... ¡NO!
```

🟤 **Pará ahí**, porque acá está la regla de tu cátedra. En t=2 pasan dos cosas: **llega P3** y **se le termina el quantum a P1**.

> **Regla de la cátedra: entre un proceso que llega y otro que sale del estado de ejecución, el que se pone antes en la cola es el que llega.**

Así que en t=2 la cola es **[P2, P3, P1]**: P2 ya estaba, después entra P3 (que llega) y último P1 (que sale). Por eso

el Gantt sigue P2, P3, P1.

Resultado: **retorno 9,00 · espera 4,33**

	FCFS	SJF	SRTF	RR q=2
Retorno prom.	10,33	9,67	7,33	9,00
Espera prom.	5,67	5,00	2,67	4,33

No le gana a SRTF. **Y no importa**: Round Robin no está para minimizar el retorno, está para **repartir**. Su virtud es que nadie espera más de *(n–1)·q* para que le toque, sin necesitar ningún dato del futuro.

☆ Lo único verdaderamente difícil: elegir el quantum

Acá está la pregunta de teoría que más se toma del tema. Supongamos que un **cambio de contexto cuesta 1 ms**:

Quantum	Útil	Desperdiciado en cambios de contexto
4 ms	4 ms	1 ms → **20 % de la CPU tirada**
100 ms	100 ms	1 ms → **1 % de la CPU tirada**

Entonces, ¿quantum grande y listo? No:

```
quantum DEMASIADO CHICO              quantum DEMASIADO GRANDE
────────────────────────             ──────────────────────────────
muchos cambios de contexto           pocos cambios: eficiente
→ se desperdicia CPU                 → pero si llegan 50 pedidos
→ baja la EFICIENCIA                    juntos, el último espera
                                        50 × 100 ms = 5 segundos
                                     → mala RESPUESTA interactiva
```

Conclusión del libro: un quantum demasiado corto produce demasiados cambios de contexto y baja la eficiencia; uno demasiado largo puede producir una mala respuesta a las peticiones interactivas cortas. Un quantum de entre 20 y 50 ms es a menudo un arreglo razonable.

Y lo podés verificar con tus propios números

Corrí el mismo juego de tres procesos con todos los quantums:

q	Gantt (cantidad de tramos)	Retorno prom.	Respuesta prom.
1	11 tramos	9,33	**0,33** ← la mejor respuesta
2	6 tramos	**9,00**	1,00
3	6 tramos	10,33	2,00
4	4 tramos	9,67	3,00
8	**3 tramos**	10,33	**5,67** ← la peor respuesta

Mirá las dos columnas de los extremos: **la respuesta mejora cuando el quantum baja, y la cantidad de cambios de contexto empeora**. Eso es exactamente el compromiso del libro, con números.

🔑 **Y fijate en q = 8**

```
q=8  →  P1[0-8] P2[8-12] P3[12-14]      retorno 10,33   espera 5,67
FCFS →  P1[0-8] P2[8-12] P3[12-14]      retorno 10,33   espera 5,67
```

Idénticos. Si el quantum es más grande que la ráfaga más larga, ningún proceso llega nunca al fin de quantum, nadie se desaloja, y Round Robin **degenera en FCFS**.

Round Robin con quantum → ∞ es FCFS. Es una frase para tener lista: muestra que entendés que la apropiación es lo único que los diferencia.

(Tanenbaum, cap. 2.4.3, págs. 156–157, fig. 2-42.)

🎮 Para probar ahora

En el simulador, preset **"El de la clase"**, algoritmo **Round Robin**:

1. Movelo a **q = 8** y compará la tabla de abajo con la fila de FCFS. Son la misma fila.
2. Bajalo a **q = 1** y contá los tramos del Gantt: 11. Ésos son los cambios de contexto que estás pagando.
3. Cargá el preset **"Parcial de la cátedra"** con **q = 2** y andá a **t = 2**. Vas a ver la regla de desempate funcionando en vivo.

Preguntas de control

1) Un administrador tiene un servidor con el quantum en **500 ms** y los usuarios se quejan de que el sistema "va a tirones". Lo baja a **1 ms** y ahora se quejan de que "va lento todo el tiempo".

¿Qué pasó en cada caso? Nombrá **qué métrica** empeoró con 500 ms y **qué se está desperdiciando** con 1 ms.

2) Dos procesos: **A** (llega 0, ráfaga 3) y **B** (llega 0, ráfaga 3), con Round Robin y **q = 3**.

¿Cuál es el Gantt? ¿Y cuánto vale el tiempo de respuesta de B? Decime además **qué otro algoritmo** de los que ya vimos da exactamente ese mismo resultado, y por qué.

Corrección

1) ✓ **Impecable.** Las dos mitades bien nombradas: con 500 ms empeora el **tiempo de respuesta** (de ahí los tirones) y con 1 ms se **desperdicia CPU en cambios de contexto** (de ahí la lentitud general). Nada que agregar.

2) ✓ **Correcto.** Gantt `A[0-3] B[3-6]`, respuesta de B = **3**, y se comporta como FCFS.

Agrego un dato que el simulador me confirma, porque es más fuerte de lo que dijiste:

```
FCFS   →  A[0-3] B[3-6]
SJF    →  A[0-3] B[3-6]
SRTF   →  A[0-3] B[3-6]
RR q=3 →  A[0-3] B[3-6]
```

Los cuatro dan lo mismo. Con ráfagas iguales y llegada simultánea, los algoritmos se vuelven indistinguibles: SJF y SRTF no tienen nada que elegir, y RR nunca llega al fin de quantum.

🔑 **Moraleja de examen: si un ejercicio te da todos los procesos con la misma ráfaga, probablemente la respuesta sea "todos los algoritmos dan el mismo resultado".** Y si te dan ráfagas distintas, es porque

quieren que se vea la diferencia.

Un detalle fino de tu redacción: dijiste *"se cumple el q=3 por lo que también se cumple A"*. En rigor A no se desaloja — **terminó exactamente en el mismo instante en que se le agotaba el quantum**, así que no hay fin de quantum, hay fin de proceso. No cambia el resultado, pero en un parcial conviene decir "A termina en t=3" y no "A se desaloja en t=3".

Tema 8.3b — Planificación por prioridades

La intuición

Round Robin trata a todos por igual, y **no todos los procesos son igual de importantes**. El demonio que manda mails no merece lo mismo que la ventana en la que estás tipeando.

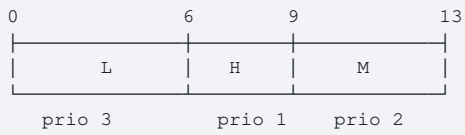
> **Planificación por prioridades**: a cada proceso se le asigna una **prioridad**, y se ejecuta el proceso listo de **mayor prioridad**.

⚠ **Convención**: en el Tanenbaum y en tu cátedra, **número más chico = más prioritario**. Prioridad 1 le gana a prioridad 5. Si un enunciado no lo aclara, aclaralo vos en la respuesta: *"asumo 1 = más prioritaria"*.

Las dos variantes

Ejemplo con tres procesos: **L**(llega 0, ráfaga 6, prio **3**), **M**(2, 4, prio **2**), **H**(4, 3, prio **1**).

No apropiativa — "esperá tu turno"

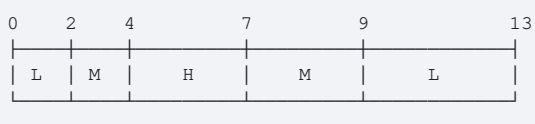


L arrancó cuando estaba solo, y **nadie lo saca**: termina sus 6 completos aunque lleguen dos más importantes. En t=6 se elige entre los que esperan, y gana **H** (prio 1), no M.

🟤 **Ésta es la que yo mismo fallé** cuando armé las pruebas del simulador: esperaba que entrara M porque llegó antes. **Prioridades NO es FIFO.** El orden de llegada solo decide los empates.

Retorno 7,33 · espera 3,00

Apropiativa — "salí, llegó alguien más importante"




```
t=0   solo L            → corre L
t=2   llega M (prio 2) vs L (prio 3)  → 💥 M DESALOJA a L
t=4   llega H (prio 1) vs M (prio 2)  → 💥 H DESALOJA a M
t=7   H termina. Esperan M(2) y L(3)  → vuelve M
t=9   M termina                       → vuelve L, el último de la fila
```

Retorno 7,67 · espera 3,33 · **respuesta 0,00**

Comparalas, porque es el tipo de pregunta que cae:

	No apropiativa	Apropiativa
Retorno prom.	**7,33**	7,67
Respuesta prom.	3,00	**0,00**
Retorno de **H** (el importante)	5	**3**
Retorno de **L** (el menos importante)	**6**	13

La apropiativa le da **respuesta 0 al proceso prioritario** y le duplica el castigo al de abajo. Es exactamente el mismo intercambio que viste en SRTF, pero decidido por importancia en lugar de por duración.

⚠ El problema: inanición otra vez

> Si siempre hay procesos de prioridad alta listos, **los de prioridad baja no se ejecutan nunca**.

Y acá está la vuelta de tuerca que te va a aparecer, porque conecta con la Unidad 2 entera:

🔗 **Inversión de prioridades** — si L (prio baja) tiene tomado un candado que H (prio alta) necesita, y el planificador por prioridades **nunca le da la CPU a L**, entonces L no puede soltar el candado y **H se cuelga para siempre**. El proceso más importante del sistema queda bloqueado por el menos importante. Es la razón por la que `mutex_lock` cede la CPU en lugar de girar.

Las soluciones

1️⃣ Prioridades dinámicas con envejecimiento

> El planificador **baja la prioridad** del proceso en ejecución en cada tic de reloj. Cuando cae por debajo de la del siguiente, se conmuta.

Así ningún proceso se queda arriba para siempre: el que usa mucha CPU se va hundiendo solo. Y al revés, al que espera mucho se le **sube** la prioridad (eso es el **envejecimiento** propiamente dicho).

La regla elegante de Tanenbaum para favorecer a los procesos de E/S — asignar la prioridad como:

$$\text{prioridad} = \frac{1}{f}$$

donde **f** es la fracción del último quantum que el proceso usó.

```
usó 1/50 del quantum (pidió E/S enseguida)  →  prioridad = 50   ⬆ altísima
usó 50/50 del quantum (cálculo puro)        →  prioridad = 1    ⬇ la más baja
```

El que menos usa la CPU es el que más prioridad recibe. Suena al revés y es justo lo correcto: el proceso de E/S necesita la CPU por un instante y después libera, así que dárselo rápido no le cuesta nada a nadie.

2️⃣ Múltiples colas

Se agrupan los procesos en **clases de prioridad**, se usa **prioridades entre las clases** y **Round Robin dentro de cada clase**.

```
Prioridad 4  →  [ ■ ■ ]      q = 1 quantum
Prioridad 3  →  [ ■ ]        q = 2 quantums
Prioridad 2  →  [ ■ ■ ■ ]    q = 4 quantums
Prioridad 1  →  [ ■ ]        q = 8 quantums
                  ▲
       se atiende la cola de arriba;
       solo se baja cuando la de arriba está vacía
```

El truco del CTSS (el sistema que inventó esto): **cada vez que un proceso agota su quantum completo, baja una clase y recibe el doble de quantum**. Un proceso de 100 quantums termina ejecutándose como 1+2+4+8+16+32+64 → **7 cambios de contexto en lugar de 100**, y mientras tanto los interactivos siguen arriba con quantum chico.

⚠ Esto tiene su propio agujero: un usuario que apretaba Enter de más conseguía que su proceso se moviera a la cola de mayor prioridad. Primera lección de que **una política de planificación es atacable**.

(Tanenbaum, cap. 2.4.4, págs. 157–158; múltiples colas 2.4.4, págs. 158–159.)

Preguntas de control

1) Un sistema usa prioridades **apropiativas** con envejecimiento. Un proceso de **cálculo puro** y uno **interactivo** arrancan los dos con prioridad 3.

Después de unos segundos, ¿cómo quedaron las prioridades de cada uno y por qué? Usá la regla del `1/f`.

2) Escenario de parcial: un servidor con prioridades apropiativas se **cuelga**. El proceso de prioridad 1 aparece como **BLOQUEADO** y nunca avanza; el de prioridad 9 está en **LISTO** y tampoco avanza porque siempre hay procesos de prioridad media corriendo.

¿Qué está pasando, cómo se llama, y cuál de las soluciones que vimos lo arregla?

"Cálculo puro" = un proceso que solo calcula y no pide E/S. Es el **limitado a CPU** del tema 8.1, en su versión extrema.

La diferencia práctica, que es lo único que le importa al planificador:

```
CÁLCULO PURO                      INTERACTIVO
____________                      ___________
arranca, calcula, calcula...      arranca, muestra algo en pantalla,
y lo sacan por FIN DE QUANTUM     pide leer el teclado y se BLOQUEA
                                  a los 2 ms

usa el quantum COMPLETO   f = 1   usa una MIGAJA del quantum   f ≈ 0,04
```

Ejemplos concretos: calcular una matriz grande, comprimir un video, minar algo, un `while` que suma. Nunca toca el disco ni la pantalla ni el teclado — **nunca se bloquea por su cuenta, siempre lo tienen que echar**.

Y el interactivo es el `nano`, el navegador, la terminal: hace un poquito y se queda esperando que vos aprietes una tecla.

El dato que usa la regla `1/f` es justamente ése: ¿el proceso devolvió la CPU solo, o hubo que sacársela?

Ahora sí, la pregunta: los dos arrancan en prioridad 3, y pasan unos segundos. ¿Dónde queda cada uno?

Dale, paso a paso. Vas a ver que son dos divisiones.

Paso 0 — Qué es `f`

```
                lo que el proceso USÓ del quantum
f    =    ──────────────────────────────────────
                      el quantum COMPLETO
```

Es una **fracción**, así que siempre está entre 0 y 1.

Pongamos números concretos: **quantum = 50 ms**.

Paso 1 — ¿Cuánto usó cada uno?

Esto ya lo sabés de la respuesta anterior:

	¿Qué hace?	¿Cuánto usó del quantum?
Cálculo puro	calcula hasta que lo echan	los **50 ms** enteros
Interactivo	muestra algo y espera el teclado	**2 ms** y se bloquea

Paso 2 — Calculá `f`

```
Cálculo puro:   f = 50/50 = 1
Interactivo:    f =  2/50 = 0,04
```

Hasta acá es solo leer la tabla. Lo importante es la intuición:

> **f grande = "chupa CPU". f chiquito = "la devuelve enseguida".**

Paso 3 — Ahora sí, aplicá la fórmula

```
              1
prioridad  =  ──
              f
```

Dividí 1 por cada uno de los dos valores del paso 2:

```
Cálculo puro:   prioridad = 1 / 1     = ?
Interactivo:    prioridad = 1 / 0,04  = ?
```

Hacé esas dos divisiones y decime los dos números.

(Pista para la segunda: dividir por 0,04 es lo mismo que multiplicar por 25... o pensalo al revés, `1 / (2/50)` = `50/2`.)

Y después de darme los números, contestame una sola cosa: **¿a cuál de los dos le quedó la prioridad más alta?**

Los dos números perfectos: 1 y 25. ✓ Y la conclusión también: **el interactivo queda con más prioridad.**

⚠ Pero la razón no es ésa

Dijiste *"porque se le sumó cada click de tiempo del cálculo puro"*. Estás mezclando dos mecanismos distintos. Separémoslos, porque son dos y los dos se toman:

	Mecanismo A — bajar en cada tic	Mecanismo B — la fórmula `1/f`
¿Cuándo actúa?	**durante** la ejecución, en cada tic de reloj	**al final** de cada quantum
¿Qué hace?	le va **bajando** la prioridad al que está en la CPU	**recalcula** la prioridad según lo que usó
¿Qué castiga?	estar mucho tiempo seguido en la CPU	haber usado todo el quantum

Los dos empujan para el mismo lado, pero la pregunta era sobre **B**, y B no suma nada: **divide**.

La razón, dicha derecho:

> **El interactivo queda arriba porque usó MUY POCO del quantum (f = 0,04), y `1/f` premia el f chico. El cálculo puro queda abajo porque usó TODO (f = 1), y `1/1` es el valor mínimo que la fórmula puede dar.**

No es que al cálculo puro le hayan sumado nada. Es que **la fórmula le da el peor número posible al que se queda con la CPU entera**. Y es justo: el interactivo necesita la CPU 2 ms y la devuelve, así que atenderlo primero no le cuesta nada a nadie.

🟤 Y un aviso importante, que es culpa mía

Fijate el choque:

```
Te dije:      número MÁS CHICO = más prioritario   (prio 1 le gana a prio 5)
La fórmula:   prioridad 25 > prioridad 1           ¡al revés!
```

Las dos convenciones existen y conviven en la materia. El Tanenbaum usa la fórmula `1/f` con *"número más grande = más prioritario"*, y los ejercicios con procesos de prioridad 1, 2, 3 usan *"número más chico = más prioritario"*.

> **Regla de supervivencia en el parcial: antes de resolver cualquier ejercicio de prioridades, escribí qué convención estás usando.** Una línea: *"asumo que 1 es la prioridad más alta"*. Si el enunciado la aclara, usás la del enunciado. Si no la aclara, la aclarás vos y nadie te puede bajar puntos.

Lo que **nunca** cambia, y es lo que hay que entender:

```
el que usa POCA CPU  →  se lo FAVORECE
el que usa MUCHA CPU →  se lo CASTIGA
```

Esa dirección es siempre la misma, le pongas el número que le pongas.

Volvamos entonces a la que quedó pendiente:

2) Un servidor con prioridades apropiativas se **cuelga**. El proceso de **prioridad 1** (la más alta) aparece como **BLOQUEADO** y nunca avanza. El de **prioridad 9** (la más baja) está en **LISTO** y tampoco avanza, porque siempre hay procesos de prioridad media corriendo.

Tres cosas:

1. ¿Qué está pasando? (pensá en qué puede estar esperando el de prioridad 1, y qué podría tener el de prioridad 9)
2. ¿Cómo se llama ese fenómeno?
3. ¿Cuál de las soluciones que vimos lo arregla?

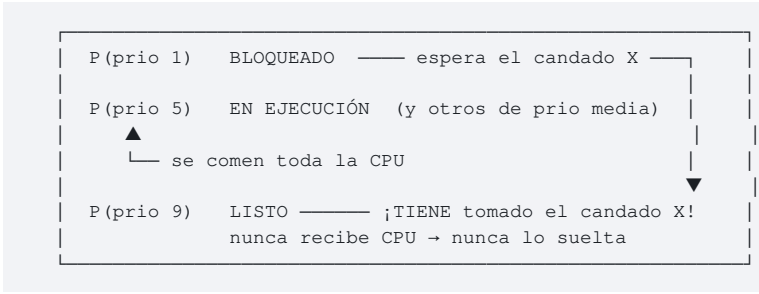
Corrección

2) ✓ Inanición y ✓ **envejecimiento**. La solución es la correcta. Pero te falta la mitad más importante de la película.

Lo que te falta: los dos problemas son UNO

Dijiste que el de prioridad 1 *"puede estar esperando E/S"*. Si fuera solo eso, **el servidor no se colgaría**: la E/S termina, llega la interrupción, el proceso se despierta y sigue. Un bloqueo por E/S se resuelve solo.

La pregunta decía que **nunca avanza**. Entonces no está esperando un disco: está esperando **algo que alguien tiene que soltar**.



La cadena completa, que es lo que hay que poder recitar:

```
1. El de prioridad 9 tomó el candado X.
2. Antes de soltarlo, lo desalojaron (es el menos prioritario).
3. El de prioridad 1 pidió X → se BLOQUEÓ.
4. Los de prioridad media corren eternamente.
5. El de prioridad 9 nunca recibe CPU → nunca suelta X.
6. El de prioridad 1 nunca se despierta.

→ El proceso MÁS importante del sistema quedó frenado
  por el MENOS importante.
```

El nombre

Inanición ✓ es correcto, pero es el nombre de lo que le pasa al de prioridad 9. El fenómema completo tiene nombre propio:

> **Inversión de prioridades**: un proceso de prioridad alta queda bloqueado esperando un recurso que tiene un proceso de prioridad baja, al que el planificador no le da la CPU. La prioridad queda **invertida**: el importante depende del que no lo es.

Si en el parcial te preguntan esto, la respuesta que vale los puntos es **"inversión de prioridades causada por la inanición del proceso de prioridad baja"**. Nombrá las dos.

🔗 Y acá se cierra el círculo con el tema 7

Es el mismo problema que vimos con `enter_region` hace tres clases, con una diferencia:

	Tema 7 — espera ocupada	Ahora — inversión de prioridades
¿Quién gira o se cuelga?	H gira en el `while`, quemando su quantum	H está BLOQUEADO, no gasta nada
¿Por qué no avanza L?	H no le cede la CPU	el planificador prefiere a los de prioridad media
¿Cómo se arregla?	`thread_yield` (ceder la CPU)	**envejecimiento** (subirle la prioridad a L)

Dos caminos distintos, el mismo final: el que tiene el candado tiene que llegar a ejecutarse.

🟣 Y guardate este caso, porque en la **Unidad 6 (interbloqueos)** vamos a necesitar distinguirlo:

```
INANICIÓN   →  no avanza porque el ALGORITMO siempre prefiere a otro.
               Se arregla cambiando la política (envejecimiento).

INTERBLOQUEO →  no avanza porque cada uno espera un recurso del otro.
                Cambiar la política NO lo arregla.
```

Esto que acabamos de ver es inanición, no interbloqueo. Con envejecimiento se destraba solo.

Por qué el envejecimiento lo arregla

```
t=0   prio 9, esperando...
t=1   prio 8, esperando...        ← el envejecimiento le va subiendo
...                                  la prioridad por esperar
t=5   prio 4  →  ¡ahora le gana a los de prioridad media!
      → recibe CPU → suelta el candado X
      → el de prioridad 1 se despierta → el servidor se destraba
```

El envejecimiento garantiza que **nadie espera para siempre**, y eso es precisamente lo que rompe la cadena.

Perfecto. Te falta solo la última tanda del tema 8: los algoritmos interactivos que quedan y la planificación de tiempo real. Antes de seguir dejame verificar un ejemplo numérico que necesito.

Tema 8.3c — Más corto a continuación, garantizada, lotería y partes equitativas

Son más cortos. Cada uno resuelve una queja distinta.

1️⃣ El proceso más corto a continuación

> SJF aplicado a sistemas interactivos: se ejecuta el proceso cuya **próxima ráfaga** se estima más corta.

La queja que resuelve: SJF daba el mejor retorno pero necesitaba el futuro. Acá se lo **estima** a partir del pasado.

Cómo se estima — promedio exponencial (envejecimiento):

$$\text{estimación nueva} = a \cdot \text{estimación vieja} + (1 - a) \cdot \text{ráfaga real}$$

Con **a = ½** queda la versión que se usa: *"la mitad de lo que creía más la mitad de lo que pasó"*.

Ejemplo verificado, arrancando con una estimación de 10 ms:

Estimación vieja	Ráfaga real	Cuenta	Estimación nueva
10	6	½·10 + ½·6	**8**
8	4	½·8 + ½·4	**6**
6	14	½·6 + ½·14	**10**
10	6	½·10 + ½·6	**8**

Fijate la fila 3: la ráfaga saltó a 14 y la estimación solo subió a 10. **No persigue el último valor, lo amortigua.** Y los pesos de las ráfagas viejas van cayendo:

```
la última ráfaga pesa  1/2
la anterior            1/4
la anterior            1/8        ← se van "olvidando" solas
la anterior            1/16
```

Eso es lo que significa **envejecimiento**: *estimar el siguiente valor de una serie tomando el promedio ponderado del valor actual y la estimación previa*.

⚠ No lo confundas con el otro envejecimiento. **Son dos cosas con el mismo nombre:**

Envejecimiento de prioridades	Envejecimiento como estimación
Subirle la prioridad al que espera	Predecir la próxima ráfaga
Sirve contra la **inanición**	Sirve para **adivinar la duración**

2️⃣ Planificación garantizada

El sistema **promete** a cada proceso una fracción de la CPU — con *n* procesos, **1/n** — y lleva la cuenta de lo que cada uno realmente recibió.

$$\text{razón} = \frac{\text{CPU que YA consumió}}{\text{CPU que le CORRESPONDÍA}}$$

Se ejecuta el de razón más baja: el más atrasado respecto de su promesa.

Ejemplo: 5 procesos, 100 ms transcurridos → a cada uno le corresponden **20 ms**.

	Consumió	Razón	
A	10 ms	10/20 = **0,50**	⬅ el más atrasado → **corre A**
E	15 ms	0,75	
C	20 ms	1,00	justo en lo prometido
D	25 ms	1,25	adelantado

B	30 ms	1,50	el que más abusó

Razón 1,00 = exactamente lo prometido. Menos de 1 = atrasado. Más de 1 = recibió de más.

3️⃣ Planificación por lotería ★

> Se dan a los procesos **boletos de lotería** para los recursos. Cada vez que hay que decidir, se **sortea** un boleto y gana el proceso que lo tiene.

La queja que resuelve: garantizada es prometer y después tener que cumplir, lo cual es complicado. La lotería consigue lo mismo **sin llevar ninguna contabilidad**: con más boletos, más chances.

```
A: 10 boletos  ─┐
B: 20 boletos   ├─ total 100   →   A obtiene 10 %
C: 70 boletos  ─┘                  B obtiene 20 %
                                   C obtiene 70 %  de la CPU
```

Las tres propiedades que la hacen elegante:

- **Es proporcional**: la fracción de CPU es la fracción de boletos. Directo, sin cuentas.
- **Es cooperativa**: **los procesos pueden regalarse boletos**. Un cliente que le pide algo a un servidor le pasa sus boletos para que lo atienda rápido, y los recupera después. Imposible de expresar con prioridades.
- **Responde al instante**: si llega un proceso nuevo con boletos, participa del próximo sorteo. No hay que recalcular nada.

4️⃣ Planificación por partes equitativas (*fair-share*)

> Se reparte la CPU **por usuario**, no por proceso.

La queja que resuelve: todos los anteriores reparten entre procesos. Si vos tenés 1 proceso y yo tengo 9, Round Robin me da el 90 % de la máquina... **por el solo hecho de haber abierto más ventanas.**

```
SIN partes equitativas              CON partes equitativas (50 % y 50 %)
──────────────────────              ────────────────────────────────────
Usuario 1:  A                       Usuario 1:  A  A  A  A  A
Usuario 2:  B C D E                 Usuario 2:  B  C  D  E  B

A B C D E A B C D E ...             A B A C A D A E A B ...
U1 recibe 20 %  ✗                   U1 recibe 50 %  ✓
U2 recibe 80 %                      U2 recibe 50 %
```

La unidad de reparto es el usuario, y los procesos de ese usuario se reparten su mitad entre ellos.

(Tanenbaum, cap. 2.4.4, págs. 156–161. "Dos niveles" y "tres niveles" figuran en el programa pero no los ubiqué todavía en el Tanenbaum de la cátedra; los busco en los apuntes antes de dártelos, no te los invento.)

🏷 El cuadro completo del tema 8 — para memorizar

Algoritmo	¿Apropia?	¿Qué optimiza?	¿Su defecto?

FCFS	✗	nada, es justo y simple	efecto convoy
SJF	✗	retorno promedio (si llegan juntos)	necesita el futuro
SRTF	✓	retorno y respuesta	inanición del largo
Round Robin	✓	**respuesta**, reparto equitativo	elegir el quantum
Prioridades	ambas	**importancia**	inanición / inversión
Múltiples colas	✓	respuesta **y** eficiencia	atacable (Enter de más)
Más corto a continuación	✗	retorno, **estimando**	la estimación puede fallar
Garantizada	✓	cumplir una **promesa** (1/n)	hay que contabilizar todo
Lotería	✓	proporción, **sin contabilidad**	es probabilístico
Partes equitativas	✓	justicia **entre usuarios**	—

Mnemotecnia para el orden: los tres primeros son **por lotes** (nadie mira la pantalla, importa el retorno). Los demás son **interactivos** (alguien espera, importa la respuesta).

Preguntas de control

1) Un proceso tiene estimación actual de **20 ms**. Sus próximas tres ráfagas reales son **8, 8 y 8 ms**. Con **a = ½**, calculá las tres estimaciones sucesivas.

Después contestame: ¿la estimación llega alguna vez a valer exactamente 8? ¿Y hacia dónde va?

2) Un sistema usa **lotería**. Hay tres procesos: A con 5 boletos, B con 15 y C con 30.

a) ¿Qué porcentaje de CPU recibe cada uno? b) C le **regala 20 boletos a A** (porque le pidió un servicio y quiere que lo atienda ya). ¿Cómo quedan los porcentajes? c) ¿Por qué esto **no se puede hacer** con planificación por prioridades?

Dale. Primero la herramienta, después la usamos tres veces.

La fórmula, con los nombres puestos

```
NUEVA  =  ½ · VIEJA  +  ½ · REAL
```

Traducida a castellano: **"la mitad de lo que yo creía, más la mitad de lo que pasó"**.

Los datos que tenés:

```
estimación inicial (VIEJA) = 20 ms
las tres ráfagas REALES    = 8, 8 y 8 ms
```

Ronda 1 — te la hago yo entera

```
VIEJA = 20        REAL = 8

NUEVA = ½ · 20  +  ½ · 8
```

```
      =   10     +    4
      =   14 ms
```

La estimación bajó de 20 a 14. El proceso resultó más rápido de lo que el sistema creía, así que el sistema le baja la expectativa — pero **solo a mitad de camino**: de 20 a 8 hay 12 de diferencia, y bajó 6.

🔑 **Cada ronda recorre la mitad de la distancia que falta.** Guardá esa idea, que es la respuesta a la segunda parte de la pregunta.

Ronda 2 — ahora vos

Lo único que cambia es el valor de VIEJA: **ya no es 20, es el 14 que acabamos de calcular**. La REAL sigue siendo 8.

```
VIEJA = 14        REAL = 8

NUEVA = ½ · 14  +  ½ · 8
      =    ?     +    ?
      =    ?
```

Hacé esa cuenta y decime el número.

(Y si querés adelantarte: fijate cuánta distancia hay de 14 a 8, y dividila por dos. Tiene que coincidir con lo que te dé la fórmula — ése es el chequeo.)

11. ✓ Y mirá que el chequeo cierra:

```
de 14 a 8 hay 6 de distancia  →  la mitad es 3  →  14 − 3 = 11   ✓
```

Ronda 3 — la última

VIEJA pasa a ser **11**. REAL sigue siendo 8.

```
VIEJA = 11        REAL = 8

NUEVA = ½ · 11  +  ½ · 8
      =   5,5    +    4
      =    ?
```

Ésta te la dejo servida: **9,5 ms**.

Así que la serie completa es:

```
20  →  14  →  11  →  9,5
     −6−   −3−   −1,5−        cada salto es la mitad del anterior
```

Ahora la parte conceptual, que es la que vale

La pregunta era: **¿llega alguna vez a valer exactamente 8? ¿Y hacia dónde va?**

Seguí la serie unos pasos más, aplicando siempre "la mitad de lo que falta":

```
20
14        falta 6
```

```
11        falta 3
 9,5      falta 1,5
 8,75     falta 0,75
 8,375    falta 0,375
 8,1875   falta 0,1875
 8,09375  ...
```

Decime las dos cosas:

a) ¿Llega a valer **exactamente 8** en algún momento? Sí o no, y por qué.

b) ¿**Hacia dónde** va la serie? (¿sube, baja, se queda quieta, se acerca a algo?)

Y te agrego una tercera, que es la que te pueden preguntar en el final:

c) Si en la ronda 8 el proceso de golpe hace una ráfaga de **40 ms** en lugar de 8, ¿la estimación salta a 40? ¿Qué le pasa?

Las tres bien. Y la (c) la calculaste vos: `½·8,09 + ½·40 = 4,05 + 20 ≈ 24`. Exacto.

Afinemos la (a) y la (b)

(a) *"Va a estar súper cerquita"* ✓. El **por qué** decílo así en un final:

> Cada ronda recorre **la mitad** de la distancia que falta. Si siempre queda la mitad, **nunca queda cero**. La estimación se acerca a 8 **asintóticamente**: tan cerca como quieras, pero nunca exactamente.

(b) *"Va decreciendo"* ✓, pero es media respuesta. Decrece **porque arrancó arriba**. Si hubiera arrancado en 2 ms con ráfagas de 8, la serie sería `2 → 5 → 6,5 → 7,25...` y **crecería**.

```
arranca en 20, ráfagas de 8   →   20 ▼ 14 ▼ 11 ▼ 9,5 ▼ ... → 8
arranca en  2, ráfagas de 8   →    2 ▲  5 ▲ 6,5 ▲ 7,25 ▲ ... → 8
```

> **No decrece: CONVERGE.** Si las ráfagas son estables, la estimación **tiende al valor real** desde donde sea que arranque. Eso es lo que hace útil al promedio exponencial: **se autocorrige**.

Y la (c) es la joya del tema

Mirá lo que acabás de descubrir:

```
la ráfaga real salta de 8 a 40     (×5, un salto brutal)
la estimación salta de 8,09 a 24   (ni ignora el salto, ni lo cree del todo)
```

> **El promedio exponencial amortigua los picos.** No se cree el valor nuevo entero, pero tampoco lo descarta: le da la mitad de crédito. Si el proceso realmente cambió de comportamiento, en dos o tres ráfagas más la estimación llega a 40. Si fue un pico aislado, vuelve a bajar a 8.

El parámetro `a` — lo que controla todo

```
estimación  =  a · VIEJA  +  (1 − a) · REAL
               └────────┘    └──────────┘
                MEMORIA        NOVEDAD
```

Valor de `a`	Qué pesa	Comportamiento
a = 0	solo la REAL	*"la próxima ráfaga va a ser igual a la última"* — cero memoria, **nerviosísimo**
a = ½	mitad y mitad	el arreglo razonable
a = 1	solo la VIEJA	*"la estimación nunca cambia"* — **no aprende nada**

```
a chico  →  reacciona RÁPIDO a los cambios,  pero se marea con los picos
a grande →  es ESTABLE frente a los picos,   pero tarda en darse cuenta
```

Es **el mismo compromiso que el quantum** de Round Robin: reacción contra estabilidad. En esta materia ese patrón aparece una y otra vez — vas a volver a encontrarlo en el tamaño de página de la Unidad 3 y en el tamaño de bloque de la Unidad 5.

Queda pendiente la segunda, que es cortita:

2) Lotería, tres procesos: **A = 5 boletos**, **B = 15**, **C = 30**.

a) ¿Qué porcentaje de CPU recibe cada uno?

b) C le regala 20 boletos a A. ¿Cómo quedan los tres porcentajes?

c) ¿Por qué ese regalo **no se puede hacer** con planificación por prioridades?

Corrección — hay un error en (a) y (b)

El error: los boletos no son porcentajes

```
A = 5 boletos      ¿cuántos boletos hay EN TOTAL?
B = 15 boletos
C = 30 boletos     5 + 15 + 30 = 50 boletos
                                 ──
                   ¡NO son 100!
```

Dijiste "A recibe el 5 %" porque tiene 5 boletos. Pero un boleto no vale 1 %: vale **1 chance entre 50**.

$$\%\ \text{de CPU} = \frac{\text{boletos del proceso}}{\text{boletos TOTALES}} \times 100$$

(a) correcto:

	Boletos	Cuenta	% de CPU
A	5	5/50	**10 %**
B	15	15/50	**30 %**
C	30	30/50	**60 %**
			100 % ✓

Chequeo que te salva siempre: los porcentajes tienen que sumar 100. Los tuyos sumaban 50, y eso solo ya te avisaba que faltaba dividir.

(b) — acá en realidad acertaste lo difícil

Los números que escribiste — **A = 25, B = 15, C = 10** — son **los boletos correctos**. Entendiste perfectamente la transferencia:

```
ANTES:   A = 5    B = 15   C = 30     total 50
         C le da 20 a A:  C pierde 20, A gana 20
DESPUÉS: A = 25   B = 15   C = 10     total 50  ← ¡EL MISMO!
```

Lo que te faltó fue el último paso, dividir por 50:

	Boletos	% antes	% después
A	5 → **25**	10 %	**50 %** ⬆
B	15 → 15	30 %	**30 %** =
C	30 → **10**	60 %	**20 %** ⬇

🔑 Y mirá el detalle hermoso de este ejercicio

El total siguió siendo 50 y **B no se movió del 30 %**, aunque B no hizo nada.

```
Un REGALO de boletos  →  el total NO cambia  →  los demás no se ven afectados.
Si C hubiera CREADO 20 boletos nuevos (total 70),
B habría caído de 30 % a 15/70 ≈ 21 % sin haber hecho nada.
```

Transferir boletos redistribuye. Crear boletos diluye a todos. Por eso la lotería funciona: los procesos se prestan boletos entre ellos y el sistema no se desbalancea.

(c) — la idea está, el argumento no

Dijiste *"es por prioridad de proceso y solo se mide el 1/f"*. Ojo: `1/f` es **una** forma de asignar prioridades, no la definición de prioridad. No es el motivo.

El motivo real son dos cosas:

1. Una prioridad es un PUESTO, no una cantidad.

```
BOLETOS          →  son una CANTIDAD: 25 boletos es 5 veces 5 boletos.
                    Se suman, se restan, se parten, se transfieren.

PRIORIDAD        →  es un ORDEN: "prioridad 3" significa
                    "voy antes que el 4 y después del 2".
                    ¿Qué sería "darte la mitad de mi prioridad"? Nada.
```

No podés sumar prioridades. Si A tiene prioridad 5 y C tiene 2, **"A + C" no significa nada**. Con boletos sí: 5 + 20 = 25, y eso **sí** significa algo.

2. La prioridad la pone el sistema, no el proceso. Un proceso no es dueño de su prioridad — si pudiera cambiársela, se pondría en 1 y listo. Los boletos, en cambio, **son suyos y los administra él**.

Un dato que vale puntos

Igual, el sistema operativo **sí** tiene un mecanismo parecido, y es exactamente el parche para la inversión de prioridades que vimos hace un rato:

> **Herencia de prioridades**: cuando un proceso de prioridad alta se bloquea esperando un candado, el sistema le **presta su prioridad** al que lo tiene, para que corra ya y lo suelte. Cuando lo suelta, vuelve a su prioridad original.

```
H (prio 1) se bloquea esperando el candado de L (prio 9)
     → el SO le presta la prioridad 1 a L
     → L corre, suelta el candado, vuelve a prio 9
     → H sigue.  Destrabado.
```

La diferencia con la lotería: acá el **sistema operativo** decide el préstamo y es temporal; en la lotería lo decide **el proceso** y es voluntario.

Dejame verificar los números del último tema antes de dártelo.

Tema 8.4 — Planificación de tiempo real

La intuición

Hasta acá el objetivo era *"que termine rápido"* o *"que responda rápido"*. En tiempo real el objetivo cambia de naturaleza:

> **Que termine ANTES DE un instante determinado. Llegar tarde es tan malo como no haber hecho nada.**

Un reproductor de CD que entrega los datos tarde produce un chasquido. Un airbag que se infla 200 ms tarde no sirve. **No es que "ande lento": está mal.**

Duro y blando

	Tiempo real **duro** (*hard*)	Tiempo real **blando** (*soft*)
El plazo...	es **absoluto**, no se puede perder	se puede perder **de vez en cuando**
Si se pierde...	**catástrofe**	molestia tolerable
Ejemplos	control industrial, airbag, frenos ABS, piloto automático	audio y video digital, telefonía IP

Las dos clases de eventos

```
PERIÓDICOS      →  ocurren a intervalos regulares
                   (el reproductor de CD pide datos cada 1/75 de segundo)

APERIÓDICOS     →  ocurren de manera impredecible
                   (el usuario aprieta el botón de parada de emergencia)
```

Esto importa porque solo con eventos periódicos se puede garantizar algo por anticipado. Si no sabés cuándo va a llegar el trabajo, no podés prometer que lo vas a terminar a tiempo.

☆ La condición de planificabilidad

Ésta es **la fórmula del tema**, y es de las que caen seguro.

Con *m* eventos periódicos, donde el evento *i* ocurre con **período P_i** y requiere **C_i segundos de CPU**:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Cómo leerla

Cada término C_i/P_i es **la fracción de CPU que ese evento consume de manera permanente**.

```
50 ms de CPU cada 100 ms   →   50/100 = 0,5   →   el 50 % de la CPU, para siempre
```

Y la suma es **el total de CPU comprometida**. Si pasa de 1, estás prometiendo más del 100 % de una máquina que solo tiene 100 %.

> **Un sistema de tiempo real que cumple esta condición se dice PLANIFICABLE.**

Ejemplo del libro — verificado

Tres eventos periódicos:

Evento	Período P_i	CPU C_i	C_i/P_i
1	cada 100 ms	50 ms	0,500
2	cada 200 ms	30 ms	0,150
3	cada 500 ms	100 ms	0,200
		Suma	**0,850**

0,850 ≤ 1 → ES PLANIFICABLE. Usa el 85 % de la CPU y le queda un 15 % libre.

Ahora rompámoslo de tres maneras

① **Subiéndole el trabajo a un evento** — el evento 3 pasa de 100 a 300 ms de CPU:

```
0,500 + 0,150 + 0,600 = 1,250   →  ✗ NO PLANIFICABLE
necesitaría el 125 % de la CPU
```

② **Agregando un evento nuevo**, aunque parezca livianito — 200 ms de CPU cada 1000 ms:

```
0,500 + 0,150 + 0,200 + 0,200 = 1,050   →  ✗ NO PLANIFICABLE
```

🔑 **Fijate que el evento nuevo solo pide el 20 % y alcanza para romper el sistema**, porque solo quedaba el

15 % libre. En tiempo real **no hay "un poquito más": o entra, o no entra.**

③ **Agregando un quinto** (40 ms cada 250 ms):

```
0,500 + 0,150 + 0,200 + 0,200 + 0,160 = 1,210   →  ✗ NO PLANIFICABLE
```

⚠ Las dos trampas de esta fórmula

Trampa 1 — no confundas cuál va arriba.

```
  CPU que necesita        Cᵢ            ✓ CORRECTO
─────────────────── =   ────
  cada cuánto ocurre      Pᵢ

  Pᵢ / Cᵢ                               ✗ al revés: te daría > 1 cuando está bien
```

Memoria rápida: **el resultado tiene que ser una fracción menor que 1 para cada evento**. Si un solo término te sale mayor que 1, lo invertiste (o ese evento solo ya es imposible: pide más CPU de la que tiene disponible en su período).

Trampa 2 — unidades. Si los períodos están en segundos y los tiempos de CPU en milisegundos, la cuenta sale 1000 veces mal. **Pasá todo a la misma unidad antes de dividir.**

Estáticos y dinámicos

```
ESTÁTICOS   →  las decisiones se toman ANTES de que el sistema arranque.
               Necesitan saberlo todo con anticipación.
               Solo funcionan con información perfecta y eventos periódicos.

DINÁMICOS   →  las decisiones se toman EN TIEMPO DE EJECUCIÓN.
               No necesitan saberlo todo de antemano.
               Son los que se usan cuando hay eventos aperiódicos.
```

(Tanenbaum, cap. 2.4.5, págs. 161–162. Los algoritmos concretos de tiempo real — RMS y EDF — el Tanenbaum los desarrolla en el capítulo de multimedia, no acá; si tu programa los pide, los buscamos ahí.)

Preguntas de control

1) Un sistema de tiempo real tiene que atender tres eventos periódicos:

Evento	Período	CPU que requiere
A	cada 50 ms	15 ms
B	cada 100 ms	20 ms
C	cada 200 ms	40 ms

a) ¿Es planificable? Mostrame la cuenta. b) ¿Cuánta CPU queda libre? c) Si quisieras agregar un evento **D que ocurre cada 500 ms**, ¿cuál es el **máximo** tiempo de CPU que puede requerir para que el sistema siga siendo planificable?

2) Un técnico dice: *"nuestro sistema de tiempo real cumple la condición de planificabilidad con suma 0,92, así*

*que **nunca** va a perder un plazo"*.

¿En qué se está confiando demasiado? Pensá en los **eventos aperiódicos** y en qué supone la fórmula.

Las tres partes de la 1 correctas, incluido el despeje de la (c).

(c) `1 − 0,7 = 0,3` → `x/500 = 0,3` → **x = 150 ms**. Perfecto. Y notá lo que significa: un evento que ocurre cada 500 ms puede permitirse ser **largo** (150 ms de CPU), mientras que uno que ocurre cada 50 ms tiene que ser cortísimo. **Cuanto más seguido ocurre un evento, menos puede trabajar cada vez.**

Pregunta 2 — vas bien encaminado, pero hay tres cosas

✓ *"Que no surja ninguno desprevenido"* — ésa es la primera, y es la que te pedía: **los eventos aperiódicos no están en la suma**. La fórmula solo cuenta los periódicos. Si llega una emergencia que consume CPU, come del 8 % libre... y si consume más, rompe los plazos de todos los demás.

Pero faltan dos, y la tercera es la que separa un 7 de un 10:

② **La fórmula supone que los C_i son del PEOR CASO.** Si el técnico midió "20 ms" y en realidad alguna vez tarda 35 porque la caché falló, la suma real se le va arriba de 1 sin que él se enterara. En tiempo real **los tiempos se calculan en el peor caso, no en el promedio**.

③ 🔑 **La condición es NECESARIA pero NO SUFICIENTE.**

Esto suena abstracto hasta que lo ves. Dos eventos:

Evento	Período	CPU	C_i/P_i
A	cada 100 ms	50 ms	0,50
B	cada **10 ms**	2 ms	0,20
		Suma	**0,70** ✓ planificable

Suma 0,70, bien debajo de 1. Ahora supongamos un planificador **no apropiativo** y que A arranca primero:

```
t=0    llegan A y B.  Arranca A.
t=0..50   A ocupa la CPU y nadie lo puede sacar.
t=10   ⏰ ¡B tenía que haber terminado acá!
t=50   A termina. B arranca.
t=52   B termina... 42 ms TARDE.
```

Plazo perdido con la condición cumplida.

> La condición te dice que **la CPU alcanza en total**. No te dice que **el algoritmo que elegiste vaya a llegar a cada plazo**. Hace falta la capacidad *y* un planificador que la use bien — en tiempo real, prácticamente siempre apropiativo y con prioridad para el de plazo más cercano.

Analogía: tener plata suficiente para pagar todas las cuentas del mes no garantiza que no te corten la luz. También tenés que pagarlas **en orden de vencimiento**.

La respuesta completa en un parcial sería: *"se confía en que (1) no habrá eventos aperiódicos, (2) los C_i medidos son el peor caso y (3) la condición garantiza el cumplimiento de los plazos, cuando solo garantiza que la*

capacidad total alcanza".

Dejame confirmar las páginas del último bloque del tema 8 antes de dártelo.

Encontré el texto, y con un regalo: **el ejercicio que te di es prácticamente el del libro.**

> *"Si se agrega un cuarto evento con un periodo de 1 segundo, el sistema seguirá siendo planificable mientras que este evento no requiera más de **150 mseg** de tiempo de la CPU por evento."*

El libro: `(1 − 0,85) × 1000 = 150`. Vos: `(1 − 0,70) × 500 = 150`. Mismo despeje, misma respuesta. Ese tipo de cuenta ya lo tenés.

Y hay una cuarta cosa para la pregunta 2, que está en el libro justo al lado:

> *"En este cálculo está implícita la suposición de que **la sobrecarga por la conmutación de contexto** es tan pequeña que se puede ignorar."*

La fórmula no cuenta lo que cuesta cambiar de proceso. Si tenés eventos muy frecuentes, los cambios de contexto se comen una porción real de ese 8 % libre. Con suma 0,92 el técnico tiene muy poco margen para pagarlos.

Tema 8.5 — Política contra mecanismo

El problema

Todo lo que vimos supone que los procesos son de **usuarios distintos que compiten**. Pero mirá este caso:

```
Administrador de base de datos  (proceso padre)
     |
     ├── hijo 1: analiza consultas
     ├── hijo 2: accede al disco
     ├── hijo 3: analiza consultas
     └── hijo 4: mantiene índices
```

¿Quién sabe cuál de esos hijos es el más urgente? El padre. Él sabe que el hijo 2 está trayendo los datos que los otros tres están esperando.

¿Y quién decide? El planificador del kernel, que no tiene la menor idea de qué hace cada uno.

> *"Ninguno de los planificadores antes descritos acepta entrada de los procesos de usuario acerca de las decisiones de planificación. Como resultado, raras veces el planificador toma la mejor decisión."*

La solución

> **Separar el MECANISMO de planificación de la POLÍTICA de planificación.** El algoritmo de planificación está **parametrizado**, y los procesos de usuario **llenan los parámetros**.

```
┌──────────────────────────────────────────────────────┐
│  MECANISMO  (vive en el KERNEL)                      │
│  "ejecuto siempre el proceso de mayor prioridad"     │
│  Es fijo. No se discute. Es el motor.                │
│                              ▲                       │
│                              | parámetros            │
│  POLÍTICA  (la pone el PROCESO DE USUARIO)           │
│  "mi hijo 2 tiene prioridad 1, los demás prioridad 5"│
│  Se cambia. La decide quien sabe.                    │
└──────────────────────────────────────────────────────┘
```

Concretamente: el kernel usa prioridades, **y ofrece una llamada al sistema para que un proceso establezca y modifique las prioridades de sus hijos**.

> **El mecanismo está en el kernel, pero la política la establece un proceso de usuario.**

El padre controla cómo se planifican sus hijos sin encargarse él de la planificación.

🔗 Por qué esto es más grande que el tema

Es un **principio de diseño** que ya viste disfrazado:

Mecanismo (fijo, en el kernel)	Política (configurable, afuera)
El planificador por prioridades	Qué prioridad tiene cada hijo
El microkernel (Unidad 1)	Los servidores que corren arriba
El TSL del hardware (tema 7)	Qué protocolo armás con él

Si te preguntan "¿por qué separar política de mecanismo?": porque el que tiene la información (el proceso) no es el que tiene el poder (el kernel), y parametrizar es la forma de juntarlos.

(Tanenbaum, cap. 2.4.5, págs. 161–162.)

Tema 8.6 — Planificación de hilos

Acá hay **dos niveles de paralelismo**: procesos e hilos. Y cambia todo según dónde vivan los hilos.

Hilos a nivel usuario — el kernel elige PROCESO

```
1. El kernel elige un proceso (A) y le da SU quantum de 50 ms.
2. El sistema en tiempo de ejecución, DENTRO de A, elige un hilo (A1).
3. Como NO HAY INTERRUPCIONES DE RELOJ para los hilos,
   A1 se ejecuta TODO EL TIEMPO QUE QUIERA.
```

Esto ya lo discutimos cuando me preguntaste por el `mutex_lock`, y acá está la consecuencia completa:

```
Si cada hilo trabaja 5 ms y el quantum del proceso es 50 ms:

┌──────────── quantum de A (50 ms) ────────────┐┌── quantum de B ──┐
```

```
| A1 A2 A3 A1 A2 A3 A1 A2 A3 A1              || B1 B2 B3 ...     |
└──────────────────────────────────────────┘└──────────────────┘

POSIBLE:    A1, A2, A3, A1, A2, A3
IMPOSIBLE:  A1, B1, A2, B2, A3, B3     ← ¡no se puede intercalar!
```

El kernel no puede meter a B1 en medio del quantum de A, porque no sabe que los hilos de A existen.

🔑 **Y el dato tranquilizador**: si A1 es antisocial y se queda con los 50 ms enteros, *"su comportamiento antisocial no afectará a los demás procesos"*. **Se perjudica a sí mismo y a sus hermanos, no al resto del sistema.** El daño queda encerrado en el proceso.

Hilos a nivel kernel — el kernel elige HILO

```
1. El kernel elige un HILO específico, de cualquier proceso.
2. Le da un quantum y lo SUSPENDE OBLIGATORIAMENTE si se excede.
```

```
POSIBLE:  A1, B1, A2, B2, A3, B3     ← ¡ahora SÍ se puede intercalar!
```

Ésa es la diferencia observable, y es la que te pueden poner como pregunta: *"dada esta secuencia, ¿son hilos de usuario o de kernel?"*. Si los hilos de distintos procesos se alternan, **son de kernel**.

☆ El cuadro comparativo — esto es lo que se toma

	Hilos a nivel usuario	**Hilos a nivel kernel**
¿Quién elige el hilo?	el sistema en tiempo de ejecución	**el kernel**
¿Hay reloj para los hilos?	**no** — el hilo corre hasta que cede	**sí** — se lo suspende obligatoriamente
Costo de conmutar hilo	**muchas instrucciones de máquina**, rapidísimo	**conmutación de contexto completa**: cambiar el mapa de memoria e invalidar la caché → **varios órdenes de magnitud más lento**
Si un hilo se bloquea por E/S	**se suspende TODO el proceso** ✕	**solo ese hilo** ✓
¿Planificador propio de la aplicación?	**sí** ✓	no
Intercalar hilos de procesos distintos	imposible	posible

⚠ **La trampa de este cuadro**: los dos tienen una ventaja y una desventaja, **y están cruzadas**.

```
Nivel usuario:  conmuta RAPIDÍSIMO  pero  una E/S frena a todos sus hermanos.
Nivel kernel:   una E/S frena solo a UNO  pero  conmutar sale carísimo.
```

Si te preguntan "¿cuál es mejor?", la respuesta es **"depende de si la aplicación conmuta mucho o se bloquea mucho"**, nunca "el de kernel".

Las dos cosas finas del tema

① **El kernel puede aprovechar la afinidad.** Conmutar de un hilo de A a otro hilo de A es más barato que saltar a B (no hay que cambiar el mapa de memoria ni arruinar la caché). Así que **entre dos hilos de igual importancia, el kernel puede preferir el que está en el mismo proceso que el hilo que se acaba de bloquear.**

🔗 Esto es **afinidad de caché**, y vuelve en el tema 10 (multiprocesadores).

② **Los hilos de usuario pueden tener un planificador hecho a medida de la aplicación.** El ejemplo del servidor web:

```
Se bloquea un hilo trabajador. Están listos: el DESPACHADOR y 2 trabajadores.

¿Quién conviene que corra?   →  el DESPACHADOR
¿Por qué?  porque va a poner a trabajar a OTRO trabajador,
           con lo cual maximiza el paralelismo.

El sistema en tiempo de ejecución SABE qué hace cada hilo.
El kernel NUNCA lo sabe.
```

> *"En general los planificadores de hilos específicos para la aplicación pueden optimizar mejor una aplicación de lo que el kernel puede."*

🔗 **Y mirá qué cierra:** esto es **política contra mecanismo otra vez**, pero resuelto de la otra manera. En 8.5 el kernel se deja parametrizar; acá la aplicación se queda con **todo** el planificador. Son las dos salidas al mismo problema: *el que tiene la información no es el que tiene el poder.*

(Tanenbaum, cap. 2.4.6, págs. 162–163, fig. 2-43.)

🏁 Tema 8 terminado

Repasá el cuadro de los diez algoritmos que te di antes, sumale la fórmula $\Sigma\ C_i/P_i \leq 1$ y el cuadro de hilos usuario/kernel, y tenés el tema completo. **Es el que más puntos vale de la unidad.**

Preguntas de control

1) En un examen te dan esta secuencia de ejecución observada en un sistema con dos procesos de tres hilos cada uno:

```
A1, A2, B1, A3, B2, B3, A1, B1
```

¿Son hilos a nivel usuario o a nivel kernel? **Justificá con un solo dato de la secuencia.**

2) Una aplicación de servidor hace **muchísima E/S de disco** y sus hilos conmutan poco entre ellos. Otra hace **cálculo puro y conmuta entre hilos miles de veces por segundo**.

¿Qué tipo de hilos le conviene a cada una, y por qué? Nombrá **la fila exacta del cuadro** que decide cada caso.

Las dos bien. Un ajuste en la justificación de la segunda.

1) ✓ Perfecto. El dato es exactamente ése: **se intercalan hilos de procesos distintos** (`A2, B1, A3`). Con hilos de usuario, el kernel le da el quantum a A y hasta que A no lo agote no aparece ningún `B`. Un solo `B` en medio del quantum de A alcanza para probarlo.

2) ✓ Las dos elecciones correctas, y la primera justificada perfecto: hilos de kernel **para que el bloqueo por E/S no suspenda todo el proceso**. Ésa es la fila *"si un hilo se bloquea por E/S"*.

⚠ Pero la segunda la justificaste con la fila equivocada. Dijiste *"puede ceder fácilmente"* — eso es cierto pero no es lo que decide. Lo que decide es el **precio de cada conmutación**:

```
conmuta MILES de veces por segundo
     × nivel usuario:  unas pocas instrucciones de máquina
     × nivel kernel:   conmutación de contexto COMPLETA
                       (cambiar el mapa de memoria + invalidar la caché)
                       → varios ÓRDENES DE MAGNITUD más lento

miles de conmutaciones × "órdenes de magnitud más caro" = la aplicación se arrastra
```

Y la ventaja del kernel **no le sirve para nada**, porque hace cálculo puro: **no se bloquea nunca por E/S**, así que paga el costo alto sin cobrar el beneficio.

La regla de decisión, en una línea: ¿qué hace esta aplicación MUCHAS veces? Si se bloquea mucho → kernel. Si conmuta mucho → usuario.

Caso	Fila que decide	Elección
Mucha E/S	*si un hilo se bloquea por E/S*	**kernel**
Muchas conmutaciones	*costo de conmutar hilo*	**usuario**

Tema 8 cerrado. Dejame traer el texto del que sigue.

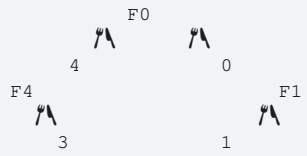
Tema 9.1 — Los filósofos comelones

9.1 — Los filósofos comelones

El planteo

Dijkstra, 1965. **Cinco filósofos** en una mesa redonda. Cada uno tiene un plato de espagueti, y entre cada par de platos hay **un tenedor** — o sea, **cinco tenedores en total**.

El espagueti es tan resbaloso que hace falta tomar DOS tenedores para comer.



```
                F3   🍴2   F2

   5 filósofos, 5 tenedores.
   El filósofo i usa el tenedor i (izquierdo) y el (i+1)%5 (derecho).
   Cada tenedor es COMPARTIDO por dos vecinos.
```

La vida de un filósofo: alterna entre **pensar** y **comer**. Cuando tiene hambre, trata de tomar sus dos tenedores **uno a la vez**, come, los devuelve y vuelve a pensar.

> **La pregunta clave: ¿podés escribir un programa para cada filósofo que haga lo que debe y NUNCA SE TRABE?**

Por qué este problema importa tanto

> *"Desde ese momento, todos los que inventaban otra primitiva de sincronización se sentían obligados a demostrar qué tan maravillosa era, al mostrar con qué elegancia resolvía el problema de los filósofos comelones."*

Es **el banco de pruebas** de la sincronización. Y lo que modela es muy concreto: **procesos que compiten por el acceso exclusivo a un número limitado de recursos**, como dispositivos de E/S. Los tenedores son impresoras, unidades de cinta, archivos.

🔗 Y de paso es el ensayo general de la **Unidad 6**: acá vas a ver el interbloqueo en su forma más pura.

Intento 1 — la solución obvia (y está MAL)

```
#define N 5

void filosofo(int i) {
    while (TRUE) {
        pensar();
        tomar_tenedor(i);            /* toma el IZQUIERDO  */
        tomar_tenedor((i+1) % N);    /* toma el DERECHO    */
        comer();
        poner_tenedor(i);
        poner_tenedor((i+1) % N);
    }
}
```

`tomar_tenedor` **espera** hasta que ese tenedor esté disponible y lo toma. Parece impecable. Y falla:

> **Suponga que los cinco filósofos toman sus tenedores izquierdos al mismo tiempo. Ninguno podrá tomar su tenedor derecho, y habrá un INTERBLOQUEO.**

```
   F0 tomó el tenedor 0  →  espera el 1, que tiene F1
   F1 tomó el tenedor 1  →  espera el 2, que tiene F2
   F2 tomó el tenedor 2  →  espera el 3, que tiene F3
   F3 tomó el tenedor 3  →  espera el 4, que tiene F4
   F4 tomó el tenedor 4  →  espera el 0, que tiene F0
                              └──────────────────────┘
                                ESPERA CIRCULAR 🔁
```

Nadie suelta lo que tiene, y todos esperan lo del vecino. Para siempre.

🔑 **Y acá está el detalle que vale puntos:** los cinco están **BLOQUEADOS**, en la cola de bloqueados de un semáforo. **No consumen un solo ciclo de CPU.** La máquina está libre y los filósofos están muertos.

> Acordate de la frase que armamos en el tema 7: **"interbloqueo = no avanza y NO gasta CPU"**. Éste es el ejemplo canónico.

Intento 2 — "si no puedo el derecho, devuelvo el izquierdo"

Parche razonable: después de tomar el izquierdo, **mirá si el derecho está libre**. Si no, **devolvé el izquierdo**, esperá un rato y volvé a intentar.

Y falla **por una razón distinta**:

```
t=0   los cinco toman el izquierdo
t=1   los cinco ven que el derecho no está
t=2   los cinco devuelven el izquierdo
t=3   los cinco esperan el mismo rato
t=4   los cinco toman el izquierdo otra vez
t=5   los cinco ven que el derecho no está
       ... y así eternamente.
```

> *"Una situación como ésta, en la que **todos los programas continúan ejecutándose en forma indefinida pero no progresan**, se conoce como **inanición** (starvation)."*

☆ La comparación que SE TOMA

Éste es el único lugar del libro donde los dos fracasos aparecen uno al lado del otro. Aprendete esta tabla:

	Intento 1 — interbloqueo	Intento 2 — inanición
¿En qué estado están?	**BLOQUEADOS**	**EN EJECUCIÓN / LISTOS**
¿Gastan CPU?	**no, cero**	**sí, toda**
¿Se mueven?	no, están congelados	sí, toman y sueltan sin parar
¿Progresan?	**no**	**no**
¿Se puede detectar fácil?	sí: todos quietos	**no**: parece que funciona

```
INTERBLOQUEO →  parálisis total. Se ve enseguida.
INANICIÓN    →  actividad frenética sin resultado. Se ve en el monitor
                la CPU al 100 % y nadie termina nunca nada.
```

🟣 **Ésta es la distinción que confundiste en el tema 7**, cuando dijiste que dos procesos interbloqueados *"se bloquean entre sí ocupando tiempo de CPU"*. Acá tenés las dos cosas separadas, con nombre y apellido, en el mismo problema.

Intento 3 — esperar un tiempo ALEATORIO

Si el problema es que los cinco hacen todo sincronizado, que cada uno espere un rato **distinto**:

```
F0 espera 3 ms       →  se rompe la simetría
F1 espera 17 ms      →  alguno llega primero, toma los dos, come
F2 espera 8 ms
...
```

Y en la práctica funciona. Es exactamente lo que hace **Ethernet**: si dos máquinas transmiten a la vez, cada una espera un tiempo aleatorio y reintenta.

Pero no es una solución, es una apuesta:

> *"En algunas cuantas aplicaciones sería preferible una solución que funcione siempre y que no pueda fallar debido a una serie improbable de números aleatorios.* ***Piense acerca del control de la seguridad en una planta de energía nuclear."***

```
"la probabilidad de que falle una hora seguida es bajísima"
     → aceptable en una red de oficina
     → INACEPTABLE en una central nuclear, un marcapasos o un freno ABS
```

🔗 Es el mismo criterio del **tiempo real duro** que vimos hace un rato: *"casi siempre a tiempo"* no es estar a tiempo.

Intento 4 — un mutex para toda la mesa

Protejamos **todo el acto de comer** con un semáforo binario: `down(&mutex)` antes de tomar tenedores, `up(&mutex)` después de devolverlos.

```
down(&mutex);
tomar_tenedor(i);
tomar_tenedor((i+1) % N);
comer();
poner_tenedor(i);
poner_tenedor((i+1) % N);
up(&mutex);
```

> **Desde un punto de vista teórico, esta solución es adecuada.** No hay interbloqueo ni inanición.

Pero tiene un error de rendimiento, y es grave:

```
Solo UN filósofo puede comer a la vez.

              F0 comiendo
          F4 🍝    (mesa     🍝 F1     ← cuatro filósofos esperando
          F3 🍝  bloqueada)  🍝 F2       con 3 tenedores LIBRES sobre la mesa

Con 5 tenedores, DOS filósofos deberían poder comer simultáneamente.
(F0 y F2, por ejemplo: no comparten ningún tenedor)
```

Serializamos todo para evitar un problema que solo afectaba a los vecinos. Convertimos un problema de concurrencia en un problema de rendimiento.

🔑 **La lección general, que aparece en toda la unidad:**

```
Un candado DEMASIADO GRANDE es correcto pero mata el paralelismo.
Un candado DEMASIADO CHICO es rápido pero deja agujeros.
```

(Tanenbaum, cap. 2.5.1, págs. 163–166, figs. 2-44 y 2-45.)

Nos queda la **solución buena** (figura 2-46), que usa un arreglo de estados y **un semáforo por filósofo**, y permite el máximo paralelismo. Te la doy en el próximo bloque y te armo la visualización interactiva, porque este problema se entiende diez veces mejor viéndolo girar.

Preguntas de control

1) En el **intento 1**, el interbloqueo pasa si los cinco toman el izquierdo a la vez. Un compañero propone:

> *"Que el filósofo **4** tome primero el DERECHO y después el izquierdo. Los otros cuatro siguen igual."*

¿Funciona? Probalo: hacé que los cinco intenten tomar su primer tenedor simultáneamente y mirá quién se queda con qué. **Decime si se rompe la espera circular y por qué.**

2) En el **intento 4** (el mutex para toda la mesa), decime **qué pares de filósofos podrían estar comiendo al mismo tiempo** si el mutex no estuviera, y cuántos pares distintos hay. (Pista: dos filósofos pueden comer juntos si **no comparten ningún tenedor**.)

Verificado con una búsqueda exhaustiva. **La 1 está mal, y vale la pena que veas por qué.**

Pregunta 1 — ✕ sí funciona

Tu razonamiento fue: *"el derecho ya fue tomado por el F3"*. Ahí está el error: **confundiste cuál es el tenedor derecho de F4.**

```
Numeración del libro:  el filósofo i usa el tenedor i (IZQUIERDO)
                                    y el tenedor (i+1)%5 (DERECHO)

F4  →  izquierdo = tenedor 4        derecho = tenedor (4+1)%5 = tenedor 0
                                                ──────────────
                                              ¡el tenedor 0, no el 4!
```

El tenedor 4 es el **izquierdo** de F4 (y el derecho de F3). El **derecho** de F4 es el tenedor **0**, que es el izquierdo de F0.

Rehagamos la jugada

Los cinco piden su primer tenedor al mismo tiempo. Pero ahora **F4 pide el 0**, igual que F0:

```
F0 pide t0  ─┐
             ├─ ¡CHOCAN! Uno de los dos gana. Digamos F0.
F4 pide t0  ─┘
```

```
F1 pide t1  → lo consigue
F2 pide t2  → lo consigue
F3 pide t3  → lo consigue

Reparto:  F0:t0   F1:t1   F2:t2   F3:t3   F4: NADA (bloqueado en t0)

                      ✯ el tenedor 4 quedó LIBRE ✯
```

Y ahí se rompe todo:

```
F0 tiene t0, espera t1 (de F1)   →  espera
F1 tiene t1, espera t2 (de F2)   →  espera
F2 tiene t2, espera t3 (de F3)   →  espera
F3 tiene t3, espera t4  ———————— ¡LIBRE! ————————→  🍝 F3 COME

F3 termina → suelta t3 y t4 → F2 come → suelta t2 → F1 come → ...
La cadena se desarma sola.
```

La verificación

Recorrí **todos los estados alcanzables** de los dos esquemas:

	Estados explorados	Estados de interbloqueo
Intento 1 (los cinco el izquierdo)	82	**1**
Propuesta (F4 invierte el orden)	70	**0**

Y el único estado de interbloqueo del intento 1 es exactamente el del libro:

```
F0 tiene t0 y espera t1
F1 tiene t1 y espera t2
F2 tiene t2 y espera t3
F3 tiene t3 y espera t4
F4 tiene t4 y espera t0
```

🔑 Por qué funciona — esto es oro para el examen

> **Un solo filósofo que pide en el orden inverso rompe la espera circular, y sin espera circular no hay interbloqueo.**

```
TODOS iguales  →  F0→F1→F2→F3→F4→F0   un ANILLO cerrado 🔒
F4 invertido   →  F0→F1→F2→F3→t4 libre   una CADENA abierta ✓
```

Se lo conoce como **"el filósofo zurdo"** o **romper la simetría**. Es una solución real, y es la más barata de todas: **no agrega ni un semáforo, solo cambia dos líneas de orden.**

🔗 En la **Unidad 6** vas a ver que el interbloqueo necesita **cuatro condiciones simultáneas**, y una es la **espera circular**. Acabás de ver la técnica de prevención más usada: **ordenar los recursos**. Si todos piden los tenedores en orden numérico creciente, nunca hay ciclo. Guardalo, porque ahí lo vas a volver a necesitar.

Pregunta 2 — 4 de 5 ✓

Encontraste **F0+F2, F2+F4, F4+F1, F1+F3**. Te falta **uno**:

```
F0+F2   F0+F3   F1+F3   F1+F4   F2+F4
        ─────
      éste te faltó
```

F0 usa los tenedores **0 y 1**. **F3** usa los tenedores **3 y 4**. No comparten nada → pueden comer juntos.

La cuenta ordenada, que es la forma de no perderte ninguno:

```
pares totales          C(5,2) = 10
pares de VECINOS            −5     (comparten un tenedor: 0-1, 1-2, 2-3, 3-4, 4-0)
                       ────────
pares que pueden comer       5
```

En una mesa redonda de 5, cada filósofo tiene 2 vecinos y 2 no-vecinos. Cada uno aparece en 2 pares válidos → 5×2/2 = 5 pares. ✓

Y el dato extra que verifiqué

TRÍOS que podrían comer a la vez: 0.

Tres filósofos necesitarían **6 tenedores distintos**, y solo hay 5. Así que:

> **El máximo paralelismo posible en la mesa es 2 filósofos comiendo simultáneamente.**

Eso convierte en número exacto la queja del libro contra el intento 4: el mutex global permite **1**, y lo óptimo es **2**. **Se desperdicia la mitad del paralelismo disponible.**

Ahora sí, la solución buena. Y te la armo para que la veas girar: dejame construir el simulador.

Vos sos el planificador: hacés clic en un filósofo y ejecuta su próximo paso. Los cuatro esquemas, el código de cada uno al lado, y un botón 💥 **Forzar el interbloqueo**. Con el teclado: `0`–`4` para cada filósofo, `espacio` para un paso al azar.

Verificado explorando **todos los estados alcanzables** de cada esquema (`_trabajo/verificar_motor_filosofos.js` y `_trabajo/verificar_html_filosofos.js`): el intento 1 tiene exactamente **1** estado de interbloqueo y es el del libro; los otros tres tienen **0**; en ningún esquema comen dos vecinos a la vez; y con el mutex global el máximo de comensales simultáneos es **1**, mientras que con Dijkstra es **2**.

La solución buena — figura 2-46

La idea

Los intentos anteriores fallaban porque los filósofos **tomaban los tenedores de a uno**. Entre el primero y el segundo había una ventana, y ahí se colaba el desastre.

> **La solución de Dijkstra elimina la ventana: un filósofo toma los DOS tenedores o NINGUNO.**

Y para decidir si puede, **no mira los tenedores: mira el estado de sus vecinos.**

```
   Un filósofo sólo pasa al estado COMIENDO si NINGUNO de sus dos vecinos está comiendo.
```

Si ningún vecino come, **sus dos tenedores están necesariamente libres** (porque solo los vecinos pueden tenerlos). Reemplazó una pregunta sobre recursos por una pregunta sobre estados.

Las tres piezas

```
int estado[N];        /* PENSANDO | HAMBRIENTO | COMIENDO, uno por filósofo */
semaforo mutex = 1;   /* protege SÓLO la consulta del arreglo estado[] */
semaforo s[N];        /* UN SEMÁFORO POR FILÓSOFO, todos arrancan en 0 */
```

⚠ **Fijate la diferencia con el intento 4**: ahí el mutex protegía **todo el acto de comer** (incluido `comer()`, que puede durar horas). Acá el mutex protege **solo unas pocas líneas** que consultan un arreglo. **El candado se hizo chiquito, y por eso vuelve el paralelismo.**

El código, explicado

```
void tomar_tenedores(int i) {
    down(&mutex);              /* ① entro a la región crítica */
    estado[i] = HAMBRIENTO;    /* ② anuncio que quiero comer  */
    probar(i);                 /* ③ ¿puedo? si sí, me autorizo */
    up(&mutex);                /* ④ salgo de la región crítica */
    down(&s[i]);               /* ⑤ si NO me autoricé, acá me bloqueo */
}
```

🔑 **El paso ⑤ es lo más fino de todo el programa.** Pensalo:

```
   Si probar(i) tuvo éxito  →  hizo up(&s[i])  →  s[i] vale 1
                            →  el down(&s[i]) pasa DE LARGO, sin bloquear.

   Si probar(i) falló       →  s[i] sigue en 0
                            →  el down(&s[i]) me BLOQUEA.
```

El mismo `down` sirve para las dos cosas, y por eso no hay `if`. Y está **fuera del mutex** a propósito: si me bloqueara adentro, me dormiría con el candado tomado y **nadie podría volver a entrar**.

🟡 Ésa es exactamente la trampa que ya discutimos con el productor-consumidor: **nunca te duermas con el candado en la mano.** Acá el libro lo evita poniendo el `down(&s[i])` después del `up(&mutex)`.

```
void poner_tenedores(int i) {
    down(&mutex);
    estado[i] = PENSANDO;      /* ya terminé */
    probar(IZQUIERDO);         /* ¿puede comer mi vecino izquierdo ahora? */
    probar(DERECHO);           /* ¿y el derecho? */
    up(&mutex);
}
```

Al terminar de comer, el filósofo no se va sin más: revisa a sus dos vecinos y despierta al que ahora pueda comer.

🔗 **Y acá está la asimetría del tema 7 otra vez.** `probar` solo hace `up(&s[i])` **si el vecino está HAMBRIENTO**.

¿Por qué se puede condicionar este `up`, cuando te dije que un `up` de semáforo va siempre incondicional?

Porque `s[i]` **no es un contador de recursos**: es un semáforo que se usa como **señal dirigida a una persona concreta**. Funciona como el `signal` de un monitor, no como el `up` de `vacias`. Si el vecino no tiene hambre, no hay a quién despertar, y acumular la señal rompería la cuenta la próxima vez.

```
void probar(int i) {
    if (estado[i] == HAMBRIENTO
        && estado[IZQUIERDO] != COMIENDO
        && estado[DERECHO]   != COMIENDO) {
        estado[i] = COMIENDO;
        up(&s[i]);
    }
}
```

Las tres condiciones del `if` son el corazón del algoritmo:

```
   estado[i] == HAMBRIENTO        →  ¿éste quiere comer? (si no, no hay nada que hacer)
   estado[IZQUIERDO] != COMIENDO  →  ¿su tenedor izquierdo está libre?
   estado[DERECHO]   != COMIENDO  →  ¿y el derecho?
```

Un detalle de implementación que se pregunta

> *"Observe que cada proceso ejecuta el procedimiento `filosofo` como su código principal, pero los demás procedimientos (`tomar_tenedores`, `poner_tenedores` y `probar`) son **procedimientos ordinarios y no procesos separados**."*

`probar` no es un proceso ni un hilo: es una función que ejecuta quien la llama. Cuando F3 termina de comer y llama a `probar(2)`, **el que corre ese código es F3**, actuando en nombre de F2. Si te preguntan "¿quién ejecuta `probar`?", la respuesta es **el filósofo que acaba de dejar los tenedores**.

🏷️ El cuadro de los cuatro intentos

	¿Interbloqueo?	¿Inanición?	Máx. comiendo	¿Sirve?
1. Todos el izquierdo	**sí** 🔒	—	2	**no**
2. Devolver y reintentar	no	**sí** (todos corren sin progresar)	2	**no**
3. Esperar al azar	no	improbable, no imposible	2	solo si tolerás riesgo
4. Mutex para la mesa	no	no	**1** ✗	correcto, **lento**
5. Dijkstra (fig. 2-46)	no	no	**2** ✓	**sí**

(Tanenbaum, cap. 2.5.1, págs. 163–166, fig. 2-46.)

🎮 Tres cosas para probar en el simulador

1. **Esquema 1** → botón 💥 **Forzar el interbloqueo**. Mirá que los cinco queden con 🔒 y leé el cartel rojo:

nadie consume CPU.

2. **Esquema 2 (el zurdo)** → el mismo botón. No se traba, y en el panel vas a ver que **queda un tenedor libre**: el que desarma la cadena.
3. **Esquema 4 (Dijkstra)** → hacé comer a **F0**, y después tocá **F2** y hacelo comer también. Vas a ver "Comiendo ahora: 2". Después intentá que coma **F1** (vecino de los dos): su `probar` falla, se bloquea en `down(s[1])`, y queda esperando hasta que **F0 o F2 llamen a `probar(1)`** al dejar los tenedores.

Preguntas de control

1) En el esquema de Dijkstra, F1 está **HAMBRIENTO y bloqueado** en `down(&s[1])` porque F0 estaba comiendo. F0 termina y llama a `probar(DERECHO)` = `probar(1)`.

Pero resulta que **F2 también está comiendo** en ese momento. ¿Qué pasa con F1? ¿Y **quién** va a tener que despertarlo, y **cuándo**?

2) Un compañero quiere "simplificar" `tomar_tenedores` y lo escribe así:

```
void tomar_tenedores(int i) {
    down(&mutex);
    estado[i] = HAMBRIENTO;
    probar(i);
    down(&s[i]);        /* ← lo movió ACÁ ADENTRO */
    up(&mutex);
}
```

Movió el `down(&s[i])` **dentro** de la región crítica. **¿Qué pasa la primera vez que un filósofo no consigue comer?** Describí la secuencia y decime cómo se llama el problema.

Las dos correctas, y la 1 impecable: **F0 llama a `probar(1)`, falla porque `estado[2] == COMIENDO`, y F1 queda bloqueado hasta que F2 termine y llame a `probar(IZQUIERDO)`**.

El detalle que hace que eso no sea un bug

> **El último de los dos vecinos en terminar es el que lo despierta.** Siempre hay alguien que lo va a hacer, porque los dos vecinos van a terminar en algún momento, y el que termine último ya no tiene al otro comiendo.

Y fijate **por qué el `probar(1)` fallido de F0 no pierde nada**:

```
F0 llama a probar(1) → falla → NO hace up(s[1])
Pero estado[1] SIGUE valiendo HAMBRIENTO en el arreglo.
→ La información de que F1 quiere comer NO se perdió.
→ Cuando F2 pregunte, la va a encontrar ahí.
```

🔑 **`estado[]` es la memoria que impide el "despertar perdido".** Con `sleep`/`wakeup` el aviso se evaporaba; acá el deseo de comer queda **escrito en una variable compartida**, así que cualquiera que pase después lo puede atender. Es la misma idea que el contador del semáforo, aplicada a un deseo en lugar de a una cantidad.

Pregunta 2 — ✓ y le ponemos la secuencia

Tu diagnóstico es correcto. La secuencia completa, que es lo que te van a pedir que escribas:

```
1. F1 hace down(&mutex)      →  mutex = 0, la región crítica es suya
2. estado[1] = HAMBRIENTO
3. probar(1) FALLA           →  s[1] sigue en 0
4. down(&s[1])               →  💥 F1 SE BLOQUEA... CON EL MUTEX TOMADO
5. F0 termina de comer y quiere llamar a poner_tenedores:
      down(&mutex)           →  mutex = 0  →  F0 SE BLOQUEA
6. F2 quiere lo mismo        →  F2 SE BLOQUEA
7. Los cinco se bloquean uno a uno en down(&mutex).

El up(&mutex) que los liberaría está en la LÍNEA SIGUIENTE
del código de F1... que nunca se va a ejecutar.
```

El nombre: *dormirse con el candado tomado*, y el resultado es un **interbloqueo total del sistema**. ✓

⚠ **Y la lección que vale para toda la materia**

```
El algoritmo de los filósofos estaba PERFECTO.
Lo único que se movió fue UNA LÍNEA de lugar.
Resultado: el sistema entero se cuelga.
```

Regla sin excepciones: dentro de una región crítica nunca se ejecuta una operación que pueda bloquear. Ni un `down` de otro semáforo, ni un `read` de disco, ni un `receive`, ni nada que pueda dormir al proceso.

🔗 Y mirá el patrón del tema completo:

Dónde está el `down` que bloquea	Qué pasa
Fuera del `up(&mutex)` (el libro)	solo se bloquea ese filósofo ✓
Dentro del mutex (la "simplificación")	se bloquea **todo el sistema** 🔒

Una línea de distancia entre un algoritmo correcto y un sistema muerto. Por eso este problema es el banco de pruebas de la sincronización.

Vamos al último problema clásico. Dejame traer el texto.

Tema 9.2 — Lectores y escritores

El planteo

Courtois y colaboradores, 1971. Modela **el acceso a una base de datos** — por ejemplo **un sistema de reservación de aerolíneas**, con muchos procesos que quieren leer y escribir.

La regla del problema:

```
|  VARIOS LECTORES a la vez      →  ✓ permitido              |
|     (nadie modifica nada, todos ven lo mismo)              |
|                                                            |
|  UN ESCRITOR escribiendo       →  ✗ NADIE más puede entrar  |
|     ni otro escritor, NI SIQUIERA UN LECTOR                |
└────────────────────────────────────────────────────────────┘
```

¿Por qué el lector tampoco puede entrar mientras alguien escribe? Porque leería datos a medio actualizar: la reserva descontada del vuelo pero todavía no cargada en el pasajero.

⚠ En qué se diferencia de todo lo anterior

	Exclusión mutua clásica	Lectores y escritores
¿Cuántos adentro?	**siempre 1**	**N lectores o 1 escritor**
Tipo de candado	uno solo	**dos tipos de acceso**

Esto no es exclusión mutua: es exclusión mutua ASIMÉTRICA. Y es justo lo que hacen las bases de datos reales con sus *shared locks* y *exclusive locks*.

La solución — figura 2-47

Las piezas:

```
semaforo mutex = 1;   /* controla el acceso a 'cl' */
semaforo bd = 1;      /* controla el acceso a la BASE DE DATOS */
int cl = 0;           /* cuántos procesos están leyendo o quieren leer */
```

⚠ **Dos semáforos con trabajos distintos.** Confundirlos es el error típico:

```
   mutex  →  protege UNA VARIABLE (el contador cl). Se toma por microsegundos.
   bd     →  protege EL RECURSO (la base de datos). Se puede tener tomado minutos.
```

El escritor — trivial

```
void escritor(void) {
    while (TRUE) {
        pensar_datos();          /* región NO crítica */
        down(&bd);               /* acceso exclusivo */
        escribir_base_de_datos();
        up(&bd);
    }
}
```

Es exclusión mutua pura. No sabe que los lectores existen.

El lector — acá está toda la gracia

```
void lector(void) {
    while (TRUE) {
        down(&mutex);            /* ① acceso exclusivo a cl */
        cl = cl + 1;             /* ② un lector más */
```

```
        if (cl == 1) down(&bd);  /* ③ ¡SÓLO EL PRIMERO toma la base de datos! */
        up(&mutex);              /* ④ libera cl */

        leer_base_de_datos();    /* ⑤ varios lectores están ACÁ a la vez */

        down(&mutex);            /* ⑥ */
        cl = cl - 1;             /* ⑦ un lector menos */
        if (cl == 0) up(&bd);    /* ⑧ ¡SÓLO EL ÚLTIMO la suelta! */
        up(&mutex);              /* ⑨ */
        usar_lectura_datos();    /* región NO crítica */
    }
}
```

🔑 El mecanismo, en una frase

El primer lector en entrar hace `down(&bd)`. Los siguientes sólo incrementan `cl`. A medida que salen, lo decrementan, y el último hace `up(&bd)`.

```
   cl=0  ──┐
           │  llega L1:  cl=1  →  cl==1  →  down(&bd)   🔒 la BD queda tomada
           │  llega L2:  cl=2  →  NO toca bd            ← solo suma
           │  llega L3:  cl=3  →  NO toca bd
           │
           │  sale  L2:  cl=2  →  NO toca bd
           │  sale  L1:  cl=1  →  NO toca bd
           │  sale  L3:  cl=0  →  cl==0  →  up(&bd)     🔓 recién ahora se libera
   cl=0  ──┘
```

El primero abre la puerta para todos y el último la cierra. Los del medio pasan gratis. Se lo llama el patrón del **molinete** (*turnstile*).

🔎 Y mirá la elegancia: `bd` **nunca se usa como contador**, siempre vale 0 o 1. El contador es `cl`, una variable común protegida por `mutex`. **Separaron "contar" de "bloquear"**, que son las dos cosas que el semáforo mezcla.

☆ La decisión oculta: esto favorece a los lectores

"La solución que se presenta aquí contiene en forma implícita una decisión sutil que vale la pena observar."

Seguí la película:

```
   1. L1 está leyendo.                        cl=1, bd tomado
   2. Llega L2 → no hay problema, entra.      cl=2
   3. Llega un ESCRITOR → down(&bd) → 🔒 SE BLOQUEA (bd está tomado)
   4. Llega L3 → ¿se fija si hay un escritor esperando?  ¡NO!
      Sólo hace cl=3. ENTRA TRANQUILAMENTE, pasándole por encima al escritor.
   5. Llega L4 → entra. L5 → entra. L6 → entra...
```

*"Mientras que haya un suministro continuo de lectores, todos entrarán tan pronto lleguen. **El escritor estará suspendido hasta que no haya un lector presente.**"*

El ejemplo del libro, verificado

> *"Si llega un nuevo lector, por decir **cada 2 segundos**, y cada lector requiere **5 segundos** para hacer su trabajo, **el escritor nunca entrará**."*

Lo simulé 200 segundos:

Llega un lector cada	Cada lectura dura	Huecos con `cl == 0`	¿El escritor entra?
2 s	5 s	**0**	**nunca** 🔒
4 s	5 s	**0**	**nunca** 🔒
5 s	5 s	**0**	**nunca** 🔒
6 s	5 s	33	**sí**, el primero en t=5 ✓
10 s	5 s	20	sí ✓

La condición exacta, que es la que te conviene saber:

```
intervalo de llegada  ≤  duración de cada lectura   →   INANICIÓN del escritor
intervalo de llegada  >  duración de cada lectura   →   el escritor entra
```

Si los lectores se superponen aunque sea por un instante, `cl` nunca llega a 0 y el `up(&bd)` nunca se ejecuta. El escritor no está interbloqueado — **está muerto de inanición**, esperando una condición que el sistema no le va a dar nunca.

🔗 Es el **mismo fenómeno** que vimos en prioridades y en el intento 2 de los filósofos, por tercera vez y en otro disfraz: *el algoritmo siempre prefiere a otro*.

El parche, y lo que cuesta

> *"Cuando llega un lector **y hay un escritor en espera**, el lector se suspende **detrás** del escritor, en vez de ser admitido de inmediato."*

```
Versión original:   L1 L2 [E espera] L3 L4 L5 ... E nunca
Versión parcheada:  L1 L2 [E espera] │ L3 L4 L5 van A LA COLA DETRÁS DE E
                    → terminan L1 y L2 → entra E → después L3, L4, L5
```

La garantía que da: un escritor espera a los lectores que **ya estaban activos cuando llegó**, pero **no a los que llegaron después**. Eso acota la espera y mata la inanición.

Lo que cuesta: *"logra una menor concurrencia y por ende, un menor rendimiento"*. L3, L4 y L5 podrían haber leído en paralelo con L1 y L2, y en cambio se quedan esperando.

```
Favorecer a los LECTORES    →  máximo rendimiento, pero el escritor puede morir de hambre.
Favorecer a los ESCRITORES  →  nadie muere de hambre, pero se pierde paralelismo.
```

No hay solución gratis: es un intercambio entre rendimiento y equidad. Y es la misma disyuntiva del mutex de los filósofos (candado grande = correcto pero lento).

Courtois y colaboradores también publicaron una solución que **da prioridad a los escritores**; el libro remite a la bibliografía para los detalles.

(Tanenbaum, cap. 2.5.2, págs. 166–168, fig. 2-47.)

Preguntas de control

1) Un compañero mira el código del lector y dice:

> *"El `if (cl == 1) down(&bd)` está al revés. Si cada lector hiciera su propio `down(&bd)`, sería más prolijo y más seguro."*

¿Qué pasaría si **cada** lector hiciera `down(&bd)` al entrar y `up(&bd)` al salir? Pensá qué valor tiene `bd` y qué le ocurre al **segundo** lector que llega.

2) En el código del lector, el `down(&bd)` del paso ③ está **dentro** de la región crítica protegida por `mutex` (entre el `down(&mutex)` y el `up(&mutex)`).

Acabamos de establecer la regla de que **nunca hay que bloquearse dentro de una región crítica**. ¿Está el libro rompiendo su propia regla? ¿Es esto un bug o hay algo que lo salva?

(Pista: ¿quién es el único lector que ejecuta ese `down(&bd)`, y qué puede haber tomado la `bd` en ese momento?)

Pregunta 1 — ✓ exacta

"Los demás quedarían bloqueados... dejando de lado la lectura en paralelo." Eso es precisamente lo que pasa:

```
L1 llega:  down(&bd)  →  bd = 0, entra a leer
L2 llega:  down(&bd)  →  bd ya vale 0  →  🔒 L2 SE BLOQUEA
L3 llega:  down(&bd)  →  🔒 se bloquea
```

Los lectores se serializan: entran de a uno. Y fijate el detalle que lo vuelve interesante:

> **La "simplificación" sería CORRECTA — nadie corrompe nada — pero destruiría la única razón de ser del problema.** Si los lectores no pueden leer en paralelo, no hay problema de lectores y escritores: hay exclusión mutua común y ya lo resolvimos en el tema 7.

🔗 Es **el mismo error que el intento 4 de los filósofos**: un candado demasiado grande. Correcto y lento.

Y la contracara: `cl` no es decoración. **Es lo que permite que N procesos compartan UN solo `down`.**

Pregunta 2 — ✓ en la conclusión, ✗ en el razonamiento

Acertaste que no está rompiendo nada. Pero *"solo lo duerme hasta que esté liberado"* es **describir** lo que pasa,

no explicar **por qué es seguro**. Y hace un rato vimos que exactamente eso colgaba el sistema de los filósofos. Entonces: ¿cuál es la diferencia?

Comparemos los dos casos

```
╔═ FILÓSOFOS (la "simplificación" que cuelga) ═══════════════╗
║  F1 se duerme en down(&s[1])  teniendo el mutex             ║
║  ¿Quién lo tiene que despertar?  F0 o F2, con probar(1)     ║
║  ¿Qué necesitan F0 y F2 para hacerlo?  ¡EL MUTEX!           ║
║        → que F1 tiene tomado → 🔒 INTERBLOQUEO              ║
╚═════════════════════════════════════════════════════════════╝

╔═ LECTORES Y ESCRITORES (el código del libro) ══════════════╗
║  L1 se duerme en down(&bd)  teniendo el mutex               ║
║  ¿Quién lo tiene que despertar?  el ESCRITOR, con up(&bd)   ║
║  ¿Qué necesita el escritor para hacerlo?  NADA.             ║
║        → mirá su código: nunca toca mutex                   ║
║        → el escritor termina, hace up(&bd), L1 despierta,   ║
║          hace up(&mutex) y se destraba todo  ✓              ║
╚═════════════════════════════════════════════════════════════╝
```

```
void escritor(void) {
    while (TRUE) {
        pensar_datos();
        down(&bd);                  ← solo bd
        escribir_base_de_datos();
        up(&bd);                    ← solo bd.  mutex no aparece NUNCA.
    }
}
```

🔑 La regla precisa

La que te di antes (*"nunca te bloquees en una región crítica"*) es una **regla de seguridad**: si la seguís, nunca te equivocás. Pero la condición exacta es ésta, y es la que te deja analizar código ajeno:

> **Dormirse con un candado tomado es un interbloqueo SOLO SI el que te tiene que despertar necesita ese mismo candado.**

```
El que me despierta NO necesita mi candado  →  seguro (aunque feo)
El que me despierta SÍ necesita mi candado  →  🔒 interbloqueo garantizado
```

Para el examen: usá la regla de seguridad al escribir código. Usá la regla precisa al **justificar** por qué un código dado se cuelga o no.

El precio que igualmente se paga

Que no sea un interbloqueo no significa que sea gratis. Mientras L1 duerme con el `mutex` tomado:

```
L2, L3, L4 ... llegan y hacen down(&mutex)  →  se bloquean en MUTEX
                                               (no en bd)
```

Se amontonan en el semáforo equivocado. No se pierde la corrección, se pierde un poco de concurrencia durante la escritura — que de todos modos no podían usar, porque la BD estaba tomada. **Por eso el libro se lo permite.**

🏁 Tema 9 cerrado

El libro trata **tres** problemas clásicos de IPC, y ya tenés los tres:

Problema	Qué modela	Qué enseña
Productor-consumidor (tema 7)	un búfer compartido de tamaño fijo	semáforos como contadores; el despertar perdido
Filósofos comelones	procesos que compiten por **varios recursos**	**interbloqueo** vs **inanición**; candado grande vs chico
Lectores y escritores	acceso a una **base de datos**	exclusión **asimétrica**; rendimiento vs equidad

Si el programa de tu cátedra pide el barbero dormilón, avisame: **no está en esta edición del Tanenbaum** (el libro trata estos tres y nada más), así que lo tendría que desarrollar como complemento marcado, citando la bibliografía del programa. No te lo voy a mezclar con el libro sin avisarte.

Nos queda el **tema 10: multiprocesadores**. Dejame ubicar las páginas.

Tema 10.1 — Hardware de multiprocesador

Mapa del tema

```
10. MULTIPROCESADORES  (cap. 8.1)
│
├── 10.1 HARDWARE          ¿cómo se conectan las CPUs a la memoria?
│        UMA / NUMA · bus · barras cruzadas · multinúcleo
│        coherencia de cachés
│
├── 10.2 TIPOS DE SO       ¿quién ejecuta el sistema operativo?
│        cada CPU su propio SO · maestro-esclavo · SMP ★
│
├── 10.3 SINCRONIZACIÓN    ¿sigue sirviendo TSL con varias CPUs?
│        bloqueo del bus · thrashing de caché · espera activa vs conmutar
│
└── 10.4 PLANIFICACIÓN     ahora hay DOS preguntas, no una
         tiempo compartido · afinidad · dos niveles
         espacio compartido · planificación por pandillas ★
```

Todo este tema reusa lo que ya sabés y le agrega una vuelta: la Unidad 1 (caché, buses), el tema 7 (TSL, regiones críticas) y el tema 8 (planificación) vuelven, pero con *varias* CPUs.

La definición

> **Multiprocesador con memoria compartida:** un sistema de cómputo en el que **dos o más CPUs**

comparten todo el acceso a una RAM común.

Un programa que corre en cualquiera de las CPUs ve un espacio normal de direcciones virtuales. **La única propiedad inusual:**

> *"La CPU puede escribir cierto valor en una palabra de memoria y después puede volver a leer esa palabra y obtener un valor distinto (tal vez porque otra CPU lo cambió)."*

🔑 **Y eso no es un defecto: es el canal de comunicación.** *"Una CPU escribe ciertos datos en la memoria y otra lee esos datos."*

🔑 Es la **condición de carrera del tema 7 elevada a categoría de hardware**. En un uniprocesador la carrera la causaba la interrupción de reloj; acá la causa **otra CPU que escribe de verdad al mismo tiempo**.

☆ UMA y NUMA — la distinción que se pregunta siempre

```
┌─ UMA  (Uniform Memory Access — Acceso Uniforme a la Memoria) ──────┐
│  CADA palabra de memoria se lee a la MISMA velocidad que           │
│  cualquier otra.  No importa dónde esté el dato.                   │
└────────────────────────────────────────────────────────────────────┘

┌─ NUMA (Non-Uniform Memory Access — Acceso NO Uniforme) ────────────┐
│  El acceso a la memoria LOCAL es más rápido que a la REMOTA.       │
│  Importa MUCHO dónde esté el dato.                                 │
└────────────────────────────────────────────────────────────────────┘
```

Las tres características de NUMA — memorizalas, son tres y el libro las numera

1. **Hay un solo espacio de direcciones** visible para todas las CPUs.
2. **El acceso a la memoria remota es mediante instrucciones `LOAD` y `STORE`** (no con mensajes).
3. **El acceso a la memoria remota es más lento** que a la local.

⚠ **La trampa**: los puntos 1 y 2 son los que la distinguen de una **multicomputadora**. En NUMA seguís programando con memoria compartida normal; lo único que cambia es la velocidad.

> *"Todos los programas UMA se ejecutarán sin cambios en las máquinas NUMA, pero el rendimiento será peor."*

NUMA no te obliga a reescribir el programa. Te castiga si lo escribiste mal.

Las dos variantes de NUMA

```
NC-NUMA  (No Cache NUMA)         →  sin caché: el costo remoto se ve siempre
CC-NUMA  (Cache-Coherent NUMA)   →  con cachés coherentes: se disimula
```

El CC-NUMA grande se construye con **multiprocesadores basados en directorios**: una base de datos en hardware que dice **dónde está cada línea de caché y en qué estado**. Tiene que responder *"en una fracción de un ciclo del bus"*, así que va en hardware dedicado carísimo.

🔑 **Y acá hay una frase que es perfecta para una pregunta de desarrollo:**

*"Observe que hasta un «multiprocesador con memoria compartida» realiza muchas operaciones de **paso de mensajes** de manera interna."*

La memoria compartida es una ilusión que el hardware mantiene mandando mensajes por abajo. Conectalo con el tema 7: arriba creés que escribís una variable; abajo viajan paquetes por la red de interconexión.

El problema del bus único, y los tres parches

```
(a) SIN CACHÉ
    CPU  CPU  CPU ──────┬────── Memoria compartida
                       │
                todas pelean por el MISMO bus

    2 o 3 CPUs  →  manejable
    32 o 64     →  "será imposible"
    → "el sistema estará totalmente limitado por el ancho de banda del bus,
       y la mayoría de las CPUs estarán inactivas la mayor parte del tiempo"
```

```
(b) CON CACHÉ POR CPU
    [CPU+caché] [CPU+caché] ──┬── Memoria compartida
    Muchas lecturas se sirven de la caché local
    → mucho menos tráfico en el bus → soporta más CPUs
    → PERO aparece el problema de la COHERENCIA
```

```
(c) CACHÉ + MEMORIA PRIVADA LOCAL
    Cada CPU tiene además memoria privada por un bus dedicado.
    El COMPILADOR pone ahí el código, las constantes, la pila
    y las variables locales. La memoria compartida queda SOLO
    para las variables compartidas.
    → "requiere de una cooperación activa por parte del compilador"
```

🔗 **(c) es la misma idea que las variables locales del tema 7**: lo que no se comparte no necesita protección *ni* bus. Acá eso se vuelve una decisión de arquitectura.

El protocolo de coherencia de cachés

La caché no trae palabras sueltas: trae **líneas de caché** de **32 o 64 bytes**. Y cada bloque está marcado:

```
SÓLO LECTURA     →  puede estar en VARIAS cachés a la vez
LECTURA-ESCRITURA →  NO puede estar en otras cachés
```

Cuando una CPU escribe una palabra que otras tienen en caché, **el hardware del bus detecta la escritura y avisa a todas las demás cachés**:

La otra caché tiene...	Qué hace
una copia **limpia** (igual a memoria)	**descarta** su copia
una copia **sucia** (modificada)	la **escribe de vuelta** a memoria, o se la pasa directo a la CPU que escribe

A este conjunto de reglas se le llama protocolo de coherencia de cachés.

⚠ **Guardate "línea de caché de 32 o 64 bytes"**, porque en el próximo bloque esa unidad produce un problema de rendimiento espectacular.

Más allá del bus: barras cruzadas

> *"Aun con la mejor caché, el uso de un solo bus limita el tamaño de un multiprocesador UMA a **16 o 32 CPUs** aproximadamente."*

Para más se usa el **conmutador (interruptor) de barras cruzadas**: una matriz de *n* CPUs × *k* memorias con un **punto de cruce** en cada intersección. Viene de las centrales telefónicas. Su defecto es el costo: **n × k puntos de cruce** crece al cuadrado.

(Tanenbaum, cap. 8.1.1, págs. 526–534, figs. 8-2 a 8-6.)

Preguntas de control

1) Un sistema tiene 2 CPUs, cada una con su caché, sobre un bus compartido. La CPU 1 tiene la variable `x` en su caché marcada **sólo lectura**, y la CPU 2 también la tiene.

Ahora **la CPU 2 escribe** en `x`. Describí qué pasa: **qué detecta el hardware, qué le pasa a la copia de la CPU 1, y por qué era necesario**.

2) Un compañero dice:

> *"NUMA y una multicomputadora son lo mismo: en las dos, acceder a la memoria de otro nodo cuesta más."*

¿En qué se equivoca? Usá **dos de las tres características de NUMA** para refutarlo, y decime qué cambia eso **para el programador**.

Pregunta 1 — ✕ y la diferencia importa

Dijiste que la copia de la CPU 1 **se actualiza** con el valor nuevo. El libro dice lo contrario:

> *"Si otras cachés tienen una copia «limpia»... **sólo tienen que descartar sus copias** y dejar que la CPU que va a escribir obtenga el bloque de la caché de la memoria antes de modificarla."*

```
✕ ACTUALIZAR:  la CPU 2 manda el valor nuevo a todas las cachés
✓ INVALIDAR:   la CPU 2 manda "tiren lo que tienen de esta línea"
```

La copia de la CPU 1 no se corrige: se tira a la basura. Si la CPU 1 vuelve a necesitar `x`, va a fallar en caché y la va a traer de memoria, ya actualizada.

¿Por qué invalidar y no actualizar? (esto es lo que se pregunta)

```
Si actualizara:  CADA escritura tendría que difundir el valor nuevo por el bus
```

```
                a todas las cachés. Y las escrituras son constantes.
                → el bus se saturaría, que es justo lo que la caché venía a evitar.

Al invalidar:   se manda UNA señal corta, una sola vez.
                Y si la CPU 1 nunca vuelve a leer x, no se pagó NADA.
```

Invalidar paga solo cuando el otro realmente necesita el dato. Actualizar paga siempre.

Y hay una segunda razón, que es la que cierra la lógica

Acordate de la regla de los bloques:

```
SÓLO LECTURA      →  puede estar en varias cachés
LECTURA-ESCRITURA →  NO puede estar en otras cachés
```

Para escribir, la CPU 2 necesita el bloque en modo **lectura-escritura**, y ese modo **exige que no esté en ninguna otra caché**. Así que invalidar no es una optimización: **es la condición para poder escribir.**

Lo que sí hay que pasar de mano en mano es la copia **sucia**: si la CPU 1 ya había modificado el bloque, no se puede tirar, hay que **escribirlo de vuelta a memoria** o pasárselo directo a la CPU 2.

```
copia LIMPIA  →  descartar          (no hay nada que perder)
copia SUCIA   →  escribir de vuelta (hay cambios que nadie más tiene)
```

Pregunta 2 — te la explico

La diferencia está en **la característica 1 y la 2** de NUMA:

```
╔═ NUMA ═══════════════════════════════════════════════════════╗
║  1. UN SOLO espacio de direcciones para todas las CPUs       ║
║  2. La memoria remota se accede con LOAD y STORE             ║
║  3. La memoria remota es más lenta                           ║
╚══════════════════════════════════════════════════════════════╝

╔═ MULTICOMPUTADORA ═══════════════════════════════════════════╗
║  1. Cada nodo tiene su PROPIA memoria privada.               ║
║     NO hay un espacio de direcciones común.                  ║
║  2. Para llegar a datos de otro nodo: send / receive EXPLÍCITOS ║
╚══════════════════════════════════════════════════════════════╝
```

Tu compañero solo mira la característica **3** (que la remota cuesta más), y es la única que comparten.

Qué cambia para el programador — ésta es la respuesta que vale los puntos

```
/* En NUMA */
x = x + 1;        /* la variable x puede estar en OTRO nodo.
                     El hardware la trae solo. Vos no te enterás.
                     Si está lejos, tarda más. Eso es TODO. */

/* En una multicomputadora */
x = x + 1;        /* ✗ si x vive en otro nodo, esto NO COMPILA
                     o modifica una x distinta. No hay x compartida. */
send(nodo2, pedido);        /* tenés que escribir ESTO */
```

```
receive(nodo2, &respuesta); /* y esto */
```

🔑 **En NUMA, ubicar mal un dato es un problema de RENDIMIENTO: el programa anda, más lento. En una multicomputadora, no hay dónde ubicarlo mal: o escribís el paso de mensajes, o el programa NO FUNCIONA.**

La mnemotecnia

```
NUMA              →  "misma casa, habitación lejana"
                      caminás más, pero llegás solo.

MULTICOMPUTADORA  →  "otra casa"
                      tenés que mandar una carta. No hay pasillo.
```

🔗 Y conectalo con el tema 7: **la multicomputadora es el único caso donde el paso de mensajes no es una opción sino la única posibilidad**, porque es el único mecanismo que no requiere memoria compartida. Ahí tiene todo el sentido lo que vimos: *"es el único mecanismo que sirve para procesos en máquinas distintas"*.

Tema 10.2 — Los tres tipos de SO multiprocesador

La pregunta que resuelven los tres es la misma: **¿quién ejecuta el código del sistema operativo?** Y el libro aclara que los tres *"se pueden aplicar de igual forma a los sistemas multinúcleo"* — no es historia antigua, es tu notebook.

① Cada CPU tiene su propio sistema operativo

```
┌─────────┬─────────┬─────────┬─────────┐
│ CPU 1   │ CPU 2   │ CPU 3   │ CPU 4   │
│ SO      │ SO      │ SO      │ SO      │   ← cuatro copias
│ privado │ privado │ privado │ privado │
└─────────┴─────────┴─────────┴─────────┘
     Memoria dividida ESTÁTICAMENTE
     [Datos 1][Datos 2][Datos 3][Datos 4][Código del SO compartido]
```

En efecto, las n CPUs operan como n computadoras independientes.

Los cuatro problemas, que el libro numera (y te los pueden pedir así):

```
1°  Cada llamada al sistema se atrapa y se maneja EN SU PROPIA CPU,
    con las tablas de ese SO.

2°  NO hay compartición de PROCESOS. Cada SO planifica sus propios procesos.
    → "Si un usuario se conecta a la CPU 1, todos sus procesos se ejecutan en la CPU 1."
    → 💥 La CPU 1 puede estar INACTIVA mientras la CPU 2 está saturada.

3°  NO hay compartición de PÁGINAS.
    → 💥 La CPU 1 puede tener páginas de sobra mientras la CPU 2 pagina sin parar,
```

```
         y no se las puede prestar: la asignación de memoria es FIJA.

4º  (y lo PEOR) Cada SO mantiene SU PROPIA caché de búfer de bloques de disco.
    → 💥 El mismo bloque puede estar SUCIO en varias cachés a la vez
         → resultados INCONSISTENTES. Datos corruptos.
    → La única forma de evitarlo es ELIMINAR las cachés de búfer,
       lo cual "daña el rendimiento en forma considerable".
```

⚠ **Fijate la escala del daño: 2º y 3º son desperdicio; el 4º es CORRUPCIÓN.** Por eso es el peor.

> *"Es muy raro que se utilice este modelo."* Se usó al principio, solo para portar rápido sistemas existentes.

② Maestro-esclavo

```
┌──────────┬──────────┬──────────┬──────────┐
│  CPU 1   │  CPU 2   │  CPU 3   │  CPU 4   │
│ MAESTRO  │ esclavo  │ esclavo  │ esclavo  │
│ ejecuta  │ procesos │ procesos │ procesos │
│  el SO   │ usuario  │ usuario  │ usuario  │
└──────────┴──────────┴──────────┴──────────┘
        Una sola copia del SO, solo en la CPU 1
        TODAS las llamadas al sistema se redirigen ahí
```

Resuelve los tres problemas del modelo 1 de un golpe, porque hay **una sola estructura de datos** de procesos listos:

```
✓ Cuando una CPU queda inactiva, le PIDE un proceso al maestro.
  → "nunca puede ocurrir que una CPU esté inactiva mientras otra esté sobrecargada"
✓ Las páginas se asignan dinámicamente entre todos los procesos.
✓ Hay UNA SOLA caché de búfer → nunca hay inconsistencias.
```

☆ Pero tiene un cuello de botella, y el libro lo cuantifica

> *"Si, por ejemplo, el **10% del tiempo** está manejando llamadas al sistema, entonces con **10 CPUs** el maestro casi se saturará y con **20 CPUs** estará completamente sobrecargado."*

```
cada CPU usa el 10 % del maestro
     × 10 CPUs  =  100 %  →  el maestro está al límite
     × 20 CPUs  =  200 %  →  imposible: cola de espera permanente
```

🔑 **La cuenta es simplísima y es muy preguntable:** `nº de CPUs × fracción de tiempo en el SO ≤ 1`. ¿Te suena? Es **la misma forma que la condición de planificabilidad** $\Sigma\ C_i/P_i \leq 1$ del tema 8.4. Un recurso único no puede comprometerse más del 100 %.

> *"Este modelo es simple y funciona para pequeños multiprocesadores, pero no para los grandes."*

③ SMP — Multiprocesador simétrico ★

```
┌──────────┬──────────┬──────────┬──────────┐
│  CPU 1   │  CPU 2   │  CPU 3   │  CPU 4   │
```

```
| ejecuta procesos de usuario Y EL SO        |  ← cualquiera, todas iguales
└──────────┴──────────┴──────────┴──────────┘
     UNA copia del SO en memoria + un juego de BLOQUEOS
```

Hay una copia del sistema operativo en memoria, pero CUALQUIER CPU puede ejecutarlo. Cuando se hace una llamada al sistema, **la CPU que la hizo** atrapa al kernel y la procesa ella misma.

Elimina la asimetría: no hay maestro, no hay cuello de botella, y equilibra procesos y memoria dinámicamente porque hay un solo juego de tablas.

💥 Pero introduce su propio problema

> *"Si dos o más CPUs están ejecutando código del sistema operativo al mismo tiempo, **se puede producir un desastre**."*

Los dos ejemplos del libro:

```
Dos CPUs seleccionan EL MISMO PROCESO para ejecutarlo.
Dos CPUs reclaman LA MISMA PÁGINA de memoria libre.
```

🔗 **Es la condición de carrera del tema 7, pero ahora dentro del propio kernel.** Y es peor: en un uniprocesador el kernel podía desactivar las interrupciones para protegerse. **Con otra CPU corriendo de verdad en paralelo, desactivar interrupciones no sirve de nada.**

La solución simple, y por qué es mala

> *"Asociar un mutex con el sistema operativo, convirtiendo **a todo el sistema en una gran región crítica**."*

```
Cualquier CPU que quiera ejecutar código del SO:  down(&mutex_gigante)
→ cualquier CPU puede ejecutar el SO, pero SÓLO UNA A LA VEZ
```

> *"Este modelo funciona, pero **casi tan mal como el modelo maestro-esclavo**. Con 20 CPUs habrá largas colas de CPUs esperando entrar."*

🟡 **¡Es el mutex de toda la mesa de los filósofos otra vez!** Correcto y lento, por la misma razón: **el candado es demasiado grande.**

La solución buena, y es la misma de siempre

> *"Muchas partes del sistema operativo son **independientes** unas de otras. No habría problema si una CPU ejecutara el planificador mientras otra manejara una llamada al sistema de archivos y una tercera procesara un fallo de página."*

```
→ Dividir el SO en VARIAS regiones críticas INDEPENDIENTES
  que no interactúan entre sí.
→ Cada una protegida por SU PROPIO mutex.
```

```
→ Sólo una CPU a la vez por región, pero VARIAS CPUs en el SO simultáneamente.
```

```
✗  [ ———————— UN mutex para todo el SO ———————— ]     1 CPU en el kernel

✓  [ mutex planificador ][ mutex archivos ][ mutex memoria ]   3 CPUs a la vez
```

Candado grande = correcto y lento. Candados chicos e independientes = correcto y rápido.

Y esto es **exactamente** el paso del intento 4 (mutex de la mesa) a la solución de Dijkstra (mutex sólo para `estado[]`) en los filósofos. **Es el mismo movimiento conceptual, aplicado al kernel de Linux.**

(Tanenbaum, cap. 8.1.2, págs. 534–538, figs. 8-7, 8-8 y 8-9.)

🏷 Cuadro de los tres modelos

	① SO por CPU	② Maestro-esclavo	③ SMP
Copias del SO	**n**	1 (en el maestro)	1 (compartida)
¿Quién atiende la llamada?	la CPU propia	**siempre el maestro**	**la CPU que la hizo**
¿Equilibra la carga?	**no** ✗	sí	sí
¿Comparte páginas?	**no** ✗	sí	sí
Caché de búfer	**n, inconsistentes** ✗	una	una
Escala a muchas CPUs	no	**no** (cuello de botella)	**sí** ✓
Problema propio	inconsistencias	el maestro se satura	carreras dentro del kernel

Mnemotecnia: ① *cada uno en su casa* → ② *todos le preguntan al jefe* → ③ *todos iguales, con candados*.

Preguntas de control

1) Un multiprocesador **maestro-esclavo** tiene 8 CPUs, y los procesos pasan **el 15 % de su tiempo** dentro del sistema operativo.

a) ¿Está saturado el maestro? Mostrá la cuenta. b) ¿Cuántas CPUs como máximo soporta este sistema? c) Si pasás a **SMP con un solo mutex global**, ¿mejora algo? ¿Por qué?

2) Un sistema usa el modelo ① (cada CPU con su propio SO). Un usuario conectado a la CPU 1 lanza 20 procesos pesados, mientras las CPUs 2, 3 y 4 están sin hacer nada.

a) ¿Se reparten los procesos? ¿Por qué? b) El usuario copia un archivo grande desde la CPU 1 y otro usuario lo lee desde la CPU 3. ¿Qué puede salir mal, y **cuál de los cuatro problemas** del libro es?

Corrección

1a) ✓ **Perfecto.** `8 × 0,15 = 1,20` → el maestro necesitaría **120 %** de una CPU. Imposible.

1b) ✓ **Correcto, y la cuenta exacta es ésta:**

```
n × 0,15 ≤ 1     →     n ≤ 1 / 0,15 = 6,67     →     n = 6 CPUs
```

```
6 CPUs →  90 %   ✓ anda, con poco margen
7 CPUs → 105 %   ✗ ya está saturado
```

🔑 Y notá la forma: `n × fracción ≤ 1`. **Es la misma estructura que $\Sigma\ C_i/P_i \le 1$** del tiempo real. Un recurso único no se puede comprometer más del 100 %. Si la ves así, no tenés que memorizar nada.

1c) ⚠ **Mitad y mitad — y la mitad que falla es importante.**

Dijiste *"la saturación no existe pero deberían esperar a su turno"*. Esas dos frases se contradicen: **si tienen que esperar su turno, la saturación es exactamente la misma.**

```
MAESTRO-ESCLAVO:   el trabajo de kernel se ejecuta en 1 CPU (la maestra)
SMP + 1 MUTEX:     el trabajo de kernel se ejecuta en 1 CPU A LA VEZ

→ En los dos casos el kernel está SERIALIZADO.
→ La demanda sigue siendo 120 %. La cuenta no cambió.
```

El libro lo dice casi con esas palabras:

> *"Este modelo funciona, pero **casi tan mal como el modelo maestro-esclavo**. Con 20 CPUs habrá largas colas de CPUs esperando entrar."*

¿Entonces qué SÍ mejora?

Dos cosas, y son reales:

```
✓ No hay una CPU condenada a ser maestra: cuando ninguna necesita el kernel,
  las 8 corren procesos de usuario. En maestro-esclavo la maestra está
  semi-dedicada al SO.
✓ Desaparece el viaje de ida y vuelta: la CPU que hace la llamada la atiende
  ella misma, no se la manda a otra y espera la respuesta.
```

Pero **el techo de trabajo de kernel por segundo es idéntico.** Y el arreglo de verdad ya lo dijimos: **partir el mutex gigante en varios mutexes independientes** (planificador, archivos, memoria). Ahí sí 3 CPUs pueden estar en el kernel a la vez y el 120 % se reparte.

> ⚠ **Trampa de examen:** "SMP resuelve el cuello de botella del maestro-esclavo" es **FALSO si el SO tiene un solo candado global**. Lo resuelve la **granularidad de los candados**, no el modelo SMP por sí solo.

2a) ✓ **Correcto.** No se reparten: *"Si un usuario se conecta a la CPU 1, todos sus procesos se ejecutan en la CPU 1"*. Las CPUs 2, 3 y 4 quedan **inactivas mientras la 1 está saturada**, y el SO de la CPU 1 no tiene forma de enterarse ni de pedir ayuda. La división de memoria y de procesos es **estática**.

2b) ⚠ Bien el síntoma, te falta el mecanismo. Dijiste "se leen cosas desactualizadas" — la dirección es correcta, pero la pregunta era **cuál de los cuatro** y **por qué**.

Es el **problema 4º: la caché de búfer de bloques de disco**. Y fijate el detalle que lo hace posible:

```
LA MEMORIA está dividida estáticamente  →  no se comparte
EL DISCO está COMPARTIDO                →  las 4 CPUs leen y escriben
                                            los MISMOS bloques
```

Ahí está la grieta: cada SO cachea en **su propia** memoria los bloques del disco **común**.

```
CPU 1: escribe el bloque 7   →  queda SUCIO en la caché de búfer del SO 1
CPU 3: lee el bloque 7       →  va al DISCO, trae la versión VIEJA
                                (la CPU 3 no sabe que existe la caché del SO 1)
CPU 3: modifica y escribe    →  💥 se pisan: se perdió la escritura de la CPU 1
```

> **El mismo bloque está sucio en dos cachés a la vez → resultados inconsistentes.**

Y por eso el libro remata que la única salida es **eliminar las cachés de búfer**, *"lo cual daña el rendimiento en forma considerable"* — o sea, el remedio es casi tan malo como la enfermedad. **Ése es el motivo real de que este modelo esté abandonado.**

🔗 Y es la **condición de carrera del tema 7 otra vez** (dos escritores sobre el mismo dato sin exclusión mutua), pero ahora el dato compartido es un **bloque de disco** y los que compiten son **sistemas operativos enteros**.

Tema 10.3 — Sincronización de multiprocesadores

Por qué el truco del uniprocesador ya no sirve

```
UNIPROCESADOR:      el kernel deshabilita las interrupciones
                    → nadie puede entrometerse  ✓

MULTIPROCESADOR:    "deshabilitar las interrupciones sólo afecta a esa CPU"
                    → las demás CPUs SE SIGUEN EJECUTANDO
                    → y pueden tocar la misma tabla crítica  💥
```

🟤 **Esto hay que tenerlo clavado**, porque en la Unidad 2 aprendiste que desactivar interrupciones es una forma (fea, pero válida) de lograr exclusión mutua. **En un multiprocesador NO LO ES.** Las interrupciones son un recurso *por CPU*; el paralelismo es real, no simulado.

Entonces hace falta **un protocolo de mutex que todas las CPUs respeten**, y su corazón es una instrucción atómica: **TSL**.

💥 TSL también se rompe, si el bus no se bloquea (fig. 8-10)

Acordate de qué hace TSL: **lee** una palabra a un registro **y escribe un 1** en ella. Son **dos ciclos de bus**.

```
Palabra 1000 (el candado) = 0

paso 1 | CPU 1  LEE   → obtiene 0   ("está libre, es mío")
paso 2 | CPU 2  LEE   → obtiene 0   ("está libre, es mío")   ← ¡se metió en el medio!
paso 3 | CPU 1  ESCRIBE 1
paso 4 | CPU 2  ESCRIBE 1

→ las DOS recibieron 0 de su TSL
→ las DOS entran a la región crítica
→ 💥 falla la exclusión mutua
```

🔑 **La atomicidad de TSL en un uniprocesador venía de que la instrucción no se puede partir por una interrupción. Eso acá no alcanza**: nadie interrumpió a la CPU 1, simplemente **otra CPU se metió entre sus dos ciclos de bus**. La indivisibilidad tiene que ser *del bus*, no *de la instrucción*.

La solución

> *"La instrucción TSL debe primero **bloquear el bus**, para evitar que otras CPUs lo utilicen; después debe realizar ambos accesos a la memoria, y luego **desbloquear el bus**."*

```
TSL  =  [ bloquear bus ] → [ LEER ] → [ ESCRIBIR 1 ] → [ desbloquear bus ]
                           ── nadie más puede usar el bus acá ──
```

Se hace **declarando una línea especial del bus** (ponerla en 1 lógico) hasta terminar los dos ciclos.

📌 **Dato de examen:** esto *exige hardware*. Los buses modernos lo tienen; **los primeros no**, y en esas máquinas TSL no se podía implementar correctamente. **Por eso se inventó el protocolo de Peterson: para sincronizar enteramente por software** (Peterson, 1981).

🩹 ¡Ahora cierra Peterson del tema 7! No era un ejercicio académico: era **la única opción en hardware sin bus bloqueable**.

El segundo problema: TSL destroza las cachés

TSL funciona, pero es un **bloqueo de giro** (*spin lock*): la CPU evalúa el candado en un ciclo estrecho, lo más rápido que pueda.

> *"No sólo se desperdicia por completo el tiempo de la CPU que hace la petición, sino que también se puede imponer **una carga masiva en el bus o la memoria, reduciendo considerablemente a todas las demás CPUs** que tratan de realizar su trabajo normal."*

⚠ **Y acá viene la trampa fina.** Uno diría: "bueno, la CPU gira sobre *su copia en caché*, así que no molesta a nadie". **Falso, y el motivo es hermoso:**

```
Las cachés trabajan en BLOQUES de 32 o 64 bytes.

TSL es una ESCRITURA  →  necesita el bloque en modo EXCLUSIVO
                      →  invalida la copia del DUEÑO del candado
                      →  el dueño toca una palabra VECINA (en el mismo bloque)
                      →  el bloque vuelve a viajar a la máquina del dueño
```

```
                        →  la otra CPU hace TSL otra vez...
```

El bloque de caché se transporta CONSTANTEMENTE entre el dueño del candado y el que lo pide → MÁS tráfico de bus que si cada uno leyera la palabra suelta.

Eso es el **thrashing de caché**. 🔑 Es **compartición falsa**: las dos CPUs no pelean por el mismo dato, pelean por el mismo **bloque**, porque el candado y las variables del dueño cayeron juntos.

Los tres arreglos, en orden de calidad

① Lectura pura antes del TSL

```
while (TRUE) {
    while (candado != 0) ;       /* 1. SONDEO con LECTURAS puras  */
    if (TSL(&candado) == 0)      /* 2. sólo ahora intento tomarlo */
        break;
}
```

*"La mayoría de los sondeos son ahora **lecturas** en vez de escrituras"* → todas las CPUs pueden tener el bloque en **sólo lectura compartido** → **cero transferencias** mientras el candado esté ocupado. Cuando el dueño libera, **esa** escritura invalida todo y recién ahí se recarga.

⚠ **La pregunta que te van a hacer:** *"si dos CPUs ven el candado libre al mismo tiempo, ¿no es una condición de carrera?"* → **NO.**

*"La verdadera adquisición la realiza la instrucción TSL, y esta instrucción es **atómica**. Sólo una de estas operaciones tendrá éxito. **El éxito en la lectura pura es tan sólo una sugerencia** de que sería un buen momento de adquirir el bloqueo, pero no constituye una garantía."*

🔑 **La lectura pura es una optimización, no el mecanismo de exclusión.** El mecanismo sigue siendo TSL. Y "no importa quién gane" — lo único que importa es que gane **uno solo**.

② Backoff exponencial binario (el de Ethernet)

```
intento 1 → espero  1 instrucción
intento 2 → espero  2
intento 3 → espero  4
intento 4 → espero  8  ...  hasta un MÁXIMO
```

```
máximo BAJO  →  respuesta rápida al liberarse, pero más thrashing de caché
máximo ALTO  →  menos thrashing, pero tarda en darse cuenta de que se liberó
```

Se puede combinar con ① o usar solo. 🔑 **Es el mismo backoff del azar en los filósofos comelones** — y el mismo de Ethernet. La idea reaparece en todos lados: *si choqué, espero cada vez más antes de reintentar*.

③ Un candado privado por CPU — Mellor-Crummey y Scott, 1991 (fig. 8-11) ★

Es la mejor, y resuelve el thrashing de raíz:

```
La CPU que no puede adquirir el candado:
   1. se asigna UNA variable de bloqueo PROPIA
      (en un bloque de caché NO UTILIZADO, para evitar conflictos)
   2. se encola al FINAL de una lista de CPUs que esperan

CPU1 ——— CPU2 ——— CPU3 ——— CPU4
(tiene    (gira     (gira     (gira
 el real)  en SU     en SU     en SU
           privado)  privado)  privado)

Cuando la CPU 1 sale de la región crítica:
   libera el candado PRIVADO de la CPU 2  →  la CPU 2 entra
   la CPU 2 al salir libera el de la CPU 3  →  y así sucesivamente
```

Por qué es mejor, en dos puntos que valen puntos:

```
✓ Cada CPU gira sobre SU PROPIA variable, en SU PROPIA caché
  → CERO tráfico de bus mientras esperan. Se murió el thrashing.
✓ Es una COLA FIFO  →  es LIBRE DE INANICIÓN
```

🔑 **Eso último es enorme.** El spin lock con TSL **no garantiza nada**: una CPU desafortunada puede girar para siempre. Con la lista, cada uno tiene su turno asegurado. 🔗 Es **lo mismo que la cola de bloqueados del semáforo** (tema 7): convertir una espera anárquica en una **cola ordenada**.

🏷️ Resumen de los cuatro

	Tráfico de bus	¿Sin inanición?	Complejidad
TSL puro (spin)	**masivo** + thrashing	✗ no	trivial
Lectura pura + TSL	bajo mientras esté ocupado	✗ no	trivial
Backoff exponencial	regulable	✗ no	baja
Candado privado (MCS)	**casi nulo** ✓	✓ **sí**	alta

Espera activa contra conmutación

La pregunta: la CPU quiere un mutex que está tomado. ¿**Gira** esperándolo, o **conmuta** a otro hilo?

Primero: a veces NO hay opción

> *"Suponga que cierta CPU está inactiva y necesita acceder a la **lista compartida de procesos listos** para elegir un proceso y ejecutarlo. Si la lista de procesos listos está bloqueada, la CPU no puede sólo decidir suspender lo que está haciendo y ejecutar otro proceso, **ya que para ello tendría que leer la lista de procesos listos**."*

```
Para conmutar a otro hilo necesito leer la lista de listos.
La lista de listos está bloqueada.
```

```
→ No puedo conmutar PARA PODER conmutar.
→ ESTOY OBLIGADO a girar.
```

🔥 **Esto es casi un acertijo y es carne de examen.** El candado de la lista de listos es especial: **es el único que no se puede esperar durmiendo**, porque dormir requiere tomarlo. En cambio, si el candado es el de la **caché de búfer**, sí se puede conmutar.

🔑 Y es **exactamente el criterio que ya aprendimos**: *dormirse con un candado tomado es un problema sólo si el que te tiene que mover necesita ese mismo candado*. Acá el planificador **es** quien te tiene que mover, y necesita **ese** candado.

⚠ **Y ojo con esto:** *"esta cuestión no ocurre en un uniprocesador"*. En un uniprocesador la espera activa **nunca** tiene sentido: **no hay otra CPU que pueda liberar el candado** mientras vos giras. Si fallás al adquirirlo, te bloqueás y punto.

El cálculo de la decisión

Las dos opciones cuestan:

```
GIRAR      →  ciclos tirados evaluando un candado. Trabajo improductivo puro.

CONMUTAR   →  guardar el estado del hilo
            +  tomar el candado de la lista de listos
            +  elegir un hilo, cargar su estado, arrancarlo
            +  💥 la caché queda llena de BLOQUES INCORRECTOS → fallos de caché
            +  💥 fallos de TLB
            +  y después el cambio DE VUELTA, con más fallos de caché
```

🔑 **El costo escondido no son los dos cambios de contexto: es que la caché y la TLB quedan frías.** 🔑 Y acá vuelve la **afinidad de caché**, que es justo lo que viene en el tema 10.4.

El ejemplo numérico del libro

```
Costo de conmutar = 1 ms (ir) + 1 ms (volver) = 2 ms

Caso A: el mutex se tiene 50 µs = 0,05 ms
        girar 0,05 ms   vs   conmutar 2 ms      →  🏆 GIRAR (40× mejor)

Caso B: el mutex se tiene 10 ms
        girar 10 ms     vs   conmutar 2 ms      →  🏆 CONMUTAR (5× mejor)
```

> **Regla: girá si la espera esperada es menor que el costo de los dos cambios de contexto.**

El problema real: *"las regiones críticas pueden variar de manera considerable en cuanto a su duración"*. No sabés de antemano cuál caso te tocó.

Los tres diseños, y cómo se mide quién gana

```
① girar siempre
② conmutar siempre
③ decidir CADA VEZ que te encontrás un mutex bloqueado
```

Para saber qué tan bueno es un algoritmo, se usa un **algoritmo de retrospección**: se rastrea toda la actividad, y **después, fuera de línea**, se calcula cuál habría sido la decisión óptima en cada caso. Eso da el **punto de referencia** contra el que se miden los algoritmos reales.

🔑 **Es idéntico a SJF/Óptimo**: un algoritmo que necesita el futuro, irrealizable, que sirve **sólo como cota de comparación**. 🔑 El mismo papel que juega *el algoritmo óptimo* en reemplazo de páginas (Unidad 3).

Y la solución que gana: ¡envejecimiento otra vez!

El modelo: el hilo **gira hasta un umbral**, y si lo pasa, conmuta. El umbral puede ser **fijo** (= el costo conocido de ir y volver) o **dinámico**.

> *"Los mejores resultados se obtienen cuando el sistema **lleva el rastro de los últimos tiempos de espera activa observados**, y asume que éste será similar a los anteriores."*

```
Conmutar cuesta 1 ms  →  umbral de giro = 2 ms

Si las 3 veces anteriores esperó 200 µs en promedio
     →  GIRA hasta 2 ms antes de conmutar    (apuesta a que será cortito)

Si las 3 veces anteriores giró los 2 ms COMPLETOS
     →  CONMUTA DE INMEDIATO, sin girar nada  (ya sabe que es largo)
```

🔥 **Esto es el promedio exponencial del tema 8.5, aplicado a candados en vez de ráfagas de CPU.** *"Asume que éste será similar a los anteriores"* = la misma apuesta que `estimación = a·vieja + (1−a)·real`. Si te preguntan "¿dónde más aparece la idea de predecir el futuro con el pasado reciente?", la respuesta ahora tiene **tres** lugares: estimar ráfagas, envejecer prioridades, y **decidir si girar o conmutar**.

(Tanenbaum, cap. 8.1.3, págs. 538–542, figs. 8-10 y 8-11.)

Preguntas de control

1) Un programador escribe este spin lock para un multiprocesador:

```
while (TSL(&candado) != 0) ;   /* giro hasta conseguirlo */
/* región crítica */
candado = 0;
```

Funciona correctamente (nunca entran dos CPUs a la vez), pero cuando lo corren con 8 CPUs **todo el sistema se vuelve lentísimo, incluso las CPUs que NO usan ese candado**.

a) ¿Por qué se frenan las CPUs que **no tienen nada que ver** con ese candado? b) ¿Qué agregarías para arreglarlo **sin cambiar el algoritmo**? Mostrá el código. c) ¿Ese arreglo elimina el riesgo de que **dos CPUs vean el candado libre a la vez**? Si no, ¿por qué no es un problema?

2) Un sistema SMP tiene un candado para la **caché de búfer del sistema de archivos** y otro para la **lista de procesos listos**. Conmutar de hilo cuesta **1,5 ms** de ida y **1,5 ms** de vuelta.

a) Una CPU encuentra ocupado el candado de la **caché de búfer**, y el sistema midió que ese candado se tiene

por 400 µs en promedio. ¿Gira o conmuta? Hacé la cuenta. b) La misma CPU encuentra ocupado el candado de la **lista de procesos listos**, y mide que se tiene **por 20 ms** en promedio. Por la cuenta debería conmutar. **¿Puede?** ¿Por qué?

Las respuestas del 10.3 — todas bien

Repasé las cinco y **están correctas**: la saturación del bus por *thrashing*, el código de la lectura pura, el argumento de que la carrera no importa porque TSL es atómico, y las dos cuentas de espera activa (400 µs < 3000 µs → girar; y la paradoja de la lista de listos). Nada que corregir ahí.

Pero revisando el resto del material encontré **tres cosas que te van a costar puntos si las repetís así en el examen**. Las marco y seguimos:

🟡 1. La lotería contra prioridades

La explicación que trajiste dice que no se puede transferir prioridad porque *"el Proceso B con prioridad media 15 se metería en el medio"*. **Eso está mal**: confunde los **boletos** (5, 15, 30) con **números de prioridad**. B no tiene prioridad 15, tiene 15 boletos.

Las dos razones reales son las que ya trabajamos:

```
① Una prioridad es un PUESTO (ordinal), no una CANTIDAD.
   No existe "regalarle 20 puntos de prioridad": sos 3° o sos 7°.
   Los boletos son una cantidad DIVISIBLE y TRANSFERIBLE.

② La prioridad la fija EL SISTEMA, no el proceso.
   Si un proceso pudiera subirse la prioridad solo, el esquema se cae.
   Los boletos, en cambio, son tuyos y los regalás a quien quieras.
```

🟡 2. El ejercicio de prioridades dinámicas usa la convención opuesta a tu cátedra

Ese Gantt está armado con **"número más grande = más prioritario"** y con una regla de aging inventada (±1 por tic) que **no está en el Tanenbaum**.

```
Tu cátedra:  1 = MÁS prioritario
Ese ejemplo: 5 = MÁS prioritario
```

El mecanismo está bien explicado, pero ⚠ **en el examen declará siempre tu convención en la primera línea de la respuesta.** Lo que nunca cambia: *poca CPU → favorecer; mucha CPU → castigar*.

🟢 3. Lectores y escritores: la conclusión es correcta, pero falta la regla

Se concluyó bien que el `down(&bd)` adentro del `mutex` **no es un bug**. Pero el motivo quedó en "funciona igual". El motivo preciso es el que armamos:

> **Dormirse con un candado tomado es un interbloqueo SÓLO SI el que te tiene que despertar necesita ESE MISMO candado.**

```
El escritor NUNCA toca 'mutex'  →  puede hacer up(&bd) aunque mutex esté cerrado
                                →  despierta al primer lector  →  SEGURO ✓
```

```
Los filósofos F0 y F2 SÍ necesitan el mutex para llamar a probar(1)
                                → si F1 se duerme con el mutex, nadie lo despierta
                                → INTERBLOQUEO ✖
```

Esa frase es la que te resuelve cualquier ejercicio de este tipo. Memorizala tal cual.

Tema 10.4 — Planificación de multiprocesadores

☆ Lo primero, y es lo que más se pregunta: ahora hay DOS dimensiones

```
╔═ UNIPROCESADOR ═══════════════════════════════════════╗
║  UNA pregunta:   ¿cuál hilo ejecuto a continuación?   ║
╚═══════════════════════════════════════════════════════╝

╔═ MULTIPROCESADOR ═════════════════════════════════════╗
║  DOS preguntas:  ¿cuál hilo ejecuto?                  ║
║                  ¿y EN CUÁL CPU lo ejecuto? ★         ║
╚═══════════════════════════════════════════════════════╝
```

> *"Esta dimensión adicional complica de manera considerable la planificación en los multiprocesadores."*

🔑 **Si te preguntan "¿qué cambia en la planificación al pasar a un multiprocesador?", ésa es la respuesta en una línea.** Todo el resto del tema son consecuencias de esa segunda pregunta.

¿Y qué se planifica: procesos o hilos?

Esto ya lo sabés del tema 8.6, y acá se aplica directo:

```
Hilos de USUARIO  →  el kernel ni sabe que existen
                  →  "es muy difícil que pueda planificarlos"
                  →  se planifica POR PROCESO, como siempre

Hilos de KERNEL   →  el kernel conoce todos los hilos
                  →  "el kernel selecciona un hilo, y el proceso al que
                      pertenece sólo tiene un pequeño papel (o ninguno)"
```

El segundo factor: ¿los hilos están relacionados o no?

Y de acá salen **las dos familias de algoritmos** del tema:

```
HILOS NO RELACIONADOS          →  TIEMPO COMPARTIDO
usuarios independientes            (reparto en el TIEMPO)
lanzan procesos independientes

HILOS RELACIONADOS             →  ESPACIO COMPARTIDO
trabajan juntos en la misma        (reparto en el ESPACIO = las CPUs)
aplicación y se comunican
```

El ejemplo del libro para el segundo caso es `make` paralelo, y la justificación es muy buena:

*"Si se necesitan 10 compilaciones, **no tiene caso programar 9 de ellas para que se ejecuten de inmediato y dejar la última hasta mucho después**, ya que el usuario no percibirá el trabajo como terminado sino hasta que termine la última compilación."*

🔑 **Cuando el resultado depende del ÚLTIMO hilo, optimizar el promedio no sirve: hay que optimizar el máximo.** 🔗 Es exactamente la lógica de la **barrera** del tema 7: el grupo avanza al ritmo del más lento.

A) Tiempo compartido — una sola cola global (fig. 8-12)

```
UNA SOLA estructura de datos para todo el sistema:
un conjunto de listas de hilos listos, por prioridad.

prioridad 7 | A
prioridad 5 | B  C
prioridad 4 | D  E
prioridad 2 | F  G  H  I
prioridad 1 | J  K  L
prioridad 0 | M  N
            └──────────────────  14 hilos esperando

┌────┬────┬────┬────┐
| 0  | 1  | 2  | 3  |
| 4  | 5  | 6  | 7  |   16 CPUs, todas ocupadas
| 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 |
└────┴────┴────┴────┘
```

La secuencia del libro, paso a paso:

```
(a) las 16 CPUs ocupadas, 14 hilos en cola

(b) la CPU 4 termina su trabajo (o su hilo se bloquea)
    → BLOQUEA LAS COLAS DE PLANIFICACIÓN
    → toma el hilo de mayor prioridad: A

(c) la CPU 12 queda inactiva
    → toma el siguiente: B
```

⚠ **Fijate el "bloquea las colas":** la cola global es una **región crítica compartida entre 16 CPUs**. 🔗 Es *literalmente* el candado de la lista de procesos listos del tema 10.3 — el único sobre el que **hay que girar obligatoriamente**.

Lo bueno

```
✓ Las CPUs se comparten en el tiempo, igual que en un uniprocesador.
✓ BALANCE AUTOMÁTICO DE CARGA:
  "no puede haber una CPU inactiva mientras otras están sobrecargadas"
✓ "es muy simple de implementar con eficiencia"
```

Las dos desventajas — el libro dice "dos", contalas así

```
1. CONTENCIÓN por la estructura de datos, que crece con el nº de CPUs.
2. La SOBRECARGA del cambio de contexto cuando un hilo se bloquea por E/S.
```

💥 La anomalía del spin lock — y esto es carne de examen

> *"Suponga que el hilo contiene un **bloqueo de espera activa** al momento en que **expira su quantum**. Las otras CPUs que esperan a que se libere el bloqueo **sólo desperdician su tiempo en una espera activa hasta que se vuelve a programar ese hilo** y se libera el bloqueo."*

```
CPU 0 | hilo A, con un SPIN LOCK tomado ——— 💥 fin de quantum → A sale
CPU 1 | gira esperando el candado de A ·············· sigue girando
CPU 2 | gira esperando el candado de A ·············· sigue girando
CPU 3 | gira esperando el candado de A ·············· sigue girando
         ↑
         A no está corriendo, así que NO PUEDE SOLTARLO.
         Tres CPUs quemándose al 100 % por nada.
```

🟤 **¿Te suena?** Es **la inversión de prioridades** del tema 8, pero ahora con tres CPUs reales girando. Y el libro marca por qué no pasaba antes:

> *"En un uniprocesador, los bloqueos de espera activa se utilizan raras veces, por lo que si un proceso se suspende mientras contiene un mutex, y otro hilo trata de adquirirlo, **se bloqueará de inmediato**, por lo que no se desperdicia mucho tiempo."*

La solución: planificación inteligente (Zahorjan, 1991) ★

```
1. El hilo que adquiere un spin lock  →  ESTABLECE UNA BANDERA
                                         a nivel de proceso
2. Al liberar el bloqueo              →  BORRA la bandera
3. El planificador NO DETIENE a un hilo que tenga la bandera puesta:
   "le da un poco más de tiempo para que complete su región crítica
    y libere el bloqueo"
```

🔥 **Y acá se cierra algo grande.** En el tema 8 el libro decía:

> *"el planificador comúnmente **no tiene la información necesaria** para tomar en cuenta esta dependencia"*

La bandera ES esa información. El planificador por fin sabe quién tiene el candado, y no lo desaloja. 🩹 **Es el primo pobre de la herencia de prioridades**: en vez de subirle la prioridad al que tiene el candado, simplemente *no se lo saca*. Mismo problema, solución más barata.

Afinidad de caché y planificación de dos niveles

> *"Aunque todas las CPUs son iguales, **ciertas CPUs son más iguales que otras**."*

```
El hilo A corrió mucho tiempo en la CPU k
→  la caché de k está LLENA de bloques de A
```

```
→  si A se vuelve a ejecutar pronto, conviene que sea EN k
→  más aciertos de caché  +  la TLB ya tiene las páginas correctas
                              (menos fallas de TLB)
```

Eso es **planificación por afinidad** (Vaswani y Zahorjan, 1991): *"esforzarse en serio por hacer que un hilo se ejecute en la misma CPU en que se ejecutó la última vez"*.

🔗 Ya la vimos dos veces: en **8.6** (el kernel prefiere un hilo del mismo proceso) y en **10.3** (la caché fría es el costo escondido de conmutar). **Es la tercera aparición: ya es un tema propio, no un detalle.**

☆ Planificación de dos niveles — cómo se implementa la afinidad

```
╔═ NIVEL SUPERIOR ═══════════════════════════════════════════════╗
║  Al CREAR el hilo, se lo ASIGNA A UNA CPU                      ║
║  (por ejemplo, a la que tenga menor carga en ese momento).     ║
║  → cada CPU adquiere SU PROPIA COLECCIÓN de hilos              ║
╚════════════════════════════════════════════════════════════════╝
                              ↓
╔═ NIVEL INFERIOR ═══════════════════════════════════════════════╗
║  Cada CPU planifica SUS hilos POR SEPARADO,                    ║
║  con prioridades o cualquier otro algoritmo.                   ║
╚════════════════════════════════════════════════════════════════╝

⚠  Y la válvula de escape:
   "si una CPU no tiene hilos qué ejecutar, OBTIENE UNO DE OTRA CPU
    en vez de quedar inactiva"   ← robo de trabajo (work stealing)
```

```
UNA COLA GLOBAL            vs      UNA COLA POR CPU
_____________________                ____________________

[A B C D E F G H...]                 CPU0: [A D G]   CPU1: [B E H]
       ↑                             CPU2: [C F]     CPU3: [I J]
16 CPUs peleando                     cada una en su casa
por el mismo candado                 candados independientes
```

📌 Los TRES beneficios — el libro los numera, aprendételos así

```
1°  Distribuye la carga de manera casi uniforme sobre las CPUs.
2°  Aprovecha la AFINIDAD DE CACHÉ siempre que sea posible.
3°  Al dar a cada CPU su propia lista, MINIMIZA LA CONTENCIÓN por las listas
    ("es muy poco frecuente que una CPU intente usar la lista de otra").
```

🔑 **El 3º es el mismo movimiento de siempre: partir un candado grande en candados chicos.** Igual que el mutex global del SMP → mutexes por subsistema, igual que el mutex de la mesa → el `estado[]` de Dijkstra. **Ya es un patrón, no una coincidencia.**

B) Espacio compartido (fig. 8-13)

> **Planificar varios hilos al mismo tiempo, esparcidos sobre varias CPUs.**

El algoritmo más simple:

```
Se crea un grupo de N hilos relacionados.

¿Hay N CPUs libres?
   SÍ  →  cada hilo recibe SU PROPIA CPU DEDICADA (¡SIN multiprogramación!)
          y todos arrancan.
   NO  →  NINGUNO arranca, hasta que haya suficientes CPUs.

Cada hilo se aferra a su CPU HASTA QUE TERMINA.
Al terminar, la CPU vuelve a la reserva.
```

```
32 CPUs repartidas en particiones:

┌──────┬──────────┬──────┬──────────────────┬─────┐
│  4   │    6     │  8   │       12         │ 2   │
│ CPUs │   CPUs   │ CPUs │       CPUs       │libres│
└──────┴──────────┴──────┴──────────────────┴─────┘

El número y el tamaño de las particiones CAMBIA con el tiempo.
```

💥 El defecto, y es gordo

> *"Si un hilo se bloquea en espera de una operación de E/S, **de todas formas retiene a la CPU**, que simplemente **se queda inactiva** hasta que el hilo se despierta."*

```
✓ Elimina la multiprogramación → ELIMINA la sobrecarga de cambio de contexto
✗ Pero una CPU bloqueada por E/S queda TIRADA, sin hacer nada
```

Es el intercambio exacto: ganás en sobrecarga, perdés en utilización.

📌 El dato de examen: ¿qué algoritmo se usa para decidir?

```
El análogo de SJF en multiprocesador:
elegir el hilo cuyo  (cuenta de CPUs × tiempo de ejecución)  sea MENOR

→ "en la práctica esta información raras veces está disponible"
→ "los estudios han demostrado que, en la práctica,
   ES DIFÍCIL VENCER AL ALGORITMO PRIMERO EN LLEGAR, PRIMERO EN SER ATENDIDO"
   (Krueger y colaboradores, 1994)
```

🔥 **Esto es buenísimo y es muy preguntable: FCFS, el algoritmo más tonto de todos, gana en la práctica.**
🔗 Y el motivo es el mismo de SJF en el tema 8: **el algoritmo óptimo necesita un dato del futuro que nadie tiene.**

La variante dinámica (Tucker y Gupta, 1989)

En vez de pedir N CPUs y quedarse con ellas, **la aplicación administra activamente su grado de paralelismo**:

```
Un SERVIDOR CENTRAL lleva la cuenta de:
  · qué hilos se están ejecutando
  · qué hilos quieren ejecutarse
  · los requerimientos MÍNIMOS y MÁXIMOS de CPU de cada uno

Cada aplicación SONDEA al servidor: "¿cuántas CPUs puedo usar?"
```

```
y después AUMENTA O REDUCE su número de hilos.

Ejemplo: un servidor Web corre con 10 hilos.
         Sube la demanda → le dicen que baje a 5.
         → cuando los próximos 5 hilos terminen su trabajo actual,
           DEJAN DE EJECUTARSE en vez de recibir más trabajo.
```

🔑 **Esto es política contra mecanismo (8.5) una vez más**: el servidor central da el mecanismo (cuántas CPUs hay), y la aplicación pone la política (cuántos hilos quiere tener).

C) Planificación por pandillas ★ (Ousterhout, 1982)

Es **la estrella del tema** y lo que más se pregunta. Sale de la **co-planificación**.

💥 El problema que resuelve (fig. 8-14)

```
A0 y B0 comparten el tiempo en la CPU 0
A1 y B1 comparten el tiempo en la CPU 1
A0 y A1 necesitan comunicarse MUCHO (petición → respuesta → petición → ...)

Arrancan A0 y B1:

          0      100     200     300     400     500     600
          ├───────┼───────┼───────┼───────┼───────┼───────┤
CPU 0 |   A0      B0      A0      B0      A0      B0
CPU 1 |   B1      A1      B1      A1      B1      A1

          |       |       |       |
    A0 manda    A1 la   A0 recibe
    PETICIÓN 1  recibe  la RESPUESTA 1
                y responde
                al toque
```

La cuenta del desastre:

```
t=0    A0 manda la petición 1 ...pero A1 NO ESTÁ CORRIENDO
t=100  A1 arranca, recibe la petición, responde DE INMEDIATO
       ...pero A0 no está corriendo
t=200  A0 arranca y recién ahí recibe la respuesta

→ UNA secuencia solicitud-respuesta CADA 200 ms
→ "No es algo muy bueno."
```

🔑 **Los dos hilos están corriendo en paralelo, sobre dos CPUs, y comunicándose a la velocidad de un fax.** El problema no es la CPU: es que **están fuera de fase**.

Las TRES partes — el libro las numera, es tabla de examen

```
1. Los grupos de hilos relacionados se planifican COMO UNA UNIDAD, o pandilla.

2. Todos los miembros de una pandilla se ejecutan EN FORMA SIMULTÁNEA,
   en distintas CPUs de tiempo compartido.
```

```
3. Todos los miembros de la pandilla INICIAN Y TERMINAN SUS INTERVALOS
   EN CONJUNTO.
```

El truco que lo hace funcionar

```
· El tiempo se divide en QUANTUMS DISCRETOS.
· Al inicio de cada quantum, TODAS las CPUs se reprograman a la vez,
  y arranca un hilo nuevo en cada una.
· DENTRO del intervalo NO se planifica nada.
· ⚠  Si un hilo se bloquea, SU CPU QUEDA INACTIVA hasta el final del quantum.
```

📌 **Ojo con una cosa**: la edición en español dice *"todas las CPUs se planifican en forma asíncrona"*, pero el mecanismo que describe acto seguido es **sincrónico** (todas se reprograman juntas al inicio de cada quantum). Es un desliz de la traducción. **Describí el mecanismo, no la palabra**, y nadie te puede marcar nada.

El ejemplo del libro (fig. 8-15) — 6 CPUs, 5 procesos, 24 hilos

```
ranura | CPU0  CPU1  CPU2  CPU3  CPU4  CPU5
───────┼──────────────────────────────────────
   0   |  A0    A1    A2    A3    A4    A5     ← toda la pandilla A
   1   |  B0    B1    B2    C0    C1    C2     ← pandillas B y C
   2   |  D0    D1    D2    D3    D4    E0     ← pandilla D + E0
   3   |  E1    E2    E3    E4    E5    E6     ← el resto de E
───────┼──────────────────────────────────────
   4   |  = la ranura 0 otra vez, y el ciclo se repite
```

```
A: 6 hilos   B: 3   C: 3   D: 5   E: 7   →  24 hilos, 4 ranuras
```

🔑 Por qué arregla el problema

> *"Como **todos los hilos de A se están ejecutando en conjunto**, durante un quantum pueden enviar y recibir **una gran cantidad de mensajes**, con lo cual se elimina el problema de la figura 8-14."*

```
SIN pandillas:  1 ida y vuelta cada 200 ms      🐌
CON pandillas:  decenas de idas y vueltas DENTRO de un solo quantum  🚀
```

(Tanenbaum, cap. 8.1.4, págs. 542–548, figs. 8-12 a 8-15.)

🏷 Cuadro de los tres enfoques

	Tiempo compartido	Espacio compartido	Pandillas
¿Para qué hilos?	**no relacionados**	relacionados	**relacionados**
Reparte en...	el **tiempo**	el **espacio** (CPUs)	**tiempo Y espacio**
¿Multiprogramación?	sí	**no**	sí, sincronizada

¿Sobrecarga de conmutar?	sí	**cero** ✓	sí
CPU de un hilo bloqueado	se le da a otro ✓	**queda inactiva** ✗	**inactiva hasta fin de quantum** ✗
Problema propio	contención por la cola	CPUs tiradas	desperdicio en los huecos

Mnemotecnia: *tiempo compartido = cada uno por su lado · espacio compartido = cada grupo su cancha · pandillas = todos a la vez, al mismo paso.*

Preguntas de control

1) Un sistema SMP de 8 CPUs usa **una sola cola global** de hilos listos. El administrador nota que al pasar de 8 a 32 CPUs el rendimiento **casi no mejora**, aunque haya hilos listos de sobra.

a) ¿Cuál de las dos desventajas del tiempo compartido explica esto? b) ¿Qué cambiarías, y cuáles son **los tres beneficios** de ese cambio? c) Con ese cambio, ¿qué pasa si la CPU 5 se queda sin hilos y la CPU 2 tiene 20?

2) Una aplicación científica tiene **4 hilos** que se mandan mensajes entre sí **miles de veces por segundo**. Corren en un multiprocesador de 4 CPUs con **tiempo compartido** y cola global, con quantum de 50 ms. El programador mide el rendimiento y le da **200 veces peor** de lo esperado, aunque las 4 CPUs están al 100 % de uso.

a) ¿Qué está pasando? Nombrá la figura del libro y explicá el mecanismo. b) ¿Qué algoritmo lo arregla y cuáles son sus **tres partes**? c) ⚠ Esa solución tiene un costo. ¿Cuál? ¿En qué caso **no** conviene usarla?

Corrección de las preguntas del 10.4

Las dos tienen casi todo bien. Hay un error en la 2a que **ya te marqué antes** y que en el final te descuentan.

Pregunta 1

a) ✓ **Correcta.** Es la contención por la cola global. Con 32 CPUs, todas giran sobre el mismo candado.

b) ✓ **Correcta.** Los tres beneficios están bien. El libro los da en otro orden (carga uniforme → afinidad → contención), pero el orden no cambia nada.

c) ✓ **Correcta.** Te sumo un detalle que te puede dar puntos extra:

```
El hilo que la CPU 5 le roba a la CPU 2 llega con la CACHÉ FRÍA:
pierde la afinidad.

¿Por qué igual conviene robar?
CPU ociosa       → pierde el 100 % del tiempo
hilo con caché fría → pierde algunos fallos de caché al arrancar
→ la segunda pérdida es mucho menor.
```

Es un **trade-off**: la afinidad es una preferencia, no una regla. Se rompe cuando la otra opción es tener una CPU ociosa.

Pregunta 2

a) ⚠ **Parcial.** La figura (8-14) y el desfase están bien. Pero hay **dos frases incorrectas**:

```
✗ "Cuando al Hilo 0 se le acaba el quantum, SE VA A DORMIR"
✓ Cuando se le acaba el quantum, el Hilo 0 pasa a LISTO.
  Nunca pasa a BLOQUEADO ni "se duerme".
```

🟡 **Este error ya lo tuviste en el tema 6** (H "pasaba a bloqueado" al terminar el quantum). Fin de quantum lo decide el planificador y el hilo va a **LISTO**. Dormirse lo decide el hilo, cuando pide algo que no tiene.

```
✗ "Las CPUs figuran al 100 % porque se la pasan haciendo cambios
   de contexto pesados e inútiles"
✓ Las CPUs están al 100 % porque ejecutan OTROS hilos (B0, B1),
  y ese trabajo es útil.
```

Esta segunda corrección es la más importante. En la figura 8-14 **el hardware no desperdicia nada**: las CPUs nunca están ociosas. El problema es la **latencia** de A, porque sus hilos nunca coinciden en el tiempo. Por eso el dato del 100 % era una **trampa**: la utilización es perfecta y aun así la aplicación anda 200 veces más lenta.

Un detalle menor: no hay "red". Los hilos se comunican dentro de la misma máquina.

b) ✓ **Correcta.** Las tres partes están exactas.

c) ✓ **Correcta.** El costo (la CPU de un hilo bloqueado queda ociosa hasta el fin del quantum) y el caso I/O-bound están bien. Te falta el caso más directo:

> **No conviene con hilos NO relacionados.** Si los hilos no se comunican entre sí, ejecutarlos juntos no les da **ninguna ventaja**, y el sistema igual paga los huecos. La planificación por pandillas solo sirve cuando hay comunicación.

📝 **Un comentario sobre cómo respondiste.** Las respuestas tienen las marcas `[1.1]` del otro chat. Para aprender está perfecto usarlo. Para el cuestionario de la unidad, respondé **solo**. Si no, la nota mide al otro asistente y no a vos, y en el final vas a estar solo.

Cierre de la Unidad 2

Ya vimos los **10 temas**. Estos son los 8 puntos que tenés que llevar al examen:

1. Proceso ≠ hilo. El proceso **agrupa recursos** y el hilo **ejecuta**. Cada hilo tiene su propio PC, sus registros, su pila y su estado. Los hilos de un mismo proceso comparten todo lo demás. `fork` **duplica** el proceso y `execve` **reemplaza** el programa. Con **n** `fork` seguidos quedan 2^n procesos.

2. Tres estados y cuatro transiciones. Fin de quantum → LISTO, nunca BLOQUEADO. **No existe la transición BLOQUEADO → EJECUCIÓN.**

3. Región crítica: las 4 condiciones y cuál viola cada intento fallido:

```
deshabilitar interrupciones → 2      variable candado → 1
alternancia estricta        → 3      Peterson / TSL   → funcionan
```

4. Espera ocupada contra bloqueo.

```
espera ocupada = no avanza Y gasta CPU
interbloqueo   = no avanza y NO gasta CPU
```

La **inversión de prioridades** es culpa de la espera ocupada, no de TSL.

5. Semáforos. Un `up` sin destinatario se guarda; un `wakeup` o un `signal` sin destinatario se pierde. En productor-consumidor se usan tres semáforos, con `vacias + llenas = N`. Primero va el `down` de sincronización y después el de exclusión mutua.

6. ★ Planificación: el tema que más puntos vale.

```
retorno = fin - llegada
espera = retorno - ráfaga
respuesta = primer arranque - llegada
```

Los algoritmos son FCFS, SJF, SRTF, **Round Robin**, prioridades y lotería. En tiempo real, el sistema es planificable si $\mathbf{\Sigma\ C_i/P_i \leq 1}$.

7. Problemas clásicos. En los filósofos, la solución de Dijkstra deja comer a **2 como máximo**. En lectores y escritores, **el escritor sufre inanición si l ≤ D**. Y la regla que resuelve los dos: **dormirse con un candado tomado causa interbloqueo SOLO SI el que te tiene que despertar necesita ese mismo candado.**

8. Multiprocesadores. Toda la unidad repite el mismo patrón:

```
un candado grande        →  correcto, pero lento
candados chicos e        →  correcto y rápido
independientes

SMP con un mutex global → SMP con un mutex por subsistema
mutex de toda la mesa   → estado[] de Dijkstra
cola global             → una cola por CPU (dos niveles)
```

En multiprocesador la planificación tiene **dos dimensiones**: qué hilo y en qué CPU. Para hilos relacionados se usa la **planificación por pandillas**.

🎯 **Lo que más se toma de esta unidad (fotos de los exámenes de la cátedra)**

- **Round Robin con quantum 1 y 2**: el Gantt, las métricas y **la cola de preparados en un instante t**. Desempate de la cátedra: *entre un proceso que llega y otro que sale de ejecución, primero entra el que llega.*
- **SJN y SRTF** (la cátedra los llama así).
- Comandos de Linux: `ps -ef`, `pgrep`, `kill -STOP` y `kill -CONT`, `strace`.

Practicá esto con el simulador de Gantt. Ya trae cargados los ejercicios de la cátedra.
